1. Tên đề tài — Title

Phân công: TV1 tổng hợp bìa; cả nhóm xác nhận thông tin định danh trước khi xuất bản nộp.

BOMB IT MULTIPLAYER

Thiết kế và đánh giá đồng bộ trạng thái theo delta trên kiến trúc server authoritative

Báo cáo project ứng dụng mạng — ngày tổng hợp 26/10/2026.

Thông tin bìa	Nội dung
Trường/khoa	Chưa được cung cấp thông tin chính thức
Học phần/mã học phần/lớp	Chưa được cung cấp thông tin chính thức
Giảng viên hướng dẫn	Chưa được cung cấp thông tin chính thức
Nhóm/số nhóm	Chưa xác nhận
Học kỳ/năm học	Chưa xác nhận; không suy từ ngày chạy thực nghiệm
Repository khảo sát	BombItMultiplayer-Structure

Danh sách theo tên gọi trong phân công nội bộ; chưa thay cho họ tên/MSSV trên bìa chính thức:

Vai trò	Tên gọi đã có trong tài liệu	Họ tên chính thức	MSSV
Thành viên 1	Mai	Chưa xác nhận	Chưa xác nhận
Thành viên 2	Chinh	Chưa xác nhận	Chưa xác nhận
Thành viên 3	Duy	Chưa xác nhận	Chưa xác nhận
Thành viên 4	Giang	Chưa xác nhận	Chưa xác nhận

Nguồn tên gọi: nhiemvu.md. Nhóm cần thay các ô chưa xác nhận bằng thông tin thật trước khi nộp; không dùng tên gọi ngắn làm xác nhận tác giả đầy đủ.

MỤC LỤC

1. Tên đề tài — Title 1
2. Tóm tắt — Abstract 8
3. Từ khóa — Keywords 9
4. Giới thiệu — Introduction 10
4.1. Bối cảnh và động lực 10
4.2. Vấn đề đồng bộ game nhiều người chơi 10
4.3. Mục tiêu và phạm vi 10
4.4. Đóng góp chính và tổ chức báo cáo 11
5. Cơ sở lý thuyết và công trình liên quan 12
5.1. TCP, framing, kết nối và session 12
5.2. Server authoritative, GameLoop và concurrency 13
5.3. Full/delta, prediction, reconciliation và interpolation 14
5.4. Tile map, bom, timer và event gameplay 16
5.5. Các cách tiếp cận liên quan, đối chứng và lựa chọn thiết kế 17
6. Định nghĩa bài toán 19
6.1. Input, output và đối tượng sử dụng 19
6.2. Luật game và arena hiện hành 20
6.3. Yêu cầu chức năng 23
6.4. Yêu cầu phi chức năng, mô hình mạng và giả định 24
6.5. Phạm vi chưa triển khai và giới hạn kết luận 26
7. Giải pháp đề xuất 27
7.1. Ý tưởng server authoritative và luồng input → state → client 27
7.2. Quản lý connection, session và room 29
7.3. Single-writer state, command queue và runtime theo phòng 29
7.4. Movement/collision và đồng bộ chuyển động phía client 30
7.5. State Delta Compression và phục hồi baseline 31
7.6. Bom/item/event, damage batch, respawn và kết quả 36
7.7. Protocol, match scope, routing và backpressure 37
8. Kiến trúc hệ thống 39
8.1. Sơ đồ tổng thể Client–Server 40
8.2. Tầng transport, session và room 41
8.3. Model, state, runtime và movement 42
8.4. Gameplay extension và world adapters 43
8.5. Client lobby, movement, renderer và results 44
8.6. Router, worker, queue, quyền sở hữu thread và cleanup 46
9. Thiết kế giao tiếp mạng 50
9.1. TCP framing, UTF-8, size limit và handshake protocol 2 50
9.2. Session, room, match identity và vòng đời kết nối 51
9.3. Message lobby và room 53

9.4. MOVE, input sequence/ACK, snapshot v2–v5 và full/delta/resync 54
9.5. PLANT_BOMB, event, dedupe và GAME_OVER 56
9.6. Sơ đồ tuần tự các luồng chính 58
9.7. Packet lỗi, timeout, client chậm, queue đầy và stale match 63
10. Triển khai hệ thống 66
10.1. Java 25, Swing, Maven, cấu trúc project và bootstrap 66
10.2. TCP server/client lifecycle, room/lobby/chat/heartbeat 68
10.3. Model/map/state, command queue, GameLoop và runtime 71
10.4. Movement, collision, snapshot builder và delta codec 74
10.5. Input, prediction, replay, interpolation và respawn reset 75
10.6. Bom, chain reaction, block, item, lives/shield/damage/result 78
10.7. Renderer bom/item, grid/HUD và bảng kết quả 81
10.8. Router, ClientNetwork, metrics, benchmark và vận hành 82
11. Thiết lập thực nghiệm — Experimental Setup 87
11.1. Revision, phần cứng, OS/JDK và cấu hình 87
11.2. Môi trường loopback, nhiều process và LAN nếu có 88
11.3. Map, luật game, input workload và seed 88
11.4. Kịch bản chức năng và lỗi 89
11.5. Đối chứng full mỗi tick so với delta + full định kỳ 90
11.6. Metric, đơn vị, công cụ, số lần lặp và cách tái lập 91
12. Kết quả — Results 96
12.1. Chức năng phòng/kết nối 96
12.2. Movement/runtime/client synchronization 97
12.3. Gameplay bomb/item/lives/result 98
12.4. Concurrency, fault handling và cleanup 99
12.5. Full so với delta 101
12.6. Latency/ACK/throughput/message rate theo workload 105
12.7. Test chưa đạt và phần chưa có bằng chứng 108
13. Thảo luận — Discussion 110
13.1. Mức đáp ứng mục tiêu và điều kiện thực nghiệm 110
13.2. Hiệu quả/chi phí và trường hợp delta kém lợi 110
13.3. Tick, FIFO, backlog, prediction correction và độ trễ 112
13.4. Tính nhất quán gameplay, event và respawn 113
13.5. Bottleneck, lỗi quan sát được và giới hạn kết luận 113
14. Điểm mới và đóng góp — Novelty and Contributions 115
14.1. Baseline kỹ thuật, phần kế thừa và phần tự phát triển 115
14.2. State Delta Compression: phương pháp và bằng chứng 116
14.3. Các đóng góp triển khai bổ sung được chứng minh 116
14.4. Bảng Contribution — Description — Evidence 117
15. Hạn chế — Limitations 119
15.1. Hạn chế chức năng và reconnect 119

15.2. GUI/LAN/tải/soak chưa nghiệm thu 120
15.3. Giới hạn số liệu lịch sử, fixture và công cụ đo 121
16. Hướng phát triển — Future Work 122
16.1. Hoàn thiện và mở rộng chức năng 122
16.2. Tối ưu đồng bộ/client correction và tài nguyên 123
16.3. Reconnect có xác thực/generation an toàn 123
16.4. Đánh giá nhiều máy, tải lớn hơn và vận hành dài hạn 124
17. Kết luận — Conclusion 125
18. Tài liệu tham khảo — References 126
19. Phụ lục — Appendix 127
A. Đặc tả message và sequence bổ sung 127
B. Cấu hình/build/run/test/demo 128
C. Test cases, log, CSV và kết quả bổ sung 129
D. Mã giả/đoạn code quan trọng 130
E. Bảng đóng góp thực tế — nhóm xác nhận 131

DANH MỤC HÌNH

Hình 7.1. Luồng phối hợp của giải pháp. Nhóm tự vẽ từ thiết kế và code project. 28
Hình 7.2. Tạo delta, pending và commit. Nhóm tự vẽ từ StateDeltaBuilder/GameLoop. 33
Hình 8.1. Các thành phần chính và ranh giới tiến trình. Nhóm tự vẽ từ code project. 40
Hình 8.2. Luồng command và snapshot qua các thread/queue. Nhóm tự vẽ từ code project. 46
Hình 8.3. Một lượt input đi qua runtime và trở về client. Nhóm tự vẽ từ code project. 48
Hình 9.1. Kết nối và bắt đầu một trận hai người. Nhóm tự vẽ từ code project. 59
Hình 9.2. Input sequence và nhánh phục hồi baseline. Nhóm tự vẽ từ code project. 60
Hình 9.3. Nhánh đặt bom thành công và vụ nổ sau đó. Nhóm tự vẽ từ code project. 61
Hình 9.4. Kết thúc gameplay bình thường và mở lượt mới. Nhóm tự vẽ từ code project. 62
Hình 12.1. Tổng frame ứng dụng theo cấu hình; phần trăm ghi trên cột delta là tỷ lệ giảm so với full của cùng hàng. Đơn vị trục là KiB = 1024 byte; D0 có 200 tick, D1 có 600 tick. Nguồn hình: script phân tích dữ liệu thực chạy. 103
Hình 12.2. Byte tích lũy của các trace D1 bốn player. Đường dọc ở trace sự kiện chỉ tick 100/160/172/220; đường delta vẫn có chi phí full định kỳ và update timer. Trục tick là thời gian mô phỏng, không phải thời gian đo mạng. 103
Hình 12.3. Mean RTT có trọng số và tổng RX client; cột là median qua năm run, thanh là min–max. Nhãn TCP tương ứng mode transport, Move tương ứng movement. Đo một JVM/loopback; không suy thành tải tối đa hoặc LAN. 107

DANH MỤC BẢNG

Bảng 5.1. So sánh các kỹ thuật đồng bộ theo mục tiêu sử dụng. 15
Bảng 5.2. Đối chiếu các cách tổ chức trạng thái và điều khiển. 17
Bảng 5.3. Phần kế thừa từ tài liệu và phạm vi áp dụng. 17
Bảng 6.1. Các nhóm input của hệ thống. 19
Bảng 6.2. Các nhóm output và người nhận. 20
Bảng 6.3. Cấu hình gameplay mặc định của ứng dụng. 21
Bảng 6.4. Các loại item và hiệu ứng. 22
Bảng 6.5. Danh sách yêu cầu chức năng. 23
Bảng 6.6. Các thuộc tính chất lượng cần kiểm chứng. 24
Bảng 6.7. Các giả định của phạm vi đề tài. 25
Bảng 7.1. Vấn đề cần giải quyết và cơ chế được chọn. 28
Bảng 7.2. Quy tắc phát hiện và biểu diễn thay đổi. 31
Bảng 7.3. Ví dụ logic về baseline, không phải log thực nghiệm. 34
Bảng 7.4. Minh họa byte phần giá trị tile theo layout hiện tại, không phải byte toàn packet. 35
Bảng 7.5. Lựa chọn hiện hành và hệ quả cần đánh giá. 37
Bảng 7.6. Phần kế thừa, phần nhóm hiện thực và cách kiểm chứng. 38
Bảng 8.1. Các thành phần hạ tầng server. 41
Bảng 8.2. Phân biệt dữ liệu model, state và dữ liệu bàn giao. 42
Bảng 8.3. Các lớp liên kết gameplay với state và đầu ra. 43
Bảng 8.4. Trách nhiệm các thành phần client. 44
Bảng 8.5. Thread, dữ liệu sở hữu và cơ chế giới hạn. 47
Bảng 8.6. Kiến trúc đáp ứng các nhóm yêu cầu ở mục 6. 49
Bảng 9.1. Frame ứng dụng trên TCP. 50
Bảng 9.2. Định danh và phạm vi ý nghĩa. 51
Bảng 9.3. Các message phiên/control đang được sử dụng. 52
Bảng 9.4. Các message phòng đang nối vào server/client mặc định. 53
Bảng 9.5. Message đồng bộ trạng thái, đều có match envelope ở protocol 2. 54
Bảng 9.6. Thứ tự các khối binary cơ sở v2. 54
Bảng 9.7. Các version snapshot được decoder hiện tại hỗ trợ. 55
Bảng 9.8. Body inner của các thông báo gameplay và kết quả. 57
Bảng 9.9. Các giới hạn mặc định liên quan đến giao tiếp. 63
Bảng 9.10. Cách xử lý theo lớp lỗi thực tế. 63
Bảng 10.1. Stack và cấu hình được khai báo trong project. 66
Bảng 10.2. Các script hiện có và đầu ra/chức năng. 67
Bảng 10.3. Class/API chính và điểm nối vào ứng dụng. 68
Bảng 10.4. Ý nghĩa các SessionState trong luồng hiện tại. 69
Bảng 10.5. API runtime và ranh giới dữ liệu TV2. 73
Bảng 10.6. Các lớp đồng bộ movement và trách nhiệm. 75
Bảng 10.7. Thành phần TV3 và cầu nối gameplay. 78

Bảng 10.8. ItemType, mã truyền và effect mặc định. 80
Bảng 10.9. Các lớp hạ tầng TV4 và vai trò hiện thực. 82
Bảng 10.10. Công cụ đo và phạm vi dữ liệu được ghi. 85
Bảng 11.1. Môi trường khảo sát đã có bằng chứng và thông tin cần bổ sung. 87
Bảng 11.2. Topology và kết luận được phép rút ra. 88
Bảng 11.3. Workload và nhịp phát của runner. 89
Bảng 11.4. Ma trận kiểm chứng chức năng và fault handling. 89
Bảng 11.5. Phép đo delta và mức sẵn sàng của công cụ. 91
Bảng 11.6. Metric và cách diễn giải. 91
Bảng 11.7. Kế hoạch số lần chạy, chưa phải số lần đã hoàn thành. 92
Bảng 12.1. Kết quả thực chạy và phạm vi đếm. 96
Bảng 12.2. Kết quả phòng, session và lobby. 96
Bảng 12.3. Kết quả movement và runtime. 97
Bảng 12.4. Kết quả kiểm tra đồng bộ. 98
Bảng 12.5. Kết quả gameplay và tích hợp presentation. 98
Bảng 12.6. Kết quả lifecycle và process smoke. 99
Bảng 12.7. Test concurrency/reliability theo lớp và số lần lặp. 100
Bảng 12.8. Full so với delta trên cùng trace; số byte của một lần chạy đại diện. 102
Bảng 12.9. Chi phí phát packet và phân biệt byte/tick với byte/packet. 104
Bảng 12.10. Latency theo cấu hình, đơn vị ms. 105
Bảng 12.11. Tổng lưu lượng phía client; median (min–max) theo run. 106
Bảng 12.12. Trạng thái và số đếm qua năm run/cấu hình. 107
Bảng 12.13. Bằng chứng cần bổ sung trước khi nộp. 108
Bảng 13.1. Mục tiêu, bằng chứng và mức kết luận được hỗ trợ. 110
Bảng 13.2. Cách đọc hiệu quả delta theo loại dữ liệu. 111
Bảng 13.3. Phân biệt quan sát thực tế với khả năng cần đo thêm. 113
Bảng 14.1. Phương pháp kế thừa và phần nhóm hiện thực trong project. 115
Bảng 14.2. Đóng góp được nhận trong phạm vi báo cáo. 117
Bảng 15.1. Các giới hạn cơ chế hiện hành. 119
Bảng 15.2. Khoảng trống nghiệm thu và khả năng khái quát kết quả. 120
Bảng 16.1. Các nhóm công việc đề xuất gắn với hạn chế. 122
Bảng 16.2. Ma trận kiểm tra tối thiểu đề xuất cho resume. 124
Bảng A.1. Nhóm message dùng để đối chiếu log và source. 127
Bảng B.1. Điểm vào tái lập các nhóm thực nghiệm đã công bố. 129
Bảng C.1. Nguồn dữ liệu thực tế và giới hạn sử dụng. 129
Bảng C.2. Các cột cần để đọc hoặc phân tích lại. 130
Bảng E.1. Trách nhiệm theo tài liệu; phần xác nhận cá nhân còn thiếu. 131

2. Tóm tắt — Abstract

Phân công: TV2 chủ trì; TV1/TV3 xác nhận chức năng, TV4 xác nhận số liệu.

Game nhiều người chơi cần thống nhất chuyển động và diễn biến trận giữa các client trong khi hạn chế dữ liệu lặp. Project Bomb It Multiplayer xây dựng ứng dụng Java 25/Swing qua TCP, với server quyết định trạng thái, runtime riêng theo room và command queue có giới hạn. Client dùng prediction/reconciliation cho movement, nội suy remote và replica full/delta. Cải tiến chính là State Delta Compression: so sánh tile/player/entity với baseline đã xuất bản, gửi thay đổi và xóa, bỏ update rỗng, đồng thời giữ full khởi tạo/định kỳ và cơ chế resync. Thực nghiệm đối chứng cùng trace cho thấy fixture v2 giảm từ 110076 xuống 16411 byte, tương ứng 85,09%; sáu cấu hình v5 có match envelope giảm tổng frame snapshot 35,98–97,54% tùy workload. Toàn bộ 18.000 đối chiếu state chuẩn hóa v5 đạt. Lượt suite mới đạt 133/133 JUnit test; ba lượt reliability bổ sung đạt 84/84 lần thực thi test. Benchmark TCP loopback hoàn tất 45/45 scenario-run với transport 1/2/4 client và gameplay 2/4 player. Đóng góp nằm ở hiện thực, tích hợp và đánh giá đồng bộ delta trong game, không phải phát minh delta encoding. Kết quả còn giới hạn ở fixture và loopback; chưa nghiệm thu GUI/LAN, soak dài hạn hoặc resume trận, và chưa có đối chứng TCP chứng minh compression cải thiện latency tương ứng.

3. Từ khóa — Keywords

Phân công: TV2 đề xuất; TV4 rà soát thuật ngữ.

Game nhiều người chơi; TCP; server authoritative; đồng bộ trạng thái; State Delta Compression; client-side prediction.

4. Giới thiệu — Introduction

Phân công: TV1 chủ trì bối cảnh/phạm vi; TV2 mô tả bài toán đồng bộ và đóng góp chính; TV3 xác nhận gameplay; TV4 rà soát nguồn và bằng chứng.

4.1. Bối cảnh và động lực

Một game đặt bom nhiều người chơi không chỉ cần vẽ bản đồ và xử lý phím. Các client còn phải thống nhất người chơi đang ở đâu, ô nào bị phá, bom nào đã nổ, ai nhận item và khi nào kết quả trận được chốt. Nếu các thành phần cập nhật khác thứ tự hoặc khác nền state, người chơi có thể quan sát thế giới không nhất quán dù mỗi màn hình vẫn đang hoạt động.

Project dùng TCP làm kênh truyền byte có thứ tự và tin cậy [1], đồng thời tự định nghĩa framing và message contract. Tính chất transport không thay thế việc quản lý room/match, kiểm tra quyền thao tác hoặc baseline snapshot. Nguồn nền: RFC 9293.

Mô hình server authoritative tập trung quyết định world ở server, client gửi intent rồi nhận kết quả [2]. (Gambetta, Client-Server Game Architecture). Trong Bomb It, cách tổ chức này cho phép movement, collision, bom/item và kết quả dùng cùng state của trận. Thách thức còn lại là cập nhật đủ kịp thời và nhất quán mà không lặp lại quá nhiều dữ liệu không đổi.

4.2. Vấn đề đồng bộ game nhiều người chơi

Gửi full snapshot mỗi tick là một đối chứng rõ ràng: client nhận đủ dữ liệu tại mỗi mốc cập nhật, nhưng map và các thực thể không đổi vẫn bị truyền lại. Với map theo tile, thay đổi thường có tính cục bộ: một bom được thêm/xóa, một block bị phá hoặc một player dịch chuyển. Do đó project áp dụng delta theo baseline, một kỹ thuật đã có trong tài liệu về snapshot compression [3]. (Fiedler, Snapshot Compression). Phần cần giải quyết trong implementation là chọn baseline, biểu diễn xóa, giữ thứ tự xuất bản, phát hiện nền không khớp và phục hồi bằng full mà không áp lại effect gameplay.

Ở client, chỉ chờ state server có thể làm điều khiển phụ thuộc độ trễ phản hồi; prediction/reconciliation dùng input chưa ACK là hướng tiếp cận đã có [4], còn nội suy dùng các mẫu trước để hiển thị remote giữa các lần cập nhật [5]. Những phương pháp này phải gắn với contract tick/sequence và được reset khi respawn hoặc đổi match. Chúng không cấp quyền cho client tự quyết định sát thương hoặc kết quả trận.

Từ đó, câu hỏi chính của đề tài là: **có thể giảm tổng byte snapshot so với full mỗi tick trên cùng workload, trong khi giữ replica đúng theo contract và ghép được vào luồng game nhiều người chơi hay không?** Các câu hỏi bổ sung là hệ thống xử lý đồng thời/lifecycle/lỗi như thế nào và latency/lưu lượng quan sát được ở số client đã thử nằm trong phạm vi nào. Báo cáo phân biệt các câu hỏi này thay vì coi ít byte hơn đồng nghĩa ít latency hơn.

4.3. Mục tiêu và phạm vi

Mục tiêu của project gồm tổ chức lobby/room, xác lập authority server, cập nhật gameplay theo loop, đồng bộ full/delta tới client và kiểm tra các ranh giới lỗi/cleanup. Implementation dùng Java 25, Swing/AWT, TCP và custom Java 2D renderer; server tạo runtime riêng theo room. Một match lobby có 2–4 player, map arena hiện tại 17 × 15 ô, bước mô phỏng mục tiêu 50 ms.

Luật cụ thể, input/output và yêu cầu chức năng được xác định ở mục 6, không suy từ hình ảnh minh họa.

Đánh giá gồm ba nhóm: kiểm thử logic/tích hợp; đối chứng frame full/delta trên cùng trace; và benchmark TCP loopback theo mode/số client. Mỗi kết quả được gắn với source/artifact, môi trường, workload và dữ liệu gốc như mục 11–12. Full-every-tick là chính sách đối chứng trong harness, không khẳng định đó là một commit sản phẩm cũ đã được đo.

Phạm vi kết quả hiện tại chưa gồm nghiệm thu GUI/LAN nhiều máy, tải tối đa, soak dài hạn hoặc resume trận. Disconnect/leave được xử lý theo chính sách kết thúc lượt hiện hành. Các giới hạn không được thay bằng giả định tính năng đã có hoặc bằng số liệu dự kiến.

4.4. Đóng góp chính và tổ chức báo cáo

Đóng góp chính là hiện thực và đánh giá cơ chế State Delta Compression cho state Bomb It: diff tile/player/entity, tombstone, baseline commit khi enqueue, full policy và replica/resync. Các đóng góp bổ sung là tích hợp runtime/gameplay theo room, contract movement/client qua ACK/respawn/match scope, và bộ bằng chứng có raw data cùng công cụ kiểm tra lại. Mức độ đóng góp và giới hạn từng claim được trình bày ở mục 14; delta, TCP và prediction/interpolation là các kỹ thuật kế thừa, không được nhận là phát minh của nhóm.

Báo cáo được tổ chức theo các phần: mục 5 trình bày kiến thức nền/công trình liên quan; mục 6 định nghĩa bài toán; mục 7–10 mô tả giải pháp, kiến trúc, giao thức và implementation. Mục 11–12 trình bày thiết kế thực nghiệm và kết quả; mục 13–14 thảo luận và gắn đóng góp với bằng chứng. Mục 15–17 nêu hạn chế, hướng phát triển và kết luận. References ở mục 18 và phụ lục ở mục 19 cung cấp nguồn, contract/lệnh tái lập cùng bản đồ dữ liệu phục vụ kiểm tra.

5. Cơ sở lý thuyết và công trình liên quan

Chương này trình bày cơ sở giao tiếp và đồng bộ cho Bomb It Multiplayer: truyền thông TCP, vòng cập nhật phía server, biểu diễn trạng thái, xử lý input tại client và mô hình gameplay trên bản đồ ô. Các phương pháp liên quan được đối chiếu theo yêu cầu của đề tài, gồm tính nhất quán, phản hồi điều khiển, dung lượng dữ liệu và khả năng triển khai trong phạm vi nhóm. Số nguồn trong chương dùng chung với danh mục tài liệu tham khảo.

Phân công: TV1 và TV4 phụ trách mục 5.1.

5.1. TCP, framing, kết nối và session

5.1.1. Đặc tính TCP và biên thông điệp

TCP cung cấp dịch vụ truyền luồng byte tin cậy, theo thứ tự giữa các đầu kết nối. Biên các lần ghi dữ liệu của ứng dụng không được bảo toàn thành biên các lần đọc ở phía nhận. Vì vậy, một thông điệp có thể cần nhiều lần đọc; một lần đọc cũng có thể chứa dữ liệu của nhiều thông điệp. Ứng dụng phải tự xác định ranh giới thông điệp, thay vì giả định mỗi lần đọc trả về một gói hoàn chỉnh. [1]

Trong đề tài, một **frame ứng dụng** gồm độ dài, loại thông điệp và payload JSON UTF-8: `[length:int32][type:int16][payload]`. Frame khác với TCP segment ở tầng vận chuyển. Quy ước độ dài cho phép bộ giải mã biết số byte còn phải nhận; trường loại cho phép chuyển thông điệp đến chức năng tương ứng. Giới hạn độ dài và kiểm tra payload là yêu cầu của protocol Bomb It, không phải thuộc tính tự có của TCP. Đặc tả và hiện thực nằm trong protocol và PacketEncoder.

Truyền tin cậy cũng có chi phí: dữ liệu phía sau có thể phải chờ phần dữ liệu trước được truyền lại. Do đó, nhóm không xem TCP là bảo đảm mọi cập nhật đến đúng hạn của tick. Đây là hệ quả cần cân nhắc từ cơ chế truyền lại và giao dữ liệu theo thứ tự của TCP. [1]

5.1.2. Kết nối vật lý và trạng thái phiên ở mức ứng dụng

Trong mô hình của đề tài, **connection** là kênh socket, **session** gắn hoạt động ứng dụng với một người chơi, **room** tổ chức nhóm trước trận và **match** xác định một lượt chơi cụ thể. Các định danh này có vai trò khác nhau: kênh còn hoạt động không có nghĩa người chơi được phép gửi lệnh cho mọi room hoặc match. Việc ánh xạ danh tính và kiểm tra phạm vi thông điệp được quy định trong protocol của nhóm.

Java cung cấp các API kết nối, luồng đọc/ghi và đóng socket. `setSoTimeout` đặt giới hạn thời gian chặn cho thao tác đọc; giá trị 0 cho phép chờ vô hạn. Khi xảy ra read timeout, API không mặc nhiên làm socket mất hiệu lực. Giới hạn này khác với deadline ghi hoặc deadline nhận trọn một frame. [6]

Với Bomb It, heartbeat, chính sách timeout và xử lý kết thúc kết nối thuộc tầng ứng dụng. Các định danh phiên/trận còn giúp từ chối thông điệp không thuộc lượt đang chơi. Bản hiện hành chưa cung cấp chức năng reconnect để tiếp tục trận; có token hoặc API session không đủ để khẳng định tính năng này đã hoàn thiện. Phạm vi đó được ghi trong hồ sơ khảo sát.

5.1.3. Độ trễ và ý nghĩa phép đo

RTT là độ trễ khứ hồi; đường đi và đường về có thể bất đối xứng. Vị trí lấy timestamp và thời gian xử lý ở các đầu đo ảnh hưởng kết quả, nên không mặc nhiên suy ra độ trễ một chiều bằng RTT chia hai. [7]

Trong thiết kế thực nghiệm của nhóm, RTT PING/PONG được hiểu ở mức ứng dụng, bao gồm các bước xử lý giữa hai thời điểm ghi nhận. Cần phân biệt chỉ số này với thời gian input được server xác nhận và độ trễ hiển thị. Định nghĩa, cách lấy mẫu và môi trường đo sẽ được trình bày ở mục 11; chương này chưa đưa ra số đo hiệu năng.

Phân công: TV2 phụ trách mục 5.2.

5.2. Server authoritative, GameLoop và concurrency

5.2.1. Quyền quyết định trạng thái

Trong kiến trúc server authoritative, client gửi ý định thao tác và server xác định kết quả hợp lệ. Gambetta dùng mô hình này để giải thích vì sao vị trí hoặc sức khỏe do client tự khai báo không nên được coi là trạng thái cuối cùng. Server vẫn có thể có lỗi hoặc lỗ hổng; kiến trúc này không phải bảo đảm chống được mọi gian lận. [2]

Bomb It áp dụng nguyên tắc trên cho movement, collision, đặt bom và kết quả gameplay. Chẳng hạn, lệnh di chuyển biểu thị hướng và trạng thái di chuyển, còn server tính vị trí sau khi kiểm tra vật cản. Client có thể dự đoán hình ảnh điều khiển nhưng không thay thế quyết định của server. Luồng xử lý này được hiện thực tại GameLoop và BombGameplayExtension.

5.2.2. Vòng cập nhật và bước thời gian

Nystrom mô tả game loop như vòng lặp tiếp nhận input, cập nhật trạng thái và phục vụ hiển thị, qua đó tách tiến trình game khỏi tốc độ phát sinh input. Với server của đề tài, phần hiển thị nằm ở client; đầu ra của vòng lặp là dữ liệu trạng thái được công bố cho tầng mạng. [8]

Bước thời gian biến đổi sử dụng khoảng thời gian thực giữa các lần cập nhật; bước cố định giữ nguyên Δt mô phỏng. Fiedler phân tích ảnh hưởng của cách chọn bước đến mô phỏng và nguy cơ quá tải khi phải chạy bù nhiều bước. Bước cố định không tự bảo đảm chương trình luôn theo kịp thời gian thực. [9]

Để mô tả vòng cập nhật của nhóm, gọi S_k là state ở tick k, C_k là các command được chọn xử lý và F là toàn bộ phép cập nhật movement/gameplay. Có thể viết:

$$S_{k+1} = F(S_k, C_k, \Delta t).$$

Đây là ký hiệu mô tả thiết kế, không khẳng định F đã được chứng minh deterministic giữa mọi máy. Trong cấu hình mặc định, $\Delta t = 0{,}05\ s$, tương ứng nhịp mục tiêu 20 tick/giây. Một tick xử lý command, cập nhật movement, gọi gameplay extension và tạo full/delta snapshot. Nếu tick trễ, loop điều chỉnh mốc chạy tiếp; code hiện tại không chạy bù bằng một accumulator đầy đủ. Các chi tiết này lấy từ GameLoop của project, không phải tần số hay thuật toán bắt buộc do tài liệu tham khảo quy định.

5.2.3. Queue hữu hạn và quyền sở hữu state

`BlockingQueue` hỗ trợ các cách xử lý khi chưa thể thêm/lấy phần tử: trả kết quả thất bại, chặn hoặc chờ có thời hạn. `offer` không chờ đến khi có chỗ trống như `put`. Queue an toàn khi dùng từ nhiều thread không đồng nghĩa mọi đối tượng mutable đưa qua queue cũng an toàn. [10]

Từ đó, nhóm chọn một luồng cập nhật state cho mỗi runtime: network tiếp nhận và đưa command vào queue; game thread quyết định khi nào áp dụng; tầng gửi sử dụng snapshot bất biến. Queue hữu hạn cho phép nhận biết quá tải, còn giới hạn command mỗi tick ngăn bước đọc queue chiếm toàn bộ vòng cập nhật. Tuy nhiên, FIFO và giới hạn này vẫn có thể tạo backlog. Thiết kế không mặc nhiên bảo đảm mọi command được xử lý ngay ở tick kế tiếp. Các tham số và chính sách cụ thể sẽ được giải thích trong phần kiến trúc/triển khai.

Ở client Swing, phần lớn thao tác với thành phần giao diện phải thực hiện trên Event Dispatch Thread (EDT), trừ các ngoại lệ API cho phép; `invokeLater` hỗ trợ chuyển công việc sang EDT. [11] Do đó, đồng bộ dữ liệu và chuyển kết quả sang giao diện là hai trách nhiệm cần phối hợp trong thiết kế client.

Phân công: TV2 phụ trách mục 5.3; TV4 đối chiếu format và cách tính dung lượng.

5.3. Full/delta, prediction, reconciliation và interpolation

5.3.1. Full snapshot và delta snapshot

Snapshot là ảnh dữ liệu cần thiết để phía nhận tái dựng trạng thái hoặc biểu diễn game tại một thời điểm. Trong cách tiếp cận snapshot của Fiedler, client dựng biểu diễn từ dữ liệu server thay vì phải mô phỏng giống hệt server. [12]

Full snapshot mang dữ liệu nền đầy đủ theo contract; delta chỉ biểu diễn thay đổi so với **baseline**. Fiedler chỉ ra rằng phía nhận phải có đúng baseline mới giải mã được delta tương ứng. [3]

Nhóm dùng ký hiệu `D_(b→t) = diff(S_b, S_t)` và `apply(S_b, D_(b→t)) = S_t` để mô tả yêu cầu khôi phục đối với **phần state được contract đồng bộ**. Phép `diff` phải biểu diễn đủ cả thay đổi, thêm mới và xóa; đây không nhất thiết là phép trừ số học. Trong Bomb It, delta bao gồm tile, player và entity theo contract, không chỉ các ô nền thay đổi. Client ghép delta vào replica trước khi dùng dữ liệu cho hiển thị hay reconciliation. Hiện thực tham chiếu là StateDeltaBuilder và ClientWorldState.

Baseline không được đồng nhất với "tick ngay trước". Project có thể không công bố update rỗng; delta chỉ áp dụng khi `baseTick` khớp replica. Khi không khớp, client yêu cầu full state. Server hiện gửi full lúc khởi tạo, theo chu kỳ và khi được yêu cầu; baseline được commit sau khi enqueue update thành công. Commit này chưa phải xác nhận client đã nhận hoặc áp dụng. Vì vậy, cơ chế project khác ACK baseline trong ví dụ UDP của Fiedler. [3]

Base64 là phép biểu diễn dữ liệu nhị phân bằng ký tự, với 3 byte được biểu diễn bởi 4 ký tự. Từ quy tắc đó, dạng có padding có độ dài `4 × ceil(n/3)` cho `n` byte đầu vào. [13] Trong codec của nhóm, binary được bọc Base64 trong JSON. Do đó, phần giảm dữ liệu phải được đánh giá từ delta sau khi tính chi phí biểu diễn/envelope ở cùng một lớp; không gọi Base64 là thuật toán nén.

5.3.2. Client-side prediction và server reconciliation

Prediction cho phép client mô phỏng input local trước khi nhận kết quả server. Reconciliation sử dụng số thứ tự input và số input cuối server đã xử lý: client trở về trạng thái authoritative, bỏ input đã được xác nhận, rồi phát lại các input còn chờ. Cơ chế này xử lý chênh lệch giữa trạng thái server trong quá khứ và trạng thái client đã dự đoán tiếp. [4]

Áp dụng vào project, một chu kỳ hiệu chỉnh gồm:

1. Áp dụng snapshot hợp lệ vào replica và cập nhật map/entity dùng kiểm tra va chạm.
2. Đọc `lastProcessedInput` của nhân vật local.
3. Khôi phục dữ liệu nhân vật từ server, bỏ input đã ACK và replay phần lịch sử còn lại.

Các bước được hiện thực trong PredictionController và ReconciliationController. Client dùng lại movement/collision nhưng chỉ có world mới nhất đã nhận; ví dụ bom hoặc block mới thay đổi có thể chưa xuất hiện trong dữ liệu dự đoán. Vì vậy, vẫn cần hiệu chỉnh. Cơ chế này cải thiện phản hồi local theo mục tiêu thiết kế; mức cải thiện trải nghiệm thực tế cần được đo hoặc nghiệm thu riêng.

5.3.3. Interpolation cho thực thể từ xa

Client không có input tương lai của những người chơi khác. Gambetta đối chiếu dead reckoning, ước lượng tiếp từ trạng thái cũ, với interpolation, hiển thị giữa các mẫu đã biết. Nội suy phù hợp khi cần hạn chế dự đoán sai do đối tượng đổi hướng hoặc dừng đột ngột, đổi lại phía hiển thị phải lùi thời gian. [5]

Với hai mẫu vị trí `p_a`, `p_b` ở tick `a`, `b`, công thức nội suy tuyến tính được dùng trong project là:

`α = clamp((r − a)/(b − a), 0, 1)`, với `b > a`;

`p(r) = (1 − α)p_a + αp_b`.

Trong đó `r` là tick muốn hiển thị. Buffer snapshot giúp có các mẫu bao quanh thời điểm render, nhưng thêm buffer cũng tạo đánh đổi giữa tính liên tục và độ trễ. [12]

InterpolationController của nhóm nội suy tuyến tính vị trí, giữ mẫu biên khi thiếu dữ liệu và xóa lịch sử khi phát hiện respawn. Đây là biểu diễn phục vụ client, không thay đổi vị trí authoritative trên server. Project chưa hiện thực rewind phục vụ lag compensation; cũng không sử dụng Hermite/quaternion như một số ví dụ tham khảo.

5.3.4. Phân biệt mục tiêu các kỹ thuật

Bảng 5.1. So sánh các kỹ thuật đồng bộ theo mục tiêu sử dụng.

Kỹ thuật	Mục tiêu	Điều kiện/chi phí	Vai trò trong Bomb It
Full snapshot	Tái dựng nền đầy đủ	Dữ liệu lặp lại; không phụ thuộc baseline delta	Khởi tạo và phục hồi
Delta snapshot	Tránh gửi lại phần không đổi	Cần baseline khớp và metadata thay đổi [3]	Đồng bộ tăng dần
Prediction + reconciliation	Phản hồi input local [4]	Lịch sử input, replay và khả năng hiệu chỉnh	Nhân vật local
Interpolation	Làm mượt dữ liệu từ xa [5]	Buffer và độ trễ hiển thị	Nhân vật remote

Các kỹ thuật có thể kết hợp vì tác động đến những khâu khác nhau. Giảm dung lượng không tự chứng minh phản hồi local nhanh hơn; hình ảnh mượt cũng không tự chứng minh replica luôn khớp server. Trong báo cáo, mỗi mục tiêu cần chỉ số và bằng chứng phù hợp.

Phân công: TV3 phụ trách mục 5.4; TV2 xác nhận model, collision và thứ tự runtime.

5.4. Tile map, bom, timer và event gameplay

5.4.1. Mô hình bản đồ và thực thể

Trong mô hình Bomb It, nền bản đồ là lưới `W × H`, mỗi ô có cạnh `a` pixel. Một điểm hợp lệ `(x, y)` ánh xạ sang cột `c = floor(x/a)` và hàng `r = floor(y/a)`. Tile nền xác định vùng trống, tường cứng hoặc block có thể phá; bom và item là entity riêng trên lưới. Việc tách này cho phép một ô nền trống có entity và cho phép quản lý vòng đời bom/item mà không đồng nhất chúng với terrain. Định nghĩa nằm trong TileType và EntityData.

Nhân vật có hitbox hình chữ nhật. Kiểm tra va chạm cần xét các ô hitbox phủ lên, thay vì chỉ ô chứa một điểm đại diện. Trong hiện thực của nhóm, movement chia quãng đường thành bước nhỏ, kiểm tra từng trục, giới hạn biên map và từ chối vị trí chồng vật cản. Bom chặn lối vào nhưng người đang chồng ô bom được bước ra. Đây là quy tắc cụ thể từ CollisionSystem, không phải tuyên bố rằng mọi game bản đồ ô đều dùng cùng thuật toán.

5.4.2. Timer và thứ tự xử lý bom

Java phân biệt thời gian lịch qua `currentTimeMillis` với nguồn đo khoảng thời gian qua `nanoTime`. Giá trị `nanoTime` có gốc tùy ý trong một JVM; chỉ hiệu giữa các lần đọc trong cùng JVM mới có ý nghĩa đo thời lượng. Không dùng trực tiếp giá trị đó như timestamp lịch chung giữa server và client. [14]

Trong project, deadline bom được theo dõi bằng clock nano ở phía server, còn timestamp lịch phục vụ dữ liệu sự kiện. Game thread kiểm tra bom đến hạn trong gameplay extension sau movement, trước snapshot. Điều này xác định thứ tự biến đổi state; không có thread scheduler riêng sửa world của runtime đang chạy. Nếu tick bị chậm, thời điểm áp dụng vụ nổ vẫn phụ thuộc lúc game thread xử lý; deadline không bảo đảm một callback chạy chính xác tại thời điểm đó. Các cơ chế này được kiểm tra tại BombWorldAdapter và BombGameplayExtension.

Vụ nổ được mô hình hóa bằng các tia theo bốn hướng, giới hạn bởi range và vật cản. Bom khác trong vùng ảnh hưởng có thể đưa vào hàng đợi nổ dây chuyền. BombSystem theo dõi bom đã xử lý và gom ảnh hưởng trong một batch trước khi áp dụng damage; block bị phá trong batch vẫn chắn các tia còn lại của batch đó. Đây là quy tắc nhất quán trong thiết kế gameplay của nhóm, sẽ được mô tả chi tiết ở phần triển khai TV3.

5.4.3. Trạng thái và sự kiện

Trong thiết kế của đề tài, **state** trả lời "hiện có gì", còn **event** mô tả "điều gì vừa xảy ra". Một snapshot có thể cho biết bom không còn tồn tại; thông tin vụ nổ và vùng ảnh hưởng giúp biểu diễn hành động vừa diễn ra. Hai dạng dữ liệu phục vụ mục tiêu khác nhau nên cần phối hợp, đặc biệt khi cập nhật thưa hoặc client cần resync.

Project sử dụng entity, frame hiệu ứng và gameplay notice/event để truyền phần dữ liệu tương ứng; quyền quyết định damage, item và kết quả vẫn thuộc server. Phần thiết kế protocol sẽ giải thích định danh và cách tránh áp dụng lặp theo từng loại thông điệp. Không từ việc có event suy ra hệ thống đã dùng event sourcing hoặc đã bảo đảm "exactly once" đầu cuối.

Phân công: TV4 ghép mục 5.5; TV1/TV2/TV3 xác nhận lựa chọn của module mình.

5.5. Các cách tiếp cận liên quan, đối chứng và lựa chọn thiết kế

5.5.1. Đối chiếu mô hình đồng bộ

Bảng 5.2. Đối chiếu các cách tổ chức trạng thái và điều khiển.

Cách tiếp cận	Ưu điểm theo mục tiêu thiết kế	Ràng buộc/hạn chế	Lựa chọn của đề tài
Server authoritative, client chờ state	Quyền quyết định tập trung [2]	Phản hồi local phụ thuộc đường input–server–state [4]	Dùng quyền quyết định server; bổ sung prediction
Deterministic lockstep	Đồng bộ bằng input với mô phỏng giống nhau	Cần tính deterministic; có thể chờ input của phía chậm [12]	Không dùng làm cơ chế đồng bộ toàn world
Snapshot + hỗ trợ phía client	Truyền kết quả mô phỏng; client có thể dựng hình từ state [12]	Chi phí snapshot, hiệu chỉnh và buffer	Dùng full/delta, prediction/reconciliation và nội suy vị trí remote

Các hàng thể hiện những lựa chọn cần phối hợp về quyền quyết định, dữ liệu truyền và biểu diễn; chúng không phải ba loại kiến trúc loại trừ nhau hoàn toàn. Nhóm chọn truyền state từ server để không yêu cầu mọi client mô phỏng toàn world giống hệt nhau. Việc dùng chung movement/collision hỗ trợ prediction local, chưa phải bằng chứng game chạy theo deterministic lockstep.

5.5.2. Giá trị và giới hạn của tài liệu liên quan

Bảng 5.3. Phần kế thừa từ tài liệu và phạm vi áp dụng.

Tài liệu/cách tiếp cận	Nội dung kế thừa	Giới hạn khi áp dụng vào Bomb It
Nystrom: Game Loop [8]	Tổ chức input và cập nhật liên tục	Server xuất snapshot, client đảm nhiệm render
Fiedler: timestep [9]	Xem xét bước mô phỏng và quá tải	Code không chạy bù bằng accumulator đầy đủ
Gambetta: multiplayer, Parts I–III [2]–[5]	Quyền server, replay input và nội suy	Áp dụng vào movement trên tile map; cần xử lý thay đổi world/respawn
Fiedler: Snapshot Interpolation [12]	Đánh đổi buffer với độ trễ	Chỉ dùng nội suy tuyến tính vị trí 2D
Fiedler: Snapshot Compression [3]	Delta theo baseline	TCP + `baseTick`/full/resync; không sao chép ACK UDP hay mọi kỹ thuật nén trong bài

Đây là các tài liệu kỹ thuật do tác giả xuất bản và chuẩn/API chính thức; báo cáo không gọi toàn bộ chúng là bài báo phản biện. Nhóm dùng chúng làm cơ sở phương pháp, rồi mô tả phần hiện thực của mình qua model, queue, snapshot contract và tích hợp client. Các thông số, tần số hay mức tiết kiệm trong ví dụ nguồn ngoài không được dùng làm kết quả của Bomb It.

5.5.3. Đối chứng đánh giá State Delta Compression

Đối chứng thực nghiệm dự kiến là **gửi full mỗi tick**, so với **delta kết hợp full theo chính sách của nhóm**. Hai chế độ cần dùng cùng world, workload, thời lượng, cấu hình và phép đếm byte. Chi phí full ban đầu, full định kỳ và resync của chế độ delta phải được tính, nếu phát sinh trong kịch bản. Đây là thiết kế đối chứng của nhóm; chưa phải kết quả đã đo trong chương này.

Gọi `B_full` là tổng byte ở chế độ đối chứng và `B_delta` là tổng byte ở chế độ có delta, đều đo tại cùng lớp dữ liệu. Khi `B_full > 0`, tỷ lệ tiết kiệm được định nghĩa:

`Saving(%) = 100 × (1 − B_delta/B_full).`

Giá trị âm biểu thị chế độ delta dùng nhiều byte hơn trong kịch bản đó. Để phân tích chi phí protocol, cần chỉ rõ đo binary snapshot, JSON payload hay frame ứng dụng; không trộn các lớp hoặc gọi byte ứng dụng là toàn bộ lưu lượng TCP/IP trên đường truyền. Ngoài dung lượng, thực nghiệm cần kiểm tra việc replica khôi phục đúng phần state được đồng bộ và hành vi khi sai baseline.

Delta compression đã là kỹ thuật được tài liệu trước mô tả [3]. Vì vậy, đóng góp của đề tài được đặt ở việc xây dựng format và cơ chế full/delta cho Bomb It, tích hợp với runtime/client và đánh giá bằng đối chứng phù hợp. Hiệu quả thực tế và giới hạn của lựa chọn này sẽ được kết luận từ các chương thực nghiệm, kết quả và thảo luận.

6. Định nghĩa bài toán

Bomb It Multiplayer đặt ra bài toán xây dựng một game đối kháng trên bản đồ ô, trong đó nhiều người chơi điều khiển nhân vật, đặt bom, phá vật cản và thu thập vật phẩm trong cùng một trận qua mạng. Hệ thống phải xác định một kết quả gameplay thống nhất dù input đến từ các client khác nhau, đồng thời hỗ trợ vòng đời từ vào phòng, sẵn sàng, bắt đầu, chơi đến kết thúc và chơi lượt mới.

Phạm vi của chương là ứng dụng lobby/gameplay mặc định của project, không phải các profile demo hoặc fixture kiểm thử. Các giá trị mặc định bên dưới được đối chiếu với code; cấu hình khác phải được ghi riêng khi thực nghiệm. Yêu cầu trong chương là tiêu chí cần đáp ứng và kiểm chứng, không phải tuyên bố toàn bộ hệ thống đã được nghiệm thu.

Phân công: TV3 chủ trì mục 6; TV1/TV4 xác nhận lobby và giao tiếp, TV2 xác nhận input và state.

6.1. Input, output và đối tượng sử dụng

6.1.1. Đối tượng sử dụng và ranh giới hệ thống

Người sử dụng trực tiếp là các người chơi chạy ứng dụng client desktop. Trong mỗi phòng, một người giữ vai trò chủ phòng (host), được yêu cầu bắt đầu hoặc kết thúc lượt chơi; các người còn lại tham gia và xác nhận sẵn sàng. Host là quyền trong phòng, không đồng nghĩa máy đó là server authoritative. Người vận hành khởi động ứng dụng server và lựa chọn cấu hình triển khai; phạm vi hiện tại chưa có giao diện quản trị riêng.

Ranh giới hệ thống gồm server, các client và giao tiếp giữa chúng. Bàn phím và thao tác giao diện tạo yêu cầu; server kiểm tra yêu cầu và áp dụng luật; client nhận kết quả để hiển thị. Session, room và match là các phạm vi định danh khác nhau. Một người chỉ được thao tác gameplay trong trận mà phiên của mình đang tham gia.

Với một trận, có thể mô tả bài toán bằng ba thành phần: trạng thái khởi tạo hợp lệ, chuỗi input theo thời gian và bộ luật đã cấu hình. Đầu ra cần có là chuỗi trạng thái có thể đồng bộ tới các client cùng kết quả cuối lượt. Phạm vi đồng bộ bao gồm map, nhân vật, bom, item, hiệu ứng và thông tin trận theo contract; không yêu cầu mọi hình ảnh client tại cùng thời điểm thực phải giống hệt nhau, vì prediction và interpolation có thể biểu diễn các thời điểm khác nhau.

6.1.2. Dữ liệu đầu vào

Bảng 6.1. Các nhóm input của hệ thống.

Nhóm input	Nội dung	Nơi phát sinh	Điều kiện xử lý
Kết nối và danh tính hiển thị	Địa chỉ/cổng server, tên hiển thị, thông tin handshake	Người chơi/client	Kết nối và handshake hợp lệ trước các thao tác được bảo vệ
Quản lý phòng	Xem danh sách, tạo phòng, chọn sức chứa, vào/rời phòng, ready, chat	Người chơi/client	Đúng trạng thái và thành viên phòng; chỉ host được bắt đầu/kết thúc lượt
Di chuyển	Hướng `UP/DOWN/LEFT/RIGHT`, cờ đi/dừng và sequence input	Client lấy mẫu điều khiển	Đúng match, nhân vật còn tham gia; server kiểm tra và tính vị trí
Đặt bom	Ý định đặt bom và mã yêu cầu	Client	Đúng trận, nhân vật còn sống, ô hợp lệ, còn quota và chưa có bom tại ô đó

Nhóm input	Nội dung	Nơi phát sinh	Điều kiện xử lý
Phục hồi đồng bộ	Yêu cầu full state khi replica cần đồng bộ lại	Client	Đúng match; tuân theo giới hạn tần suất và chính sách server
Vòng đời kết nối	Rời ứng dụng, EOF, lỗi socket, heartbeat/timeout	Client, transport và server	Server áp dụng chính sách dọn phiên/phòng/trận
Cấu hình trận	Map, spawn, tốc độ, fuse, quota, range, thời lượng, mạng sống và quy tắc item	Server khi khởi tạo	Giá trị hợp lệ; không lấy trực tiếp từ client làm luật authoritative
Thời gian và lựa chọn ngẫu nhiên	Thời lượng đã trôi qua, bom đến hạn, quyết định drop item	Server	Xử lý trong vòng cập nhật; không phụ thuộc clock hoặc kết quả random do client khai báo

Trong giao diện hiện tại, input di chuyển được lấy mẫu theo bốn hướng; nhả phím tạo ý định dừng. Đây là input thao tác, không phải tọa độ đích hoặc tốc độ do người chơi tự quyết định. Danh sách trường và mã message chi tiết thuộc mục 9; điều kiện nghiệp vụ thuộc phạm vi chương này.

6.1.3. Dữ liệu đầu ra

Bảng 6.2. Các nhóm output và người nhận.

Nhóm output	Nội dung cần cung cấp	Người nhận/mục đích
Kết quả kết nối/yêu cầu	Xác nhận phiên hoặc lỗi có thể xử lý	Client biết kết quả thao tác và trạng thái kết nối
Thông tin lobby/phòng	Danh sách phòng, thành viên, host, ready, trạng thái phòng và chat	Các client thuộc phạm vi tương ứng
Khởi tạo trận	Định danh match, roster và ánh xạ người chơi; full state ban đầu	Client tạo world của lượt mới
Đồng bộ gameplay	Full/delta map, nhân vật, entity, tick/baseTick và input ACK	Client duy trì replica, reconciliation và hiển thị
Diễn biến trận	Bom/item/hiệu ứng và thông tin mạng sống, khiên, thời gian còn lại	Client biểu diễn gameplay; không tự quyết định lại damage/pickup
Kết quả cuối lượt	Lý do kết thúc, người thắng nếu có, bảng xếp hạng khi áp dụng	Các client cùng trận hiển thị kết quả và trở về luồng phòng

Tính đúng của output không chỉ là dữ liệu hợp lệ về cú pháp: nó phải thuộc đúng room/match, phản ánh các input đã được server xử lý và không gây áp dụng lặp hiệu ứng gameplay. Phần đánh giá sẽ phân biệt tính đúng replica với độ mượt hình ảnh và thời gian phản hồi điều khiển.

Phân công: TV3 phụ trách luật gameplay; TV2 xác nhận map, spawn, movement và collision; TV1 xác nhận luật phòng.

6.2. Luật game và arena hiện hành

6.2.1. Phòng, người chơi và khởi tạo

Một phòng có sức chứa từ 2 đến 4 người. Người chơi chỉ được vào phòng đang chờ và còn chỗ; không bổ sung người vào roster giữa trận. Host chỉ bắt đầu khi phòng có ít nhất hai người và mọi người khác đã ready. Thao tác Start của host không yêu cầu thêm một bước ready riêng cho host.

Sau khi lượt kết thúc, các thành viên còn kết nối trở về trạng thái trong phòng; khách cần ready lại trước lượt mới. Lượt mới có match identity và world mới, không kế thừa bom, item hoặc nâng cấp của lượt trước. Khi host rời phòng, người còn lại vào sớm nhất nhận quyền host; phòng không còn người được xóa. Các điều kiện này được đối chiếu tại Room, RoomManager và LobbyGameServer.

6.2.2. Thông số mặc định

Bảng 6.3. Cấu hình gameplay mặc định của ứng dụng.

Thành phần	Giá trị/quy tắc mặc định
Arena	17 cột × 15 hàng, 32 pixel/ô; kích thước toàn map 544 × 480 pixel
Spawn	Bốn ô `(1,1)`, `(15,1)`, `(1,13)`, `(15,13)`, chỉ số ô bắt đầu từ 0
Nền bản đồ	Ô trống, tường cứng và block phá được; bom/item là entity riêng
Hitbox nhân vật	24 × 24 pixel, căn giữa ô spawn
Movement	Bốn hướng, tốc độ ban đầu 80 pixel/giây; không di chuyển chéo
Bom ban đầu	Thời gian chờ nổ 3000 ms, range 1, quota 1 bom đang tồn tại/người
Giới hạn nâng cấp	Quota 5, range 8, tốc độ 144 pixel/giây
Thời lượng	180 giây/trận
Mạng sống	3 mạng ban đầu/người, trần mặc định 99 khi nhặt thêm mạng
Vật phẩm đầu trận	Không có
Drop item	Mỗi block vừa phá có xác suất 75% tạo item; khi có drop, chọn đều trong 5 loại
Khiên đầu trận	Không có

Nguồn thông số: DefaultArenaMap, PlayerMovementRules, MatchRules và cấu hình server. Các property phía server có thể đổi một số giá trị. Bảng trên không mặc nhiên áp dụng cho fixture, demo rút gọn hoặc mọi lần benchmark; hiện chưa có màn hình tùy chỉnh bộ luật riêng cho từng phòng.

6.2.3. Movement và va chạm

Vị trí, hitbox và tốc độ được tính bằng pixel, còn map/entity sử dụng tọa độ ô. Nhân vật còn sống được đi trên nền trống nhưng không vượt biên map, xuyên tường cứng hoặc block chưa phá. Bom chặn nhân vật đi vào vùng chiếm chỗ của nó; người đang chồng vùng bom lúc đặt được bước ra, nhưng không được quay vào sau khi đã ra hết. Nhân vật bị loại không được tiếp tục di chuyển.

Code hiện tại không dùng collision giữa các player như một luật cản đường. Do đó, không đặt yêu cầu nhân vật phải đẩy hoặc chắn nhau trong phạm vi bản này. Quy tắc va chạm được đối chiếu tại CollisionSystem.

6.2.4. Đặt bom, nổ và sát thương

Ô đặt bom xác định theo tâm hitbox của nhân vật trên server. Bom chỉ được đặt trên nền trống, ô chưa có bom và số bom đang hoạt động của người đặt chưa vượt quota. Bom giữ range tại thời điểm được đặt; nâng cấp range sau đó không đổi ngược các bom đã tồn tại.

Vụ nổ tác động từ ô đặt theo bốn hướng, tối đa theo range. Tường cứng/biên map dừng tia; block phá được nhận tác động rồi dừng tia. Nếu vùng nổ gặp bom khác thì xảy ra nổ dây

chuyền. Với range 1 và không có vật cản, vùng ảnh hưởng gồm ô đặt và một ô mỗi hướng, tổng 5 ô. Vùng nổ có thể gây sát thương cả người đặt bom.

Các bom/tia được xử lý trong cùng một tick, kể cả nổ dây chuyền, được gom thành một đợt sát thương: một nhân vật chỉ nhận một lần damage trong đợt đó. Block đã phá trong đợt vẫn chắn các tia còn lại của cùng đợt. Bom đến hạn được xử lý trong game loop; thời điểm thực thi phụ thuộc lúc vòng cập nhật chạy, không phải một cam kết callback chính xác từng mili giây. Căn cứ là BombSystem và BombGameplayExtension.

6.2.5. Vật phẩm, mạng sống, khiên và hồi sinh

Bảng 6.4. Các loại item và hiệu ứng.

Item	Hiệu ứng	Giới hạn mặc định
`BOMB_UP`	Tăng quota bom đồng thời thêm 1	Tối đa 5
`FIRE_UP`	Tăng range thêm 1 trên mỗi hướng	Tối đa 8
`SPEED_UP`	Tăng tốc độ thêm 8 pixel/giây	Tối đa 144 pixel/giây
`LIFE_UP`	Tăng thêm 1 mạng sống	Tối đa 99; không hồi sinh người đã bị loại
`SHIELD`	Chặn một đợt damage rồi mất	Không cộng dồn số khiên

Item chỉ được xét rơi tại ô block vừa phá. Item cũ trong vùng nổ bị xóa trước khi tạo drop mới; drop mới không bị cùng đợt phá lần hai. Nhân vật còn sống nhặt item theo ô chứa tâm hitbox. Nếu nhiều người tranh cùng tick, playerId trong match nhỏ hơn được xét trước. Item được tiêu thụ một lần, kể cả khi chỉ số đã chạm trần hoặc nhân vật đã có khiên. Các quy tắc được hiện thực tại MatchGameplayExtension, ItemSystem, RandomItemSpawnPolicy và ItemEffectRegistry.

Khi nhận damage, nhân vật có khiên mất khiên và giữ nguyên mạng sống. Nếu không có khiên thì mất một mạng; còn mạng sẽ hồi sinh ngay ở spawn ban đầu và dừng movement, hết mạng mới bị loại. Các nâng cấp quota/range/speed được giữ qua hồi sinh trong cùng lượt. Không có thời gian bất tử sau hồi sinh: đợt nổ ở tick sau vẫn có thể gây damage. Lửa còn hiển thị là animation, không tự gây thêm sát thương theo mỗi frame ở client. Hành vi mạng sống được xác định trong Player.takeDamage.

6.2.6. Kết thúc lượt và chơi lại

Trong lượt đang chạy, kết quả gameplay được quyết định sau khi cập nhật tác động bom và item:

- `LAST_ALIVE`: còn đúng một người chưa bị loại; người đó thắng.
- `ALL_DEAD`: mọi người đều bị loại; không có người thắng.
- `TIMEOUT`: hết thời lượng và vẫn có nhiều hơn một người chưa bị loại; lượt hòa.

Code kiểm tra số người còn lại trước timeout, vì vậy nếu cùng tick vừa hết giờ vừa chỉ còn một người thì kết quả là `LAST_ALIVE`. Những trường hợp kết thúc hành chính như host End hoặc thành viên rời/mất kết nối khi lượt chưa có kết quả chốt dùng `ABORTED`, không coi như một chiến thắng gameplay. Trong kết quả hủy, không có người thắng và không có bảng xếp hạng thi đấu.

Sau lượt, hệ thống phải dừng runtime cũ và hỗ trợ luồng ready/start lại với world mới. Quy tắc được đối chiếu tại MatchGameplayExtension và LobbyGameServer.

Phân công: TV3 ghép yêu cầu chức năng; từng TV xác nhận dòng thuộc module mình.

6.3. Yêu cầu chức năng

Các mã FR dùng để liên kết định nghĩa bài toán với thiết kế, test case và phần kết quả. Cột kiểm chứng mô tả điều cần quan sát, không xác nhận test đã đạt hoặc GUI/LAN đã được nghiệm thu.

Bảng 6.5. Danh sách yêu cầu chức năng.

Mã	Yêu cầu	Tiêu chí kiểm chứng chính	Phụ trách
FR01	Thiết lập kết nối và phiên	Client hợp lệ nhận kết quả handshake; yêu cầu chưa đủ điều kiện bị từ chối	TV1, TV4
FR02	Xem, tạo và vào phòng	Danh sách cập nhật; sức chứa 2–4; từ chối phòng đầy/đang chơi và tham gia nhiều phòng đồng thời	TV1
FR03	Quản lý ready, host và chat phòng	Chỉ host Start/End; Start đủ điều kiện; chat gửi đúng phòng; host rời được chuyển quyền cho người còn lại	TV1, TV4
FR04	Khởi tạo và cách ly trận	Roster/spawn đúng; runtime, state và định danh match riêng; không nhận thêm người giữa trận	TV2, TV1, TV4
FR05	Di chuyển và dừng	Server xử lý hướng/đi-dừng, không vượt biên/tường/block/bom theo luật; người bị loại không di chuyển	TV2
FR06	Đặt bom hợp lệ	Kiểm tra trạng thái, ô đặt và quota; yêu cầu không hợp lệ không sinh thêm bom	TV3, TV2, TV4
FR07	Nổ, phá block và nổ dây chuyền	Đúng vùng ảnh hưởng, barrier và batch; không nổ hoặc trừ damage lặp cùng đợt	TV3
FR08	Drop và pickup item	Chỉ xét block vừa phá; một người nhận hiệu ứng, đúng loại và đúng trần chỉ số	TV3
FR09	Xử lý mạng sống, khiên và respawn	Khiên chặn một đợt; còn mạng về spawn; hết mạng bị loại; đồng bộ trạng thái mới tới client	TV3, TV2
FR10	Đồng bộ full/delta và phục hồi	Có full khởi tạo; delta áp đúng baseline; sai baseline yêu cầu full; loại dữ liệu thuộc match cũ	TV2, TV4
FR11	Xử lý điều khiển và biểu diễn client	Prediction/reconciliation dùng ACK; nội suy remote; respawn loại lịch sử cũ; HUD/gameplay nhận state server	TV2, TV3
FR12	Công bố kết quả và chơi lượt mới	Lý do/người thắng đúng luật; các client cùng nhận kết quả; lượt mới reset world, chỉ số và lịch sử input	TV3, TV1, TV2, TV4
FR13	Rời phòng, mất kết nối và dọn tài nguyên	Hủy lượt đang chơi theo chính sách; cập nhật phòng; dừng runtime/worker liên quan; phòng khác tiếp tục	TV1, TV2, TV4
FR14	Phản hồi yêu cầu lỗi và quá tải	Từ chối sai trường/quyền/match hoặc queue đầy theo loại lỗi; không áp dụng thao tác trái luật	TV4, TV1, các module

Các yêu cầu FR05–FR11 phải được đánh giá cả ở logic riêng và ở luồng tích hợp. Ví dụ, client hiển thị nhân vật đã hồi sinh chưa đủ chứng minh FR09 nếu các client còn lại chưa nhận state tương ứng. Tương tự, có codec delta chưa đủ chứng minh FR10 khi chưa kiểm tra ghép replica và phục hồi.

Phân công: TV4 tổng hợp mục 6.4; TV1 xác nhận mạng/session, TV2 runtime, TV3 luật và ngẫu nhiên.

6.4. Yêu cầu phi chức năng, mô hình mạng và giả định

6.4.1. Yêu cầu phi chức năng

Bảng 6.6. Các thuộc tính chất lượng cần kiểm chứng.

Mã	Thuộc tính	Tiêu chí/giới hạn đánh giá
NFR01	Nhất quán và quyền quyết định	Kết quả cuối do server xác định; tại cùng tick/baseline hợp lệ, replica khôi phục đúng phần state được contract đồng bộ
NFR02	Đồng thời và cách ly	Nhiều client submit an toàn; state phòng này không bị cập nhật bởi command phòng khác
NFR03	Phản hồi điều khiển	Tách I/O chặn khỏi EDT; theo dõi RTT, input ACK delay và correction; chưa đặt ngưỡng mili giây chưa có căn cứ
NFR04	Nhịp cập nhật	Bước mặc định 50 ms, nhịp mục tiêu 20 Hz; ghi nhận độ trễ/quá tải, không tuyên bố luôn đạt nhịp dưới mọi workload
NFR05	Dung lượng đồng bộ	So sánh full mỗi tick với delta + full theo cùng workload/lớp đếm byte; tính cả chi phí full/resync; không mặc định tiết kiệm ở mọi trường hợp
NFR06	Giới hạn tài nguyên	Queue và frame hữu hạn; có từ chối/counter hoặc đóng kết nối theo chính sách khi quá tải, không chờ vô hạn để có chỗ trong command queue
NFR07	Vòng đời và phục hồi lỗi	Stop runtime giải phóng thread/tài nguyên liên quan; client chậm/lỗi được xử lý có kiểm soát; resync không cộng lại item/damage
NFR08	Khả năng kiểm tra và tái lập	Ghi revision, cấu hình, môi trường, workload và dữ liệu gốc; phân biệt test logic/offscreen/loopback với GUI/LAN

Đây là các tiêu chí đánh giá của đề tài, không phải SLA của một dịch vụ sản xuất. Tải tối đa, số phòng thực tế chạy ổn định và mức sử dụng CPU/RAM cần phép đo riêng; không suy ra từ giới hạn khai báo trong code.

6.4.2. Mô hình mạng và triển khai

Một server trung tâm tiếp nhận nhiều kết nối client; các room có các lượt chơi riêng. Client/server trao đổi qua TCP với framing length/type/payload. Kênh TCP có thứ tự không thay thế kiểm tra room/match, queue ứng dụng hoặc baseline snapshot. Các chi tiết timeout, heartbeat, giới hạn tần suất và routing sẽ được mô tả trong mục 9.

Phạm vi triển khai dự kiến gồm local/loopback và các máy trong mạng có thể kết nối đến cổng server, ưu tiên kịch bản LAN để nghiệm thu nhóm. Đây là môi trường mục tiêu; hồ sơ hiện có chưa đủ để kết luận đã nghiệm thu trên nhiều máy LAN hoặc Internet. Client và server cần cùng bộ phát hành tương thích; handshake protocol version và snapshot version là hai cơ chế khác nhau, không dùng thay cho nhau.

6.4.3. Ràng buộc kỹ thuật hiện tại

- Code hiện hành sử dụng Java 25, Maven và Swing/AWT; máy chạy giao diện cần môi trường đồ họa phù hợp.
- Mỗi client tham gia tối đa một phòng tại một thời điểm; roster một trận cố định từ 2 đến 4 người.
- Server lobby hiện chặn tạo thêm khi đã có 32 phòng. Đây là giới hạn quản lý, không phải kết quả chứng minh năng lực phục vụ 32 trận đồng thời.
- Frame có `length` từ 2 đến 1.048.576 byte, tính type + payload, không tính 4 byte prefix.
- GameLoop mặc định dùng command queue 4096 và outbound queue 1024; mỗi tick đọc tối đa 256 command. Chính sách input có sequence còn giới hạn mỗi player một mẫu movement mới được áp dụng trong tick, giữ thứ tự FIFO.
- Cấu hình luật được chọn phía server và phải hợp lệ; không có giao diện để client tùy ý thay thông số authoritative trong trận.
- State gameplay chủ yếu lưu trong RAM; không đặt yêu cầu khôi phục trận sau khi tiến trình server dừng.

Các ràng buộc này lấy từ pom.xml, GameLoop, PacketEncoder và LobbyGameServer. Khi cấu hình hoặc code thay đổi, phần thiết lập thực nghiệm phải ghi đúng giá trị đã dùng.

6.4.4. Giả định và hệ quả

Bảng 6.7. Các giả định của phạm vi đề tài.

Mã	Giả định	Hệ quả cần giữ khi mô tả/đánh giá
A01	Server do nhóm vận hành và là nơi quyết định gameplay	Client có thể gửi dữ liệu sai; server vẫn phải kiểm tra yêu cầu, không lấy giả định người dùng trung thực thay validation
A02	Các máy có JDK/runtime phù hợp và kết nối được tới server	Cài môi trường, mở đường kết nối và cùng bộ phát hành là điều kiện thử nghiệm; không coi đó là tính năng tự cấu hình mạng của game
A03	Một lượt có roster cố định và không chuyển server giữa trận	Không yêu cầu join-in-progress, chuyển máy chủ hoặc tiếp tục trận sau server crash
A04	Client có thể nhận dữ liệu muộn và chưa biết toàn bộ thay đổi mới nhất	Prediction có thể sai; kết quả server được ưu tiên; khi mất baseline phải phục hồi bằng full
A05	Clock máy khác nhau không được giả định đồng bộ tuyệt đối	Server quyết định timer; RTT/ACK dùng cách đo được mô tả, không lấy chênh lệch clock hai máy làm one-way latency
A06	Drop item có ngẫu nhiên phía server	Không yêu cầu các client chạy cùng RNG; nếu so sánh định lượng cần kiểm soát seed, trace hoặc điều kiện sinh workload

Những giả định này giới hạn bài toán cần giải; chúng không thay thế bằng chứng thực nghiệm về độ ổn định, độ trễ hay chất lượng điều khiển trong từng môi trường.

Phân công: TV3 ghép mục 6.5; TV1 xác nhận reconnect/phòng, TV2 đồng bộ, TV4 phạm vi triển khai và bằng chứng.

6.5. Phạm vi chưa triển khai và giới hạn kết luận

Phiên bản được báo cáo tập trung vào một chế độ chơi tiêu chuẩn với map mặc định, phòng 2–4 người, movement, bom, năm item, mạng sống/khiên và kết quả lượt. Các nội dung sau không được đặt thành chức năng đã hoàn thành trong phạm vi hiện tại:

- **Reconnect để tiếp tục trận:** mất kết nối hiện kết thúc/hủy lượt theo chính sách, không giữ ghế và trạng thái để người chơi quay lại tiếp tục. API/token hoặc đặc tả reconnect còn tồn tại không chứng minh đã có luồng end-to-end.
- **Tham gia giữa trận hoặc spectate riêng:** roster cố định; nhân vật bị loại có thể còn trong client hiện tại nhưng không đồng nghĩa đã có chế độ spectator độc lập.
- **Tài khoản bền vững và xếp hạng dài hạn:** không yêu cầu đăng nhập bằng mật khẩu, lưu lịch sử vào database hoặc leaderboard liên trận.
- **Mở rộng vận hành:** chưa đặt yêu cầu matchmaking liên server, server migration, triển khai cụm hoặc cơ chế khôi phục sau crash.
- **Đồng bộ nâng cao:** chưa có rewind/lag compensation, không bảo đảm prediction luôn đúng hay loại bỏ mọi hiệu chỉnh.

Ngoài giới hạn chức năng, hồ sơ bằng chứng còn thiếu nghiệm thu GUI/LAN nhiều máy, đánh giá tải/soak dài hạn và đo full/delta trên các workload đại diện cho gameplay hiện hành. Khảo sát đợt khảo sát ban đầu cũng ghi nhận một process smoke test chưa đạt. Những điểm này phải được giữ trong phần kết quả/hạn chế đến khi có lần kiểm chứng mới; chúng không được thay bằng khẳng định "đạt" chỉ vì đã có code hoặc một test khác đi qua.

Các phần tiếp theo sẽ mô tả cách thiết kế và hiện thực để đáp ứng FR/NFR; phần thực nghiệm sẽ xác định mức đáp ứng bằng test, log, phép đo và minh chứng phù hợp. Nguồn đối chiếu hiện trạng là hồ sơ khảo sát và đánh giá reconnect.

7. Giải pháp đề xuất

Nhóm đề xuất giải pháp phối hợp ba phần: server quyết định gameplay bằng runtime theo phòng; client cải thiện phản hồi điều khiển bằng prediction/reconciliation và nội suy nhân vật từ xa; dữ liệu trạng thái được truyền dưới dạng full/delta với cơ chế phục hồi baseline. Mục tiêu là duy trì một kết quả gameplay thống nhất, hạn chế gửi lại dữ liệu không đổi và tổ chức xử lý lỗi mà không để I/O socket trực tiếp chi phối bước mô phỏng.

Chương tập trung vào cách giải quyết và các quyết định thiết kế. Thành phần cụ thể và format message được trình bày ở mục 8 và mục 9. Các mã giả dưới đây do nhóm diễn giải từ code hiện hành, không phải code mới cần thêm vào project; chúng lược bớt chi tiết lifecycle/exception khi đã được nêu riêng.

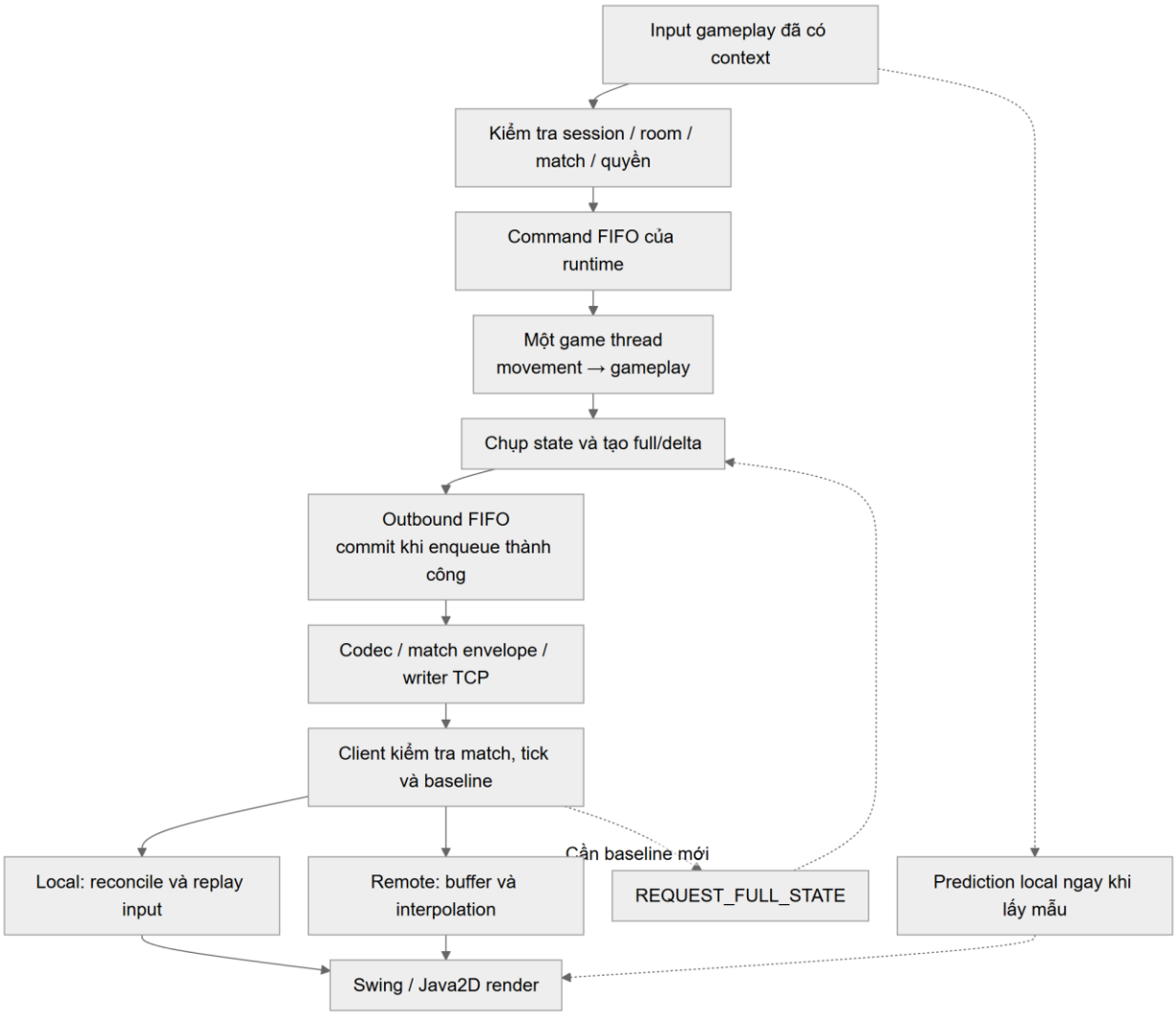
Phân công: TV2 chủ trì ý tưởng tổng thể; TV1/TV3/TV4 xác nhận các bước thuộc module mình.

7.1. Ý tưởng server authoritative và luồng input → state → client

7.1.1. Phân quyền giữa server và client

Nhóm kế thừa nguyên tắc server authoritative: client gửi ý định, server tính trạng thái hợp lệ rồi công bố kết quả. Đây là cách tổ chức đã được Gambetta mô tả, không phải thuật toán do nhóm phát minh. [2]

Trong Bomb It, MOVE mang hướng, cờ đi/dừng và sequence; PLANT_BOMB mang requestId. Server lấy player từ session/context rồi kiểm tra collision, quota, damage, pickup và kết quả. Client không tự khai vị trí authoritative, sức mạnh bom hoặc người thắng. Prediction chỉ thay biểu diễn local, không được dùng để áp lại sát thương hoặc vật phẩm vào world server.



Hình 7.1. Luồng phối hợp của giải pháp. Nhóm tự vẽ từ thiết kế và code project.

Sơ đồ tập trung luồng gameplay; các thao tác lobby như tạo/join/ready được tầng phòng xử lý trước khi có runtime, không phải mọi thao tác phòng đều trở thành GameCommand. Đường resync là yêu cầu qua handler để đặt cờ full, không phải client gọi trực tiếp builder. Bản nguồn: 07_01_giai_phap_tong_the.mmd.

7.1.2. Các mục tiêu và cơ chế tương ứng

Bảng 7.1. Vấn đề cần giải quyết và cơ chế được chọn.

Vấn đề	Cơ chế	Phạm vi tác động
Client có thể gửi yêu cầu sai hoặc cũ	Xác thực session/room/match, validation và luật server	Quyền áp dụng thao tác
Nhiều nguồn input cùng sửa world	Command FIFO và một game thread/state theo runtime	Thứ tự cập nhật
Chờ state server làm điều khiển local chậm phản hồi	Prediction, ACK và replay	Biểu diễn local, vẫn có hiệu chỉnh
Dữ liệu remote nhận theo đợt	Buffer và nội suy tuyến tính	Độ liên tục hiển thị, có độ trễ
Lặp lại dữ liệu map/entity không đổi	Delta theo baseline và full phục hồi	Dung lượng đồng bộ, phát sinh chi phí so sánh
Client chậm hoặc queue đầy	Giới hạn pending, từ chối/đóng theo lớp lỗi	Kiểm soát backlog và vòng đời

Giải pháp kết hợp các cơ chế vì chúng giải quyết các khâu khác nhau. Giảm byte snapshot không tự chứng minh giảm latency; prediction không chứng minh world luôn đúng; queue hữu hạn không thay thế phép đo tải. Mỗi mục tiêu được đối chiếu riêng với FR/NFR và bằng chứng trong các chương thực nghiệm.

Phân công: TV1 phụ trách connection/session/room; TV4 phối hợp identity, routing và vòng đời match.

7.2. Quản lý connection, session và room

Nhóm tách connection socket, session người chơi, room chờ và match gameplay. Một session tham gia tối đa một room; một lượt có roster cố định và một matchId riêng. Session không trực tiếp chứa state nhân vật đang mô phỏng và room không tự tính luật bom/movement.

Khi host Start hợp lệ, tầng điều phối chụp roster, cấp mapping session→playerId, tạo spawn/runtime rồi đưa GAME_STARTED vào writer trước snapshot gửi tới client. Client cài listener thường trực trước CONNECT, tạo context/controller khi nhận metadata và chỉ bật input sau full có local player. Giữ cùng TCP qua các lượt giúp không phải tạo lại toàn bộ kết nối mỗi lần chơi lại; matchId và world/controller mới vẫn tách dữ liệu lượt cũ khỏi lượt sau.

Server dùng các kiểm tra membership, host, room state và match scope trước khi submit gameplay. Đây là gate tiếp nhận; sau đó game thread còn kiểm tra luật nghiệp vụ. Kết quả bình thường hoặc hủy lượt đều đi qua lifecycle dừng runtime và đưa phòng về WAITING. Hiện tại mất kết nối khi đang chơi hủy lượt theo chính sách ABORTED nếu chưa có kết quả chốt; không đề xuất giả rằng token/grace đã tạo thành tính năng resume.

Lựa chọn này giữ phạm vi triển khai hữu hạn cho nhóm, với đánh đổi là người còn lại không tiếp tục trận khi có thành viên rời/mất kết nối. Nếu bổ sung reconnect sau này, cần thiết kế lại chính sách giữ world/ghế, generation socket, sequence và baseline thay vì chỉ thêm một message handler. Nguồn hiện thực: LobbyGameServer và DemoClientConnection.

Phân công: TV2 phụ trách core/runtime/queue; TV3 xác nhận extension, TV4 xác nhận handoff tới network.

7.3. Single-writer state, command queue và runtime theo phòng

7.3.1. Quyền sở hữu state và bước mô phỏng

Nhóm chọn mỗi runtime có map/state/extension riêng và một game thread cập nhật world. Network đưa command vào queue thay vì sửa Player hoặc GameMap trên luồng đọc socket. Đầu ra là snapshot bất biến qua outbound FIFO; worker gửi không cần đọc GameState mutable.

Lựa chọn bước mô phỏng cố định tham khảo các phân tích timestep của Fiedler. [9] Trong code của nhóm, bước mặc định là 50 ms và có giới hạn công việc lấy từ command queue. Loop không chạy bù toàn bộ tick đã trễ; nó điều chỉnh mốc lịch khi quá hạn. Vì vậy, đây là nhịp mục tiêu và bước mô phỏng đã chọn, không phải cam kết mọi server luôn theo kịp thời gian thực.

7.3.2. Mã giả xử lý tick

Thuật toán 7.1. Vòng cập nhật theo room.

```
Gọi extension.onStart() trên game thread
Trong khi runtime còn chạy:
    Cho từng player: beginTick()
```

```
    sampled := tập rỗng
    Lặp tối đa 256 lần:
        c := commandQueue.peek()
        Nếu không có c: dừng đọc queue
        Nếu c là movement có sequence > 0 và playerId đã ở sampled:
            dừng đọc queue, không chuyển c xuống cuối hàng
        commandQueue.poll()
        Nếu c là command lifecycle:
            áp dụng chính sách frozen/remove hoặc mở lại của core
        Ngược lại nếu c là movement và player không frozen:
            thử áp dụng c bằng MovementSystem
            Nếu thành công và sequence > 0: thêm playerId vào sampled
        extension.processCommand(c)
    Buộc player frozen dừng movement
    movement.update(state, 0,05 giây)
    extension.update(thời gian hiện tại)
    tick := tick + 1
    PUBLISH(state, tick) theo Thuật toán 7.3
    Chờ tới lịch tick tiếp; nếu trễ thì điều chỉnh mốc lịch
Khi kết thúc hoặc có lỗi:
    gọi extension.onStop() trên game thread và dọn command queue
```

Nhánh lifecycle mô tả API core hiện có, không khẳng định route reconnect đã bật ở ứng dụng. Đọc mẫu thứ hai của cùng player bằng peek rồi dừng bảo toàn FIFO toàn cục, nhưng có thể làm command của player khác phía sau cũng chờ. Đây là đánh đổi của chính sách hiện hành; rate limit phía network hỗ trợ hạn chế spam, chưa chứng minh công bằng tuyệt đối hoặc không có backlog.

Command queue mặc định 4096 và outbound queue 1024. Submit dùng offer, không chờ queue có chỗ; loop dừng thì không nhận command mới. Các tham số cần được đo dưới workload thực tế trước khi kết luận năng lực tải. Nguồn: GameLoop, GameRuntime và GameRuntimeManager.

Phân công: TV2 phụ trách movement/collision và prediction/reconciliation/interpolation.

7.4. Movement/collision và đồng bộ chuyển động phía client

7.4.1. Tính vị trí và va chạm phía server

MOVE cập nhật hướng/đi-dừng; bước movement tính quãng đường `d = speed × Δt`. Nhân vật phải còn sống và input có sequence phải được áp dụng ở tick đó mới tiến thêm bước tương ứng. Mẫu sequence không lớn hơn ACK được bỏ; không tăng tốc chỉ vì gửi lại input cũ.

Collision dùng hitbox hình chữ nhật, kiểm tra map/boundary và entity bom. Quãng đường mỗi trục được chia thành bước tối đa 1 pixel rồi kiểm tra vị trí chiếm chỗ; trục X xử lý trước Y. Cạnh phải/dưới của hitbox không được tính như phần bên trong, tránh nhận nhầm tile kề khi đứng sát mép. Người đang chồng bom được đi ra nhưng không đi trở vào sau khi đã ra hết. Đây là phương pháp đang dùng cho quy mô map và tốc độ hiện tại, chưa phải broad-phase/spatial index cho nhiều thực thể. Nguồn: MovementSystem và CollisionSystem.

7.4.2. Prediction, ACK và reconciliation

Nhóm kế thừa cách prediction/reconciliation của Gambetta: dự đoán input local, nhận state server kèm input ACK, bỏ input đã xử lý và phát lại phần chưa được xác nhận. [4]

Client dùng movement/collision chung trên state local riêng, giữ pending theo sequence tăng dần. Sau khi snapshot áp dụng hợp lệ, controller cập nhật map/entity dùng dự đoán trước khi restore player và replay. Nếu respawn counter đổi, pending trước hồi sinh được xóa nhưng sequence không quay lại số đã dùng. Nếu player bị loại, prediction không tiếp tục replay hành động cho player đó.

ACK nằm trong PlayerData sau movement, không phải ACK TCP hay xác nhận mới enqueue command. Do client chỉ có world đã nhận gần nhất, bom/block mới đổi có thể gây khác biệt prediction; reconciliation là bước cần thiết. Giới hạn 256 pending ngăn lịch sử tăng mãi, khi đầy báo lỗi cho caller; không có bảo đảm tự retry/resync mọi tình huống.

7.4.3. Nội suy nhân vật từ xa

Nhân vật remote được hiển thị giữa các mẫu đã biết, thay vì giả định đã biết input tương lai của họ. Cách tiếp cận tham khảo phần entity interpolation của Gambetta. [5]

Project giữ tối đa 32 frame vị trí; render timeline lùi 2 tick và giới hạn mức tiến của timeline khi thiếu update. Giữa hai tick khác nhau, dùng `p(r) = p_a + (p_b − p_a) × α`, với `α` được chặn trong [0,1]. Ngoài vùng mẫu thì giữ vị trí biên, không ngoại suy vô hạn; respawn làm xóa buffer để không vẽ đường dịch chuyển qua map. Render không sửa replica hoặc quyết định damage. Độ mượt, correction và độ trễ hiển thị còn cần nghiệm thu/đo riêng. Nguồn: PredictionController, MovementClientController và InterpolationController.

Phân công: TV2 chủ trì State Delta Compression và replica; TV4 phối hợp codec, resync và phép đo.

7.5. State Delta Compression và phục hồi baseline

7.5.1. Phần kế thừa và phần áp dụng của nhóm

Delta compression biểu diễn thay đổi so với baseline; phía nhận phải có baseline phù hợp để giải mã. Nguyên tắc này đã được Fiedler trình bày. [3] Nhóm áp dụng vào trạng thái Bomb It, với map/player/entity, ACK, frame gameplay và danh sách xóa. Không giới hạn giải pháp ở việc so hai mảng tile, và không nhận đây là phát minh delta compression nói chung.

Gọi `C_t` là ảnh state cần đồng bộ chụp tại tick `t`, `B_b` là ảnh baseline đã commit tại tick `b`. Update tăng dần có dạng `U_(b→t) = Diff(B_b, C_t)` và mang `baseTick=b`. Mục tiêu khôi phục là: nếu replica đang đúng ảnh ở `b`, áp dụng update hợp lệ sẽ tạo đúng phần state được contract đồng bộ tại `t`. Ký hiệu không bao gồm các đối tượng nội bộ như socket, queue hay toàn bộ lịch sử domain event.

7.5.2. Những dữ liệu được so sánh

Bảng 7.2. Quy tắc phát hiện và biểu diễn thay đổi.

Thành phần	Cách xác định thay đổi	Dữ liệu gửi/giữ
Map nền	So từng tile ở cùng `(x,y)`	Record `x,y,newType`; full thì gửi toàn map theo hàng

Thành phần	Cách xác định thay đổi	Dữ liệu gửi/giữ
Player	So toàn bộ trường PlayerData, kể cả ACK và double chính xác	Upsert cả PlayerData, chưa chỉ gửi field đổi
Player bị xóa	ID baseline không còn trong ảnh hiện tại	Danh sách removedPlayerIds
Bom/item	So EntityData theo key kind + ID và record equality	Upsert record mới/đổi; removedEntityKeys cho entity biến mất
Match state	So WAITING/RUNNING/FINISHED	State và cờ changed theo contract
MatchFrame	So timer, Stats và result	Khi phát snapshot có frame thì gửi trọn MatchFrame
BombFrame	So danh sách explosion; không chỉ vì serverTimeMillis tiến lên	Khi phát snapshot có frame thì gửi trọn effect list hiện tại
Notices	Có event chưa bàn giao	Gắn với update nội bộ để worker encode thành message riêng

Vì PlayerData có `lastProcessedInput`, nhân vật đứng yên nhưng nhận mẫu input mới vẫn có thể tạo player upsert. Match timer đổi giây, Stats, effect hoặc notice cũng có thể làm phát update dù không tile nào đổi. Do đó, không dự đoán dung lượng từ số tile thay đổi đơn thuần.

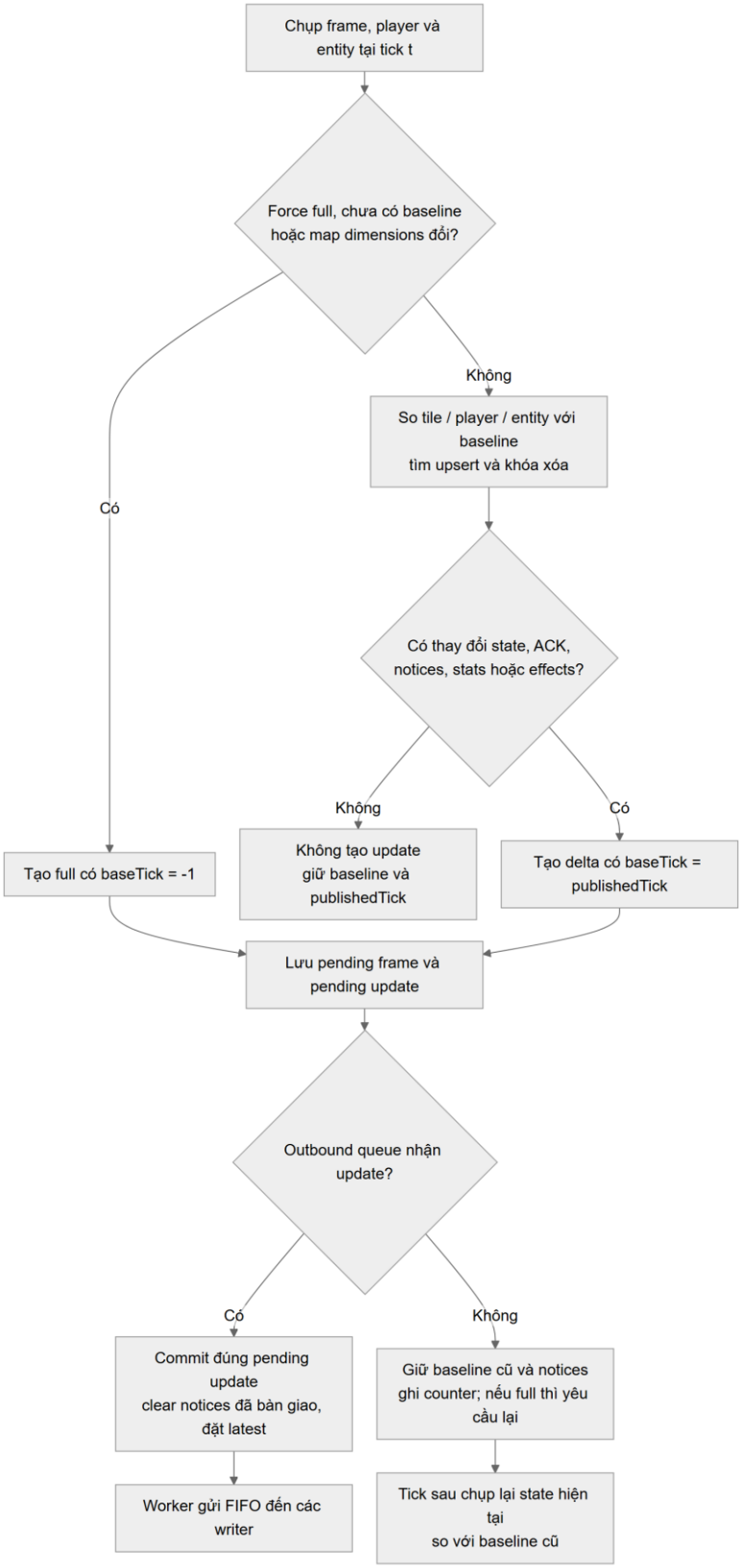
Builder giữ bản sao tile/PlayerData và EntityData bất biến làm baseline; nó không giữ Player mutable để rồi baseline vô tình đổi theo world. Không lượng tử hóa tọa độ trước so sánh và chưa chọn full/delta theo kích thước encode nhỏ hơn. Nguồn: StateDeltaBuilder và SnapshotPacketCodec.

7.5.3. Mã giả tạo full/delta

Thuật toán 7.2. Tạo update dựa trên baseline đã công bố.

```
BUILD_UPDATE(state, t, forceFull):
    C := chụp map, PlayerData và entity hiện tại trên game thread
    B := baseline đã commit, có tick b và các frame gameplay đã công bố
    Nếu forceFull hoặc B chưa tồn tại hoặc dimensions/tileSize đổi:
        U := FULL(t, baseTick=-1, toàn bộ map/player/entity)
    Ngược lại:
        tileChanges := các ô mà C.tile khác B.tile
        playerUpserts := player mới hoặc khác bất kỳ trường PlayerData nào
        removedPlayers := ID trong B nhưng không còn trong C
        entityUpserts := entity mới hoặc record khác theo key
        removedEntities := key trong B nhưng không còn trong C
        stateChanged := C.matchState khác B.matchState
        frameChanged := MatchFrame đổi hoặc danh sách explosion đổi
        Nếu mọi danh sách đổi/xóa rỗng,
           stateChanged=false, frameChanged=false và không có notices chờ:
            trả NONE, không tiến baseline
        U := DELTA(t, baseTick=b, các upsert/xóa và metadata)
    Gắn BombFrame/MatchFrame hiện tại và bản sao notices vào U
    Lưu pending frame/entity và đúng đối tượng pendingUpdate=U
    Trả U; bước này chưa commit baseline
```

Ở đây B gộp các trường published của implementation để viết gọn; code thực lưu frame, entity, BombFrame và MatchFrame ở các trường riêng. So effect chỉ xét danh sách explosion khi quyết định bỏ tick rỗng, nhưng snapshot đã phát vẫn mang serverTimeMillis mới để client biểu diễn effect theo thời lượng. Danh sách notices là handoff nội bộ, không được encode thành một phần lịch sử event trong Base64 snapshot.



Hình 7.2. Tạo delta, pending và commit. Nhóm tự vẽ từ StateDeltaBuilder/GameLoop.

Mũi tên đến worker chỉ đường handoff, không có nghĩa commit chờ socket gửi xong. Bản nguồn: 07_02_delta_va_commit.mmd.

7.5.4. Commit sau enqueue và chính sách full

Thuật toán 7.3. Công bố update mà không làm baseline tiến trước queue.

```
PUBLISH(state, t):
    requested := đọc và xóa cờ fullRequested một cách nguyên tử
    wantFull := requested hoặc t=1 hoặc t chia hết 100 hoặc state=FINISHED
    U := BUILD_UPDATE(state, t, wantFull)
    Nếu U là NONE: kết thúc bước publish
    Nếu outboundQueue.offer(U) thành công:
        builder.commit(U)              // chỉ chấp nhận đúng pendingUpdate
        state.clearGameNotices()
        latest := U
    Ngược lại:
        tăng outboundFull counter
        giữ baseline đã commit và notices chưa bàn giao
        Nếu wantFull: đặt lại cờ fullRequested
```

Nhóm dùng một consumer đọc outbound FIFO theo runtime để không bỏ delta trung gian. `latest/createUpdate()` chỉ là ảnh công bố gần nhất, không thay chuỗi queue và không bảo đảm là full. Nếu offer thất bại, lần build sau chụp state hiện tại rồi so với baseline cũ; không chỉ retry mù update cũ. Phần state trung gian có thể được gộp về ảnh cuối, còn notices chưa bàn giao được giữ trong giới hạn backlog.

Bảng 7.3. Ví dụ logic về baseline, không phải log thực nghiệm.

Tick	Điều kiện giả định	Kết quả publish	Baseline sau bước
100	Force full, queue nhận	Full có baseTick=-1	100
101	Không trường đồng bộ/ACK/frame/notice nào đổi	Không update	100
102	Có thay đổi, queue nhận	Delta baseTick=100	102
103	Có thay đổi, queue đầy	Có pending nhưng chưa công bố	102
104	Chụp state mới, queue nhận	Delta baseTick=102, gồm chênh lệch còn tồn tại	104

Ví dụ cho thấy baseTick không phải mặc định `t-1`. Điều kiện tick 101 là giả định minh họa: trong trận thật ACK/timer/notice vẫn có thể đổi dù nhân vật đứng yên. Commit chỉ xác nhận queue nhận update, chưa phải từng client xác nhận receipt hoặc apply. Nếu writer một client thất bại, transport/lobby xử lý disconnect theo chính sách; không âm thầm bỏ delta rồi tiếp tục với giả định baseline client đã tiến.

7.5.5. Ghép replica và phục hồi baseline

Thuật toán 7.4. Tiếp nhận update và đồng bộ movement phía client.

```
RECEIVE_UPDATE(packet):
    Kiểm tra match envelope; packet sai match thì bỏ
```

```
Decode và kiểm tra cấu trúc U; lỗi decode đi theo đường fail controller
Nếu U.tick <= replica.tick: bỏ stale, không lùi replica
Kiểm tra dimensions, tọa độ và dữ liệu theo contract
Nếu U là delta và (replica cần full hoặc U.baseTick != replica.tick
                   hoặc dimensions/tileSize không khớp):
    đánh dấu cần full, giữ world cũ
    yêu cầu full qua owner có giới hạn tần suất; kết thúc
Nếu các kiểm tra apply khác thất bại:
    đánh dấu cần full theo ClientWorldState, yêu cầu full qua owner; kết thúc
Nếu full: thay map/player/entity
Nếu delta: áp tile changes, upsert và các danh sách xóa
Thay các frame gameplay bằng frame đi kèm; đặt replica.tick=U.tick
Đồng bộ scene và lấy player local
Nếu respawn counter đổi: xóa pending trước hồi sinh
Cập nhật world dùng prediction, restore player server và replay input > ACK
Ghi frame vị trí cho interpolation
```

Đây là chuỗi logic tổng hợp từ controller/replica, không khẳng định tất cả validation nằm trong một hàm. Khi replica từ chối update ở một kiểm tra khác, controller cũng gửi yêu cầu full như khi sai baseline. Payload không decode được có thể làm controller fail; không coi mọi dữ liệu hỏng đều tự sửa được bằng resync.

Resync dùng handler hiện có để lưu barrier tick và đặt cờ full cho runtime. Worker duy nhất chờ full mới sau barrier rồi phát FULL_STATE_SYNC cho cả roster, vì baseline đang dùng chung room. Client owner gộp request đang chờ và hạn chế tần suất; tick client báo không được dùng để rewind world server.

Lập luận khôi phục theo contract như sau: full thiết lập ảnh nền; với delta có đúng baseline, những phần không được nêu giữ nguyên, phần upsert được thay và khóa xóa được loại bỏ. Nếu các danh sách Diff đầy đủ và thứ tự update được giữ, replica thu được ảnh cần đồng bộ của tick đích. Đây là lập luận thiết kế dưới điều kiện dữ liệu hợp lệ, không thay cho test correctness hoặc chứng minh hình thức toàn bộ hệ thống. Nguồn: ClientWorldState, MovementClientController và `LobbyGameServer`.

7.5.6. Chi phí và trường hợp delta kém lợi

Với `N=W×H`, `P` là tổng số player cần xét ở ảnh cũ/mới, `E` là tổng entity cũ/mới, `F` đại diện kích thước effect/frame/notice cần sao chép hoặc so sánh, chi phí builder thông thường có cỡ `O(N+P+E+F)` khi lookup hash có chi phí kỳ vọng hằng số. Builder vẫn quét/chụp toàn map mỗi lần, chưa dùng dirty-tile tracking để chỉ đọc ô thay đổi. Giảm số byte truyền không đồng nghĩa chi phí CPU tạo delta chỉ phụ thuộc số ô đổi.

Baseline và pending cần bộ nhớ cùng cỡ dữ liệu được giữ; queue update còn tốn bộ nhớ riêng tùy số phần tử và kích thước update. Ước lượng trên chỉ nói builder, không bao gồm encode/serialize, sắp xếp cell effect, khóa, I/O hoặc render. Đây là phân tích từ các vòng lặp code, chưa phải phép đo thời gian hay RAM.

Bảng 7.4. Minh họa byte phần giá trị tile theo layout hiện tại, không phải byte toàn packet.

Trường hợp map 17×15	Full: 1 byte/tile	Delta: 4 byte x + 4 byte y + 1 byte type/ô đổi

Trường hợp map 17×15	Full: 1 byte/tile	Delta: 4 byte x + 4 byte y + 1 byte type/ô đổi
1 tile đổi	255 byte	9 byte
29 tile đổi	255 byte	261 byte

Bảng chỉ tính vùng dữ liệu tile, bỏ các count/header, player/entity và frame khác. Nó minh họa vì sao nhiều ô đổi có thể làm phần tile của delta lớn hơn full, không đặt một ngưỡng chuyển loại tự động cho implementation. Code hiện chọn full theo policy, không encode cả hai rồi chọn nhỏ hơn. Mỗi PlayerData được gửi cả record; ACK đổi thường xuyên và toàn bộ Stats/effect gửi kèm còn làm giảm phần tiết kiệm tiềm năng.

Binary snapshot tiếp tục được bọc Base64 trong JSON. Theo quy tắc Base64, dạng có padding có độ dài `4×ceil(n/3)` với `n` byte binary. [13] Phép đo cần tính sau cùng một lớp biểu diễn ở hai chế độ, kể cả match envelope/frame và các full định kỳ/resync nếu có. Không dùng "1000 byte so với 50–100 byte" như kết quả mặc định khi chưa đo đúng workload.

7.5.7. Cách đánh giá cải tiến

Nhóm chọn đối chứng gửi full mỗi tick và chế độ delta kết hợp full theo chính sách đang dùng, trên cùng trạng thái đầu, chuỗi input, luật, thời lượng và điều kiện sinh item. Đánh giá cần tối thiểu hai mặt: replica khôi phục đúng và lượng dữ liệu/chi phí xử lý. Workload cần có idle với ACK thực, movement, bom/phá block/item và số người chơi đại diện, không chỉ fixture ít thay đổi ở map.

Tỷ lệ tiết kiệm được tính bằng `100×(1-B_delta/B_full)` với tổng byte cùng scope và `B_full>0`. Tính cả full đầu/định kỳ/phục hồi của chế độ delta; nếu đo toàn luồng, phải tính các notice/control theo cùng quy tắc ở cả hai chế độ. Giá trị âm vẫn là kết quả cần báo cáo, không loại bỏ kịch bản delta kém lợi. Chương giải pháp chưa chốt mức tiết kiệm hoặc lợi ích latency; các kết luận đó thuộc thực nghiệm/kết quả.

Phân công: TV3 phụ trách gameplay; TV2 phối hợp world adapter, runtime và respawn phía client.

7.6. Bom/item/event, damage batch, respawn và kết quả

Gameplay được đưa vào loop qua GameplayExtension để giữ cùng thứ tự cập nhật state. PLANT_BOMB đi qua gate/queue rồi mới kiểm tra placement; bom đến hạn được xét bằng thời lượng phía server trong update, không tạo thread riêng sửa world cho từng bom.

Một batch nổ gom bom gốc và nổ dây chuyền, theo dõi bom đã xử lý, các block bị phá và player trúng. Sau khi tính vùng ảnh hưởng, mỗi player nhận tối đa một hit trong batch. Quy tắc giữ block đã phá làm barrier cho các tia còn lại trong cùng batch giúp định nghĩa nhất quán trường hợp nổ đồng thời. Khiên chặn hit; còn mạng thì respawn và giữ nâng cấp, hết mạng mới bị loại.

Sau batch, extension xóa item cũ trong vùng nổ, xét drop mới từ block, thực hiện pickup theo thứ tự playerId, rồi xác định người còn sống/timer/kết quả. Điều này tránh kết luận thắng khi batch chưa xử lý hết hoặc gây damage lặp lên player vừa respawn trong cùng đợt. Kết quả được chốt trong gameplay state; tầng lobby công bố rồi dừng runtime và chuyển lượt.

Domain event được chuyển thành notices sau snapshot nhưng không làm client áp lại damage/pickup. Stats và respawn counter trong snapshot mới quyết định state client; renderer chỉ biểu diễn. Đặt bom được submit chưa có ACK nghiệp vụ riêng cho từng requestId ở callback

lobby mặc định; không bịa phản hồi lỗi gameplay mà code chưa gửi. Nguồn: BombSystem, MatchGameplayExtension và Player.

Phân công: TV4 chủ trì protocol/routing/backpressure; TV1 transport/lifecycle, TV2 và TV3 xác nhận tác động tới gameplay.

7.7. Protocol, match scope, routing và backpressure

7.7.1. Các quyết định thiết kế và đánh đổi

Bảng 7.5. Lựa chọn hiện hành và hệ quả cần đánh giá.

Lựa chọn	Lý do áp dụng trong đề tài	Chi phí/giới hạn
TCP có length/type và JSON envelope	Dùng chung kênh điều khiển/gameplay, decoder biết biên message	Giao dữ liệu có thứ tự không bảo đảm hạn tick; cần kiểm soát backlog
Một game thread/state mỗi runtime	Tập trung thứ tự movement/gameplay/snapshot	Room vẫn chia sẻ CPU/pool và khóa điều phối; không chứng minh scale tuyến tính
Bước 50 ms và giới hạn command/tick	Ràng buộc bước mô phỏng và lượng công việc	FIFO có thể làm chờ input phía sau; tick có thể trễ
Delta theo record/tile + tombstone	Tránh lặp dữ liệu, biểu diễn được xóa	Quét map, giữ baseline, record ACK và metadata vẫn tốn byte
Full đầu/định kỳ/yêu cầu/FINISHED	Tạo và phục hồi nền đồng bộ theo policy	Chi phí full, chưa có tự chọn loại theo size
Baseline commit khi enqueue	Không tham chiếu update chưa vào luồng công bố	Không phải receipt ACK của từng client; queue/writer phải giữ chuỗi
Prediction local + nội suy remote	Kết hợp phản hồi local với biểu diễn remote	Có correction, buffer delay và lịch sử cần reset khi respawn
Match envelope và context mới mỗi lượt	Loại message cũ khi replay	Không thay kiểm tra session/socket; canonical codec hạn chế dạng JSON được nhận
Queue/budget hữu hạn và đóng client lỗi	Giới hạn một số dạng backlog, không tiếp tục sai baseline	Disconnect có thể hủy lượt theo chính sách hiện hành

Các hướng kế thừa từ tài liệu đã nêu ở mục 5; thông số/policy và cách tích hợp trong bảng là lựa chọn của project. Không tuyên bố một lựa chọn tối ưu cho mọi game hoặc mọi điều kiện mạng.

7.7.2. Gate tiếp nhận và ranh giới gửi

Chuỗi kiểm tra gồm frame/UTF-8 → schema → session/room/match/quyền → queue → luật gameplay. Mã match giúp bỏ dữ liệu cũ, sequence xử lý input lặp trong match, requestId hạn chế đặt bom lặp, còn eventId phục vụ notice. Các số này không thay thế nhau và không phải bằng chứng exactly-once toàn hệ thống.

Server có một writer theo socket; producer encode/enqueue rồi writer mới ghi/flush. Client lấy mẫu trên UI nhưng gửi qua sender/control worker; callback mạng chuyển về owner UI tuần tự. Quá tải được xử lý theo lớp: command bị từ chối, outbound không commit baseline, writer đầy/timeout đóng kết nối, replica sai baseline xin full, payload hỏng có thể fail controller. Không coi mọi lỗi là một lần resync thành công.

Khóa và queue bảo vệ quyền sở hữu dữ liệu nhưng không tự bảo đảm UX hay tải tối đa. Thiết kế đầy đủ của message, giới hạn và sequence đã được mô tả ở mục 9; phần triển khai sẽ nêu các class/API và phần thực nghiệm kiểm chứng các trường hợp tương ứng.

7.7.3. Phạm vi đóng góp và bằng chứng cần có

Bảng 7.6. Phần kế thừa, phần nhóm hiện thực và cách kiểm chứng.

Nội dung	Phần kế thừa/ý tưởng nền	Phần hiện thực và bằng chứng cần dùng
Server authoritative	Mô hình client gửi intent, server quyết định [2]	Gate identity, movement/gameplay và test quyền/luật
Prediction/reconciliation/interpolation	Các phương pháp trong [4], [5]	Controllers, ACK/replay/respawn, test và nghiệm thu chuyển động
State Delta Compression	Delta theo baseline [3]	Builder/codec/replica, commit/FIFO/full-resync, correctness và so sánh byte
Runtime và giao tiếp	Kiến thức concurrency, loop và protocol thông dụng	Tích hợp theo room, queue/owner/lifecycle, test isolation/cleanup/fault và phép đo

Đóng góp dự kiến là áp dụng và tích hợp các cơ chế trong một game hoạt động theo contract đã xác định, đặc biệt phần full/delta và đánh giá đối chứng trong bối cảnh Bomb It. Mức độ novelty và hiệu quả cuối cùng chỉ được chốt từ implementation cùng bằng chứng phù hợp; không chuyển số đo fixture cũ hoặc ví dụ lý thuyết thành kết quả mọi trận. Hồ sơ ghi nhận lỗi process smoke lịch sử và khoảng trống GUI/LAN/soak như khảo sát đợt khảo sát ban đầu; chương này không thay đổi trạng thái các bằng chứng đó.

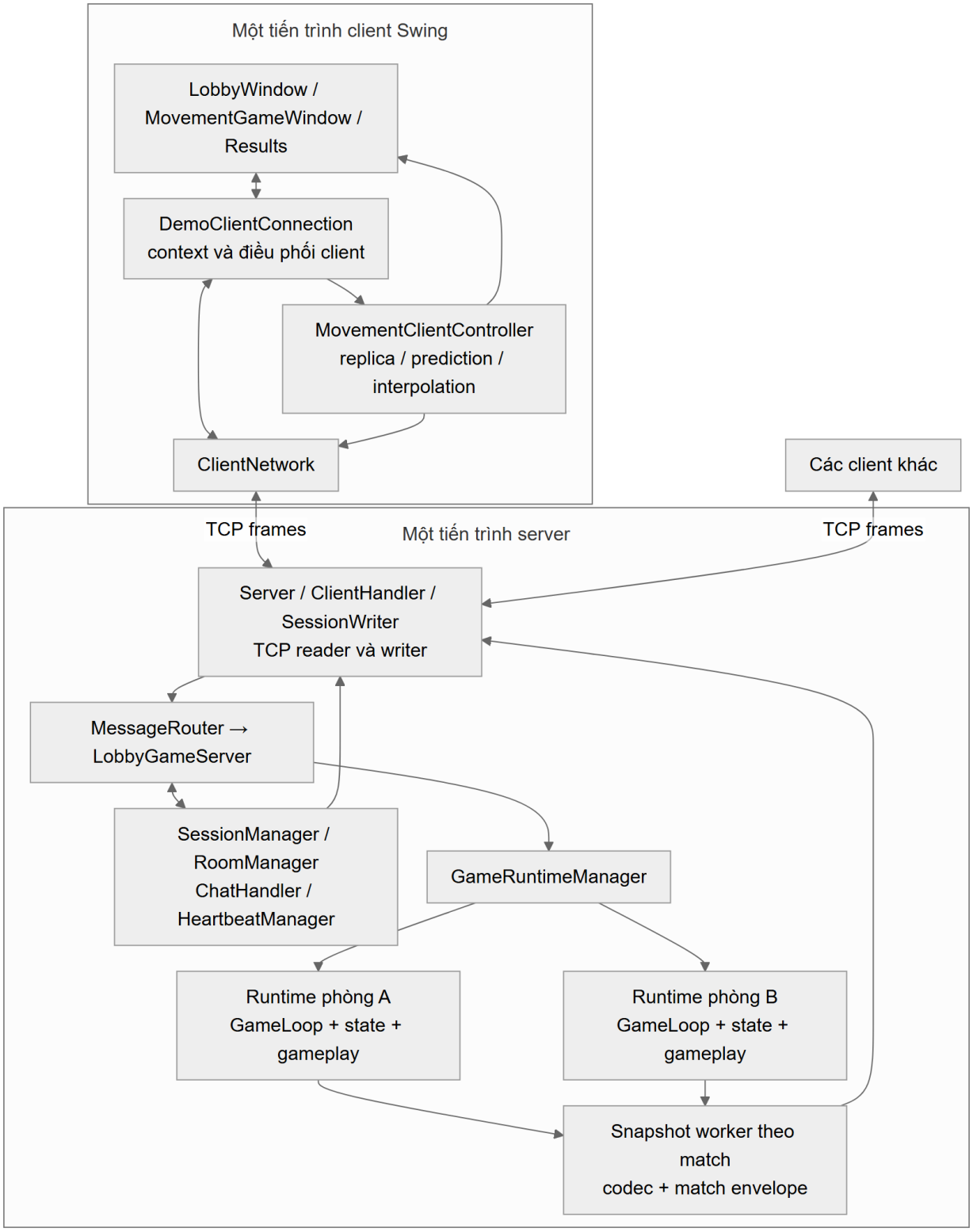
8. Kiến trúc hệ thống

Bomb It Multiplayer được tổ chức thành một tiến trình server và nhiều tiến trình client desktop. Server quản lý kết nối, phòng và trạng thái gameplay authoritative; client tiếp nhận điều khiển, duy trì dữ liệu bản sao và biểu diễn game bằng Swing/Java2D. Trong server, mỗi phòng đang chơi có runtime riêng; các runtime dùng chung hạ tầng kết nối và điều phối lobby. Đây là kiến trúc các thành phần trong cùng ứng dụng Java, không phải một tập microservice triển khai độc lập.

Chương mô tả đường chạy lobby/gameplay mặc định, nối các yêu cầu FR/NFR ở mục 6 với thành phần thực tế trong code. Thuật toán đồng bộ đã được giới thiệu ở mục 5; schema message và định dạng byte sẽ được trình bày ở mục 9.

Phân công: TV4 ghép sơ đồ tổng thể; TV1/TV2/TV3 xác nhận các thành phần thuộc module mình.

8.1. Sơ đồ tổng thể Client–Server



Hình 8.1. Các thành phần chính và ranh giới tiến trình. Nhóm tự vẽ từ code project.

Mũi tên biểu diễn trao đổi dữ liệu hoặc gọi thành phần; chỉ đường nối TCP là giao tiếp giữa tiến trình. Hai runtime A/B minh họa sự phân tách theo phòng, không phải hai server. Ô snapshot

worker đại diện cho các worker tương ứng từng match, không phải một worker duy nhất xử lý mọi trận. Bản nguồn sơ đồ được lưu tại 08_01_tong_the.mmd.

Điểm khởi tạo mặc định là ServerApplication tạo `LobbyGameServer`, và ClientApplication mở `LobbyWindow`. Các nhánh demo/headless là đường chạy phục vụ mục đích riêng; không được dùng để thay mô tả toàn bộ luồng sản phẩm.

Dữ liệu trận, phòng và session chủ yếu nằm trong RAM của tiến trình server. Kiến trúc hiện hành chưa tích hợp database cho tài khoản, lịch sử hoặc khôi phục trận sau khi server dừng. Các file cấu hình, log hoặc CSV phục vụ phát triển/thực nghiệm không được coi là tầng persistence gameplay.

Phân công: TV1 phụ trách transport/session/room; TV4 xác nhận router, codec và tích hợp.

8.2. Tầng transport, session và room

8.2.1. Phân tách trách nhiệm

Bảng 8.1. Các thành phần hạ tầng server.

Thành phần	Trách nhiệm	Ranh giới với gameplay
`Server`, `ClientHandler`, `ConnectionManager`	Accept socket, tổ chức reader/writer, decode frame, theo dõi kết nối và lỗi I/O	Không trực tiếp tính movement, damage hoặc item
`SessionManager`, `Session`	Handshake/danh tính phiên, trạng thái phiên, heartbeat metadata và cleanup	Session không thay thế player state của một match
`RoomManager`, `Room`	Thành viên, host, ready, trạng thái chờ/đang chơi và cập nhật phòng	Room không tự chứa thuật toán game loop
`MessageRouter`, `SessionHandlers`	Tra handler theo message type; xử lý các message phiên	Router gọi handler trên luồng gọi, không phải một worker queue riêng
`LobbyGameServer`	Ghép room với runtime, kiểm tra quyền/scope, ánh xạ roster và tổ chức công bố snapshot/kết quả	Là tầng điều phối giữa hạ tầng và game runtime
`ChatHandler`, `HeartbeatManager`	Worker chat theo phòng và sweep phát hiện timeout	Không tạo thread riêng cho từng thao tác gameplay

Căn cứ: Server, ClientHandler, MessageRouter và LobbyGameServer.

8.2.2. Luồng tiếp nhận và phân quyền

Sau khi accept, `Server` đăng ký session và tạo writer cho socket; `ClientHandler` được đưa vào reader pool. Handler đọc đủ frame, kiểm tra giới hạn, trạng thái handshake rồi gọi router bằng danh tính phiên server đang quản lý. Các trường định danh mà client gửi không tự cấp quyền điều khiển một player khác.

`LobbyGameServer` kiểm tra thành viên phòng, quyền host, trạng thái trận và match envelope trước khi xử lý gameplay. MOVE được đổi thành command qua `MovementHandler`; PLANT_BOMB cũng được đưa vào queue của runtime. Thao tác hợp lệ ở tầng mạng mới chỉ được tiếp nhận: kết quả nghiệp vụ còn phụ thuộc game thread và luật gameplay.

Khi Start hợp lệ, tầng điều phối chụp roster, tạo spawn và ánh xạ session UUID sang playerId số trong match, gọi `GameRuntimeManager.startGame`, tạo `MatchContext` và gửi `GAME_STARTED` trước khi worker phát snapshot. Một session có thể chơi nhiều lượt qua cùng kết nối; mỗi lượt có match identity riêng.

8.2.3. Gửi dữ liệu và lỗi kết nối

Các producer sử dụng `SessionWriter.send` để encode frame và enqueue; writer thread tương ứng socket mới thực hiện `write/flush`. Cách tổ chức này tách thao tác ghi socket khỏi bước mô phỏng và giữ các frame cùng kết nối không xen byte. Queue/byte budget và watchdog giới hạn dữ liệu tồn đọng, phát hiện ghi kéo dài và đưa lỗi về cơ chế cleanup. Chi tiết ở SessionWriter và ConnectionManager.

Reader pool, writer, heartbeat và session expiry cùng tham gia vòng đời kết nối, nhưng không tạo thành chức năng resume trận. Với ứng dụng hiện tại, callback mất kết nối đi qua tầng điều phối để hủy lượt đang chơi và bỏ ghế; metadata phiên còn tồn tại trong khoảng chờ dọn không đồng nghĩa world/roster được giữ để reconnect.

Phân công: TV2 phụ trách model/core/runtime/movement; TV1/TV4 xác nhận API tích hợp theo room.

8.3. Model, state, runtime và movement

8.3.1. Mô hình dữ liệu và quyền sở hữu

Bảng 8.2. Phân biệt dữ liệu model, state và dữ liệu bàn giao.

Nhóm	Thành phần tiêu biểu	Vai trò và cách sử dụng
Model giá trị	`Direction`, `TilePosition`, `TileType`	Enum/giá trị bất biến để thống nhất hướng, ô và loại nền
State đang mô phỏng	`GameState`, `GameMap`, `Player`, entity trong world	Mutable; sau khởi tạo được cập nhật bởi game thread của runtime
Input nghiệp vụ	`GameCommand`, `MoveCommand`, `StopMovingCommand`, `PlantBombCommand`	Handoff từ tầng tiếp nhận sang game thread; không mang tham chiếu UI/socket
Snapshot/handoff	`GameStateUpdate`, `EntityData`, `BombFrame`, `MatchFrame`, `GameNotice`	Bản dữ liệu công bố để truyền trạng thái/diễn biến; không dùng state mutable làm payload chia sẻ

`GameRuntimeManager` tạo map và state mới cho mỗi trận, kiểm tra spawn/định danh, tạo extension rồi khởi động runtime. Map factory và extension factory phải cung cấp instance riêng; manager kiểm tra việc tái sử dụng map instance. `GameRuntime` giữ roomId, matchId, mapping player và loop, cung cấp submit, poll update, request full và close. API này không trả trực tiếp `GameState` cho network thread.

Đường chạy lobby inject `DefaultArenaMap` và `MatchGameplayExtension` đầy đủ. Constructor manager không tham số còn có cấu hình tối giản phục vụ sử dụng/test khác; không lấy cấu hình tối giản đó để kết luận gameplay mặc định chưa có bom/item. Căn cứ: GameRuntimeManager và GameRuntime.

8.3.2. Trình tự của game loop

Sau `onStart` của extension, mỗi tick thực hiện:

4. Chuẩn bị cờ xử lý input của player, lấy command theo FIFO và chính sách giới hạn mỗi tick.
5. Áp dụng command movement; chuyển các command cần gameplay sang extension.
6. Cập nhật vị trí qua `MovementSystem` và `CollisionSystem` với bước mô phỏng cấu hình.
7. Gọi `GameplayExtension.update` để xử lý bom, item và kết quả.

8. Tạo full/delta update, đưa vào outbound FIFO và điều chỉnh lịch tick tiếp theo.

`MoveCommand` thay đổi ý định điều khiển; vị trí thực tế được tính trong bước movement. Trạng thái ACK phản ánh input được áp dụng vào mô phỏng, không chỉ việc packet đã đến socket. Các command cùng tick vẫn được xử lý theo chính sách FIFO; một mẫu movement mới thứ hai của cùng player có thể làm phần command còn lại đợi tick sau.

Mặc định loop có bước 50 ms, command queue 4096, outbound queue 1024 và giới hạn đọc 256 command/tick. Các giá trị là cấu hình hiện tại, không phải kết quả chứng minh tải tối đa hoặc luôn đạt 20 Hz. Tick trễ điều chỉnh mốc chạy tiếp; code không chạy bù toàn bộ bằng accumulator. Nguồn: GameLoop.

8.3.3. Snapshot và baseline

`StateDeltaBuilder` chụp dữ liệu để so sánh với baseline đã commit. Update được tạo trên game thread và bàn giao qua FIFO; chỉ enqueue thành công mới commit baseline, xóa notices đã bàn giao và công bố ảnh `latest`. Nếu outbound đầy thì baseline cũ được giữ để lần tạo tiếp vẫn tham chiếu phần đã công bố.

`createUpdate()` trả ảnh công bố gần nhất, có thể là delta; hàm không tạo snapshot mới từ network thread và không thay thế việc đọc outbound FIFO. Nếu tầng gửi chỉ lấy ảnh mới nhất rồi bỏ các delta trung gian, client có thể mất chuỗi baseline cần thiết. Nhận vào queue cũng chưa phải ACK rằng từng client đã áp dụng dữ liệu. Các ranh giới này được xác định trong StateDeltaBuilder và `GameLoop`.

Phân công: TV3 phụ trách bomb/item/event; TV2 phối hợp extension contract và các world adapter.

8.4. Gameplay extension và world adapters

8.4.1. Điểm mở rộng trong runtime

`GameplayExtension` định nghĩa `onStart`, `processCommand`, `update` và `onStop`. Loop gọi chúng trên cùng game thread. Nhờ contract này, loop quản lý lịch và movement, còn nghiệp vụ bom/item nằm trong implementation gameplay. Factory được gọi khi khởi tạo để gắn extension với state của đúng trận. Nguồn: GameplayExtension.

Trong đường chạy mặc định, `MatchGameplayExtension` phối hợp `BombGameplayExtension` và `ItemSystem`. Sau movement, extension xử lý bom đến hạn/nổ dây chuyền/damage, xóa item cũ trong vùng nổ, xét drop từ block vừa phá, pickup theo thứ tự playerId rồi quyết định thời gian/kết quả. Do các bước cùng dùng một owner thread, snapshot cuối tick phản ánh trạng thái sau chuỗi cập nhật đó.

8.4.2. Vai trò của world adapter

Bảng 8.3. Các lớp liên kết gameplay với state và đầu ra.

Thành phần	Trách nhiệm
`BombSystem` qua `BombWorldAdapter`	Kiểm tra đặt bom, truy vấn ô/player, quản lý entity bom và thời lượng đã trôi qua; áp dụng tác động vào state
`ItemSystem` qua `ItemWorldAdapter`	Tạo/xóa entity item, tìm người nhặt và áp dụng effect theo bộ luật
`ItemEffectRegistry`, `ItemSpawnPolicy`	Catalog hiệu ứng và chính sách lựa chọn drop, tách khỏi luồng pickup/runtime

Thành phần	Trách nhiệm
`MatchGameplayExtension`	Kết hợp các bước gameplay, mạng sống/respawn, timer, stats và kết quả trận

Adapter dịch giữa định danh session của API gameplay với playerId/state của runtime, đồng thời kiểm tra quyền truy cập theo game thread. Chúng không mở socket và không tự broadcast. Bom đang chạy dùng kiểm tra thời lượng trong loop; `BombScheduler` còn trong repository không phải scheduler của đường gameplay mặc định. Nguồn: MatchGameplayExtension, BombWorldAdapter, ItemWorldAdapter.

8.4.3. Domain event và thông điệp mạng

Game event không được đồng nhất với frame TCP. Một số event được ghi thành `GameNotice` để đi cùng `GameStateUpdate` nội bộ; state và các frame hiệu ứng/stats được chụp trong update tương ứng. Snapshot worker mới chuyển notices thành message chính thức và gửi sau snapshot qua writer queue của từng session. Như vậy, domain system phát sinh diễn biến nhưng tầng tích hợp quyết định cách biểu diễn trên mạng.

Repository có hai interface cùng tên `GameEventSink` ở hai package: `server.game.event` nhận domain event qua `publish`, còn `server.integration` nhận callback vòng đời session như disconnect/hard drop. Chúng không phải một event bus chung và không chứng minh kiến trúc event sourcing. Lúc cấu hình lobby, callback domain ngoài có thể là no-op; đường ghi state/notices nội bộ vẫn hoạt động.

Phân công: TV1 phụ trách lobby/results UI; TV2 movement/đồng bộ/render map-player; TV3 render bom/item/hiệu ứng; TV4 hạ tầng client dùng chung.

8.5. Client lobby, movement, renderer và results

8.5.1. Điều phối kết nối và màn hình

Bảng 8.4. Trách nhiệm các thành phần client.

Thành phần	Trách nhiệm
`LobbyWindow`, `MatchResultsPanel`	Giao diện kết nối/phòng/ready/chat và kết quả; giữ luồng chuyển giữa lobby và gameplay
`DemoClientConnection`	Owner của `ClientNetwork`, sessionId/MatchContext, heartbeat, control sender và listener ứng dụng xuyên các lượt
`ClientNetwork`	Kết nối TCP, encode/send có khóa, reader decode, phát listener và theo dõi byte; không tự cập nhật world/UI
`MovementClientController`	Nhận packet đúng match, ghép replica, điều phối prediction/reconciliation/interpolation, resync và worker gửi input
`InputController`, `PredictionController`, `ReconciliationController`, `InterpolationController`	Lấy mẫu thao tác và các bước xử lý movement tại client
`MovementGameWindow`, `TileGridRenderer`, `BombScene`, các renderer item/bom/lửa	Vẽ map, entity, nhân vật, hiệu ứng và HUD từ dữ liệu phía client

Tên `DemoClientConnection` còn mang dấu mốc phát triển, nhưng `LobbyWindow` đang dùng lớp này làm owner kết nối thật. Không có một lớp `LobbyController` riêng trong đường chạy đã khảo sát. Nguồn: LobbyWindow, DemoClientConnection và ClientNetwork.

Listener ứng dụng được cài trước CONNECT. Nó chuyển packet sang UI executor tuần tự, xử lý `GAME_STARTED` để tạo controller/context mới, sau đó chuyển snapshot/notices cho controller hiện hành. Controller được tạo với `subscribe=false` trong luồng này, tránh đăng ký thêm listener làm áp dụng packet hai lần. API controller vẫn hỗ trợ tự đăng ký typed listener cho kiểu tích hợp khác.

8.5.2. Phân biệt replica, prediction và dữ liệu render

`ClientWorldState` là replica của dữ liệu server đã áp dụng thành công. `PredictionController` giữ state local riêng và lịch sử input chưa ACK, còn interpolation giữ các mẫu vị trí remote. Ba loại dữ liệu có mục đích khác nhau; không dùng vị trí đã nội suy để sửa world authoritative hoặc tính damage.

Khi áp dụng snapshot, controller kiểm tra kết quả ghép replica. Nếu hợp lệ, nó đồng bộ scene bom, reset lịch sử khi respawn, cập nhật world cho prediction, reconcile rồi ghi mẫu nội suy. Nếu thiếu/sai baseline, controller yêu cầu full và tạm không sinh input mới theo điều kiện input-ready. Nguồn: MovementClientController.

Client tái sử dụng `GameMap`, `Player` và movement/collision từ các package mang tên `server.game` để mô phỏng local. Đây là tái sử dụng mã trong cùng repository, không phải client truy cập instance `GameState` đang chạy trên server. Snapshot codec cũng nằm ở `server.game.movement`; vì vậy, ranh giới trách nhiệm logic chưa hoàn toàn trùng với cách đặt package. Không mô tả project như đã tách mọi DTO/codec/game simulation thành thư viện common độc lập.

8.5.3. Render và kết quả

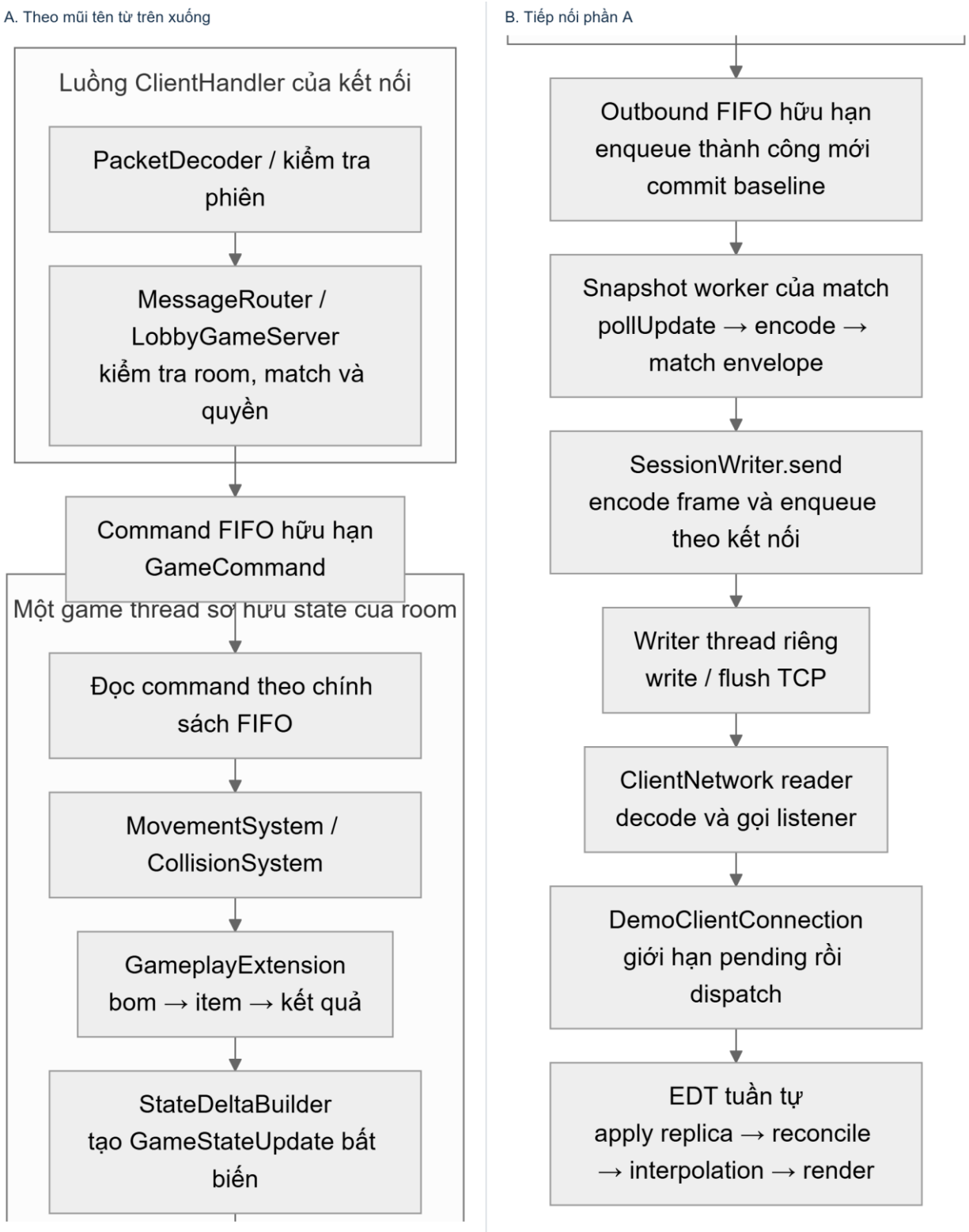
`MovementGameWindow` vẽ nhân vật local theo prediction, nhân vật khác theo interpolation, map/item từ replica và bom/lửa từ `BombScene`. HUD lấy mạng sống, khiên, quota, range và timer từ dữ liệu server. Client chỉ gửi ý định đặt bom; không tự tạo bom hoặc tự trừ damage như một kết quả authoritative.

Sau `GAME_OVER` đúng match, connection owner ghi kết quả, đóng movement controller và bỏ context trận; lobby đóng cửa sổ gameplay, hiển thị `MatchResultsPanel` và giữ kết nối để ready/start lượt khác. Render timer và input timer có vòng đời gắn với cửa sổ gameplay, không được để tiếp tục dùng controller của trận cũ. Nguồn: MovementGameWindow.

Phân công: TV4 ghép mục 8.6; TV1 xác nhận transport/session, TV2 loop và client state, TV3 gameplay cleanup.

8.6. Router, worker, queue, quyền sở hữu thread và cleanup

8.6.1. Các ranh giới bàn giao dữ liệu



Hình 8.2. Luồng command và snapshot qua các thread/queue. Nhóm tự vẽ từ code project.

Sơ đồ tập trung đường gameplay mặc định. Queue writer chứa frame đã encode; không đặt toàn bộ encode vào writer thread. Việc render chạy theo timer của UI sau các lần cập nhật state,

không phải một bước bắt buộc được gọi ngay sau mỗi packet. Bản nguồn: 08_02_thread_va_queue.mmd.

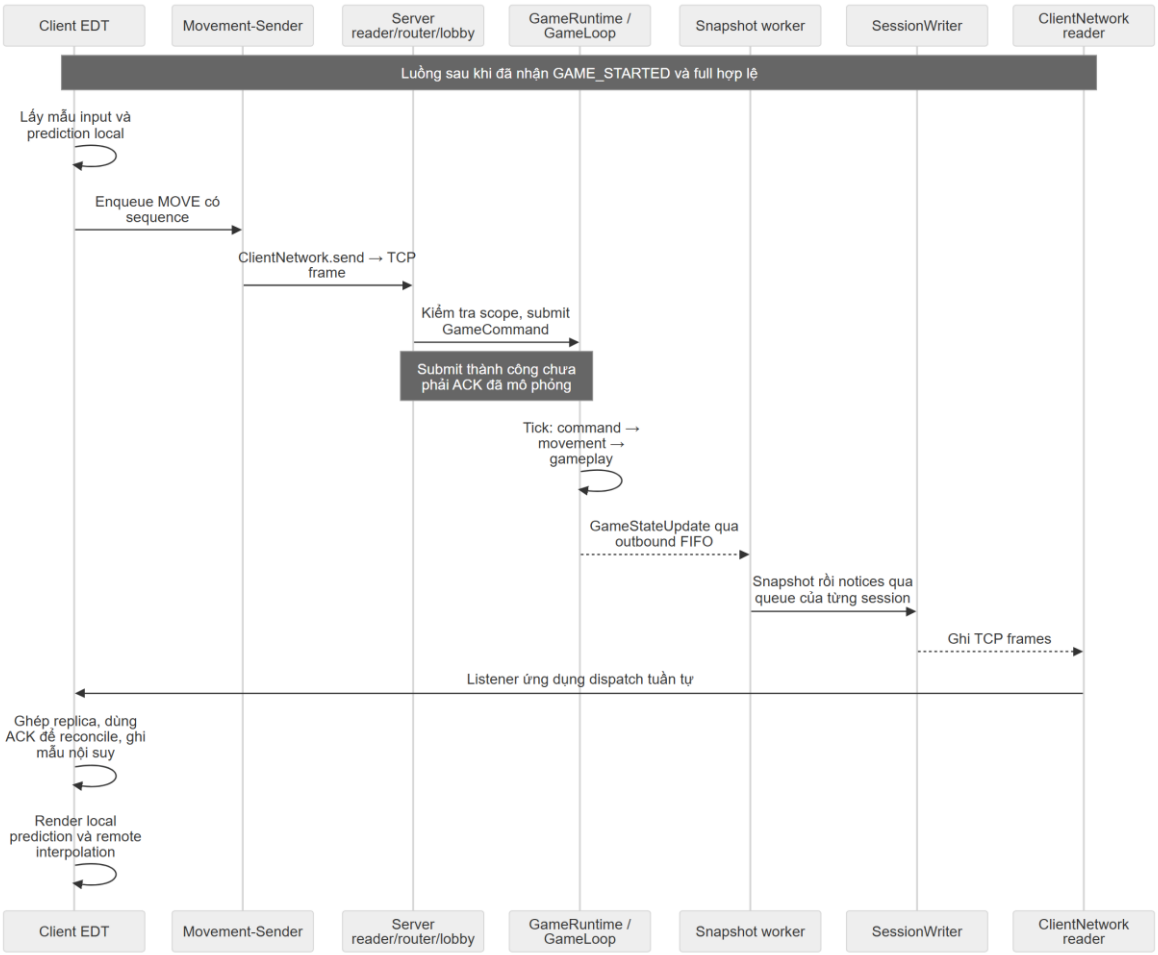
Bảng 8.5. Thread, dữ liệu sở hữu và cơ chế giới hạn.

Miền thực thi	Công việc/dữ liệu	Bàn giao và giới hạn chính
Accept và reader pool của server	Socket, đọc frame và route	Pool/queue hữu hạn; `MessageRouter` gọi đồng bộ handler
Điều phối lobby/session/room	Thành viên, match context và vòng đời	Có synchronization; một số thao tác lobby dùng khóa chung
GameLoop của từng runtime	State mutable, movement, extension và builder	Nhận command FIFO; xuất snapshot bất biến qua outbound FIFO
Snapshot worker của từng match	Poll update, codec, envelope, snapshot/notices/result	Không đọc state mutable; enqueue writer của từng session
Session writer của từng socket	Ghi/flush frame đã encode	Queue 256 frame chờ, budget 4 MiB tính cả frame đang ghi; watchdog ghi kéo dài
Worker/scheduler phụ trợ server	Chat, heartbeat, session expiry, watchdog và connection cleanup	Tách vòng đời hạ tầng khỏi bước update gameplay
Reader/control/sender của client	Decode packet, gửi control/heartbeat/resync hoặc input	`ClientNetwork.send` khóa frame; control và movement có cơ chế giới hạn pending
Swing EDT	Replica, prediction/replay, buffer nội suy và UI	Dispatch tuần tự; input/render timer; giới hạn pending dispatch trước EDT

Giới hạn pending dispatch của owner client không biến hàng đợi EDT toàn cục của Swing thành một queue hữu hạn. Đường listener mặc định tắt pull queue của `ClientNetwork` để không giữ thêm bản packet không có consumer. Nếu API `receive()` được dùng ở tích hợp khác, pull queue có giới hạn riêng.

State theo phòng được cách ly, nhưng toàn hệ thống vẫn dùng chung CPU, reader pool, manager và khóa điều phối. Không suy ra mỗi room có tài nguyên độc lập hoàn toàn hoặc throughput tăng tuyến tính theo số room.

8.6.2. Luồng dữ liệu movement và snapshot



Hình 8.3. Một lượt input đi qua runtime và trở về client. Nhóm tự vẽ từ code project.

Sơ đồ mô tả các bước logic; một snapshot có thể phản ánh nhiều biến đổi và được phát cho cả roster, không phải mỗi input luôn có ngay một packet phản hồi riêng. Gameplay notice chỉ được phát nếu có; mỗi client khác nhận qua writer riêng. Bản nguồn: 08_03_luong_movement.mmd.

Với bom, nhánh đầu vào là PLANT_BOMB → kiểm tra scope/rate/requestId → command queue. Game thread áp dụng placement và xử lý bom đến hạn qua extension; entity/map/player mới xuất hiện trong snapshot, còn diễn biến tương ứng có thể đi qua notice/effect. Với resync, client gửi REQUEST_FULL_STATE; tầng lobby đặt yêu cầu full cho runtime và snapshot worker nhận diện full mới sau mốc yêu cầu để công bố FULL_STATE_SYNC. Cả hai luồng giữ nguyên ranh giới state và I/O của hình 8.2.

8.6.3. Chính sách quá tải và lỗi

Command queue đầy làm submit thất bại, được tầng tích hợp trả lỗi thay vì chờ đến khi queue có chỗ. Outbound runtime đầy giữ baseline chưa commit để lần tạo sau còn tham chiếu đúng dữ liệu đã enqueue. Writer queue/budget đầy hoặc write deadline bị vượt có thể đóng kết nối và kích hoạt cleanup; không tùy ý bỏ một delta rồi tiếp tục như thể client đã nhận đủ.

Ở client, hết budget dispatch hoặc queue gửi được xử lý bằng dừng controller/phiên và báo lỗi theo owner. Đối với sai baseline hợp lệ ở cấp đồng bộ, client yêu cầu full thay vì tự suy đoán

phần thiếu. Các cơ chế này giới hạn một số dạng backlog; hiệu quả dưới tải và các đường lỗi vẫn cần kiểm chứng trong Results.

8.6.4. Cleanup và chuyển lượt

Kết thúc gameplay bình thường đi qua kết quả trong snapshot; snapshot worker đưa kết quả về tầng lobby để đóng match. Kết thúc hành chính hoặc mất kết nối dùng chính sách ABORTED nếu lượt chưa có kết quả chốt. Tầng điều phối đánh dấu match không còn active, dừng runtime, chuyển room về WAITING và công bố GAME_OVER/room state.

`GameLoop.stop` yêu cầu dừng, interrupt và chờ thread có thời hạn; `onStop` chạy trong phần cleanup của game thread để extension dọn bom/item/effect. Snapshot worker thoát theo cờ/lifecycle runtime và tự rời danh sách worker. Khi đóng cả server, `LobbyGameServer.close` còn đóng transport/manager và interrupt/join worker còn lại.

Transport đóng socket để đánh thức thao tác đọc/ghi đang chặn, dừng heartbeat/expiry/watchdog và reader pool/chat worker. `ConnectionManager` chuyển callback lỗi writer sang cleanup executor để tránh gọi trực tiếp logic lobby trong lúc producer còn giữ khóa. Phía client dừng timer, đóng movement sender, bỏ context trận cũ; khi đóng phiên mới đóng cả reader/control/network. Việc dọn tài nguyên là trách nhiệm theo từng owner, không chỉ gọi `dispose()` cho cửa sổ.

Một số frame đã vào writer có thể còn tới sau khi trận kết thúc. Match identity và context client giúp bỏ dữ liệu cũ; lượt chơi lại tạo runtime/world/controller mới, đồng thời giữ TCP nếu phiên vẫn hợp lệ. Các cơ chế chống dữ liệu cũ không tạo ra khả năng resume trận sau mất kết nối.

8.6.5. Liên hệ yêu cầu và giới hạn kiến trúc

Bảng 8.6. Kiến trúc đáp ứng các nhóm yêu cầu ở mục 6.

Nhóm yêu cầu	Thành phần chịu trách nhiệm chính	Nội dung cần kiểm chứng tiếp
FR01–FR03	Transport, session, room, router và lobby UI	Phân quyền, trạng thái phòng, chat đúng scope
FR04–FR09	Runtime riêng room, movement và gameplay extension/adapters	Isolation, collision, bom/item/damage và thứ tự cập nhật
FR10–FR11	Builder/codec, snapshot pump, replica và client controllers	Baseline/resync, ACK/replay, respawn và biểu diễn
FR12–FR14	Lobby lifecycle, session writer, cleanup và context theo match	Kết thúc/chơi lại, dữ liệu cũ, queue đầy và lỗi kết nối
NFR01–NFR08	Các ranh giới owner/queue và công cụ đo/test	Tính đúng, nhịp/tải/byte, giải phóng tài nguyên và tái lập

Mô tả kiến trúc giải thích cách các thành phần phối hợp, chưa xác nhận mọi thuộc tính chất lượng đã đạt. Hồ sơ khảo sát hiện còn lỗi process smoke và khoảng trống GUI/LAN/soak; không dùng sơ đồ này làm bằng chứng thay cho kiểm thử hoặc số liệu thực nghiệm. Trạng thái bằng chứng được giữ tại hồ sơ khảo sát.

9. Thiết kế giao tiếp mạng

Protocol của Bomb It Multiplayer là giao thức ứng dụng trên TCP, phục vụ kết nối, lobby/phòng và gameplay trong cùng một phiên client. Chương này trình bày đường chạy lobby mặc định theo codec và handler hiện hành; không coi mọi message có tên trong đặc tả lịch sử là chức năng đã được bật. Kiến trúc thread/queue được giải thích ở mục 8; ở đây tập trung vào dữ liệu trên đường truyền, điều kiện xử lý và thứ tự trao đổi.

Quy ước: `C→S` là client gửi server; `S→C` là server gửi một client; `S→Room` là phát tới các client thuộc phòng/trận. `S`, `R`, `M` trong schema lần lượt biểu thị UUID session, room và match, không phải chuỗi literal gửi trên mạng. Ký hiệu kiểu và trường có thể bỏ trong bảng dùng để mô tả schema; các ví dụ JSON riêng là dạng canonical khi có yêu cầu.

Phân công: TV4 phụ trách framing/codec/version; TV1 xác nhận transport và handshake.

9.1. TCP framing, UTF-8, size limit và handshake protocol 2

9.1.1. Cấu trúc frame

Bảng 9.1. Frame ứng dụng trên TCP.

Trường	Kích thước	Quy tắc
`length`	int32, 4 byte	Số byte của type + payload, không tính chính prefix này
`type`	int16, 2 byte	Mã message theo `PacketType`
`payload`	`length − 2` byte	JSON mã hóa UTF-8

Các số nhiều byte của frame dùng big-endian. Giới hạn `length` là 2..1.048.576 byte, do đó payload tối đa 1.048.574 byte và tổng frame tối đa 1.048.580 byte. Giới hạn tính sau khi đã thêm match envelope, JSON và Base64; không chỉ tính binary snapshot. Không có newline phân cách frame. TCP có thể chia hoặc gộp dữ liệu giữa các lần đọc; decoder đọc đủ 4 byte length rồi `readFully` phần body và kiểm tra UTF-8 trước khi chuyển cho handler.

Ví dụ với message PING `0x0040`, body canonical `{"pingId":1}` có 12 byte UTF-8, `length=14` và tổng frame 18 byte:

```
00 00 00 0E 00 40 7B 22 70 69 6E 67 49 64 22 3A 31 7D
```

Đây là ví dụ tính frame, chưa phải số đo packet TCP/IP. Nguồn: PacketEncoder, PacketDecoder và PacketType.

9.1.2. Phân lớp kiểm tra dữ liệu

Framing kiểm tra kích thước/UTF-8; codec kiểm tra schema; handler kiểm tra trạng thái, scope và quyền; game thread quyết định kết quả luật chơi. Frame đọc được không đồng nghĩa request đã hợp lệ hoặc đã được mô phỏng.

`SessionPacketCodec` xử lý object control nhỏ; `LobbyPacketCodec`/`LobbyJson` xử lý object lobby và một số message gameplay. MOVE, resync, metadata bắt đầu trận và snapshot còn dùng định dạng canonical riêng. Một JSON tương đương nhưng đổi thứ tự trường hoặc thêm khoảng trắng có thể không được các codec canonical chấp nhận. Client phải dùng encoder hiện có thay vì tự tạo JSON tùy ý cho các loại đó.

Request qua `LobbyPacketCodec.request` tối đa 4096 ký tự; parser `LobbyJson` có giới hạn tổng 65.536 ký tự, độ sâu 8 và tối đa 256 phần tử trong array. Control object tối đa 1024 ký tự; MOVE và resync inner body tối đa 128 ký tự. Đây là các giới hạn ở những lớp khác nhau, không thay thế giới hạn byte của frame.

9.1.3. Thương lượng phiên bản

Client mở TCP rồi gửi:

```
{"playerName":"PlayerA","protocolVersion":2}
```

Server chỉ chấp nhận handshake protocol 2 trong đường chạy tích hợp. Tên được strip, dài 1–32 code point và không chứa control character. CONNECT sai version được trả `UNSUPPORTED_VERSION`, sau đó kết nối bị dọn theo chính sách transport. `protocolVersion` không phải `snapshotVersion`: trường thứ nhất quy định contract tích hợp, trường thứ hai mô tả layout binary của snapshot.

Thông tin ACK gồm `playerId` UUID phiên, token, `serverVersion`, protocol version và các giá trị heartbeat/grace. `serverVersion` hiện là chuỗi `0.1.0`; đó không phải mã revision kiểm thử và không chứng minh tương thích với mọi binary snapshot. Nguồn: SessionPacketCodec và SessionHandlers.

Phân công: TV1 phụ trách danh tính và vòng đời phiên/phòng; TV4 kiểm tra envelope/context, TV2 xác nhận mapping runtime.

9.2. Session, room, match identity và vòng đời kết nối

9.2.1. Hệ thống định danh

Bảng 9.2. Định danh và phạm vi ý nghĩa.

Trường	Kiểu/phạm vi	Cách sử dụng
Session ID	UUID string server cấp	Field `playerId` trong CONNECT_ACK và roster phòng; danh tính kết nối đã handshake
Room ID	UUID string	Membership và quyền thao tác phòng
Match ID	UUID mới cho mỗi runtime/lượt	Lọc lệnh và state thuộc lượt cũ, kể cả chơi lại cùng phòng
Gameplay player ID	int32 dương, trong match	Ánh xạ từ session qua MatchContext/runtime; không nhận từ MOVE
Bomb/item ID	int32 dương trong namespace kind + match	Entity bom và item có thể có cùng số nhưng khác key `BOMB:id`/`ITEM:id`
Sequence, tick, baseTick, requestId, eventId	int64 Java; JSON integer khi ở JSON	Đánh số input, baseline, yêu cầu và event; không dùng float thay ID

Client lấy session UUID từ CONNECT_ACK rồi đối chiếu roster GAME_STARTED để biết local playerId số. `EntityData.ownerId` của bom vẫn là UUID chủ sở hữu, trong khi `ownerPlayerId` của event wire là int gameplay. Không chuyển hai dạng này bằng cách parse UUID thành số nguyên.

9.2.2. Match envelope

Message gameplay được bọc bằng field `matchId` ở đầu object. Ví dụ MOVE trên dây:

```
{"matchId":"11111111-1111-4111-8111-
111111111111","sequence":7,"direction":"RIGHT","moving":false}
```

`MatchEnvelopeCodec` kiểm tra type, prefix UUID kỳ vọng, header lặp và kích thước frame. Sau unwrap, codec chuyên biệt tiếp tục kiểm tra body; envelope không phải parser đầy đủ cho JSON bên trong. Server lấy danh tính từ session/socket đang xử lý và đối chiếu runtime; client kiểm tra matchId trước áp dụng state.

Envelope áp dụng cho MOVE, PLANT_BOMB, snapshot/full-sync/resync, END_GAME, GAME_OVER và các notice gameplay. GAME_STARTED đã tự chứa matchId; message session, heartbeat và lobby không bọc envelope. Các API legacy không envelope vẫn có thể phục vụ fixture, nhưng không được suy ra thành contract production protocol 2. Nguồn: MatchEnvelopeCodec.

9.2.3. Heartbeat và kết thúc kết nối

Bảng 9.3. Các message phiên/control đang được sử dụng.

Mã	Message/hướng	Body chính và ngữ nghĩa
`0x0001`	CONNECT, C→S	`playerName`, `protocolVersion:2`; phải hoàn tất trước yêu cầu lobby/gameplay
`0x0002`	CONNECT_ACK, S→C	`playerId:S`, `reconnectToken`, `serverVersion`, `protocolVersion:2`, `heartbeatIntervalMs:2000`, `heartbeatTimeoutMs:10000`, `reconnectGraceMs:15000`
`0x0040`	PING, C→S	`pingId:int32`; client mặc định gửi theo chu kỳ 2 giây
`0x0041`	PONG, S→C	Echo đúng JSON body PING đã qua kiểm tra
`0x0042`	DISCONNECT, C→S	`reason:"USER_QUIT"`; handler hỗ trợ đóng phiên chủ động
`0x00FF`	ERROR, S→C	`errorCode`, `message`, `requestId` có thể null; cách xử lý tùy lớp lỗi

Server sweep heartbeat mặc định mỗi 2 giây và timeout 10 giây; việc phát hiện không phải deadline chính xác tại giây thứ 10. Frame được route thành công có thể cập nhật liveness; dữ liệu chưa decode hoặc request bị từ chối không mặc nhiên giữ phiên sống. Client owner ghi thời điểm nhận packet đã decode và kiểm tra timeout qua control worker; không yêu cầu chỉ PONG mới được tính là hoạt động từ server.

Đường đóng client hiện tại có thể đóng socket trực tiếp, được server nhận qua EOF; không khẳng định mỗi lần người dùng đóng cửa sổ đều phát DISCONNECT. Mất kết nối trong lượt chơi hiện kết thúc/hủy lượt và bỏ ghế; metadata phiên có thể còn chờ dọn tối đa theo grace, không còn world để resume.

9.2.4. Message có khai báo nhưng chưa thành luồng sản phẩm

`RECONNECT=0x0043` và `NETWORK_LOST=0x0044` có trong PacketType/tài liệu, nhưng router sản phẩm chưa đăng ký handler. Enum reason `RECONNECT` trong request resync cũng không chứng minh đã có reconnect end-to-end. `PLAYER_DISCONNECTED=0x0032` còn dùng trong các API/nhánh

hạ tầng; đường lobby hiện tại chủ yếu cập nhật bằng GAME_OVER và ROOM_STATE, không có luồng thông báo player-disconnected trong trận giữ ghế để tiếp tục chơi.

Không đưa các schema reconnect dự kiến thành sequence đã triển khai. Sau handshake, type chưa có handler có thể trả UNKNOWN_MESSAGE; trước handshake còn chịu cổng kiểm tra CONNECT. Căn cứ: LobbyGameServer, ClientHandler, DemoClientConnection và đánh giá reconnect.

Phân công: TV1 phụ trách nội dung lobby/phòng; TV4 chuẩn hóa schema và hướng truyền.

9.3. Message lobby và room

Bảng 9.4. Các message phòng đang nối vào server/client mặc định.

Mã	Message/hướng	Body và điều kiện chính
`0x0010`	CREATE_ROOM, C→S	`roomName` 1–64 code point, `maxPlayers` 2–4; `mode` có thể bỏ hoặc là `STANDARD`
`0x0011`	JOIN_ROOM, C→S	`roomId:R`; phòng đang WAITING, còn chỗ, client chưa ở phòng khác
`0x0012`	ROOM_STATE, S→Room	`roomId,roomName,hostId,state,maxPlayers,players:[{playerId:S,username,state}]`
`0x0013`	READY, C→S	`roomId:R,ready:boolean`; đặt trạng thái ready, không phải yêu cầu toggle
`0x0014`	START_GAME, C→S	`roomId:R`; host, ít nhất hai người và mọi khách ready
`0x0015`	GAME_STARTED, S→Room	`roomId:R,matchId:M,tickMillis:50,inputStepMillis:50,mapId:"demo",players:[{sessionId:S,playerId:int,username}]`
`0x0016`	LEAVE_ROOM, C→S/S→C	Request `roomId:R`; ACK cùng type `{roomId:R,left:true}`; người còn lại nhận cập nhật phòng
`0x0017`	CHAT, C→S/S→Room	Request `roomId:R,message` 1–256 code point; response thêm `playerId:S,username` từ session
`0x0018`	LIST_ROOMS, C→S/S→C	Request `{}`; response `{rooms:[ROOM_STATE body...]}`; cũng được server chủ động phát khi danh sách đổi
`0x0035`	END_GAME, C→S	Envelope chỉ chứa `matchId:M`, inner `{}`; host của lượt hiện tại

Với READY, START_GAME, CHAT và LEAVE_ROOM, handler còn chấp nhận dạng rút gọn thiếu roomId bằng membership hiện tại; nếu có roomId thì phải khớp. JOIN_ROOM vẫn cần roomId. Sau CREATE/JOIN, trạng thái phòng và danh sách được cập nhật, không có một mã CREATE_ACK hoặc JOIN_ACK riêng.

Chat chỉ phát trong phòng, cả lúc WAITING và PLAYING. Tên tác giả lấy từ session, không tin tác giả tự khai trong request. Phòng đang chơi không cho join trực tiếp. Khi host rời, quyền host chuyển cho thành viên còn lại theo thứ tự phòng.

GAME_STARTED hiện được `DemoPacketCodec` encode với `mapId:"demo"` ngay cả ở lobby mặc định dùng DefaultArenaMap. Client không dùng chuỗi này để tự suy ra layout: map đầy đủ lấy từ full snapshot. Đây là giá trị metadata hiện tại, không phải tuyên bố ứng dụng chỉ chạy profile movement demo. Nguồn: LobbyPacketCodec, DemoPacketCodec và `LobbyGameServer`.

Phân công: TV2 phụ trách input/snapshot/baseline; TV4 phối hợp codec, match envelope và resync.

9.4. MOVE, input sequence/ACK, snapshot v2–v5 và full/delta/resync

9.4.1. Message movement và phục hồi

Bảng 9.5. Message đồng bộ trạng thái, đều có match envelope ở protocol 2.

Mã	Message/hướng	Inner body và ý nghĩa
`0x0020`	MOVE, C→S	`sequence:int64>0,direction:UP/DOWN/LEFT/RIGHT,moving:boolean`
`0x0030`	GAME_SNAPSHOT, S→Room	`snapshotVersion,data:Base64`; có thể full hoặc delta theo binary kind
`0x0033`	FULL_STATE_SYNC, S→Room	Cùng schema snapshot, nhưng binary kind bắt buộc full
`0x0034`	REQUEST_FULL_STATE, C→S	`lastAppliedTick>=-1,reason:COLD_START/BASELINE_MISMATCH/RECONNECT`

Client phát mẫu input mỗi 50 ms, kể cả mẫu đứng yên. Sequence bắt đầu 1 trong lượt mới; server bỏ mẫu không lớn hơn `lastProcessedInput` và có thể chấp nhận khoảng trống sequence. Một tick áp dụng tối đa một mẫu movement mới mỗi player theo FIFO toàn cục. Với input có sequence, khi không có mẫu mới được áp dụng thì movement không tự tiếp tục thêm một bước cho player đó.

ACK nằm trong PlayerData của snapshot sau bước movement, không có MOVE_ACK riêng và không dùng ACK TCP thay nó. Client bỏ input pending có sequence ≤ ACK, khôi phục state server rồi replay phần còn lại. Khi đã có 256 input chưa ACK, prediction từ chối tạo mẫu tiếp và báo lỗi cho caller, không tự xóa mẫu hoặc bảo đảm tự khởi động resync. Đây là khả năng của controller; không tuyên bố đã có tự động gửi lại mọi input bị từ chối bởi server. Nguồn: MovePacketCodec, MovementSystem và PredictionController.

9.4.2. Lớp binary snapshot và version

Dữ liệu trên dây có các lớp: frame → JSON match envelope → `snapshotVersion,data` → Base64 → binary snapshot. Base64 chỉ biểu diễn byte, phần tránh gửi lại dữ liệu đến từ builder full/delta. Binary dùng big-endian; int32/int64 lần lượt 4/8 byte, double 8 byte, ordinal 1 byte; encoder ghi boolean bằng một byte 0/1. Chuỗi `writeUTF` dùng modified UTF-8 có prefix độ dài uint16, khác UTF-8 của JSON bên ngoài.

Bảng 9.6. Thứ tự các khối binary cơ sở v2.

Thứ tự	Trường/khối	Kiểu và nội dung
1	kind, tick, baseTick	uint8 full=0/delta=1; int64; int64
2	mapWidth, mapHeight, tileSize	Ba int32
3	matchState, matchStateChanged	uint8 WAITING/RUNNING/FINISHED; boolean
4	Full map	count:int32, rồi tile:uint8 theo thứ tự `y*width+x`
5	Tile changes	count:int32, mỗi record `x:int32,y:int32,type:uint8`
6	Players	count:int32, các PlayerData đầy đủ hoặc upsert theo loại update

Thứ tự	Trường/khối	Kiểu và nội dung
7	Removed players	count:int32, rồi các playerId:int32
8	Entities	count:int32, các EntityData đầy đủ hoặc upsert
9	Removed entities	count:int32, rồi key:`writeUTF`

Một PlayerData gồm `id:int32`, năm double `x,y,width,height,speed`, `direction:uint8`, `moving:boolean`, `alive:boolean`, `lastProcessedInput:int64`. Một EntityData gồm `kind:uint8`, ba int32 `id,tileX,tileY`, `ownerId:writeUTF`, `variant:int32`, `expiresAtMillis:int64`. Với bom, variant là range và ownerId là UUID; với item, variant là wireId cố định và ownerId rỗng. Các key xóa dạng `BOMB:<id>/ITEM:<id>` là phần bắt buộc để biết entity đã biến mất.

Direction wire cố định `UP=0,DOWN=1,LEFT=2,RIGHT=3`; tile `EMPTY=0,HARD_WALL=1,BREAKABLE_BLOCK=2`; match state `WAITING=0,RUNNING=1,FINISHED=2`; entity `BOMB=0,ITEM=1`. OUT_OF_BOUNDS không phải tile được lưu/truyền. Item dùng mã riêng `BOMB_UP=0,FIRE_UP=1,SPEED_UP=2,LIFE_UP=3,SHIELD=4`. Thay layout, ordinal hoặc ý nghĩa trường cần migration/version tương ứng.

Bảng 9.7. Các version snapshot được decoder hiện tại hỗ trợ.

Version	Phần dữ liệu thêm so với nền v2	Tình trạng trong encoder hiện tại
2	Map/player/entity và ACK	Phát khi update không có BombFrame/MatchFrame
3	Sau v2: serverTimeMillis và danh sách toàn bộ explosion còn hiển thị	Phát khi có BombFrame nhưng không MatchFrame
4	Sau v3: remainingSeconds, Stats cũ, hasResult và result JSON qua writeUTF	Decode để tương thích; encoder hiện tại không chọn phát v4
5	Sau v3: MatchFrame như v4, nhưng Stats thêm lives/shield/respawns	Lobby gameplay hiện phát v5

Phần v3 gồm `serverTimeMillis:int64`, `explosionCount:int32`; mỗi explosion có `bombId:int32,startedAtMillis:int64,visibleUntilMillis:int64,cellCount:int32`, rồi các cặp `x,y:int32`. Danh sách này là toàn bộ effect còn hiệu lực trong cả full và delta, không phải danh sách effect chênh lệch. Giới hạn DTO là 256 explosion, mỗi explosion tối đa 1024 ô; duration hiển thị hiện 600 ms. State/effect từ snapshot mới quyết định cảnh bom/lửa, không tự xóa bom vì đồng hồ client vừa đếm về 0.

MatchFrame v5 gồm `remainingSeconds:int32,statsCount:int32`, mỗi Stats gồm `playerId:int32,bombLimit:int32,range:int32,lives:int32,shield:boolean,respawns:int32`, sau đó `hasResult:boolean`; nếu true thì `result:writeUTF` chứa inner JSON GAME_OVER. Cả full và delta mang toàn bộ Stats, không chỉ player có tọa độ đổi. V4 thiếu ba trường mới được đọc với lives=1, shield=false, respawns=0. Result trong snapshot phải thuộc state FINISHED và không có tick lớn hơn snapshot.

Decoder từ chối dữ liệu dư, count không phù hợp, số/enum không hợp lệ và body không nhất quán full/delta; replica tiếp tục kiểm tra tọa độ và tập ID Stats trước khi apply. Các bộ encode/decode hiện tại phải được phân phối tương thích giữa client/server; handshake version 2 không tự bảo đảm một client cũ đọc được v5. Nguồn: SnapshotPacketCodec, BombFrame và MatchFrame.

9.4.3. Full, delta và điều kiện áp dụng

Full có `baseTick=-1`, đủ `width*height` tile và toàn bộ player/entity; không kèm tile changes hoặc danh sách xóa. Delta có `0<=baseTick<tick`, không chứa full map, dùng record upsert và danh sách xóa rõ ràng. Không có record trong delta không có nghĩa đối tượng bị xóa. Khi một player đổi, codec gửi cả PlayerData, chưa nén riêng từng field.

Replica bỏ update có `tick<=tick` đang giữ. Delta chỉ áp dụng khi baseTick khớp tick replica, kích thước map/ô không đổi và các kiểm tra dữ liệu thành công. Nếu cần full, replica giữ dữ liệu cũ để hiển thị và báo NEED_FULL_STATE; không ghép một phần delta sai. Full hợp lệ thay dữ liệu cũ và tạo baseline mới.

Server tạo full đầu, full mỗi 100 tick, full khi được yêu cầu và full FINISHED. 100 tick tương ứng khoảng 5 giây nếu loop theo kịp nhịp 50 ms; không phải bảo đảm thời gian thực dưới mọi tải. Update không đổi có thể không được phát, nên baseline không nhất thiết là tick ngay trước. Builder chỉ commit khi enqueue vào outbound FIFO thành công, chưa phải khi client ACK đã áp dụng. Nguồn: StateDeltaBuilder, GameLoop và ClientWorldState.

9.4.4. Request full và phục hồi baseline

Ví dụ request canonical trên dây:

```
{"matchId":"11111111-1111-4111-8111-
111111111111","lastAppliedTick":40,"reason":"BASELINE_MISMATCH"}
```

`lastAppliedTick=-1` nghĩa chưa có baseline. Handler kiểm tra request nhưng không dùng tick client để rewind world. Nó lưu barrier từ update gần nhất, đặt cờ request full; consumer snapshot hiện có chờ full với tick mới hơn barrier để phát FULL_STATE_SYNC. Không thêm consumer cạnh tranh outbound FIFO và không coi `createUpdate()` là full.

Baseline chung theo runtime/room nên full phục hồi được phát cho cả roster, dù một client yêu cầu. Server giới hạn tối thiểu 1 giây giữa các request; client owner hiện lập lịch cách tối thiểu 1100 ms và gộp request đang chờ. FULL_STATE_SYNC dùng cùng body version tương ứng update, bắt buộc full. Không diễn giải resync như giao thức reconnect hoặc bảo đảm sửa được mọi payload hỏng. Nguồn: ResyncPacketCodec, LobbyGameServer và DemoClientConnection.

Phân công: TV3 phụ trách nội dung bom/item/damage/kết quả; TV4 codec và envelope, TV2 state/notice handoff.

9.5. PLANT_BOMB, event, dedupe và GAME_OVER

9.5.1. Tiếp nhận lệnh đặt bom

PLANT_BOMB `0x0021` có inner body `{"requestId":1}`, requestId int64 dương và tăng theo player/match. Không truyền tọa độ, range hoặc owner do client tự khai. Server lấy player từ session/context và vị trí từ world authoritative.

Handler bỏ requestId cũ/trùng trước khi tính giới hạn riêng của đặt bom, nhưng frame đó vẫn được tính vào tổng rate limit kết nối. Chỉ ghi nhận requestId sau khi submit command thành công; queue đầy có thể thử lại cùng ID theo contract. Không suy ra client hiện đã tự động retry mọi yêu cầu bị từ chối.

Submit thành công cũng chưa bảo đảm bom được tạo: game thread còn kiểm tra player sống, quota và ô đặt. Trong cấu hình lobby hiện tại, callback `BombCommandResult` bên ngoài là no-op,

chưa có message ACK riêng báo thành công/thất bại nghiệp vụ cho từng requestId. Bom đặt thành công được phản ánh qua snapshot và BOMB_PLANTED; các lỗi như quota đầy ở bước gameplay không mặc nhiên được gửi thành ERROR có requestId tương ứng.

9.5.2. Notice và kết quả trên dây

Mỗi notice trong bảng có thêm bộ trường chung `eventId:int64>0,tick:int64>0,occurredAtMillis:int64`, trừ GAME_OVER dùng schema riêng. Các message dưới đây là `S→Room` và có match envelope.

Bảng 9.8. Body inner của các thông báo gameplay và kết quả.

Mã	Message	Các trường ngoài bộ notice chung
`0x0022`	BOMB_PLANTED	`bombId:int,ownerPlayerId:int,x:int,y:int,fuseMs:long,explodeAtMillis:long,range:int`
`0x0023`	BOMB_EXPLODE	`bombId:int,x:int,y:int,affectedCells:[{x:int,y:int}]`
`0x0024`	ITEM_SPAWN	`itemId:int,itemType:string,x:int,y:int`
`0x0025`	ITEM_PICKED_UP	`itemId:int,playerId:int,itemType:string`
`0x0026`	PLAYER_DIE	`playerId:int,killerPlayerId:int hoặc null,cause:"EXPLOSION"`
`0x0031`	GAME_OVER	Schema riêng: `tick:long,winnerPlayerId:int hoặc null,reason,rankings:[{playerId:int,rank:int}]`

`itemType` là một trong BOMB_UP/FIRE_UP/SPEED_UP/LIFE_UP/SHIELD. PLAYER_DIE chỉ mô tả bị loại; khiên chặn hoặc mất một mạng rồi respawn được thể hiện qua snapshot Stats, không phát PLAYER_DIE cho mọi lần nhận damage. Không có packet BLOCK_DESTROYED riêng: thay đổi nền đi qua snapshot, còn event block nội bộ phục vụ gameplay/drop.

EventId được cấp trên game thread theo match; snapshot worker encode notice đã bàn giao sau snapshot tương ứng, qua cùng writer FIFO của từng session. Client không dùng notice để áp lại damage, pickup hoặc thay đổi map. Scene bom ghi nhận notice cùng tick replica và dedupe trong cache 256 ID; notice item/death được validate nhưng không cộng lại chỉ số. Không khái quát cơ chế cache này thành bảo đảm exactly-once đầu cuối cho mọi message.

Full/resync mang state và effect hiện tại, không phát lại toàn bộ lịch sử notices. Nguồn: BombPacketCodec, GameplayPacketCodec, BombScene và MovementClientController.

9.5.3. GAME_OVER và chính sách kết thúc

Reason là LAST_ALIVE, ALL_DEAD, TIMEOUT hoặc ABORTED. Chỉ LAST_ALIVE có winnerPlayerId; ABORTED không có người thắng và rankings rỗng. Lượt kết thúc bình thường có full FINISHED/result, sau đó notices nếu có và GAME_OVER. Client owner áp dụng snapshot trên cùng UI executor trước khi xử lý GAME_OVER.

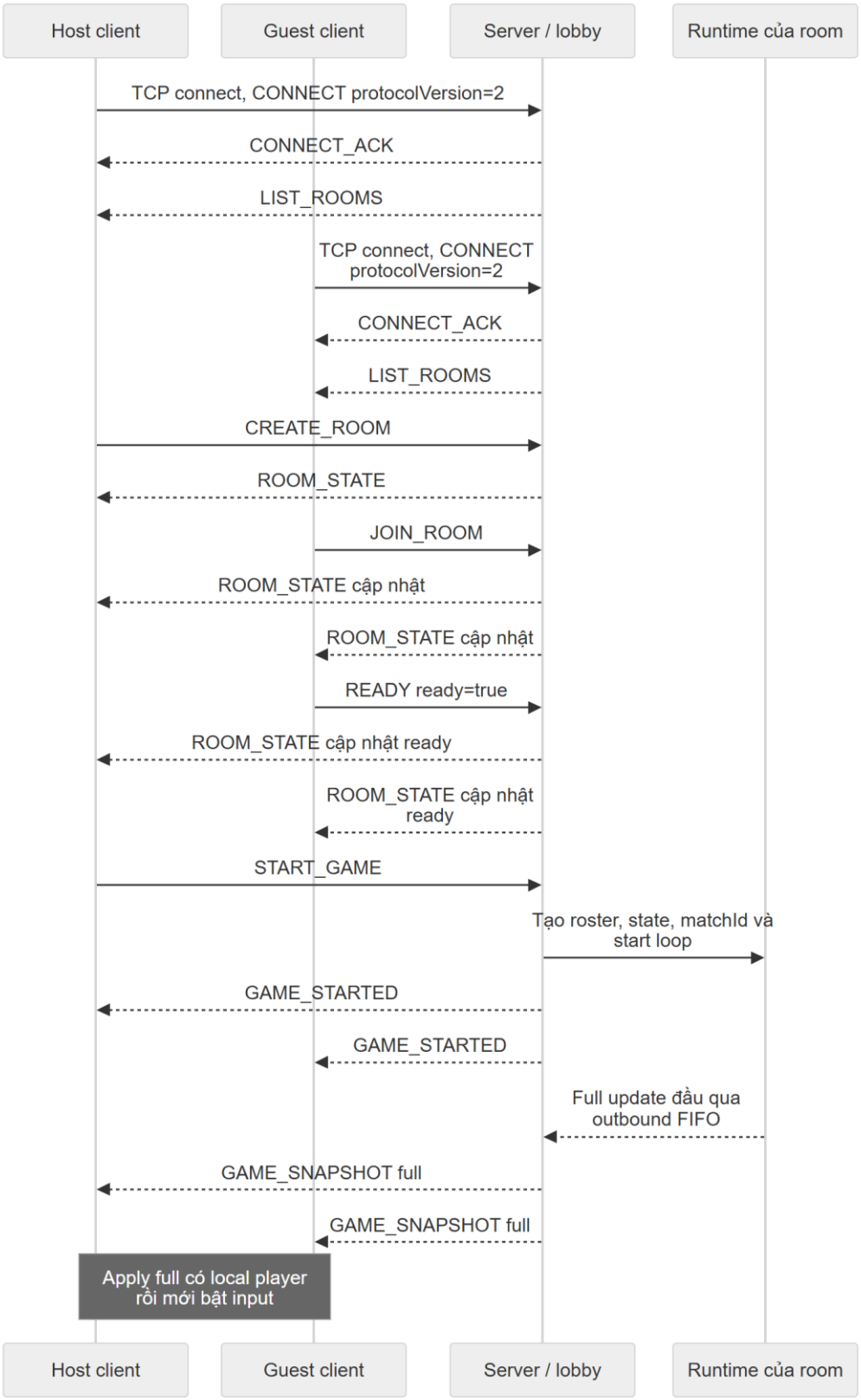
Hủy hành chính không giả tạo snapshot FINISHED; server dừng runtime và gửi GAME_OVER theo chính sách hủy nếu chưa có kết quả chốt. Client bỏ context/controller cũ, hiển thị kết quả và giữ TCP để chơi lượt sau nếu kết nối còn hợp lệ. GAME_OVER cũ/trùng không mở lại world đã đóng; match mới luôn có identity mới.

Phân công: TV4 ghép sơ đồ; TV1 xác nhận kết nối/phòng, TV2 movement/resync, TV3 gameplay/kết quả.

9.6. Sơ đồ tuần tự các luồng chính

Các mũi tên server→client dưới đây là frame ứng dụng được enqueue/ghi theo writer tương ứng; không phải mỗi mũi tên tương đương đúng một TCP segment. Các sơ đồ bỏ bớt heartbeat và cập nhật LIST_ROOMS lặp để tập trung luồng chính; không bỏ các kiểm tra của handler trong hiện thực.

9.6.1. Kết nối, vào phòng và bắt đầu

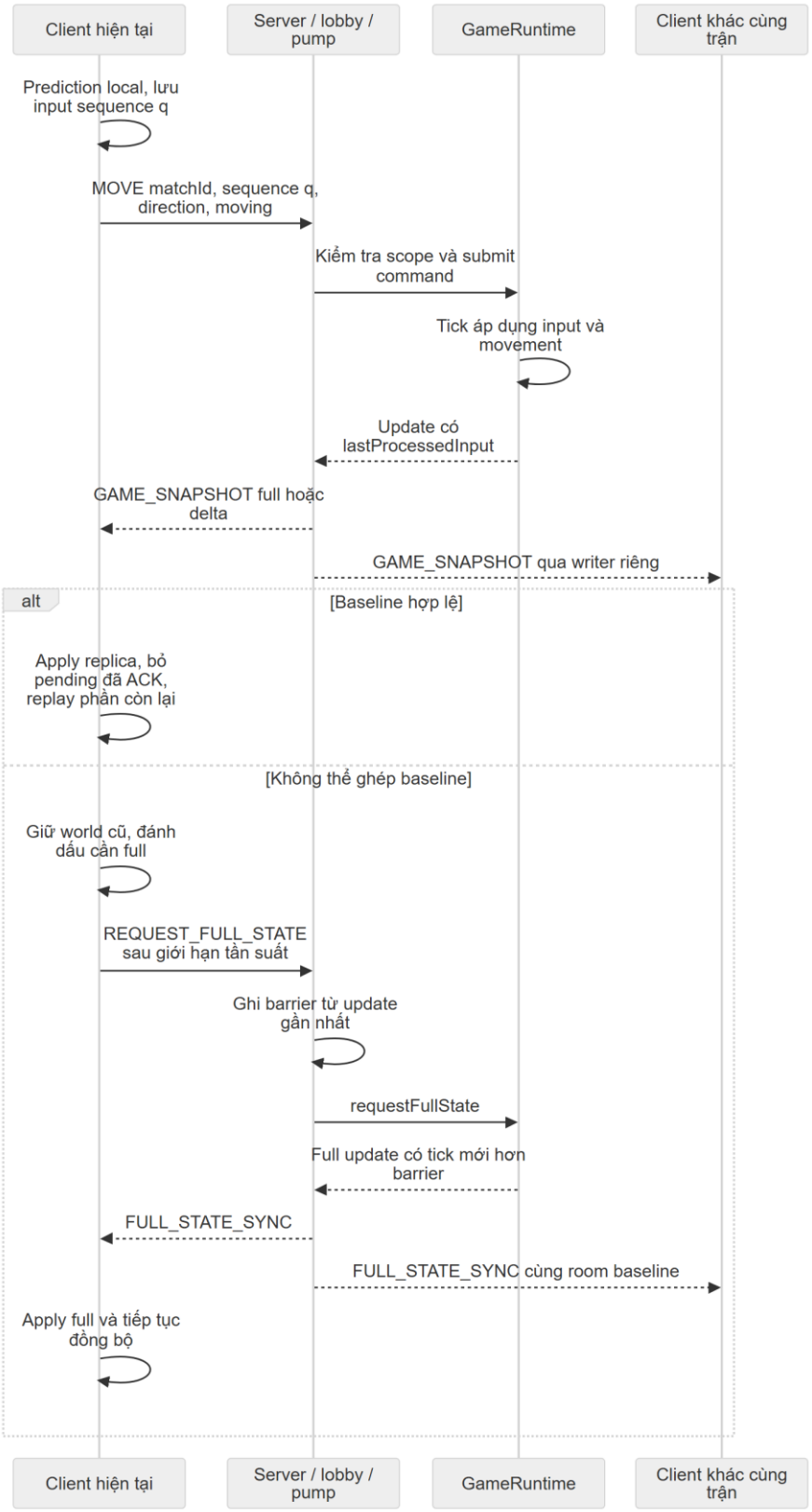


Hình 9.1. Kết nối và bắt đầu một trận hai người. Nhóm tự vẽ từ code project.

Runtime có thể đã tạo update trước khi worker bắt đầu đọc queue; bảo đảm cần thể hiện là GAME_STARTED vào writer trước full gửi cho client, không phải game thread chỉ bắt đầu mô phỏng sau khi metadata đã tới client. Server dùng dấu initialized sau enqueue full; client chỉ bật

input sau khi thực sự apply full. Không có message ACK-full riêng. Bản gốc: 09_01_ket_noi_va_start.mmd.

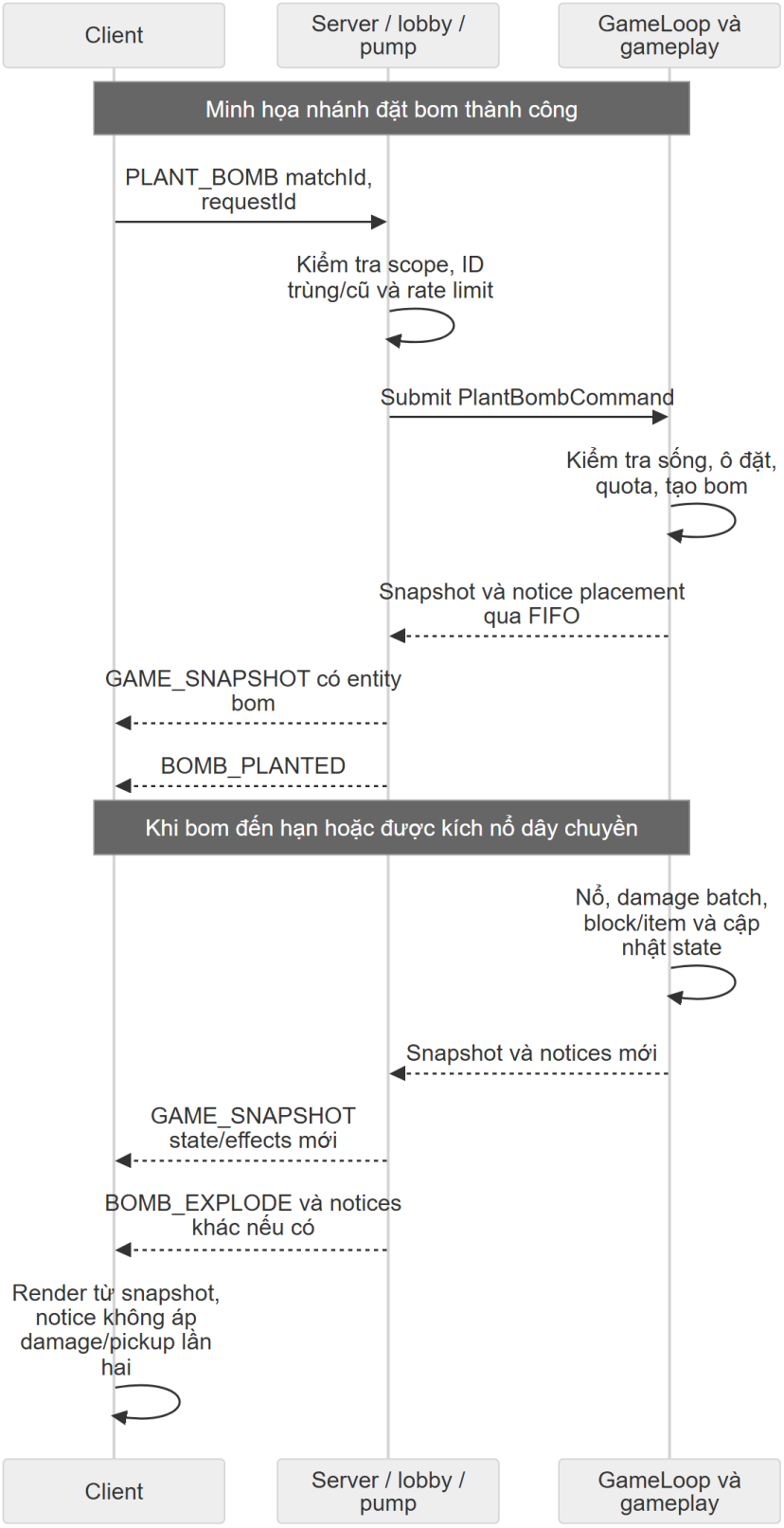
9.6.2. Di chuyển, ACK và resync



Hình 9.2. Input sequence và nhánh phục hồi baseline. Nhóm tự vẽ từ code project.

Nhánh sai baseline là lỗi/thiếu chuỗi ở lớp ứng dụng, không giả định TCP giao segment sai thứ tự như UDP. Bản gốc: 09_02_move_va_resync.mmd.

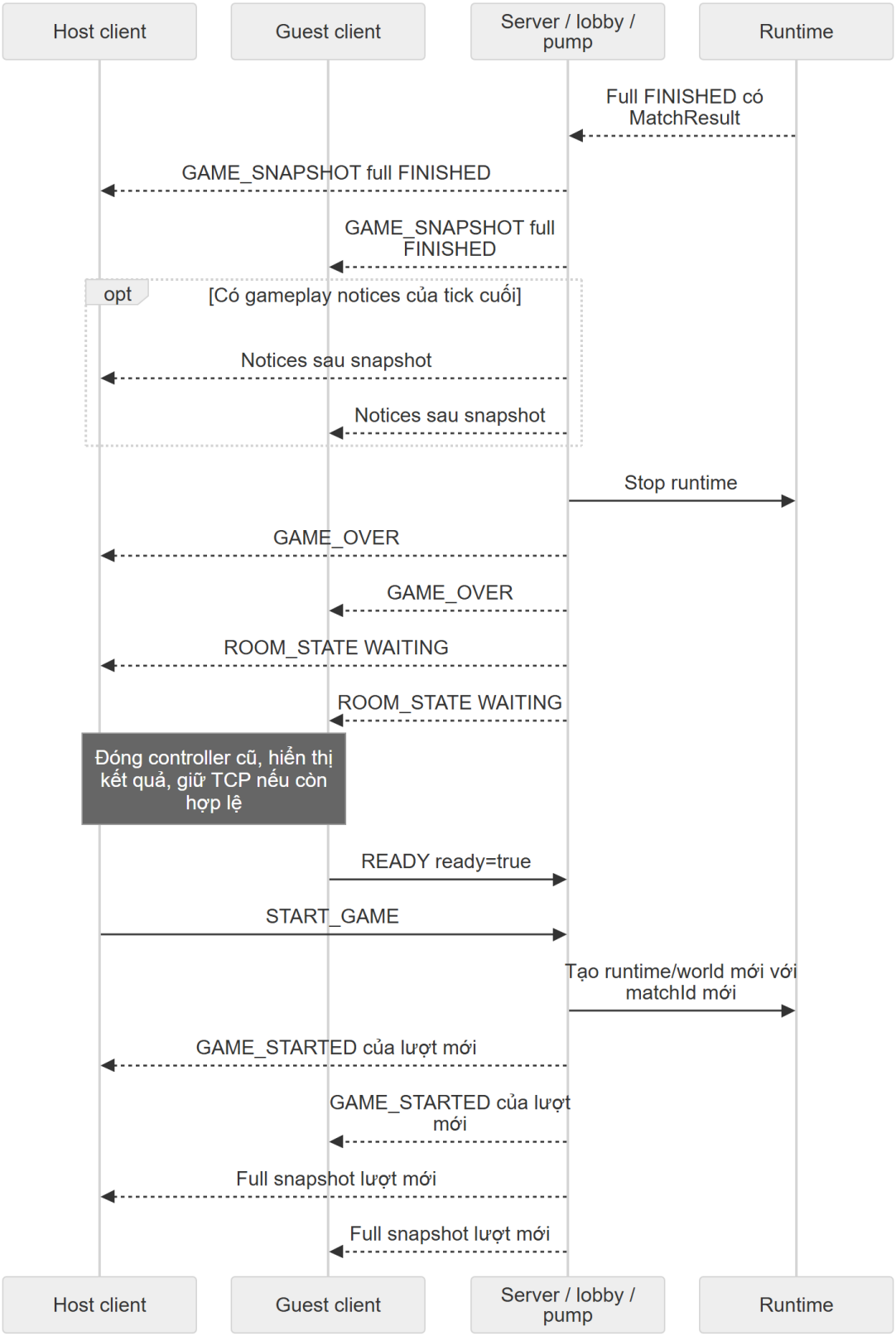
9.6.3. Đặt bom và thông báo diễn biến



Hình 9.3. Nhánh đặt bom thành công và vụ nổ sau đó. Nhóm tự vẽ từ code project.

Thông báo được phát cho cả roster, không chỉ người đặt. Request trùng không tạo thêm command; request hợp lệ ở transport nhưng không đạt luật đặt bom không đi qua nhánh tạo entity trong hình. Bản gốc: 09_03_bom_va_event.mmd.

9.6.4. Kết thúc và chơi lại



Hình 9.4. Kết thúc gameplay bình thường và mở lượt mới. Nhóm tự vẽ từ code project.

Nhánh host End/leave/disconnect bỏ phần full FINISHED nếu chưa có kết quả gameplay; nó dừng runtime và gửi ABORTED. Socket đã mất không được bảo đảm nhận thông báo cuối; các thành viên còn kết nối được cập nhật theo khả năng gửi. Bản gốc: 09_04_ket_thuc_va_choi_lai.mmd.

Phân công: TV1/TV4 phụ trách transport, timeout và validation; TV2 queue/baseline, TV3 kiểm tra tác động gameplay.

9.7. Packet lỗi, timeout, client chậm, queue đầy và stale match

9.7.1. Giới hạn vận hành của giao thức

Bảng 9.9. Các giới hạn mặc định liên quan đến giao tiếp.

Lớp/loại	Giới hạn hiện tại	Khi vượt/không đáp ứng
Tổng frame đã decode	100 frame/giây/kết nối, kể cả unknown/rejected	RATE_LIMITED và đóng sau drain có thời hạn
CHAT và LIST_ROOMS	Mỗi loại 5 request/giây/kết nối	RATE_LIMITED; chưa đóng chỉ vì vượt loại nếu chưa vượt tổng
MOVE	40 request/giây trong cửa sổ một giây	RATE_LIMITED; chưa enqueue request vượt
PLANT_BOMB	5 request mới/giây/session/match	Bỏ ID cũ/trùng trước giới hạn riêng; request mới vượt trả RATE_LIMITED
Request full	Khoảng cách tối thiểu 1000 ms ở server	RATE_LIMITED; client owner hiện dùng 1100 ms
Đọc frame server	Deadline toàn frame mặc định 10 giây	Đóng captured connection; không gia hạn vô hạn theo từng byte
Ghi socket	Watchdog mặc định 5 giây ở server/client	Đóng socket để thoát I/O chặn, báo/dọn theo owner
Writer server	Queue 256 frame chờ; byte budget 4 MiB tính cả frame đang ghi	Đóng kết nối khi đầy; không bỏ một delta rồi tiếp tục
Runtime	Command 4096, outbound 1024	Submit thất bại hoặc giữ baseline chưa commit như mục 9.4
Pending phía client	Pull queue tùy chọn 1024 packet; UI dispatch count hữu hạn, budget 4 MiB	Overflow gây lỗi/dừng; owner mặc định tắt pull queue không dùng

Các rate limit là cửa sổ/điều kiện trong handler, không phải bảo đảm throughput thực đạt. MOVE/PLANT/resync còn chịu giới hạn tổng kết nối. Control và input sender của client có giới hạn pending riêng; UI owner hiện cho tối đa 256 dispatch đang chờ, không có nghĩa toàn bộ EDT được biến thành queue hữu hạn. Watchdog/scheduler có chu kỳ kiểm tra nên các thời gian trên là cấu hình phát hiện, không phải SLA ngắt chính xác. Nguồn: InboundRateLimiter, ClientHandler, SessionWriter và ClientNetwork.

9.7.2. Phân loại lỗi và hành động

Bảng 9.10. Cách xử lý theo lớp lỗi thực tế.

Tình huống	Phản hồi/hành động server hoặc client	Lưu ý
Length/UTF-8 không hợp lệ	Server đóng kết nối có kiểm soát	Không cố tìm biên frame tiếp theo, không bảo đảm gửi ERROR

Tình huống	Phản hồi/hành động server hoặc client	Lưu ý
EOF, frame thiếu byte hoặc timeout I/O	Dọn captured connection; phiên đã handshake đi qua chính sách disconnect	Nếu đang chơi, lobby có thể hủy lượt
Request trước handshake hoặc version không hỗ trợ	NOT_CONNECTED/UNSUPPORTED_VERSION rồi đóng theo transport	CONNECT lặp sau handshake là INVALID_PACKET
JSON/field sai sau handshake	INVALID_PACKET; thông thường tiếp tục phiên	Codec, handler và trạng thái quyết định đường lỗi cụ thể
Sai phòng/quyền/trạng thái	NOT_IN_ROOM, NOT_HOST, NOT_READY, NOT_PLAYING hoặc lỗi lobby tương ứng	Không được sửa world để chấp nhận yêu cầu
Sai/mất match envelope	Handler gameplay trả STALE_MATCH; client bỏ packet sai match	UUID đúng chưa thay cho kiểm tra session/socket/membership
Command queue không nhận	INPUT_QUEUE_FULL	Chưa bảo đảm command đã được xử lý; không có replay tự động mọi request
Writer đầy/ghi quá hạn	Đóng client đó và cleanup	Theo chính sách lobby, một disconnect có thể làm hủy cả lượt của phòng
Replica không ghép được baseline	Giữ world cũ, xin full, tạm chặn input mới theo input-ready	Payload decode hỏng có thể gây fail controller thay vì đi qua resync
RuntimeException ngoài validation trong handler	SERVER_ERROR, ghi diagnostics rồi đóng kết nối	Lỗi runtime/pump được xử lý qua failure và endMatch; không hứa luôn phát MATCH_FAILED

Các lỗi lobby còn gồm ROOM_NOT_FOUND, ROOM_FULL, ROOM_PLAYING, ALREADY_IN_ROOM, INVALID_ROOM_STATE, ROOM_LIMIT, MODE_NOT_SUPPORTED, CHAT_QUEUE_FULL và MATCH_START_FAILED theo handler tương ứng. Type không có handler sau handshake trả UNKNOWN_MESSAGE; một type có tên trong PacketType không tự cho phép gửi ở mọi hướng.

Ví dụ ERROR do transport/handler hiện tạo:

```
{"errorCode":"INPUT_QUEUE_FULL","message":"INPUT_QUEUE_FULL","requestId":null}
```

Schema cho phép trường requestId, nhưng helper hiện đặt null; không mô tả ERROR như cơ chế correlation hoàn chỉnh cho từng MOVE/PLANT_BOMB. Nếu writer/socket đã hỏng thì phản hồi cuối có thể không gửi được. Ở lobby client, ERROR nghiệp vụ được chuyển lên UI để người dùng xử lý, không mặc định ngắt phiên; codec/listener hoặc I/O failure có thể đóng owner để tránh tiếp tục với state không an toàn.

9.7.3. Thứ tự, dữ liệu cũ và giới hạn kết luận

Mỗi kết nối server có writer riêng; snapshot, notice và kết quả được enqueue theo thứ tự của luồng công bố. Client dùng listener ứng dụng thường trực và UI executor tuần tự để xử lý metadata/state/kết quả. Match identity ngăn dữ liệu lượt trước ảnh hưởng lượt mới; tick và baseline kiểm tra thứ tự state trong cùng match. Không thay các tầng kiểm tra này bằng giả định "TCP tin cậy nên ứng dụng luôn đồng bộ".

Protocol hiện không có receipt ACK riêng cho snapshot, không có ACK nghiệp vụ riêng cho mọi lệnh và chưa có reconnect resume. Mức đúng/ổn định dưới phân mảnh, quá tải, lỗi, nhiều

client hoặc nhiều process cần minh chứng test/đo tương ứng. Thiết kế trên không được dùng để kết luận đã đạt GUI/LAN/soak hoặc để bỏ qua process smoke failure đã ghi ở hồ sơ khảo sát.

10. Triển khai hệ thống

Chương này mô tả cách các phương pháp và contract ở mục 7–9 được hiện thực bằng class, API, cấu hình và công cụ trong repository BombItMultiplayer-Structure. Trọng tâm là đường chạy lobby/gameplay mặc định. Tên module hoặc API legacy được phân biệt với luồng đang tích hợp, tránh lấy sự tồn tại của một phương thức làm bằng chứng một tính năng end-to-end đã hoạt động. Trách nhiệm TV1–TV4 dưới đây là phân công module/phần viết, không dùng thay xác nhận tác giả lịch sử hay tỷ lệ đóng góp.

Phân công: TV4 tổng hợp công nghệ/cấu trúc/bootstrap; cả nhóm xác nhận cấu hình và cách chạy module.

10.1. Java 25, Swing, Maven, cấu trúc project và bootstrap

10.1.1. Công nghệ và cấu hình build

Bảng 10.1. Stack và cấu hình được khai báo trong project.

Thành phần	Cấu hình hiện tại	Vai trò
Java	`maven.compiler.release=25`, source encoding UTF-8	Ngôn ngữ/runtime và API chuẩn
Maven artifact	`com.bombit:bomb-it-multiplayer:0.1.0-SNAPSHOT`	Định danh build trong POM
Compiler plugin	`maven-compiler-plugin:3.13.0`	Biên dịch theo release đã chọn
Kiểm thử Maven	`maven-surefire-plugin:3.5.2`, `useModulePath=false`	Chạy test theo cấu hình classpath
Thư viện test	`org.junit.jupiter:junit-jupiter:5.11.4`, scope test	Unit/integration tests dùng JUnit
Giao diện	Swing/AWT và Java2D trong code client	Cửa sổ, input, renderer và bảng kết quả
Network/concurrency	`java.net`, `java.io`, `java.util.concurrent`	TCP socket, framing, pool, queue và scheduler
JSON và persistence	Codec/parser nội bộ; chưa có dependency database/JSON bên ngoài trong POM	Contract hiện tại không dùng Gson/Jackson hoặc JDBC trong đường triển khai đã khảo sát

Nguồn cấu hình: pom.xml. POM hiện chỉ khai báo dependency JUnit ở scope test; production chủ yếu dùng JDK và mã trong repository. Không mô tả Java 17 là baseline build của phiên bản này từ tài liệu cũ. Môi trường JDK/Maven thực dùng khi kiểm thử được ghi riêng ở Experimental Setup; giá trị trong POM không thay cho biên bản lần chạy.

10.1.2. Cấu trúc mã nguồn và tài liệu

```
pom.xml
src/main/java/com/bombit/
    common/dto/, common/protocol/       dữ liệu và codec tích hợp
    server/app/                         bootstrap và điều phối lobby/gameplay
    server/network/, server/dispatch/   transport, writer, handler và router
    server/session/, server/room/       phiên, thành viên, host và ready
    server/chat/, server/heartbeat/     worker chat và phát hiện timeout
    server/integration/                 callback vòng đời giữa các tầng
    server/game/                        model, core, movement, runtime, bom/item/event/map
```

```
    client/app/, client/network/      connection owner, entry point và TCP client
    client/lobby/, client/game/       UI phòng/kết quả và gameplay/renderer
    metrics/, tools/                  monitor và benchmark runner
src/test/java/                        JUnit, Check/Demo/main và công cụ kiểm tra
scripts/                              build, run, test, package và benchmark
docs/                                 contract, tài liệu kỹ thuật và dữ liệu đã lưu
phancong/                             yêu cầu giảng viên và phân công
report/                               hồ sơ/bản thảo báo cáo theo từng nhiệm vụ
target/                               output sinh từ build/test/đóng gói
```

Cây trên nhóm theo trách nhiệm, không liệt kê mọi package/file. `src/test/java` còn chứa các chương trình Check/Demo/main; chúng không tự động trở thành JUnit test chỉ vì nằm trong thư mục test. `target/` là output có thể bị tạo lại; log/CSV cần dẫn làm bằng chứng báo cáo được lưu riêng kèm metadata trước khi clean build. State trận chủ yếu nằm trong RAM, chưa có tầng database tài khoản/lịch sử/khôi phục world.

10.1.3. Entry point và composition

ServerApplication mặc định tạo `LobbyGameServer` với cổng mặc định 8888. Ứng dụng cài shutdown hook gọi close, chạy accept loop và dùng try-with-resources cho owner server. Nhánh nhận số người ở đối số thứ hai chạy `DemoGameServer`; đây là profile khác, không thay thế luồng phòng mặc định.

ClientApplication đọc host/port/name, mặc định mở `LobbyWindow` trên EDT. Nhánh `--demo` và `--headless` phục vụ đường demo/smoke riêng. Kết nối từ cửa sổ lobby được đưa sang worker để không thực hiện bước connect chặn ngay trên EDT. Lớp `DemoClientConnection` vẫn được dùng làm owner kết nối trong lobby dù tên còn mang dấu mốc phát triển.

`LobbyGameServer` là nơi nối các phần: tạo `Server`, lấy RoomManager, đăng ký các handler lobby/gameplay, inject map/extension factory vào runtime manager và cài observer cho disconnect/hard drop. Constructor mặc định tạo DefaultArenaMap và MatchGameplayExtension cho từng runtime. Chi tiết state/gameplay được viết ở các phần TV2/TV3, còn chương kiến trúc đã trình bày quan hệ thành phần.

10.1.4. Build, chạy, test và đóng gói

Bảng 10.2. Các script hiện có và đầu ra/chức năng.

Script	Hành vi chính	Điểm cần ghi đúng trong báo cáo
`build-demo.cmd` → `build.ps1`	`javac --release 25`, tạo `target/demo-classes`, JAR và BUILD_INFO	Tên script còn "demo" nhưng biên dịch source production; không đồng nhất với chỉ build profile M1
`run-server.cmd`, `run-client.cmd`	Gọi entry point qua classpath đã chọn và chuyển tiếp đối số	Dùng `java-env.cmd`; chế độ thực phụ thuộc đối số entry point
`classpath.cmd`	Chọn classes/JAR đang có	Ưu tiên JAR ở root → JAR trong target → target/demo-classes; cần ghi đúng artifact đã chạy
`test.cmd`	Maven test → Check TV2 → NetworkTest main, dừng khi một bước lỗi	Nếu Maven fail thì các bước sau không tự được chạy

Script	Hành vi chính	Điểm cần ghi đúng trong báo cáo
`package.cmd` → `package.ps1`	Đóng JAR, launcher, docs, BUILD_INFO vào ZIP và tạo SHA256	Có script không có nghĩa đã tái đóng gói/nghiệm thu bản nộp
`run-benchmark.cmd`	Gọi `com.bombit.tools.NetworkBenchmark` với các tùy chọn	Là công cụ đo; số liệu thuộc lần chạy và cấu hình cụ thể

Ví dụ lệnh ở root repository, chỉ minh họa cách vận hành, không phải log của nhiệm vụ viết này:

```
scripts\build-demo.cmd
scripts\run-server.cmd 8888
scripts\run-client.cmd 127.0.0.1 8888 PlayerA
```

Server và mỗi client cần terminal/tiến trình riêng khi chơi nhiều người theo cách này. `java-env.cmd` chọn công cụ theo JAVA_HOME hoặc PATH trong phạm vi script; Maven test script cho phép đặt `BOMBIT_MVN` để chỉ binary Maven có sẵn. Build script tìm javac/jar, tạo source list UTF-8, manifest main là ClientApplication, và ghi thời gian UTC, source revision/dirty, target Java, SHA256 JAR vào BUILD_INFO. Source revision trong metadata không tự bảo đảm mọi file chưa theo dõi đã được commit.

Nguồn: build.ps1, classpath.cmd, test.cmd và package.ps1. Các script được mô tả theo source; kết quả kiểm thử/thực nghiệm nằm ở mục 12.

Phân công: TV1 phụ trách TCP/session/room/chat/heartbeat; TV4 phối hợp codec/router và tích hợp đường chạy thật.

10.2. TCP server/client lifecycle, room/lobby/chat/heartbeat

10.2.1. Các API hạ tầng TV1

Bảng 10.3. Class/API chính và điểm nối vào ứng dụng.

Class	API tiêu biểu	Chức năng triển khai
`Server`	`start`, `close`, `getRouter`, `setGameObserver`	Sở hữu accept loop, reader pool và các manager/service hạ tầng
`ClientHandler`	`run`, `readFrame`	Decode/giới hạn request, gate handshake, route, xử lý lỗi và cleanup socket đã capture
`ConnectionManager`	`open`, `find`, `release`, `setFailureHandler`	Quản lý writer theo session, watchdog và cleanup executor
`SessionWriter`	`send`, `finishAndAwait`, `close`, `expireWrite`	Encode/enqueue frame, ghi socket trên worker và dừng có thời hạn
`SessionManager`	`register`, `find`, `handleUnexpectedDisconnect`, `hardDrop`, `broadcastToRoom`	Map phiên, trạng thái disconnect/expiry và callback vòng đời
`RoomManager`	`createRoom`, `joinRoom`, `setReady`, `leaveRoom`, `publishRoomState`	Map phòng, membership và dữ liệu công bố
`ChatHandler`, `HeartbeatManager`	`enqueue/close`, `sweep/close`	Chat worker và sweep liveness theo scheduler

Đây là API đang có; cách ứng dụng gọi API quyết định tính năng thực tế. Ví dụ, RoomManager có `startGame` legacy nhưng lobby mặc định không dùng hàm đó để tạo runtime/MatchContext.

10.2.2. Accept, reader pool và cổng kiểm tra

Server mở ServerSocket trong constructor. `start()` chỉ được gọi một lần, nhận socket, đặt TCP_NODELAY, tạo session tạm và writer, rồi submit ClientHandler vào ThreadPoolExecutor. Pool có 32 reader worker và queue 32 task; số session đang giữ bị chặn tại 256, gồm cả metadata chờ expiry. Reader pool từ chối task thì session tương ứng bị hard drop. Các giới hạn này khác số người mỗi phòng và không được dùng làm kết quả benchmark số client phục vụ ổn định.

ClientHandler giữ socket/session/writer được truyền khi tạo. Trong `run`, handler đọc frame, áp rate limit tổng/riêng, chặn message khác CONNECT khi session còn ở trạng thái CONNECTED, rồi gọi MessageRouter. Sau handshake, CONNECT lặp bị từ chối; request nghiệp vụ sai không tự sửa world.

`readFrame()` tạo deadline tuyệt đối cho một frame, tính thời gian còn lại trước các lần đọc và đặt SO_TIMEOUT tương ứng. Deadline mặc định 10 giây không được gia hạn từ đầu sau mỗi byte. Ở `finally`, handler chỉ dọn theo session/socket còn khớp, release đúng captured writer và đóng socket. Đây là kiểm tra ownership của kết nối, chưa chứng minh đã có generation/reconnect resume hoàn chỉnh.

10.2.3. Writer, broadcast và cleanup kết nối

SessionWriter capture socket lúc khởi tạo. `send` chuyển OutboundMessage thành Packet, encode ra byte, kiểm tra lifecycle/byte budget rồi offer vào queue. Encode xảy ra trên producer; worker writer lấy frame và thực hiện write/flush. Queue mặc định chứa tối đa 256 frame đang chờ; budget 4 MiB còn tính frame đang ghi.

Writer lỗi hoặc đầy gọi đường fail: ngừng nhận, đóng socket để đánh thức I/O, interrupt worker và gọi callback một lần. ConnectionManager chuyển xử lý failure sang cleanup executor rồi release writer đúng socket. Watchdog kiểm tra ghi kéo dài theo write timeout mặc định 5 giây. Việc tách callback tránh chạy trực tiếp logic lobby/session ngay lúc producer có thể đang giữ khóa nghiệp vụ.

OutboundMessage clone mảng byte khi tạo và khi trả payload để caller không sửa ngầm nội dung đã bàn giao. BroadcastSender tìm outbound sender gắn với session và chuyển tiếp, không tự ghi trực tiếp cùng socket từ nhiều thread. Khi writer đã đóng/đầy, wrapper có thể bắt IllegalStateException để producer tiếp tục xử lý các session khác; việc đóng và cleanup vẫn thuộc writer/connection owner, không phải bảo đảm gửi đủ cho mọi thành viên. Nguồn: OutboundMessage và BroadcastSender.

10.2.4. Session và trạng thái ứng dụng

Session tạo UUID playerId phiên và token, giữ username, socket, roomId, outbound sender và heartbeat bằng nanoTime. State dùng AtomicReference; `completeHandshake` chuyển phiên tạm sang IN_LOBBY sau khi CONNECT đã được kiểm tra. Tên `playerId` ở lớp Session là UUID phiên, khác int playerId trong gameplay.

Bảng 10.4. Ý nghĩa các SessionState trong luồng hiện tại.

State	Ý nghĩa	Điểm chuyển tiêu biểu
CONNECTED	TCP đã đăng ký, chưa hoàn tất handshake	Tạo Session khi accept

State	Ý nghĩa	Điểm chuyển tiêu biểu
IN_LOBBY	Handshake hợp lệ, chưa ở phòng	`completeHandshake` hoặc rời phòng còn kết nối
IN_ROOM	Là thành viên phòng, chưa ready/đã hết lượt	Create/join hoặc endMatch
READY	Khách/người chơi đặt trạng thái sẵn sàng	`RoomManager.setReady`
PLAYING	Thuộc roster lượt đang chạy	`LobbyGameServer.startMatch`
DISCONNECTED	Socket mất, metadata có thể chờ expiry	`handleUnexpectedDisconnect`
CLOSED	Phiên bị loại khỏi manager/đang dọn	`hardDrop`, đóng toàn manager

SessionManager dùng map concurrent để quản lý phiên, map timer disconnect và ScheduledThreadPoolExecutor cho expiry. `handleUnexpectedDisconnect` đánh dấu DISCONNECTED, đóng socket, gọi observer rồi lên lịch hard drop theo grace. Timer có thể bị hủy khi cleanup/legacy API thay đổi trạng thái; callback lỗi được ghi vào failure để không chặn toàn bộ đường dọn.

Ở cấu hình lobby, observer được gọi trước phần RoomCleanupSink trong chuỗi callback của Server. Lobby kết thúc lượt/bỏ ghế ngay khi disconnect, nên session metadata còn 15 giây không có nghĩa phòng/world vẫn được giữ. `SessionManager.reconnect` là API legacy có token/attachSocket/restoreState; router sản phẩm chưa đăng ký RECONNECT và chương này không nhận API đó là tính năng resume đã hoàn thành.

10.2.5. Room, lobby và start/end thật

Room giữ roster, host, sức chứa và WAITING/PLAYING; các thao tác thành viên có synchronization. RoomManager quản lý map phòng concurrent, kiểm tra instance/membership khi join, đặt trạng thái ready và công bố RoomView qua broadcaster được inject. Snapshot thành viên dùng bản sao; host rời thì người còn lại đầu danh sách nhận quyền host, phòng trống được xóa.

API `setReady(session, boolean)` đặt trạng thái đích, phù hợp request READY idempotent; `toggleReady` còn tồn tại không phải handler READY được dùng trên dây. Tương tự, `RoomManager.startGame` legacy chuyển trạng thái/phát metadata cũ, còn LobbyGameServer thực hiện Start tích hợp: kiểm tra host/ready, chụp roster, tạo spawn/mapping, khởi động runtime, tạo MatchContext và gửi GAME_STARTED trước snapshot worker.

Các handler lobby được đăng ký theo bảng type; phần `handle` được synchronized trên owner để phối hợp vòng đời room/match. Quản lý phòng còn áp sức chứa 2–4, tối đa 32 phòng và từ chối join giữa trận. Shared lock này giúp tuần tự hóa một số bước nhưng vẫn là tài nguyên dùng chung, không chứng minh không có contention dưới tải.

Khi nhận kết quả bình thường hoặc End/leave/disconnect, `endMatch` dừng runtime, reset session/room, gửi GAME_OVER và công bố ROOM_STATE; lượt mới có MatchContext/world mới. Các bước tạo snapshot/gameplay được mô tả ở phần TV2/TV3, không đặt vào RoomManager.

10.2.6. Chat theo phòng

ChatHandler dùng LinkedBlockingQueue hữu hạn và một chat-worker. Handler network kiểm tra room và text, lấy senderId/tên từ session rồi enqueue ChatMessage; đường server hiện cấp queue capacity 256.

Worker không broadcast mù dữ liệu đã đợi trong queue: nó tìm lại room và kiểm tra sender còn là thành viên, không DISCONNECTED/CLOSED. Sau đó tạo CHAT JSON qua LobbyPacketCodec và gửi cho các session nhận hợp lệ. Kiểm tra lại ở thời điểm consume giúp loại message của người đã rời phòng trong thời gian chờ. Chat không phải global chat và không dùng frame plaintext legacy khác với CHAT hiện tại.

`close()` ngừng nhận, clear queue, interrupt và join worker có thời hạn. Các API `pendingMessages`, `isAlive`, `failure` hỗ trợ kiểm tra backlog/lifecycle; chúng chưa thay cho kết quả test hay khả năng lưu lịch sử chat bền vững.

10.2.7. Heartbeat và shutdown

HeartbeatManager dùng scheduler sweep mặc định 2 giây, so elapsed nanoTime với timeout 10 giây trên session. Phiên chưa handshake hết hạn bị hard drop; phiên đã handshake đi qua unexpected disconnect và callback lifecycle. Timeout được phát hiện theo lịch sweep, không bảo đảm dọn chính xác tại giây thứ 10.

Route PING hiện được đăng ký trong SessionHandlers: parse pingId, update heartbeat rồi echo PONG. `HeartbeatManager.onPing` vẫn có API nhưng không phải phương thức mà route PING sản phẩm đang gọi. Client owner có control scheduler riêng gửi PING và kiểm tra khoảng thời gian không nhận packet. Liveness của session và phép đo RTT bằng LatencyMonitor là hai trách nhiệm khác nhau.

`Server.close` ngừng accept bằng đóng ServerSocket, dừng heartbeat, đóng session/socket/writer, rồi shutdown và await reader pool; chỉ dùng shutdownNow khi chưa dừng trong thời hạn hoặc bị interrupt. Chat worker cũng được đóng. Owner `LobbyGameServer.close` còn dừng các runtime/snapshot worker trước/qua chuỗi đóng hạ tầng. Các phương thức close có guard để hạn chế thực hiện lại lifecycle, nhưng báo cáo không suy ra mọi race đã được loại bỏ.

Phân công: TV2 — model, state, command và vòng đời runtime.

10.3. Model/map/state, command queue, GameLoop và runtime

10.3.1. Model dùng chung và quy ước tọa độ

Package `server/game/model` cung cấp ba kiểu dùng chung cho core, movement và các module gameplay. Direction là enum bốn hướng với `deltaX`, `deltaY`: trục x tăng sang phải, trục y tăng xuống dưới. TileType phân biệt `EMPTY`, `HARD_WALL`, `BREAKABLE_BLOCK`, `OUT_OF_BOUNDS`; chỉ `EMPTY` cho phép đi qua. TilePosition là record bất biến. Phép đổi pixel sang ô dùng `floor`, kiểm tra số hữu hạn và tile size dương; phép dịch ô kiểm tra tràn số nguyên. Record vẫn cho phép tọa độ âm để biểu diễn các bước duyệt ra ngoài bản đồ, còn map quyết định ô đó có hợp lệ hay không.

Vị trí `Player.x/y` là góc trên trái hitbox trong đơn vị pixel. Khi cần xác định ô người chơi đang đứng, `Player.tilePosition` dùng tâm hitbox. Hai cách biểu diễn phục vụ hai mục đích: collision xét toàn bộ hình chữ nhật, còn thao tác theo lưới dùng một ô đại diện. Enum lồng

`Player.Direction` và `GameMap.TileType` vẫn còn dưới dạng API `@Deprecated` để tương thích; triển khai mới dùng model chung, không khẳng định đã xóa toàn bộ bridge cũ.

10.3.2. Map, Player và quyền sở hữu GameState

GameMap lưu mảng `tiles[y][x]` và tile size. `tileAt` trả sentinel `OUT_OF_BOUNDS` khi truy vấn ngoài map; sentinel không được lưu như một ô thật. `destroyBlock` chỉ đổi breakable block thành ô trống. API legacy `getTile` có hành vi kiểm tra biên và ném lỗi, vì vậy caller không nên coi hai API là hoàn toàn tương đương.

GameState sở hữu map, tập Player theo ID, entity theo khóa, trạng thái trận, frame gameplay và các notice chờ xuất bản. Player chỉ được thêm khi `WAITING`; `startMatch` chuyển sang `RUNNING` nếu có người chơi. Core chấp nhận một người để phục vụ API/test, trong khi điều kiện bắt đầu trận lobby là một lớp kiểm tra khác. `finishMatch` chuyển sang `FINISHED` và dừng ý định di chuyển, nhưng không tự kết thúc thread game loop. Luồng tích hợp lobby chịu trách nhiệm đóng runtime sau khi xử lý kết quả.

Player giữ vị trí, hitbox, hướng, tốc độ, trạng thái sống, movement intent, sequence input đã xử lý và dữ liệu gameplay như lives/respawns. Các trường này có thể thay đổi trong mô phỏng. `getPlayers()` trả danh sách sao chép không cho sửa cấu trúc, nhưng phần tử Player vẫn mutable; đây không phải snapshot để network thread sử dụng. Quy tắc sở hữu là một loop cập nhật state của một trận, còn các thread khác gửi command hoặc nhận update đã chụp.

Bản đồ dùng trong lobby do DefaultArenaMap tạo: 17 × 15 ô, mỗi ô 32 pixel, bốn vị trí spawn theo góc sân. Tham số ban đầu trong PlayerMovementRules là hitbox 24 pixel và tốc độ 80 pixel/giây theo cấu hình mặc định; hitbox được đặt giữa ô spawn. Không nên dùng map tối giản của constructor runtime mặc định để mô tả sân thực tế của lobby.

10.3.3. Command có kiểu và queue có giới hạn

GameCommand hiện là **sealed class**, không phải sealed interface như phương án ban đầu. Base class giữ constructor legacy đã deprecated; các subclass final biểu diễn `MoveCommand`, `StopMovingCommand`, `PlantBombCommand`, `PlayerDisconnectedCommand`, `PlayerReconnectedCommand`. Các field command là dữ liệu bất biến. Sequence dương dùng cho input mạng; sequence 0 còn phục vụ tương thích nội bộ. TV2 cung cấp contract đặt bom, còn xử lý bom thực tế do extension gameplay đảm nhiệm.

Queue đầu vào của GameLoop là `LinkedBlockingQueue` có capacity, runtime cấu hình mặc định 4096 command. `submit` kiểm tra loop đang chạy và gọi `offer`, không chờ vô hạn đến khi còn chỗ. Khi đầy queue, counter rejected tăng và kết quả là `false`; việc submit lúc loop đã dừng cũng trả `false` nhưng không được tính vào counter queue đầy. FIFO được xác định theo thứ tự các lần enqueue thành công; hai producer đồng thời không có thứ tự nghiệp vụ cố định trước khi enqueue.

Mỗi tick đọc tối đa 256 command để tránh dành toàn bộ thời gian cho input. Với movement có sequence dương, loop chỉ áp dụng một mẫu thành công cho mỗi player trong tick. Nếu command đầu queue tiếp theo là movement của player đã được lấy mẫu, loop dừng đọc tick đó thay vì bỏ qua nó. Cách này giữ FIFO nhưng có thể khiến command của player khác phía sau phải đợi. Command disconnect đóng băng movement và có thể xóa player theo cờ; reconnect

mở đóng băng và yêu cầu full state. Đây là API core, không phải bằng chứng luồng reconnect lobby đã khôi phục được trận đang chơi.

10.3.4. Tạo và quản lý runtime theo room

PlayerSpawn là dữ liệu đầu vào bất biến: session ID, gameplay player ID, vị trí, kích thước và tốc độ đã kiểm tra hợp lệ; không chứa socket. GameRuntimeManager giữ bảng room ID → runtime và đồng bộ các thao tác quản lý vòng đời.

Bảng 10.5. API runtime và ranh giới dữ liệu TV2.

API	Xử lý chính	Dữ liệu/giới hạn
`startGame(roomId, players)`	Tạo map/state/player/extension, start match và loop	1–4 spawn ở API core; kiểm tra ID trùng, spawn và room đang chạy
`findGame(roomId)`	Tìm runtime bằng room	Trả `Optional<GameRuntime>`
`submit(roomId, command)`	Chuyển command tới runtime tương ứng	Trả boolean, không trả tham chiếu GameState
`stopGame(roomId)`	Đóng runtime trước khi bỏ khỏi bảng	Đợi thread dừng trong thời hạn của loop
`GameRuntime.pollUpdate(timeout, unit)`	Lấy update từ queue đầu ra	Consumer có thể chờ có giới hạn
`GameRuntime.createUpdate()`	Đọc update gần nhất đã xuất bản	Không chụp state trực tiếp từ network thread
`requestFullState()`	Đặt cờ yêu cầu full ở tick kế tiếp	Không encode/gửi socket ngay

Manager gọi map factory cho mỗi trận và phát hiện việc tái dùng cùng một instance map; player/state/extension cũng được tạo cho từng runtime. Spawn được kiểm tra bằng collision trước khi chạy. Mỗi GameRuntime có `matchId` UUID mới, bản sao bất biến mapping session → player và một GameLoop riêng. Constructor manager mặc định dùng map tối giản và extension không làm gì; đường lobby truyền factory arena và extension gameplay đầy đủ.

10.3.5. Tick, extension và dừng thread

Runtime dùng chu kỳ 50.000.000 ns, tương ứng mục tiêu 20 tick/giây. Một tick thực hiện tuần tự: reset cờ input của player → đọc command → giữ player bị đóng băng ở trạng thái dừng → cập nhật movement với bước thời gian cố định → gọi extension update → tăng tick → tạo/xuất bản snapshot. `GameplayExtension.processCommand` nhận mỗi command đã lấy ra khỏi queue, kể cả command không phải movement; `update(nowMillis)` chạy sau movement. Hook `onStart`/`onStop` phục vụ vòng đời module. Nhờ GameplayExtension, TV3 có thể gắn bom/item mà không thêm logic gameplay vào thân GameLoop.

Loop dùng thời gian đơn điệu để tính khoảng ngủ tới tick tiếp theo, còn extension nhận thời gian millisecond. Khi tick trễ, loop đặt lại mốc lịch; hiện không chạy một chuỗi tick bù bằng accumulator. Do đó 20 Hz là mục tiêu cấu hình, không phải kết quả đo đảm bảo trong mọi tải.

`start` chỉ cho chạy một lần trên một instance loop và yêu cầu state đang RUNNING. `stop` đổi cờ, interrupt rồi join tối đa 2 giây khi được gọi từ thread khác; monitor được thả trước khi join. Nếu thread vẫn sống, phương thức báo lỗi thay vì coi là đã dừng thành công. Khối `finally` gọi extension cleanup, ghi nhận failure và xóa command còn chờ. Các API `isRunning`, `isThreadAlive`, `failure` giúp caller phân biệt cờ chạy với việc thread đã giải phóng.

Phân công: TV2; TV4 phối hợp framing, packet type và tích hợp mạng.

10.4. Movement, collision, snapshot builder và delta codec

10.4.1. Từ MOVE packet đến movement command

MovePacketCodec encode/decode payload input gồm sequence, direction và moving, giới hạn độ dài và yêu cầu sequence mạng lớn hơn 0. Client không được cung cấp tọa độ, tốc độ hoặc kết quả va chạm qua message này. MovementHandler tra Session đang PLAYING, tìm runtime theo room, suy ra gameplay player ID từ session rồi tạo Move/Stop command. Việc kiểm tra match envelope ở đường lobby diễn ra trước khi gọi phần xử lý payload này. Command được đưa vào queue; handler không trực tiếp dịch chuyển Player trên network thread.

10.4.2. Áp dụng input và bước mô phỏng

MovementSystem chỉ áp dụng input khi trận RUNNING và player còn sống. Input sequence đã xử lý bị bỏ qua; input mới cập nhật ACK cùng hướng/ý định di chuyển. ACK ở đây là sequence input authoritative đã xử lý, khác cơ chế xác nhận byte của TCP. `StopMovingCommand` tắt movement intent. Các command nội bộ sequence 0 vẫn có đường tương thích riêng.

`update(dt)` kiểm tra bước thời gian hữu hạn, không âm và chặn tối đa 0,1 giây. Khoảng dịch chuyển là speed × dt theo hướng hiện tại rồi giao cho CollisionSystem. Với input mạng có sequence, cờ lấy mẫu của Player ngăn tiếp tục mô phỏng một mẫu cũ qua nhiều tick không nhận input mới. Vì vậy client gửi cả mẫu đứng yên và giữ phím theo nhịp, thay vì chỉ gửi sự kiện nhấn phím đầu tiên. Player chết hoặc player không được phép tiến bước trong tick không được movement cập nhật vị trí.

10.4.3. Collision trên hitbox và entity

CollisionSystem kiểm tra toàn bộ hitbox nằm trong map và mọi tile bị phủ đều walkable. Biên phải/dưới dùng `Math.nextDown` trước khi đổi sang ô để hình chữ nhật vừa chạm biên không chiếm thêm ô kế bên. Hard wall, breakable block và ngoài map đều chặn movement.

Phép di chuyển chia thành bước không quá 1 pixel, xét theo từng trục và dừng trước vị trí không thể chiếm. Cách này giảm nguy cơ bước dịch chuyển bỏ qua tường mỏng trong một lần cập nhật. Danh sách entity của tick cũng tham gia kiểm tra: bom chặn việc đi vào vùng overlap mới; player đang overlap quả bom có thể đi ra khỏi nó. Hiện collision không triển khai va chạm player với player. Kết quả vị trí cuối cùng vẫn do server quyết định; client dùng cùng quy tắc để dự đoán trong world replica.

10.4.4. Snapshot bất biến ở ranh giới thread

GameStateUpdate chứa kind FULL_STATE/DELTA, tick/base tick, kích thước map, tile/player/entity thay đổi, danh sách xóa và frame gameplay. TileChange/PlayerData là giá trị bất biến; các collection được sao chép hoặc bọc không cho sửa. API bổ sung frame tạo update mới thay vì sửa instance đã xuất bản. Một số field frame nội bộ không khai báo `final`, nên tính bất biến ở đây được mô tả theo API công khai và cách xuất bản, không phải khẳng định mọi field trong class đều final.

EntityData là record dùng cho bom/item, với khóa kind + ID, tọa độ tile, owner, variant và thời điểm hết hạn. Thời điểm hết hạn là một mốc thời gian, không phải bộ đếm giảm từng tick khiến entity luôn bị coi là thay đổi. Snapshot không trả tham chiếu mutable Player hoặc mảng map gốc cho network thread. Notice được mang theo update để chuyển tiếp sự kiện; không phải một lịch sử sự kiện đầy đủ bên trong payload snapshot.

10.4.5. Capture, diff và commit baseline

StateDeltaBuilder chụp map và PlayerData của tick hiện tại, lập bảng entity, rồi so sánh với baseline đã commit. Full state được tạo khi bắt buộc hoặc chưa có baseline phù hợp. Delta chứa tile đổi loại, player đổi dữ liệu, entity thêm/sửa và ID/khóa bị xóa. So sánh player bao gồm vị trí, trạng thái và ACK; không có quantization tọa độ hoặc nén riêng từng field trong một PlayerData.

Điểm commit nằm sau `outbound.offer(update)` thành công trong GameLoop. Queue đầu ra có capacity 1024. Nếu queue đầy, builder không được chuyển baseline sang update chưa được xếp hàng; lần tạo sau vẫn so với snapshot đã được chấp nhận trước đó. `baseTick` vì vậy là tick đã xuất bản, không bắt buộc bằng current tick trừ 1. `commit` còn kiểm tra đúng instance update đang pending, tránh chốt nhầm một kết quả build cũ. Việc vào queue chưa chứng minh client đã nhận hoặc áp dụng update; đây là ranh giới xuất bản nội bộ server.

Nếu không có thay đổi có ý nghĩa, builder có thể trả null. Thời gian server trong BombFrame thay đổi một mình không đủ để bắt buộc phát delta; thay đổi explosion, match frame, notice hoặc state được xét riêng. Loop ép full ở tick đầu, theo yêu cầu, mỗi 100 tick và khi FINISHED. Yêu cầu full bị mất do queue đầy được đặt lại để thử tiếp. Khi enqueue thành công, loop mới commit baseline, xóa notice đã chuyển giao và cập nhật latest update. Implementation hiện quét/so sánh state; chưa dùng dirty tracking hoặc tự chọn full theo kích thước encode nhỏ nhất.

10.4.6. Codec và điểm nối với mạng

SnapshotPacketCodec chuyển update thành binary big-endian rồi Base64 trong packet; decode kiểm tra cấu trúc, giới hạn và dữ liệu thừa. Encoder chọn v2 khi không có BombFrame, v3 khi có BombFrame nhưng chưa có MatchFrame, v5 khi có MatchFrame; decoder giữ tương thích v2/v3/v4/v5. Không mô tả v4 là phiên bản encoder hiện phát. Match envelope và frame TCP do các lớp bên ngoài đảm nhiệm như mục 9.

TV2 cung cấp dữ liệu và codec gameplay; hạ tầng gửi packet không được đọc GameState trực tiếp. Các frame bom/trận là điểm nối với TV3, chi tiết adapter được viết ở 10.6. Dung lượng tiết kiệm thực tế phải đo sau binary/Base64/envelope theo cùng một quy tắc; sự tồn tại của thuật toán không đủ để kết luận gói tin luôn giảm xuống một khoảng byte cố định.

Phân công: TV2 — input, replica, prediction và hiển thị movement phía client.

10.5. Input, prediction, replay, interpolation và respawn reset

10.5.1. Phân lớp controller phía client

Bảng 10.6. Các lớp đồng bộ movement và trách nhiệm.

Class	Trách nhiệm	Ranh giới

Class	Trách nhiệm	Ranh giới
`InputController`	Nhận phím và lấy mẫu input bằng Swing Timer	Chỉ hoạt động khi controller đã sẵn sàng
`ClientWorldState`	Áp dụng full/delta thành world replica	Kiểm tra tick, base tick và dữ liệu trước khi áp dụng
`PredictionController`	Mô phỏng local player, giữ input chưa ACK	Không tự dự đoán bom, item hay sát thương
`ReconciliationController`	Cập nhật world, phục hồi authoritative và replay	Không thực hiện visual smoothing
`InterpolationController`	Giữ frame và lấy vị trí remote theo render tick	Nội suy có giới hạn, không ngoại suy vô hạn
`MovementClientController`	Nối network, replica, các controller và sender	Không sở hữu vòng đời ClientNetwork

10.5.2. Lấy mẫu input và gửi ngoài EDT

InputController gắn bốn phím mũi tên qua Swing key bindings và dùng Timer chu kỳ 50 ms. Tập phím đang giữ có thứ tự: phím nhấn sau cùng được ưu tiên; thả nó sẽ quay về phím còn giữ trước đó. Khi không có phím hướng, mẫu input vẫn được tạo với moving=false. Mất focus xóa phím đang giữ để tránh nhân vật tiếp tục đi khi cửa sổ không còn nhận key release. `close` dừng Timer, bỏ listener và khôi phục bindings đã thay thế.

Mỗi mẫu được prediction áp dụng trước, sau đó mới giao cho sender. MovementClientController dùng executor một thread với queue 256 tác vụ để giữ thứ tự encode/gửi; EDT không trực tiếp chờ socket write. Queue và pending input đều có giới hạn; lỗi gửi hoặc tràn giới hạn đi vào đường báo lỗi/dừng input, không phải cơ chế âm thầm bỏ input rồi tiếp tục mô phỏng. Phím Space tạo yêu cầu đặt bom theo cạnh nhấn và qua cùng đường sender, nhưng client không tạo bom dự đoán.

10.5.3. Áp dụng full/delta và yêu cầu đồng bộ lại

ClientWorldState giữ replica của map, player, entity và frame gameplay. Update có tick cũ hoặc bằng tick đã áp dụng trả STALE. Full hợp lệ thay thế dữ liệu nền; delta chỉ được áp dụng khi base tick bằng tick replica hiện tại và kích thước phù hợp. Các kiểm tra tọa độ, số lượng và dữ liệu thống kê/entity bảo vệ bước áp dụng. Khi không thể áp dụng, kết quả NEED_FULL giữ nguyên state cũ và đánh dấu cần đồng bộ lại.

Controller giải mã packet và đưa việc cập nhật world về thread UI. Nó lọc match envelope trước khi áp dụng, gộp yêu cầu full đang chờ bằng cờ resyncPending, và gửi yêu cầu qua sender. Sau update APPLIED, controller cập nhật scene gameplay, xử lý local player, reconcile prediction rồi ghi frame nội suy từ replica hoàn chỉnh. Một delta không chứa mọi player vẫn tạo frame nội suy đầy đủ vì nguồn ghi là world sau khi merge.

Controller có chế độ tự subscribe typed listener, nhưng đường ứng dụng lobby hiện tại tạo nó với subscribe=false; DemoClientConnection giữ listener ứng dụng thường trực và dispatch packet theo thứ tự UI. Không gắn thêm listener thứ hai trên đường này để tránh áp dụng hai lần cùng một message.

10.5.4. Prediction, ACK và replay

PredictionController tạo state mô phỏng local player và dùng lại MovementSystem/CollisionSystem với bước 0,05 giây. Input có sequence tăng dần được giữ trong TreeMap, tối đa 256 mẫu chưa ACK. Trước khi reconcile, bản sao map/entity được cập nhật từ replica để replay dùng các vật cản mới nhất đã biết. Nếu kích thước world thay đổi, prediction yêu cầu được tạo lại thay vì tự đổi kích thước state bên trong.

Khi nhận PlayerData authoritative, controller bỏ ACK cũ, xóa các input có sequence không lớn hơn ACK mới, phục hồi vị trí/hướng/tốc độ/trạng thái sống authoritative rồi mô phỏng lại input còn chờ theo thứ tự sequence. ACK của snapshot phản ánh công việc server đã xử lý, không phải mọi input client đã gửi. Server không phát lại đường đi cho client; bộ đệm input cục bộ cung cấp dữ liệu replay.

ReconciliationController thực hiện cập nhật world rồi gọi reconcile. Giá trị sai lệch nó trả về là khoảng cách từ vị trí predicted trước reconcile đến vị trí authoritative nhận được. Hai vị trí có thể thuộc hai thời điểm mô phỏng khác nhau; không dùng trực tiếp giá trị này làm bằng chứng sai số còn lại sau replay hoặc độ giật người dùng nhìn thấy. Đường hiển thị hiện lấy vị trí predicted sau reconcile, chưa có lớp làm mượt riêng cho correction local player.

10.5.5. Nội suy remote player và reset khi respawn

InterpolationController giữ tối đa 32 frame theo tick, bỏ frame cũ và nội suy tuyến tính giữa hai mẫu bao quanh render tick. Ngoài khoảng mẫu, nó giữ endpoint phù hợp; không dự đoán remote player chạy tiếp vô hạn. Các chuyển trạng thái sống/chết dùng endpoint để tránh trộn hai trạng thái không liên tục.

MovementClientController tính render tick từ tick replica cộng phần thời gian đã trôi qua, chặn phần cộng tối đa hai tick, rồi lùi hai tick. Vì snapshot có thể không phát khi world không đổi và có jitter, không diễn giải công thức này thành độ trễ hiển thị luôn đúng 100 ms trong mọi tình huống.

Respawn là thay đổi vị trí có tính gián đoạn. Khi bộ đếm respawns thay đổi, controller xóa input pending trước reconcile nhưng giữ sequence tăng dần; InterpolationController xóa lịch sử frame khi phát hiện respawn để tránh kéo nhân vật qua cả bản đồ. Player chết không tiếp tục được predict movement, nhưng replica/render các player khác vẫn hoạt động.

10.5.6. Đóng controller

`MovementClientController.close` đánh dấu đóng, bỏ listener do chính nó đăng ký nếu có và shutdown sender. Nó không đóng ClientNetwork dùng chung với lobby. Cửa sổ gameplay dừng Timer/input/controller khi dispose; tầng ứng dụng quyết định rời phòng hay đóng kết nối. Phân chia ownership này cho phép chuyển màn hình trong cùng connection mà không để Timer cũ tiếp tục gửi input.

Phân công: TV3 phụ trách luật bom/item, event và kết quả gameplay; TV2 phối hợp adapter/extension vào runtime; TV4 phối hợp codec và chuyển tiếp packet.

10.6. Bom, chain reaction, block, item, lives/shield/damage/result

10.6.1. Composition và điểm nối với runtime

MatchGameplayExtension là extension tổng hợp được factory của LobbyGameServer tạo riêng cho từng trận. Nó ghép BombGameplayExtension, ItemSystem, hai world adapter, luật trận và bộ sinh vật phẩm. Các hook chạy trên game thread đã mô tả ở 10.3.5; extension và adapter ghi nhận thread owner, kiểm tra khi truy cập mutable world và cleanup trên cùng thread. Không tạo thread riêng để sửa map/player. Constructor cho phép truyền clock, spawn policy, event sink và callback kết quả đặt bom, giúp tách quyết định luật khỏi TCP và hỗ trợ kiểm thử với thời gian điều khiển được.

Bảng 10.7. Thành phần TV3 và cầu nối gameplay.

Thành phần	Dữ liệu/API chính	Vai trò trong đường runtime
`BombSystem`	`plantBomb`, `explodeBombs`	Kiểm tra đặt bom, duyệt tia và xử lý nổ theo batch
`BombWorldAdapter`	Bomb theo ID/ô, truy cập player/map	Chiếu bom thành entity snapshot và thực thi thay đổi world
`BombGameplayExtension`	`processCommand`, `update`	Ghép command, deadline ngòi nổ và frame hiệu ứng
`ItemSystem`	`trySpawn`, `tryPickUp`	Sinh/nhặt vật phẩm và phát event
`ItemWorldAdapter`	Item theo ID/ô, `applyItemEffect`	Chiếu item thành entity và áp dụng effect qua registry
`MatchGameplayExtension`	Hook vòng đời, `update`	Thứ tự xử lý tổng hợp, thống kê và quyết định kết thúc

BombWorldAdapter dịch session ID của hệ thống bom sang gameplay player ID của GameState. Adapter giữ chỉ mục ID/ô để tìm bom, kiểm tra quota/range từ Player và dùng tâm hitbox để chọn ô đặt bom. Khi thêm/xóa bom, nó đồng thời thêm/xóa `EntityData.Kind.BOMB` trong GameState; variant chứa range và expiresAtMillis chứa thời điểm nổ phục vụ presentation. ItemWorldAdapter làm tương tự với `EntityData.Kind.ITEM`, dùng wire ID của ItemType làm variant. Entity là phép chiếu để collision/snapshot sử dụng, không phải một Bomb/Item mutable được chia sẻ cho network.

10.6.2. Đặt bom và quản lý thời gian

BombSystem kiểm tra lần lượt player tồn tại, còn sống, ô EMPTY, chưa có bom và số bom đang hoạt động chưa đạt giới hạn. Range/quota lấy từ world tại lúc đặt, vì chúng có thể đã được tăng bởi vật phẩm. Đặt thành công tạo Bomb có ID, owner, vị trí và thời điểm; thêm vào world rồi phát BombPlacedEvent. Thất bại trả BombError cụ thể. BombGameplayExtension kiểm tra thêm trạng thái trận và player disconnected trước khi gọi system, rồi chuyển kết quả vào callback BombCommandResult. Factory lobby hiện truyền callback rỗng; callback này không phải packet ACK placement riêng gửi cho client.

Bomb có state ARMED, QUEUED_TO_EXPLODE, EXPLODED hoặc CANCELLED. `markExploded` không cho bom đã nổ/hủy nổ lại. Đường runtime trong BombGameplayExtension

không dùng ScheduledExecutor: adapter ghi nanoTime lúc đặt, `update` chọn các bom đã đủ fuse và đưa chung vào một batch. Clock đơn điệu dùng cho kiểm tra thời hạn; timestamp epoch millisecond trong entity/event/frame dùng cho truyền dữ liệu và quy đổi presentation. Clock được inject nên test không nhất thiết cần sleep thật.

BombScheduler vẫn tồn tại như một API độc lập: task hết hạn chỉ đưa ID vào command sink, không tự cập nhật GameState. Không đưa scheduler này vào mô tả đường gọi thực tế của MatchGameplayExtension, cũng không coi nó là thread nổ bom chạy song song với loop hiện tại. Shutdown extension hủy/xóa bom còn lại cùng dữ liệu hiệu ứng của trận.

10.6.3. Nổ lan truyền, phá block và damage batch

`explodeBombs` dùng queue các bom cần nổ và tập processed ID để mỗi bom được xử lý nhiều nhất một lần trong batch. Vùng ảnh hưởng gồm ô đặt bom và tia theo bốn Direction đến giới hạn range. Hard wall hoặc ngoài map dừng tia trước ô đó. Breakable block được tính là ô chịu ảnh hưởng, bị phá rồi dừng tia; ô trống cho tia tiếp tục. Bom gặp trong vùng ảnh hưởng được đưa vào queue để chain reaction.

Tập destroyedBlocks được dùng chung cho mọi tia và mọi bom của batch. Dù block đã đổi sang EMPTY trong GameMap, những tia xử lý sau trong cùng batch vẫn coi ô đó là vật cản. Nhờ vậy, kết quả không vô tình cho tia thứ hai xuyên qua block vừa bị tia thứ nhất phá trong cùng lần cập nhật. Từng bom phát BombExplodedEvent chứa đúng tập ô mà server đã tính; BlockDestroyedEvent được phát một lần cho mỗi block đã phá.

Adapter xác định người trúng đòn bằng giao nhau giữa hitbox và ô lửa, khác phép chọn một ô theo tâm khi đặt bom/nhặt item. System gom player trúng vào một bảng trước khi áp damage: mỗi player nhận tối đa một lần damage trong batch, kể cả đứng trong nhiều tia hoặc nhiều bom chain reaction. Damage được giải quyết sau khi duyệt hết tia, tránh respawn rồi bị đánh tiếp bởi chính batch đó.

`Player.takeDamage` tiêu hao shield trước, không giảm lives trong lần đó. Nếu không có shield, lives giảm một; còn lives thì về spawn ban đầu và tăng respawns, hết lives mới chuyển alive=false. Nâng cấp được giữ khi respawn; không có cơ chế miễn thương có thời hạn được suy thêm từ animation. PlayerDiedEvent chỉ được phát khi bị loại sau damage, không phải mỗi lần mất lives. Killer được lấy từ hit đầu tiên ghi vào bảng của batch; không mô tả đây là hệ thống chấm điểm nhiều nguồn sát thương.

10.6.4. Thứ tự item và effect registry

Sau cập nhật bom/damage, MatchGameplayExtension thực hiện theo thứ tự: xóa item đang nằm trong vùng blast → thử sinh item tại block vừa phá → nhặt item → cập nhật kết quả trận. Item mới rơi sau bước xóa không bị chính bước xóa của batch hiện tại loại bỏ. Player còn sống có thể nhặt item theo vị trí sau movement/damage; không có một bước pickup độc lập trên network thread. Khi nhiều player cùng đủ điều kiện tại một ô, thứ tự gameplay player ID tăng dần quyết định lượt kiểm tra, thay vì thứ tự packet đến từ các connection.

ItemSystem gọi ItemSpawnPolicy, tạo Item nếu policy chọn một loại và ô chưa có item, thêm vào world rồi phát ItemSpawnedEvent. `tryPickUp` kiểm tra player sống, tìm item tại ô tâm player và chuyển trạng thái pickup trước khi xóa/apply effect; một item không thể cấp effect hai lần qua các lần pickup thành công. RandomItemSpawnPolicy kiểm tra xác suất rơi rồi chọn đều

một trong năm ItemType; có thể inject RandomGenerator có seed khi test. Đường lobby dùng `new Random()`, không khẳng định seed cố định hay chuỗi item giống nhau giữa các trận.

ItemEffectRegistry giữ map effect không cho thay đổi cấu trúc sau tạo và kiểm tra đủ mọi ItemType. Adapter gọi registry để thay đổi Player theo MatchRules, tránh gắn từng luật tăng chỉ số vào control flow pickup.

Bảng 10.8. ItemType, mã truyền và effect mặc định.

Loại	Wire ID	Effect authoritative
BOMB_UP	0	Tăng bombLimit một đơn vị nếu chưa tới maxBombs
FIRE_UP	1	Tăng range một đơn vị nếu chưa tới maxRange
SPEED_UP	2	Tăng speedIncrement, chặn tại maxSpeed
LIFE_UP	3	Tăng lives một đơn vị nếu chưa tới maxLives
SHIELD	4	Đặt shield=true, không cộng dồn số lớp

Mã wire được khai báo rõ trong ItemType, không suy từ ordinal khi decode. Item vẫn được tiêu thụ nếu chỉ số đã ở cap; player đã bị loại không được nhặt LIFE_UP để tự hồi sinh.

10.6.5. Luật trận, thống kê và kết quả

MatchRules.demo mặc định thời lượng 180 giây, xác suất rơi 0,75, maxBombs=5, maxRange=8, maxSpeed=144, speedIncrement=8, initialLives=3 và maxLives=99. Các giá trị được đọc từ system property tương ứng; đây là cấu hình mặc định trong code, không phải số liệu thực nghiệm. Lobby mặc định tạo BombRules với fuse 3000 ms, range=1 và quota=1. Constructor MatchRules sáu tham số còn dùng profile legacy/test một life, không đại diện mặc định lobby hiện tại. Extension từ chối cấu hình cap thấp hơn chỉ số ban đầu.

Sau pickup, extension ghi tick bị loại đầu tiên của từng player, đếm người còn lại và tính thời gian bằng nanoTime. Ưu tiên kết thúc là ALL_DEAD khi không còn ai, LAST_ALIVE khi còn một người, rồi TIMEOUT khi hết thời lượng. Chỉ LAST_ALIVE có winnerPlayerId; timeout không tự chọn người nhiều lives nhất làm winner. Người sống nhận rank 1; người bị loại xếp sau người sống và những người bị loại ở tick muộn hơn. Các player bị loại cùng tick có thể đồng hạng. Danh sách được sắp theo rank rồi player ID để truyền ổn định.

Kết quả MatchResult được giữ lại, state chuyển FINISHED và MatchFrame chứa thời gian còn lại, bombLimit/range/lives/shield/respawns cùng kết quả. Update tiếp theo mang dữ liệu này tới snapshot worker. Lobby dừng runtime và phát GAME_OVER; trường hợp rời phòng/đóng trận khi chưa có outcome có thể tạo ABORTED. UI bảng kết quả ở 10.7.3 chỉ hiển thị DTO này, không tính lại thứ hạng ở client.

10.6.6. Event nội bộ, notice mạng và snapshot

Package game/event khai báo sự kiện miền: BombPlaced, BombExploded, BlockDestroyed, ItemSpawned, ItemPickedUp và PlayerDied. Event sink là callback đồng bộ trong luồng gameplay, không phải message broker hoặc hàng đợi bền vững. Match extension dùng event để gom block/blast cho các bước item; những thay đổi map/entity/player vẫn được phản ánh qua snapshot như 10.4.

GameNotice là dữ liệu chuyển giao nội bộ có event ID, chỉ nhận các event bom, item và player bị loại; BlockDestroyed không có notice riêng trong contract này. Việc block biến mất được

truyền bằng tile delta/full state. GameState giữ notice chờ xuất bản với giới hạn backlog; GameLoop chỉ clear sau enqueue update thành công. Đây không phải event log lưu trên đĩa hoặc cam kết replay sau reconnect.

LobbyGameServer.pump gửi snapshot trước, sau đó encode các notice thuộc update bằng GameplayPacketCodec và BombPacketCodec, bọc match ID rồi đưa vào cùng writer queue của từng session. Context chuyển session ID trong event thành player ID số trên wire. Các schema, framing và match scope đã trình bày ở mục 9; ở đây chỉ xác định nơi tạo và nơi chuyển tiếp dữ liệu. Thứ tự enqueue trên một connection không thay thế kiểm tra failure/backpressure của writer hoặc xác nhận client đã áp dụng.

Client lấy snapshot đã áp dụng làm nguồn world và hình ảnh chính. Notice item/death được validate; BombScene dùng notice bom để ghi nhận thông tin theo tick/event ID, không thực thi lại phá block, xóa bom, cộng item hoặc damage. Sự phân chia này tránh nhân đôi effect khi cùng thay đổi xuất hiện ở cả snapshot và notice.

Phân công: TV2 render map/player/HUD, TV3 bom/item/hiệu ứng, TV1 bảng kết quả; TV4 phối hợp dữ liệu tích hợp.

10.7. Renderer bom/item, grid/HUD và bảng kết quả

10.7.1. Grid, nhân vật và HUD — phần TV2

TileGridRenderer vẽ ô theo thứ tự row-major từ replica: nền trống xen màu, hard wall dạng đá và breakable block dạng thùng. Renderer tạo bản sao Graphics2D rồi dispose sau khi vẽ để không làm rò trạng thái nét/màu sang thành phần khác. `markPlayerCell` đánh dấu ô theo vị trí local player đã được server xác nhận; ô đánh dấu có thể khác vị trí predicted đang hiển thị trong thời gian input chưa được ACK.

MovementGameWindow dùng Swing render Timer 16 ms và dữ liệu trong controller để repaint. Đây là chu kỳ yêu cầu vẽ, không phải cam kết đạt 60 FPS hay nhịp simulation server. Map lấy từ replica; local player lấy tọa độ prediction khi có, remote player lấy mẫu interpolation. Nhân vật được vẽ bằng hình oval theo hitbox, có màu trạng thái chết và vòng shield; không mô tả bản hiện tại là một hệ thống animation sprite hoàn chỉnh.

HUD hiển thị thời gian còn lại, lives, shield, quota bom, range và speed từ dữ liệu authoritative. Timer hiển thị, vị trí predicted và hiệu ứng không cấp quyền thay đổi kết quả trận. Cửa sổ chỉ khởi tạo InputController khi movement đã sẵn sàng; nhấn Space có cờ giữ để tránh gửi lặp liên tục, mất focus cũng reset cờ này. Bom/item/effect được nối vào cùng luồng paint và mô tả chi tiết ở 10.7.2.

10.7.2. Bom, item và hiệu ứng — phần TV3

BombScene đồng bộ danh sách BombView từ entity bom và ExplosionView từ BombFrame mỗi khi replica áp dụng snapshot thành công. Mốc epoch của server được quy đổi sang thời gian local bằng độ lệch so với `frame.serverTimeMillis` tại lúc nhận; sau đó thời gian animation tiến theo nanoTime local. Cách này tránh yêu cầu hai máy có đồng hồ epoch giống hệt nhau, nhưng không phải thuật toán ước lượng chính xác độ trễ một chiều hay bù đầy đủ network jitter. Scene bỏ hiệu ứng đã hết hạn và giữ bộ nhớ event ID notice có giới hạn 256 mục; notice lệch tick scene hiện tại không làm thay đổi hình ảnh.

BombRenderer dùng Java 2D vẽ thân bom, dây ngòi, tia lửa dao động và chữ đếm ngược. Countdown chỉ là presentation; bom không tự nổ ở client khi chữ hết thời gian. Renderer giữ chữ tối thiểu 1 trong khi bom vẫn còn trong danh sách, chờ snapshot authoritative thay đổi entity. ExplosionRenderer vẽ từng ô affectedCells do server gửi với màu/độ sáng thay đổi theo tiến độ và dừng khi hết visibleUntil. Không tự raycast lại tường hoặc tính chain reaction từ hình ảnh.

ItemRenderer vẽ đủ năm loại item bằng biểu tượng Java 2D, màu riêng và chuyển động nhấp nhô theo thời gian. Đường gameplay chính trong MovementGameWindow tạo ItemView trực tiếp từ entity replica khi paint, không dùng notice ITEM_SPAWN/ITEM_PICKED_UP để tự sửa danh sách item. BombPresentationState có API thay toàn bộ danh sách để BombScene dùng; các API cập nhật riêng lẻ vẫn phục vụ cách dùng độc lập. ItemPresentationState cũng có API spawn/pickup/replace cho presentation độc lập, nhưng không phải owner item world của đường lobby hiện tại.

Thứ tự paint chung là grid và ô đánh dấu → item → bom/lửa → player và vòng shield. Phần movement/HUD ở 10.7.1 và phần effect này dùng chung replica đã merge; không tạo một timer mô phỏng damage riêng theo số lần repaint. Các renderer tạo/dispose Graphics2D riêng và không cần tải sprite bên ngoài. Demo effect độc lập hỗ trợ xem hình thức trình bày; nó không thay bằng chứng tích hợp TCP hoặc chạy GUI nhiều client.

10.7.3. Bảng kết quả và chuyển lại phòng — phần UI TV1

MatchResultsPanel là JPanel chứa summary và JTable, dùng DefaultTableModel không cho sửa ô. `showResult(MatchResult, Map<Integer,String>)` hiển thị reason/winner/rank do server cung cấp, ánh xạ playerId sang tên hoặc fallback `P<id>`; panel không tính lại người thắng từ replica local. `clear()` xóa dữ liệu khi bắt đầu lượt mới.

LobbyWindow giữ panel kết quả trong tab riêng. Khi nhận GAME_OVER hợp lệ qua connection owner, lobby đóng cửa sổ gameplay, hiển thị result đã decode và chuyển UI về luồng ready/start lại. Room state mới điều khiển các nút; cửa sổ không tự khởi tạo world mới từ bảng kết quả.

Lobby cài các action Connect/Create/Join/Refresh/Ready/Start/Leave/End/chat. Gửi control đi qua DemoClientConnection thay vì ghi socket trực tiếp trong ActionListener. Khi đóng cửa sổ gameplay, caller quyết định host End hay người chơi Leave; đóng hẳn lobby mới đóng connection owner. `disposeGame` và `ClientNetwork.close` có phạm vi khác nhau, nên thao tác kết thúc lượt vẫn có thể giữ TCP để chơi lại.

Phân công: TV4 phụ trách protocol/router/ClientNetwork/metrics/công cụ; TV1 phối hợp transport, TV2/TV3 xác nhận dữ liệu gameplay.

10.8. Router, ClientNetwork, metrics, benchmark và vận hành

10.8.1. Hạ tầng protocol và API tích hợp

Bảng 10.9. Các lớp hạ tầng TV4 và vai trò hiện thực.

Nhóm/class	API tiêu biểu	Điểm nối
`Packet`, `PacketType`	Type constants, payload getter, name lookup	Mô hình packet dùng chung và mã contract

Nhóm/class	API tiêu biểu	Điểm nối
`PacketEncoder`, `PacketDecoder`	`encode`, `decode`	Byte frame ↔ Packet, độc lập socket owner
`SessionPacketCodec`, `LobbyPacketCodec`, `LobbyJson`	Encode/parse/validate theo message	Session/room/chat và object JSON có giới hạn
`MatchEnvelopeCodec`, `MatchContext`	`wrap`, `unwrap`, session/player lookup	Scope lượt và mapping identity
`DemoPacketCodec`, `BombPacketCodec`, `GameplayPacketCodec`, `ResyncPacketCodec`	Metadata, placement/notice/result, request full	Contract cụ thể; body movement/snapshot phối hợp TV2
`MessageRouter`, `MessageHandler`, `SessionHandlers`	Register/route và callback handler	Dispatch type tới nghiệp vụ trên thread gọi
`ClientNetwork`	Connect/send/listener/receive/disconnect/close	TCP client dùng chung, không tự điều khiển world/UI

SnapshotPacketCodec/MovePacketCodec nằm trong package `server.game.movement` và được client dùng lại; không mô tả repository như đã đưa toàn bộ protocol gameplay vào common độc lập. Vai trò module trong bảng là trách nhiệm phối hợp triển khai/viết, không xác nhận một người viết toàn bộ từng file.

10.8.2. Encoder, decoder và codec

PacketEncoder mã hóa JSON sang byte UTF-8, kiểm tra length tính type+payload, cấp mảng prefix/body và ghi big-endian. Overload ghi OutputStream dùng cùng mảng frame và flush, tránh có hai cách tính length khác nhau. Decoder dùng DataInputStream đọc length, kiểm tra trước cấp mảng, `readFully` body và giải mã UTF-8 với chế độ báo lỗi. Các lần read ngắn không bị coi là đã có đủ một packet. Nguồn: PacketDecoder.

Packet decoder không kiểm tra JSON nghiệp vụ. `SessionPacketCodec` kiểm soát object control phẳng, `LobbyJson` là parser/encoder nội bộ giới hạn kích thước/độ sâu/array, còn từng codec đặt điều kiện trường/kiểu. MOVE, metadata, resync và snapshot còn có các kiểm tra canonical riêng như đã nêu ở mục 9. Vì vậy, đổi cách tạo JSON mà không dùng encoder có thể bị từ chối dù ý nghĩa object có vẻ tương đương.

`MatchEnvelopeCodec` thêm/loại matchId canonical quanh inner object và kiểm tra kích thước qua encoder; codec không thay kiểm tra membership hoặc schema bên trong. Handler/server và client context tiếp tục kiểm tra scope. Các lớp DTO/codec chính có nguồn tại common/protocol và common/dto.

10.8.3. Router và phân loại lỗi

MessageRouter dùng ConcurrentMap type→MessageHandler, hỗ trợ register/unregister và trả false nếu type không có handler. `route(clientId, packet)` gọi handler đồng bộ trên thread hiện tại; router không tạo một queue xử lý riêng. Hạ tầng reader pool, khóa lobby và command queue mới là các ranh giới thực thi khác nhau.

SessionHandlers đăng ký CONNECT/PING/DISCONNECT. CONNECT parse/kiểm tra version, gọi completeHandshake, gửi ACK rồi callback để lobby công bố danh sách phòng.

`LobbyGameServer` đăng ký các type lobby/gameplay; RECONNECT/NETWORK_LOST không được đăng ký trong đường sản phẩm.

ClientHandler phân biệt RequestRejectedException, IllegalArgumentException, unsupported version và RuntimeException khác. Các nhánh đó quyết định ERROR/giữ phiên/đóng socket, còn frame hỏng không cố khôi phục bằng tìm byte biên tiếp theo. Error helper hiện có requestId=null; runtime failure ở snapshot pump đi qua failure/endMatch, không hứa mọi lỗi đều có message MATCH_FAILED hoặc correlation theo từng command.

10.8.4. ClientNetwork: I/O, listener và pull queue

ClientNetwork mở Socket với connect timeout 5 giây, đặt TCP_NODELAY, clear pull queue cũ rồi khởi động reader và write watchdog. `send(Packet)` capture socket, kiểm tra đang chạy, khóa sendLock rồi encode/write/flush trọn frame. Hàm này có thể chặn I/O; sender/control worker ở tầng ứng dụng phải gọi nó ngoài EDT.

Reader `listen(active, guard)` decode packet liên tục, cập nhật bộ đếm byte rồi chuyển cho global listener và typed listener. Các callback chạy trên reader, không tự chuyển sang EDT. Nếu listener ném RuntimeException, network coi đó là lỗi để kết thúc connection thay vì im lặng bỏ packet rồi tiếp tục world có thể thiếu dữ liệu.

API pull `receive(Duration)` và `receive()` dùng queue 1024 packet, budget 4 MiB; listener và pull mode có thể cùng nhận packet nên caller phải chọn cách tích hợp phù hợp. Đường lobby hiện tại gọi `setPacketQueueEnabled(false)` và dùng một listener ứng dụng thường trực, tránh giữ bản sao pull không có consumer. ReceiveLock/generation phục vụ quản lý queue/byte khi clear/thay kết nối; không thay match scope và state machine gameplay.

`disconnect()` dừng trạng thái, đóng socket để đánh thức reader, dừng watchdog, interrupt/join reader có thời hạn rồi clear queue. `close()` còn xóa listener và callback disconnect. Kiểm tra captured socket giúp reader cũ không tự đóng một socket khác; khả năng bảo vệ đó không được giới thiệu thành reconnect resume trận đã hoàn thiện.

10.8.5. Connection owner và dispatcher phía client

DemoClientConnection sở hữu ClientNetwork, control scheduler, sessionId, MatchContext và controller gameplay hiện tại. Listener được đăng ký trước connect, chuyển packet sang executor UI tuần tự có giới hạn pending count/byte. Control send được giới hạn qua semaphore trước khi submit vào executor; không phải toàn bộ hàng đợi executor/EDT của JDK đều hữu hạn.

Khi GAME_STARTED đến, owner đóng movement cũ nếu có, tạo context/controller mới với `subscribe=false`, rồi callback cho lobby tạo cửa sổ gameplay. Snapshot/notices tiếp theo được chuyển trên UI owner trước GAME_OVER. Heartbeat, gửi yêu cầu lobby và resync đi qua control scheduler; input movement có worker gửi riêng trong controller TV2. ClientNetwork sendLock bảo vệ frame khi hai nguồn gửi cùng dùng một socket.

Close owner đóng network/control/movement và bỏ context; đóng movement theo kết thúc lượt không mặc nhiên đóng cả TCP. Chi tiết pending input, replica, reconciliation/interpolation ở phần TV2 sẽ nối vào API này, không tạo thêm socket hoặc listener nhận packet trùng trong luồng mặc định.

10.8.6. Monitor và benchmark runner

Bảng 10.10. Công cụ đo và phạm vi dữ liệu được ghi.

Thành phần	Hiện thực/chức năng	Giới hạn diễn giải
`LatencyMonitor`	Ghép pingId với nanoTime, lưu duration ms, avg/min/max/p95 nearest-rank	Là RTT/timing mức ứng dụng theo caller; không phải one-way latency
`ThroughputMonitor`	Atomic counter tổng/window packet và byte; rate theo elapsed nanoTime	Byte tại điểm instrument, không tự quan sát TCP/IP header hay retransmission
`NetworkBenchmark`	Peer TCP, workload transport/idle/movement/bombs/suite, window và CSV	Có runner không đồng nghĩa đã nghiệm thu mọi workload/môi trường
`BenchmarkCsv`	Append UTF-8, escape field, đối chiếu header và tạo thư mục output	Mỗi process sở hữu file riêng; không có cơ chế ghi cạnh tranh nhiều process vào cùng CSV

LatencyMonitor giữ mặc định tối đa 10.000 mẫu, loại mẫu cũ khi đầy; pending PING được remove khi nhận PONG tương ứng, reset xóa cả pending/mẫu. Giới hạn mẫu không tự giới hạn mọi pending ping: caller cần quản lý chu kỳ/ID và vòng đời đo. Các hàm thống kê trả 0 khi chưa có mẫu, nên báo cáo phải kèm sample count thay vì coi 0 là một RTT đã đo.

ThroughputMonitor cập nhật khi ClientNetwork gửi/nhận frame. Monitor gộp traffic instrument ở instance đó; benchmark còn có bộ đếm tx/rx riêng theo cửa sổ và theo PacketEncoder. Không cộng hai bộ đếm như hai nguồn traffic độc lập.

NetworkBenchmark có local runner tự tạo server và remote/distributed runner dùng kết nối thật. Mode suite local lần lượt chạy transport với 1/2/4 client, các mode gameplay với 2/4 client; đó là danh sách kịch bản của code, chưa phải kết quả tất cả đều đạt. Runner handshake, tạo/join/ready/start và gửi input/đặt bom bằng packet, không sửa world authoritative ngoài luồng mạng.

CSV ghi run ID, nhãn/revision/scope, topology và môi trường, tx/rx byte/packet, RTT sample/statistics, MOVE ACK delay, full/delta, snapshot byte, stale/resync, quan sát bom và cleanup. `plant_ack_ms` trong runner được suy ra khi replica lần đầu thấy bom của chính session, không phải một ACK placement riêng trên protocol. Các con số trong CSV phụ thuộc nơi timestamp, poll/scheduling và cửa sổ đo.

BenchmarkCsv kiểm tra header file trước append, escape dấu phẩy/dấu nháy/xuống dòng và ghi UTF-8 theo các cột đã chọn. Khi đo nhiều process, dùng file riêng rồi tổng hợp theo scope; không suy ra đồng bộ ghi file liên process chỉ vì hàm append tồn tại. Setup và Results sẽ chọn lần chạy, số lặp và dữ liệu gốc phù hợp, không chép CSV lịch sử như vừa tái đo revision mới.

10.8.7. Vận hành, chẩn đoán và ranh giới bằng chứng

Các owner có API quan sát như connection/runtime failure, pending queue, thread alive/terminated, listener count và cleanup status để phục vụ test hoặc runner. Các hook này hỗ trợ chẩn đoán, không thay một hệ thống monitoring tập trung hay phép đo tải/soak.

Script test tổng chạy tuần tự Maven, Check TV2 và NetworkTest main. Ở khảo sát đợt khảo sát ban đầu, Maven có process smoke failure nên Check/main được kiểm riêng sau đó; không dùng bảng test cũ hoặc sự tồn tại của test class để nhận toàn bộ suite hiện đã xanh. Các kết quả thực chạy được tổng hợp ở mục 12; GUI/LAN/soak chưa được nghiệm thu.

Chương Implementation phân biệt API legacy, đường chạy thực và phần còn thiếu bằng chứng. Mục 10.2 mô tả transport/lobby, 10.3–10.5 mô tả runtime và đồng bộ movement, 10.6 mô tả luật gameplay cùng điểm ghép, 10.7 mô tả presentation và 10.8 mô tả hạ tầng/công cụ. Thuật toán tổng thể và định dạng byte được dẫn về mục 7/9 để tránh lặp lại toàn bộ trong chương này. Kết quả và hạn chế được trình bày ở các chương riêng, dựa trên hồ sơ khảo sát cùng những lần kiểm chứng mới nếu có.

11. Thiết lập thực nghiệm — Experimental Setup

Phân công: TV4 chủ trì thiết kế, lưu bằng chứng và tổng hợp; TV1 phụ trách transport/session/room và lỗi kết nối; TV2 phụ trách runtime/movement, replica và full/delta; TV3 phụ trách bom/item/damage/result. Cả nhóm xác nhận cấu hình và giới hạn của từng lần chạy.

Theo Instruction, mục 8.11 và mục 9, thực nghiệm cần mô tả phần cứng, OS, ngôn ngữ/thư viện, cấu hình mạng, dữ liệu và kịch bản; kết quả phải có số liệu thực tế. Thiết kế này đánh giá ba câu hỏi: hệ thống thực hiện đúng contract gameplay hay không; State Delta Compression giảm bao nhiêu byte trên cùng chuỗi trạng thái; latency, ACK và lượng message thay đổi thế nào với workload và số client. Các bảng kịch bản, thời lượng và số lần lặp dưới đây là **kế hoạch thu thập**, chưa phải kết quả đã chạy. Kết quả khảo sát và CSV lịch sử được giữ riêng, không đổi nhãn thành thực nghiệm mới. Kết quả thực hiện nhóm codec D0/D1, T1 và ba lượt reliability được báo cáo tại mục 12.4–12.6; các nhóm nhiều JVM/LAN có điều kiện không được tính là đã đo.

11.1. Revision, phần cứng, OS/JDK và cấu hình

11.1.1. Môi trường đã được ghi nhận

Bảng 11.1. Môi trường khảo sát đã có bằng chứng và thông tin cần bổ sung.

Thuộc tính	Thông tin đã xác minh	Phạm vi sử dụng
Revision khảo sát	`e333050997c5dbf78ec4b9b42a9bc9a73ce6a0d5`	Mốc source, không thay hash artifact và trạng thái working tree
Hệ điều hành	Windows 11, build 10.0.22631, AMD64	Máy khảo sát đợt khảo sát ban đầu
Bộ xử lý logic	16 theo manifest khảo sát	Không suy thành 16 lõi vật lý hoặc 16 người chơi đạt tải
Tên CPU, lõi vật lý, RAM	Chưa có dữ liệu được xác minh trong hồ sơ	Phải bổ sung cho từng máy trước khi công bố kết quả hiệu năng
Runtime/compiler	Java/Javac 25.0.1; HotSpot 64-Bit	Phiên bản ghi trong manifest đợt khảo sát ban đầu
Maven	3.9.12; dependency lấy từ local cache ở lần khảo sát	Đường cài Maven là cấu hình máy, không hard-code cho mọi thành viên
Build/test	Release 25, compiler plugin 3.13.0, Surefire 3.5.2, JUnit Jupiter 5.11.4	Theo POM; JUnit là test dependency
Giao diện/giao tiếp	Swing/AWT, Java 2D, TCP socket	Runner headless không đánh giá trải nghiệm GUI

Căn cứ: manifest khảo sát, hồ sơ môi trường/lệnh và pom.xml. Thiết kế mục 11 tách khỏi kết quả suite/benchmark ở mục 12. Truy vấn phần cứng ở lượt biên soạn này không lấy được CPU/RAM; không dùng suy đoán để lấp khoảng trống. Khi chạy chính thức, bổ sung tên CPU, RAM vật lý, OS/JDK, nguồn điện và topology thực tế cho server lẫn client. Không dùng thông số máy hiện tại để gán ngược cho CSV lịch sử.

11.1.2. Khóa cấu hình và artifact

Mỗi đợt đo ghi `git rev-parse HEAD`, `git status --short`, hash source/config liên quan, hash JAR hoặc class inventory thực sự chạy, `java -version`, `javac -version`, `mvn -version` và toàn bộ lệnh. Nếu có sửa source chưa commit, ghi dirty cùng diff hoặc bản sao source; chỉ ghi revision là chưa đủ để tái lập. Giữ mặc định JVM trừ khi có lý do cụ thể; mọi heap/GC/system

property override phải nằm trong manifest. Hạn chế ứng dụng nền, dùng cùng chế độ nguồn điện và ghi các sự cố tải nền quan sát được. Không cần cài framework/game engine hoặc database mới để thực hiện các phép đo này.

classpath.cmd ưu tiên JAR ở root rồi JAR trong target trước `target/demo-classes`. Vì vậy build source mới xong chưa đủ để kết luận wrapper benchmark đang dùng class mới. Đợt đo từ source ưu tiên chạy trực tiếp classpath `target/demo-classes` sau build thành công; đợt đo bản phát hành dùng đúng JAR đã hash. Không trộn hai loại artifact trong một nhóm so sánh. Không cần xóa artifact cũ để lựa chọn classpath đúng.

11.2. Môi trường loopback, nhiều process và LAN nếu có

Bảng 11.2. Topology và kết luận được phép rút ra.

Mã	Bố trí	Cách dùng	Giới hạn
T0	State/codec trong một process, không socket	So sánh byte và tính đúng full/delta	Không đo TCP latency hoặc băng thông đường truyền
T1	`--local`, một JVM chứa server và các client, TCP qua 127.0.0.1	Nhóm benchmark chính có thể tái lập bằng runner hiện tại	Chia sẻ CPU/GC; không đại diện mạng LAN
T2	Server JVM riêng, runner JVM riêng trên cùng máy	Kiểm tra ảnh hưởng phân process và process smoke	Vẫn dùng tài nguyên máy chung, vẫn là loopback
T3	Server và client/runner ở nhiều máy qua LAN	Nhóm bổ sung khi có máy và biên bản thực tế	Phải ghi loại mạng, IP/port, Wi-Fi/Ethernet và bố trí máy

T1 dùng ephemeral port khi runner tự tạo server. T2/T3 dùng port cố định được ghi trong manifest; ví dụ lệnh bên dưới dùng 8888. Runner dùng TCP thật và luồng lobby/match hiện tại, không thay world qua một đường debug để tạo số liệu mạng. Không bật giả lập delay/loss trong nhóm chính. Nếu làm nhóm bổ sung có impairment, ghi công cụ, cấu hình và tách label; runner chưa có tùy chọn tự tạo packet loss/delay.

Mỗi file CSV chỉ có một process ghi. Với distributed runner, host giữ room và công bố ROOM_ID, guest dùng `--clients 1 --expected-clients N --room-id UUID`; mỗi máy lưu CSV riêng. Chỉ gom những file có cùng run ID/roster/topology sau khi đối chiếu log. Runner phân tán bắt đầu cửa sổ đo riêng từng process, không có barrier thời gian chung toàn máy; không coi tổng các hàng đó là một cửa sổ server throughput được đồng bộ tuyệt đối. Timestamp UTC giúp ghép hồ sơ, còn duration trong một process dùng nanoTime, không trừ timestamp hai máy để suy one-way delay.

11.3. Map, luật game, input workload và seed

11.3.1. Dữ liệu đầu vào

Không có dataset huấn luyện hoặc dữ liệu người dùng ngoài hệ thống. Dataset thực nghiệm là cấu hình map/luật, roster, chuỗi input/sự kiện có mô tả, snapshot và output test/CSV/log được tạo trong từng lần chạy. Map gameplay chuẩn lấy từ DefaultArenaMap: 17 × 15 ô, tile 32 pixel, bốn spawn cố định. Lưu hash factory/map cùng source để phát hiện đổi layout. Hitbox/tốc độ ban đầu lấy theo PlayerMovementRules; mọi property override phải được lưu.

11.3.2. Cấu hình của benchmark hiện có

NetworkBenchmark ở chế độ local khởi tạo BombRules(3000, 1, 1), MatchRules với dropChance=0,75, cap bom/range/speed lần lượt 5/8/144, speedIncrement=8, initialLives=3 và maxLives=99. Thời lượng trận local là `seconds + warmup + 30` giây, không phải mặc định 180 giây của server lobby thông thường. Chạy remote phải cấu hình và ghi luật ở process server; không suy runner tự đổi luật server từ CLI.

Bảng 11.3. Workload và nhịp phát của runner.

Mode	Số client/người hỗ trợ trong nhóm chính	Input trong cửa sổ đo	Mục đích
transport	1, 2, 4 client; không tạo match	PING khoảng mỗi 200 ms	RTT và chi phí message của tầng kết nối
idle	2, 4 người	Có PING, không phát MOVE/đặt bom	Lưu lượng nền và snapshot của trận đứng yên
movement	2, 4 người	MOVE khoảng mỗi 50 ms, đổi hướng sau mỗi 20 bước gửi	ACK input và lưu lượng khi player di chuyển
bombs	2, 4 người	Mỗi client đặt một bom đầu cửa sổ; movement theo đường thoát rồi dừng	Bom/lửa được mọi peer quan sát, thời gian thấy bom authoritative

Nhịp trên là lịch gửi mục tiêu của runner; scheduling có thể khiến số mẫu thực khác phép tính lý tưởng. Warm-up runner chủ yếu poll/PING, chưa làm nóng đầy đủ workload movement/bombs. Khi phân tích phải nêu giới hạn này, không gọi đó là steady-state warm-up đầy đủ cho mọi module. Mode bombs không phải stress đặt bom liên tục và thường chỉ có một plant observation mỗi client mỗi lần chạy. Idle cũng không đồng nghĩa không có snapshot: timer trận, full định kỳ hoặc thay đổi frame có thể tạo update.

Đường local dùng Random không có seed CLI và không xuất trace input thô. Giữ nguyên cách chạy hiện có khi báo cáo benchmark hệ thống, ghi seed là không điều khiển/không được lưu; không khẳng định hai lần chạy có chuỗi drop giống nhau. Các test gameplay cần đối chứng chính xác dùng policy/clock được inject theo fixture. Phép so sánh codec ở 11.5 dùng **cùng chuỗi state** cho cả hai nhánh, không phụ thuộc hai trận ngẫu nhiên độc lập.

11.4. Kịch bản chức năng và lỗi

Bảng 11.4. Ma trận kiểm chứng chức năng và fault handling.

Mã	Kịch bản/điều kiện	Điều kiện kiểm tra	Phụ trách và loại bằng chứng
F01	CONNECT, room, host/ready/start, chat và leave	State transition/roster đúng; message không sang phòng khác	TV1; JUnit/TCP log
F02	Hai runtime và nhiều producer submit	State tách room, FIFO enqueue, queue có giới hạn, không hỏng dữ liệu	TV2; RuntimeContractTest và log check
F03	MOVE/STOP tại tường, block, biên và bom	Không đi xuyên; player chết không di chuyển; ACK không lùi	TV2; movement/collision test
F04	Full/delta, thêm/xóa player/entity, bỏ một update ở replica	Snapshot không bị sửa; baseline lệch được phát hiện; full phục hồi world	TV2/TV4; codec/replica assertions

Mã	Kịch bản/điều kiện	Điều kiện kiểm tra	Phụ trách và loại bằng chứng
F05	Prediction/replay, interpolation và respawn	Bỏ input đã ACK, không replay input trước respawn hoặc kéo nhân vật qua map	TV2; controller test; GUI bổ sung
F06	Placement lỗi, fuse, chain và block chung batch	Đúng quota/ô, bom không nổ hai lần, tia không xuyên block vừa phá trong batch	TV3; BombSystem/BombRuntime Integration test
F07	Damage/lives/shield/item/result	Một damage/player/batch, pickup không lặp, cap và ranking/reason đúng	TV3; MatchGameplay/ExtendedItems test
F08	Frame sai/cắt dở, timeout, client chậm và disconnect	Bị từ chối/cleanup theo contract; không treo vô hạn hoặc làm hỏng phòng khác	TV1/TV4; transport/writer/lifecycle test
F09	Start/stop lặp và kết thúc trận	Runtime/reader/writer/worker được giải phóng theo timeout test	TV2/TV4; thread/cleanup assertion
F10	GUI hai/bốn người, mất focus, bom/item/HUD và trở lại phòng	Input/render theo snapshot, kết quả khớp server	Cả nhóm; biên bản người kiểm tra, ảnh/video và log cùng run

Các mã F là nhóm kịch bản, không thay số lượng JUnit test. Tên test/API cụ thể đã được truy vết ở hồ sơ khảo sát và chương 10. Không lấy việc lớp test tồn tại làm kết quả pass. Bỏ update ở fixture F04 là fault injection tại tầng replica, không phải bằng chứng TCP tự làm mất message. Reconnect resume trận không nằm trong tiêu chí nghiệm thu vì đường sản phẩm chưa bật tính năng đó; disconnect/leave có thể kết thúc lượt ABORTED như contract hiện tại.

Kế hoạch chạy full suite một lần và từng check/main riêng một lần để ghi nhận tổng quan. Các nhóm concurrency/lifecycle/fault có tính thời gian được chạy thêm ba lần độc lập với log riêng; báo cả ba kết quả, không chỉ chọn lần đạt. Nếu full suite dừng khiến các bước check/main phía sau không chạy, chạy riêng để thu bằng chứng và giữ trạng thái full suite thất bại. Lỗi process smoke đã ghi nhận ở đợt khảo sát ban đầu cần hiện diện trong Results nếu chưa có lượt xác minh mới giải quyết nó. Không tự tăng timeout, sửa test hoặc đổi source trong một đợt đo để làm kết quả đẹp hơn.

11.5. Đối chứng full mỗi tick so với delta + full định kỳ

11.5.1. Hai nhánh so sánh và phạm vi byte

Nhánh B0 gửi FULL_STATE ở mọi tick. Nhánh B1 dùng StateDeltaBuilder, full tại tick 1 và mỗi 100 tick, delta khi có thay đổi, không phát packet khi builder trả null; full do resync được tính nếu kịch bản có resync. Hai nhánh nhận cùng state ở cùng tick, cùng phiên bản codec và cùng nội dung gameplay. Commit baseline B1 theo đúng điều kiện xuất bản của harness; không bỏ các full định kỳ khỏi tổng byte. Kết quả này đo cả lợi ích chọn dữ liệu thay đổi và việc bỏ update rỗng, không gán toàn bộ mức giảm riêng cho tile diff.

Thước đo chính là tổng độ dài frame ứng dụng sau encode: 4 byte length + 2 byte type + payload UTF-8, bao gồm binary snapshot đã Base64/JSON và match envelope nếu harness có dùng. Không gọi số này là tổng byte trên dây TCP/IP: chưa gồm header mạng, ACK TCP, retransmission hoặc mã hóa tầng khác. Nếu cần so sánh riêng binary payload, ghi thành cột bổ sung và giữ phạm vi đo hai nhánh giống nhau.

11.5.2. Fixture hiện có và mở rộng được thiết kế

Bảng 11.5. Phép đo delta và mức sẵn sàng của công cụ.

Mã	Chuỗi đầu vào	Phạm vi đo	Trạng thái công cụ
D0	200 tick; map trống 17 × 15, một player; di chuyển tick 1–80, thêm bom tick 40, xóa tick 100, thêm item tick 120	Snapshot v2 + PacketEncoder, không match envelope/BombFrame/MatchFrame	Có trong DeltaCompressionCheck
D1	600 tick, 2/4 player, arena; các trace idle, movement và bomb/block/item được lưu cố định	Snapshot v5 + match envelope giống đường gameplay	Harness báo cáo đã được hiện thực và đo ở mục 12.5; không thay runtime production

DeltaCompressionCheck đồng thời kiểm tra codec, baseline, xóa entity/player và full phục hồi trước vòng đo D0. Vòng đo D0 không phá block và không mô phỏng đầy đủ lives/shield/notice của trận hiện tại; 200 tick tương ứng 10 giây **thời gian mô phỏng**, không phải benchmark chạy đủ 10 giây đồng hồ. Không suy trực tiếp mức tiết kiệm D0 thành mức tiết kiệm bandwidth v5 ở LAN.

D0 chạy năm lần bằng process mới để kiểm tra output byte có lặp lại; đây là fixture xác định nên năm số bằng nhau không tạo bằng chứng biến thiên thống kê về mạng. D1 chỉ được công bố khi đã có harness lưu trace và kiểm tra equality replica sau mỗi tick. Trace cần ghi player input/ACK, map change, entity add/remove, BombFrame/MatchFrame và thời điểm sự kiện; nhánh full/delta phải đọc cùng trace, dùng cùng match ID hoặc envelope cùng chiều dài. Nếu sinh trace bằng RNG, lưu thuật toán, seed và trace thực tế. Không sửa bộ sinh state khác nhau cho hai nhánh.

Mỗi tick có dòng `tick,full_frame_bytes,delta_frame_bytes,kind,base_tick` ở harness D1; D0 hiện chỉ xuất ba cột đầu và dòng PASS cuối, nên lưu stdout nguyên trạng như log rồi tách đúng các dòng số thành CSV. Không đọc toàn stdout D0 như một file CSV thuần hoặc tự bịa kind/base_tick cho fixture hiện có. Equality replica là điều kiện chấp nhận phép đo: giảm byte mà world cuối/từng tick sai thì không được nhận là cải tiến đạt.

11.6. Metric, đơn vị, công cụ, số lần lặp và cách tái lập

11.6.1. Định nghĩa metric và cửa sổ quan sát

Bảng 11.6. Metric và cách diễn giải.

Metric	Định nghĩa/đơn vị	Công cụ hoặc bằng chứng	Giới hạn
Byte full/delta	`F = Σ full_frame_bytes`, `D = Σ delta_frame_bytes`, byte	D0/D1 sau PacketEncoder	Null update tính 0; giữ cả full định kỳ
Tỷ lệ giảm	`100 × (1 − D/F)` %, khi F > 0	Tính từ tổng byte cùng trace	Không lấy trung bình tỷ lệ giảm từng packet
Snapshot rate	`(full_snapshots + delta_snapshots) / T`, message/s	CSV runner, T là window_seconds thực đo	Snapshot nhận/được poll, không bảo đảm mọi snapshot đã APPLIED
Message rate	`tx_packets/T`, `rx_packets/T`, message/s	CSV theo peer	Message ứng dụng, không phải số TCP segment

Metric	Định nghĩa/đơn vị	Công cụ hoặc bằng chứng	Giới hạn
Throughput ứng dụng	`tx_bytes/T`, `rx_bytes/T`, byte/s	CSV theo peer	Có framing/envelope, chưa tính TCP/IP
Ping RTT	Thời gian startPing đến xử lý PONG trên cùng client, ms	LatencyMonitor, mean/min/p95/max và sample count	Bao gồm poll/scheduling, không chia đôi để tuyên bố one-way latency
Input ACK delay	Từ ghi timestamp gửi MOVE đến poll snapshot APPLIED có ACK ≥ sequence, ms	move_ack_samples/mean/p95	Một snapshot có thể xác nhận nhiều input; không phải render latency
Plant observation	Từ yêu cầu đặt bom đến replica thấy bom của local session, ms	plant_ack_ms	Không có ACK placement packet riêng; ít mẫu ở mode bombs
Pending cuối cửa sổ	pending_ping_end, pending_move_end, số request	CSV runner	Chưa hoàn tất ở mốc kết thúc, không tự kết luận packet loss
Stale/resync	Số snapshot cũ/yêu cầu full trong cửa sổ	stale_snapshots, resync_requests	Phân tích nguyên nhân, không thay bằng tỷ lệ loss mạng
Tính đúng	Pass/fail assertion, equality replica và đúng result	Log test/harness	Không suy từ throughput cao
Cleanup/error	status, error, cleanup_ok và exit code	CSV/log/thread assertion	Local kiểm tra server; remote runner không tự chứng minh server đã dừng sạch

T trong các tỷ suất là thời gian đo thực tế bằng nanoTime, không lấy cố định `--seconds` nếu process bị trễ. Byte runner đếm ở send/poll ứng dụng, không phải timestamp capture NIC. Mẫu nằm sát cuối cửa sổ có thể còn pending; giữ số pending để tránh chỉ báo latency của các request hoàn tất nhanh. Nếu một peer không có RTT/ACK sample, ghi thiếu dữ liệu hoặc FAIL theo runner, không thay bằng 0 ms. P95 trong LatencyMonitor dùng nearest-rank: sắp n mẫu và chọn vị trí `ceil(0,95n)`, đánh số từ 1.

Không cộng cả TX phía client với RX phía server để báo lượng traffic duy nhất của cùng message. Trong nhóm này tổng TX hoặc tổng RX của client là metric tổng hợp riêng có tên rõ ràng. Trung bình mean giữa peer chỉ hợp lệ khi ghi đó là mean theo peer; muốn pooled mean cần trọng số số mẫu, còn pooled p95 cần raw sample. CSV hiện chỉ có thống kê tổng hợp, nên không tái dựng pooled p95 bằng cách lấy trung bình các p95. Chỉ số correction local, frame time/FPS, CPU/RAM và queue rejection không tự có trong CSV runner hiện tại; nếu đo bổ sung cần instrumentation/log riêng và tách khỏi các cột sẵn có.

11.6.2. Số lần chạy và quy tắc tổng hợp

Bảng 11.7. Kế hoạch số lần chạy, chưa phải số lần đã hoàn thành.

Nhóm	Cấu hình	Lặp độc lập	Cửa sổ/chú ý
TCP T1	9 ô: transport 1/2/4; idle/movement/bombs 2/4	5/ô	Warm-up 2 s; đo 10 s; process runner mới cho mỗi vòng suite
TCP T2	Cùng 9 ô, khi đủ thời gian triển khai server process riêng	5/ô	2 s + 10 s; tách label với T1
LAN T3	Ưu tiên movement/bombs 2/4; thêm transport nếu thực hiện	5/ô thực sự đo	Ghi số máy và runner/peer ở mỗi máy
D0	Fixture 200 tick hiện có	5	Kiểm tra byte lặp lại; không coi là năm thử nghiệm latency

Nhóm	Cấu hình	Lặp độc lập	Cửa sổ/chú ý
D1	2/4 player × idle/movement/bomb-block-item	5/trace sau khi có harness	600 tick; trace xác định, full mỗi 100 tick
F01–F09	Suite/check tổng và nhóm concurrency/fault/lifecycle	1 tổng; 3 lần nhóm nhạy thời gian	Ghi từng attempt, lỗi và exit code
F10	GUI hai người và bốn người nếu thực hiện	3 lượt/kịch bản	Biên bản người kiểm tra, không chỉ ảnh màn hình

T1 chính có 45 scenario-run; vì CSV có một hàng cho mỗi client, năm vòng suite đầy đủ dự kiến 125 hàng, không phải 125 lần chạy độc lập. Một suite gồm 9 scenario tuần tự, mỗi scenario local tạo server/client mới; chúng không phải 9 scenario chạy song song. Thứ tự suite cố định là một hạn chế cần ghi nhận. Các nhóm T2/T3/D1 là bổ sung có điều kiện; không tính số dự kiến vào Results nếu chưa chạy. Năm lần lặp là lựa chọn thiết kế của nhóm để quan sát độ ổn định, không phải con số bắt buộc do giảng viên đưa ra.

Cho mỗi ô, báo số attempt, số thành công/thất bại và cleanup trước khi tổng hợp latency/byte. Với dữ liệu hiệu năng hợp lệ, báo median và min–max của metric theo lần chạy, kèm từng hàng gốc. Có thể tính một chỉ số tổng hợp peer trong từng run trước khi lấy median qua năm run, nhưng phải nêu công thức. Không coi các ping, peer hoặc tick cùng run là các lần lặp độc lập. Giữ FAIL trong dữ liệu; lần chạy lại dùng attempt ID mới và không ghi đè. Nếu thay code/cấu hình để sửa lỗi, tạo đợt đo mới, chạy lại hai nhánh đối chứng và ghi lý do thay đổi.

11.6.3. Lệnh tái lập và kiểm soát classpath

Các ví dụ sau dành cho PowerShell ở root repository, **chưa được thực thi trong nhiệm vụ viết này**. Máy cần JDK 25 và Maven đã cài; JAVA_HOME/BOMBIT_MVN cấu hình theo máy. scripts/test.cmd chạy Maven rồi Check TV2/NetworkTest main khi các bước trước đạt; scripts/check-tv2.cmd có thể chạy riêng để biên dịch check và xuất D0.

```
# Lưu output và exit code vào thư mục run trước khi phân tích.
cmd /c scripts\test.cmd
cmd /c scripts\check-tv2.cmd

# Build production từ source; chỉ đo sau khi build trả exit code 0.
cmd /c scripts\build-demo.cmd
```

Ví dụ một vòng suite T1, với classpath chỉ định rõ để không lấy nhầm JAR cũ:

```
$experimentJava = if ($env:JAVA_HOME) {
    Join-Path $env:JAVA_HOME 'bin/java.exe'
} else { 'java' }
$experimentRevision = git rev-parse HEAD
$experimentRun = 'T1-r01'  # Đổi r01..r05; lưu dirty/diff riêng trong manifest.
$experimentDir = Join-Path 'report/evidence/experiments' $experimentRun
New-Item -ItemType Directory -Force -Path $experimentDir | Out-Null
& $experimentJava -cp 'target/demo-classes' com.bombit.tools.NetworkBenchmark `
    --local --mode suite --seconds 10 --warmup 2 `
    --label 'single-jvm-loopback' --revision $experimentRevision --run-id $experimentRun `
    --csv (Join-Path $experimentDir 'runner.csv') *> (Join-Path $experimentDir
'runner.log')
```

```
$LASTEXITCODE | Set-Content (Join-Path $experimentDir 'exit-code.txt')
```

Tạo r02–r05 bằng các process Java mới và thư mục mới, giữ log nguyên trạng. Không chạy lại với cùng thư mục/ID rồi để append tạo bộ dữ liệu không phân biệt attempt. CLI runner hiện giới hạn seconds 1–300, warmup 0–30; bombs/suite cần ít nhất 4 giây đo, gameplay cần 2/4 người. Thiết kế 10 giây nằm trong các giới hạn này. run-benchmark.cmd là wrapper tương đương nếu đã xác nhận đúng artifact/classpath.

D0 có thể lấy từ `target/delta-compression-results.csv` do check-tv2 tạo, nhưng phải sao lưu trước lần check tiếp theo. Sau check compile thành công, chạy riêng fixture để lưu mỗi attempt:

```
& $experimentJava -cp 'target/tv2-check' `
    com.bombit.server.game.movement.DeltaCompressionCheck `
    *> (Join-Path $experimentDir 'delta-raw.log')
$LASTEXITCODE | Set-Content (Join-Path $experimentDir 'delta-exit-code.txt')
```

D0 dùng thư mục/ID riêng thay cho T1-r01 trong ví dụ. Để thực hiện T2, khởi chạy ServerApplication trong process riêng rồi bỏ `--local`, truyền `--host 127.0.0.1 --port 8888 --clients N --mode MODE` cho runner. T3 thay host bằng IP LAN và lưu log server/runner từng máy; nếu chia peer ra nhiều runner, dùng expected-clients/room-id như 11.2. Đây là hướng dẫn bố trí, không xác nhận server remote/LAN đã được chạy.

11.6.4. Hồ sơ bằng chứng và điều kiện chấp nhận

Mỗi run lưu ngoài `target` để không mất khi clean. Cấu trúc đề xuất:

```
report/evidence/experiments/<batch>/<run-id>/<machine-id>/
    manifest.json           revision, dirty, artifact/source hash, hardware, topology,
rules
    commands.txt            cwd, lệnh đầy đủ, giờ bắt đầu/kết thúc
    runner.csv              một process sở hữu file
    runner.log              stdout/stderr nguyên trạng
    server.log              nếu server là process do máy này sở hữu
    exit-code.txt
    delta-raw.log           chỉ với nhóm D
    trace.csv               chỉ khi harness D1 đã xuất trace
    screenshots/            nếu nghiệm thu GUI
    checksums.sha256        hash các file evidence sau khi kết thúc
```

Tên `<batch>/<run-id>/<machine-id>` tránh trộn nguồn giữa máy và lần lặp; ví dụ ngắn T1-r01 ở trên có thể đặt bên trong batch tương ứng khi tổ chức đợt đo. Manifest ghi loại run, mode, số client thực tế, seed hoặc lý do không có seed, cấu hình luật, thời lượng yêu cầu/thực đo, sample count, trạng thái và file gốc. Raw CSV giữ nguyên; dữ liệu tổng hợp và script phân tích nằm thư mục khác, có hash và lệnh chạy riêng. CSV theo BenchmarkCsv là UTF-8, header cố định; không ghép trực tiếp file khác header hoặc gộp hàng từ nhiều revision mà bỏ metadata.

Một run hiệu năng chỉ hợp lệ khi setup đúng roster/topology, có mẫu phù hợp, không mất baseline chưa phục hồi, không có lỗi runtime/runner và cleanup đạt theo phạm vi công cụ kiểm tra. Lần FAIL vẫn được lưu để phân tích reliability, không xóa khỏi mẫu attempt. Remote runner chỉ xác minh cleanup client do nó sở hữu; server cần log/assertion riêng. Đối chứng D cần thêm equality replica từng tick và cùng trace/codec/byte scope. GUI cần người kiểm tra ký nhận kết quả quan sát, không suy từ headless PASS.

Hồ sơ đợt khảo sát ban đầu ghi nhận 133 JUnit test với 132 pass, 1 process smoke failure, các check/main được chạy riêng; CSV ngày 22/10 thuộc revision lịch sử khác. Những dữ liệu đó có thể được trình bày với đúng ngày/revision ở Results, nhưng chưa đáp ứng số lần lặp mới trong bảng 11.7. Chương 12 sẽ tách dữ liệu đã thu, lỗi còn tồn tại và nhóm chưa thực hiện khỏi thiết kế thực nghiệm này.

12. Kết quả — Results

Phân công tổng hợp: TV4; TV1 xác nhận phòng/kết nối, TV2 xác nhận movement/runtime/đồng bộ, TV3 xác nhận gameplay. Mỗi thành viên chịu trách nhiệm giải thích assertion và log của phần mình.

Ngày 26/10/2026, nhóm chạy lại bộ test từ source tại revision `e333050997c5dbf78ec4b9b42a9bc9a73ce6a0d5`, không thay production code, test, luật game hoặc timeout. Maven dùng JDK 25 tại máy khảo sát và dependency cache offline. Log và XML mới được sao lưu ngoài target tại hồ sơ đợt kiểm thử chức năng. Lượt suite đầy đủ này là một lần thực thi tự động; các lần lặp mạng/reliability bổ sung được trình bày ở 12.4/12.6. Nghiệm thu GUI/LAN của thiết kế mục 11 chưa được thực hiện.

Bảng 12.1. Kết quả thực chạy và phạm vi đếm.

Lượt	Kết quả thực tế	Cách đọc
Gọi wrapper test trong sandbox	Dừng trước khi chạy test do không tạo được local Maven repository	Lỗi môi trường thực thi; không tính là test case fail
Maven offline, gọi trực tiếp sau khi được phép truy cập môi trường	38 class, 133 test; 133 pass, 0 failure/error/skipped; BUILD SUCCESS; Maven báo 39,743 s	Tổng suite, có cả test công cụ/protocol, không phải 133 kịch bản GUI
Check TV2 độc lập	Exit 0; bốn nhóm check API, movement, runtime và delta đều PASS	Không cộng bốn dòng PASS vào số 133 JUnit
NetworkTest main độc lập	Exit 0; codec/framing/validation/router/metrics/TCP loopback PASS	Main độc lập, không cộng vào số JUnit
Khảo sát đợt khảo sát ban đầu, cùng revision	133 test, 132 pass, 1 failure; smoke đa tiến trình thất bại cả lần chạy riêng	Dữ liệu lịch sử được giữ; một lần PASS mới không chứng minh đã sửa nguyên nhân

Nguồn tổng hợp: junit-summary.json, Maven log, TV2 log, NetworkTest log và khảo sát trước. Số giây Maven là thời gian công cụ báo cho build/test, không phải latency gameplay hay throughput.

Trong các bảng dưới đây, "đạt" nghĩa assertion của lớp test được chạy ở lượt mới đều đạt. Một dòng có thể gom nhiều assertion; số test của một lớp không phải số tính năng độc lập. Các lớp tích hợp có thể xuất hiện ở nhiều bảng vì kiểm tra nhiều phần của cùng luồng; không cộng lại số của các bảng để tính tổng suite.

Phân công: TV1; TV4 phối hợp contract và đối chiếu log TCP.

12.1. Chức năng phòng/kết nối

Bảng 12.2. Kết quả phòng, session và lobby.

Nguồn test	Đầu vào/kịch bản thực thi	Điều kiện đã kiểm tra	Kết quả lượt mới
RoomManagerTest	Tạo phòng, thêm guest, lấy snapshot roster	Đủ hai thành viên; snapshot không cho sửa cấu trúc	1/1 đạt
SessionManagerTest	Các fixture quản lý phiên trong lớp	Quản lý session theo assertion đã khai báo	3/3 đạt; không suy tính năng resume trận
LobbyContractTest	Payload/request lobby hợp lệ và không hợp lệ	Encode/decode và kiểm tra contract	4/4 đạt

Nguồn test	Đầu vào/kịch bản thực thi	Điều kiện đã kiểm tra	Kết quả lượt mới
LobbyLifecycleTest	Phòng bốn người, người thứ năm; quyền/ready/start; hai phòng; chat/resync; concurrent start; leave/disconnect; chơi lượt mới cùng TCP	Roster/quyền authoritative, room tách biệt, match epoch mới, quay lại phòng và cleanup	7/7 đạt qua TCP loopback
ChatFaultTest	Fixture lỗi luồng chat	Hành vi từ chối/cleanup theo assertion	1/1 đạt

Nguồn: RoomManagerTest, SessionManagerTest, LobbyContractTest, LobbyLifecycleTest và ChatFaultTest. Trạng thái thực chạy từng lớp nằm trong summary/XML của lượt mới.

Đáng chú ý, `fourPlayerRoomRejectsFifthAndCanonicalRequestsWork` kiểm tra capacity qua đường request thật; `independentRoomsChatMovementResyncAndCleanup` kiểm tra nhiều chức năng cùng ranh giới phòng; `realClientControllersReturnToRoomAndReplayOnSameTcpConnections` dùng controller ứng dụng để kiểm tra vòng đời. Những kết quả này hỗ trợ kết luận lobby/room đã tích hợp qua TCP trong các fixture đã chạy. Chúng chưa chứng minh bốn người ở bốn máy thao tác giao diện thành công, hoặc reconnect khôi phục trận đang chơi.

Phân công: TV2; TV4 phối hợp packet và các kiểm tra tích hợp.

12.2. Movement/runtime/client synchronization

12.2.1. Movement, collision và ranh giới runtime

Bảng 12.3. Kết quả movement và runtime.

Nguồn test/check	Tình huống kiểm tra	Kết quả assertion tiêu biểu	Kết quả lượt mới
RuntimeContractTest → RuntimeContractCheck	Queue nhỏ bị đầy, hai producer, hai runtime, STOP, wall/block/biên, player chết, extension lỗi	Reject được đếm; giữ thứ tự từng producer; đủ 200 command identity; A dừng không làm B dừng; snapshot cũ giữ nguyên; thread được join	1/1 JUnit đạt; check riêng cũng PASS
MovementFeatureTest → MovementFeatureCheck	Hai bước prediction, ACK/replay, interpolation, collision bom và runtime/TCP	Replay chỉ input chưa ACK; giữ endpoint ngoài buffer; đi ra ô bom được nhưng không đi vào lại; không chạy tiếp vô hạn khi thiếu mẫu input	1/1 JUnit đạt; check riêng cũng PASS
MovementBalanceTest	Fixture cân bằng hitbox/tốc độ và movement	Các assertion cấu hình/chuyển động của lớp đạt	4/4 đạt
MovementDemoTest	Hai client MOVE/STOP/biên; hard/soft wall qua TCP; resync/rate gate; roster và cleanup	Client nhận state authoritative; controller nhận full và tự yêu cầu resync theo fixture	8/8 đạt
RespawnPredictionTest	Respawn local và remote	Xóa input pending cũ nhưng không dùng lại sequence; remote teleport thay vì nội suy xuyên map	2/2 đạt

Nguồn: RuntimeContractCheck, MovementFeatureCheck, MovementBalanceTest, MovementDemoTest và RespawnPredictionTest. RuntimeContractTest và MovementFeatureTest là wrapper JUnit cho check tương ứng, vì vậy việc chạy lại check độc lập không tạo thêm coverage độc lập tương đương một nhóm tính năng mới.

FIFO được kiểm tra theo enqueue/thứ tự producer, không phải tuyên bố thứ tự thời gian tuyệt đối giữa hai thread. Các mốc vị trí được assert trong fixture với map/tốc độ cụ thể; không suy mọi tốc độ/map đều không có lỗi va chạm. Prediction/interpolation được kiểm tra bằng giá trị state, chưa có phép đo độ mượt, correction nhìn thấy hoặc FPS GUI.

12.2.2. Snapshot và khôi phục replica

Bảng 12.4. Kết quả kiểm tra đồng bộ.

Nguồn	Tình huống	Kết quả lượt mới và phạm vi
IntegrationContractTest	Contract packet/match scope và dữ liệu snapshot theo fixture	8/8 đạt
DeltaCompressionTest → DeltaCompressionCheck	Full đầu, delta tile/entity, xóa entity/player, build chưa commit, bỏ update, full phục hồi và full cũ	1/1 JUnit đạt; check riêng PASS
MovementDemoTest	First full và tự resync của client controller, xử lý request theo match/rate gate	Đạt trong 8 test của lớp; không đếm lại thành 8 test mới
MatchLifecycleIntegrationTest	Stats/pickup/result, respawn/shield, full resync và lượt mới qua hai client TCP	5/5 đạt; dữ liệu client/server khớp assertion

Nguồn: IntegrationContractTest, DeltaCompressionCheck và MatchLifecycleIntegrationTest.

Fixture delta kiểm tra build không commit giữ baseline cũ, client phát hiện base tick thiếu và giữ state cũ khi từ chối, sau đó full state phục hồi được replica. Vòng trace còn đối chiếu map, số entity và vị trí x của player với nhánh full trên từng tick của fixture. Không mô tả assertion chọn lọc này là kiểm tra equality mọi field của mọi snapshot v5. Thao tác bỏ update diễn ra có chủ ý trong test replica, không phải đo packet loss trên TCP. Phần byte được in cùng check sẽ được phân tích riêng ở 12.5, tránh trộn kiểm chứng tính đúng với kết luận hiệu năng.

Phân công: TV3; TV2 phối hợp world adapter và runtime, TV4 phối hợp wire contract.

12.3. Gameplay bomb/item/lives/result

Bảng 12.5. Kết quả gameplay và tích hợp presentation.

Nguồn test	Kịch bản chính	Điều kiện đạt theo assertion	Kết quả lượt mới
BombSystemTest	Trùng ô bom, block dừng tia, chain và death event	Mỗi bom nổ một lần; block bị phá đúng; death event theo fixture	4/4 đạt
BombGameplayTest	Các fixture luật placement/explosion	Các nhánh luật được lớp test kiểm tra đạt	10/10 đạt
BombRuntimeIntegrationTest	Loop thật đặt/nổ/xóa entity, room tách kho bom, lỗi callback, hai client TCP	Entity/map/alive được xuất bản; lỗi vẫn cleanup trên game thread; resync thấy trạng thái bom	4/4 đạt
ItemSystemTest	Spawn và pickup qua ItemWorld fixture	Sinh/nhặt và effect/event theo assertion	2/2 đạt
MatchGameplayTest	Drop/pickup cùng ô, upgrade quota, chết cùng tick, last alive, timeout, cap và foreign thread	Một người thắng pickup; không xuất winner trung gian khi cùng chết; rank/reason đúng; adapter từ chối thread khác	5/5 đạt

Nguồn test	Kịch bản chính	Điều kiện đạt theo assertion	Kết quả lượt mới
ExtendedItemsTest	Năm loại item, shield, blast chồng, respawn, LIFE_UP tranh chấp, range và seeded drop	Shield không cộng dồn; một damage/batch; upgrade giữ khi respawn; không hồi sinh người đã bị loại	6/6 đạt
BombClientIntegrationTest	Hai client trên đường giao tiếp bom	Các assertion tích hợp trong hai fixture đạt	2/2 đạt
MatchLifecycleIntegrationTest	Kết quả cuối, replay/disconnect và trạng thái mở rộng	Final snapshot, GAME_OVER và state client theo assertion	5/5 đạt; trùng lớp đã dẫn ở 12.2
BombSceneTest	Snapshot/notice và presentation qua controller	Hình ảnh scene theo state authoritative trong fixture	2/2 đạt
Java2DRendererTest	Vẽ BufferedImage và năm icon item	Có pixel hiển thị; năm icon có chữ ký pixel khác nhau	2/2 đạt; chưa phải ảnh nghiệm thu GUI
TileGridRendererTest / MatchResultsPanelTest	Grid và bảng kết quả theo fixture	Assertion renderer/UI component đạt	Mỗi lớp 1/1 đạt

Nguồn chi tiết: BombSystemTest, BombGameplayTest, BombRuntimeIntegrationTest, ItemSystemTest, MatchGameplayTest, ExtendedItemsTest, BombClientIntegrationTest, BombSceneTest, Java2DRendererTest, TileGridRendererTest và MatchResultsPanelTest.

Các test gameplay dùng world/policy/clock được điều khiển để kiểm tra quy tắc cụ thể; các test TCP bổ sung bằng chứng các lớp đó được ghép vào runtime và client. Sự kết hợp này hỗ trợ kết luận logic bom/item/lives/result đã có kiểm chứng ở cả fixture và một số luồng tích hợp. Nó chưa chứng minh mọi thứ tự nhận input hoặc mọi vị trí respawn đều đã được kiểm tra. Test renderer dùng ảnh trong bộ nhớ, không lưu PNG và không đánh giá bố cục cửa sổ, trải nghiệm điều khiển hoặc khả năng đọc HUD. Không lấy một ảnh dựng lại sau test làm screenshot của một trận mạng đã chạy.

Phân công: TV4 tổng hợp; TV1/TV2/TV3 xác nhận cleanup tài nguyên do module mình sở hữu.

12.4. Concurrency, fault handling và cleanup

12.4.1. Kết quả lifecycle chức năng đã có

Bảng 12.6. Kết quả lifecycle và process smoke.

Nguồn	Nội dung đã chạy	Kết quả lượt mới	Giới hạn kết luận
RuntimeContractTest	Queue, producer, stop từng room và extension failure	1/1 đạt	Không phải stress vô hạn
StabilityLifecycleTest	20 vòng lượt chơi; injected runtime fault; 10 chu kỳ start/stop server	3/3 đạt	Vòng lặp nằm trong fixture, không phải 20 lần thực nghiệm độc lập hoặc soak dài hạn
LobbyLifecycleTest	Concurrent start, replay match epoch, disconnect/host transfer và start failure	7/7 đạt	Luồng TCP loopback đã kiểm tra, chưa phải LAN
MovementDemoProcessTest	Một server và hai headless client ở ba Java process	1/1 đạt; hai client READY players=2, FINAL ack=78	Profile demo/headless; không thay GUI/lobby nhiều máy

Nguồn: StabilityLifecycleTest, MovementDemoProcessTest, client A log, client B log và server log. Hai client cùng ghi match ID `f69d2c20-5415-4906-b5be-733d95b8669e`; đây là định danh của lần chạy, không hard-code vào test. ACK=78 là output quan sát được, không dùng làm ngưỡng hiệu năng hoặc mong đợi cố định cho các lần chạy sau.

Process smoke trước đây thất bại với `No full snapshot before smoke deadline`, kể cả lần retry ở đợt khảo sát ban đầu. Lần mới pass với source không đổi và assertion/timeout không đổi. Do điều kiện môi trường/chạy process khác nhau có thể ảnh hưởng, kết luận hiện tại là **chưa tái hiện lỗi trong lần chạy mới**, chưa xác định nguyên nhân và chưa có bằng chứng một bản sửa đã giải quyết lỗi. Các lần lặp bổ sung được trình bày ở 12.4.2, giữ riêng với lượt suite đầy đủ và log lịch sử.

12.4.2. Ba lượt concurrency/fault/lifecycle độc lập

Sau nhóm benchmark mạng, bảy lớp test được chạy lại bằng ba Maven invocation độc lập, giữ nguyên assertion, timeout và source. Mỗi lượt có 28 test; XML mới và log ba process smoke được lưu riêng trước lần tiếp theo. Các lượt này không chạy đồng thời với cửa sổ đo latency.

Bảng 12.7. Test concurrency/reliability theo lớp và số lần lặp.

Class	Test/lượt	Số lượt	PASS / tổng lần thực thi test	Điều kiện đã kiểm tra
RuntimeContractTest	1	3	3/3	FIFO/2 producer, queue đầy, room isolation, snapshot và stop/failure
SessionWriterTest	3	3	9/9	Budget byte, queue đầy, validation trước enqueue
TransportStabilityTest	6	3	18/18	Heartbeat, absolute frame deadline, blocked writer, lỗi handler và rate limit
ClientStabilityTest	7	3	21/21	Pull/UI queue đầy, byte budget, listener lỗi, write deadline và cleanup
StabilityLifecycleTest	3	3	9/9	20 lượt trận, lỗi runtime được inject và 10 chu kỳ server start/stop trong fixture
MovementDemoProcessTest	1	3	3/3	Server và hai headless client ở ba process riêng
LobbyLifecycleTest	7	3	21/21	Capacity/quyền, concurrent start, replay, disconnect và lỗi start
Tổng	28	3	84/84	Không cộng các lần lặp này vào 133 test của suite đợt kiểm thử chức năng

Nguồn: reliability-summary.json, RuntimeContractCheck, SessionWriterTest, TransportStabilityTest, ClientStabilityTest và StabilityLifecycleTest. Các lớp còn lại đã dẫn ở 12.1/12.4.1. Raw log/XML của từng attempt nằm trong hồ sơ đợt thực nghiệm mạng/reliability.

Ở RuntimeContractCheck, hai producer gửi tổng 200 command; assertion kiểm tra đủ identity và thứ tự từng producer. Fixture queue nhỏ kiểm tra submit không vượt capacity và counter reject tăng. SessionWriter/ClientStability chủ động làm chậm consumer, đầy queue hoặc vượt byte budget, rồi kiểm tra kết nối đóng/báo lỗi và tài nguyên được giải phóng thay vì âm thầm bỏ snapshot. TransportStability kiểm tra peer gửi frame dở không kéo dài absolute deadline, writer bị chặn vẫn timeout dù peer tiếp tục PING và client khỏe khác vẫn dùng được.

Các exception như `injected runtime fault` hoặc `injected handler failure` là đầu vào chủ động, không phải lỗi ngoài dự kiến của lượt test. Test PASS nghĩa hệ thống đã phản ứng theo assertion của fixture. Một số deadline của fixture được rút ngắn để kiểm tra nhánh timeout; không dùng chúng làm thông số timeout production. 20 lượt trận hoặc 10 chu kỳ server bên trong một test không phải 20/10 run hiệu năng độc lập, và không thay soak dài hạn hoặc profiler chứng minh không còn mọi memory leak.

12.4.3. Lịch sử lỗi và lỗi môi trường thực thi

Smoke ba process đạt cả ba lần lặp mới, ngoài một lần đạt ở đợt kiểm thử chức năng. Tuy nhiên lỗi `No full snapshot before smoke deadline` của đợt khảo sát ban đầu vẫn chưa có nguyên nhân được xác minh; không có bản sửa source/assertion/timeout để gán kết quả PASS mới cho một fix cụ thể. Kết luận giới hạn là lỗi không tái hiện trong bốn lần chạy mới đã lưu, không phải xác suất lỗi bằng không.

Lần đầu gọi script reliability bị policy chặn trước khi script chạy. Lần gọi PowerShell tiếp theo truyền sai JAVA_HOME nên ba invocation Maven đều dừng ở bước môi trường, chưa chạy test và chưa sinh XML mới. Sau khi sửa cách truyền biến môi trường của process, ba lượt thực thi test ở bảng trên hoàn tất. Những log lỗi chuẩn bị vẫn được giữ riêng, không tính chúng là test case fail hoặc xóa để tạo bộ bằng chứng chỉ có PASS.

Phân công: TV2 thiết kế trace, baseline và kiểm tra replica; TV4 chạy lặp, lưu dữ liệu, tính tổng và vẽ biểu đồ. TV3 đối chiếu ý nghĩa dữ liệu bom/item/frame.

12.5. Full so với delta

12.5.1. Phép đo thực hiện và đối chứng

Ngày 26/10/2026, thực nghiệm chạy năm process Java mới cho fixture D0 và năm process cho harness D1; mỗi process D1 chạy sáu cấu hình tuần tự, tổng cộng 35 scenario-run. Không sửa source production hoặc bật nén ở transport. DeltaExperiment.java và analyze_delta.py là công cụ thực nghiệm nằm trong thư mục báo cáo. Raw data, log, trace và bảng tổng hợp được giữ tại hồ sơ đợt thực nghiệm delta.

D0 chạy lại DeltaCompressionCheck với 200 tick, map trống 17 × 15 và một player, dùng snapshot v2 không match envelope. D1 dùng arena 17 × 15, 2/4 player, 600 tick, BombFrame/MatchFrame để encoder phát v5, cùng match UUID cố định cho hai nhánh và framing PacketEncoder. Mỗi tick lấy cùng một GameState làm đầu vào: B0 tạo full, B1 tạo full/delta hoặc bỏ tick rỗng. Full bắt buộc ở tick 1 và các bội 100; D0 có ba full, D1 có bảy full. Không có full do mất baseline, queue đầy hoặc disconnect trong phép đo này.

Byte được đo là frame ứng dụng hoàn chỉnh, gồm length/type, JSON/Base64 và envelope nếu có; không gồm TCP/IP, retransmission hoặc traffic notice riêng. Không so trực tiếp tổng D0 với D1 như hai cách encode cùng thời lượng: chúng khác phiên bản, số player và số tick. Mỗi tỷ lệ giảm chỉ so B0 và B1 trong cùng một hàng cấu hình.

12.5.2. Trace D1 và kiểm tra tính đúng

Ba trace D1 được dựng xác định, không dùng RNG:

- **Idle:** không phát movement input, MatchFrame giảm remainingSeconds mỗi 20 tick.

- **Movement:** tất cả player nhận input sequence tăng mỗi tick, đi ngang theo hướng đổi sau 40 tick, dùng MovementSystem với dt=0,05 giây; ACK vì vậy thay đổi liên tục.
- **Bom/block/item theo script:** movement ở tick 1–80, STOP ở tick 81; thêm bom tại (3,2) ở tick 100; tick 160 xóa bom, phá block (3,3), thêm item và frame lửa; tick 172 bỏ frame lửa; tick 220 xóa item và tăng bombLimit player 1. Những tick còn lại chủ yếu chỉ thay timer trận.

Trace thứ ba trực tiếp dựng thay đổi state để đo kích thước serialization, không chạy BombSystem/ItemSystem và không kiểm tra player có thật sự đứng ở ô pickup. Không diễn giải nó thành log của một trận gameplay tự nhiên. Luật placement/damage/pickup thực tế có bằng chứng kiểm thử riêng ở 12.3. Mỗi trace lưu full frame dạng Base64 theo từng tick cùng nhãn sự kiện, đủ để đối chiếu lại dữ liệu đầu vào.

Harness D1 decode cả full và delta qua PacketDecoder, match envelope, snapshot codec rồi áp vào hai ClientWorldState. Sau mỗi tick, kể cả tick B1 không gửi, harness so sánh map/dimensions, match state, mọi field PlayerData được codec truyền, entity, MatchFrame và danh sách explosion. So sánh chuẩn hóa thứ tự player/entity, bỏ metadata tick và serverTimeMillis của BombFrame: replica không nhận packet có thể giữ tick/clock cũ dù state gameplay vẫn đúng. Timestamp bắt đầu/kết thúc và cells của explosion vẫn được giữ trong phép so sánh. Không so notice vì nó không nằm trong payload snapshot.

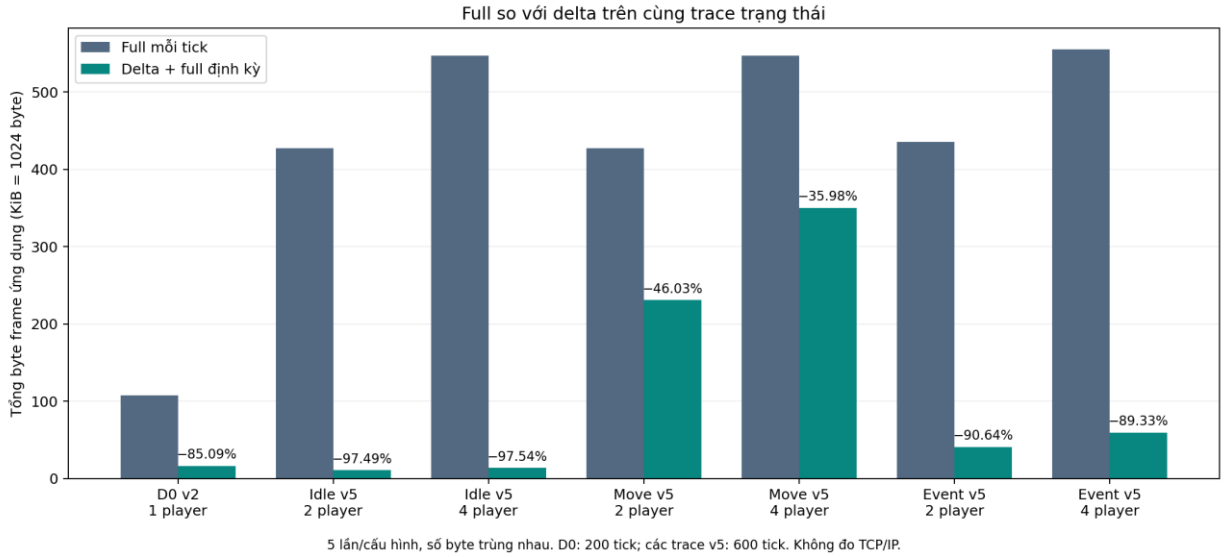
Cả 30 scenario-run D1 hoàn tất, **18.000/18.000 lần so sánh state chuẩn hóa đạt**. D0 cũng hoàn tất đủ năm lần với assertion gốc, nhưng kiểm tra equality của D0 chỉ bao phủ các field fixture đã assert, không được gộp thành kiểm tra mọi field v5. Đây là kiểm tra harness, không làm tăng số 133 JUnit ở 12.1.

12.5.3. Tổng byte và tỷ lệ giảm

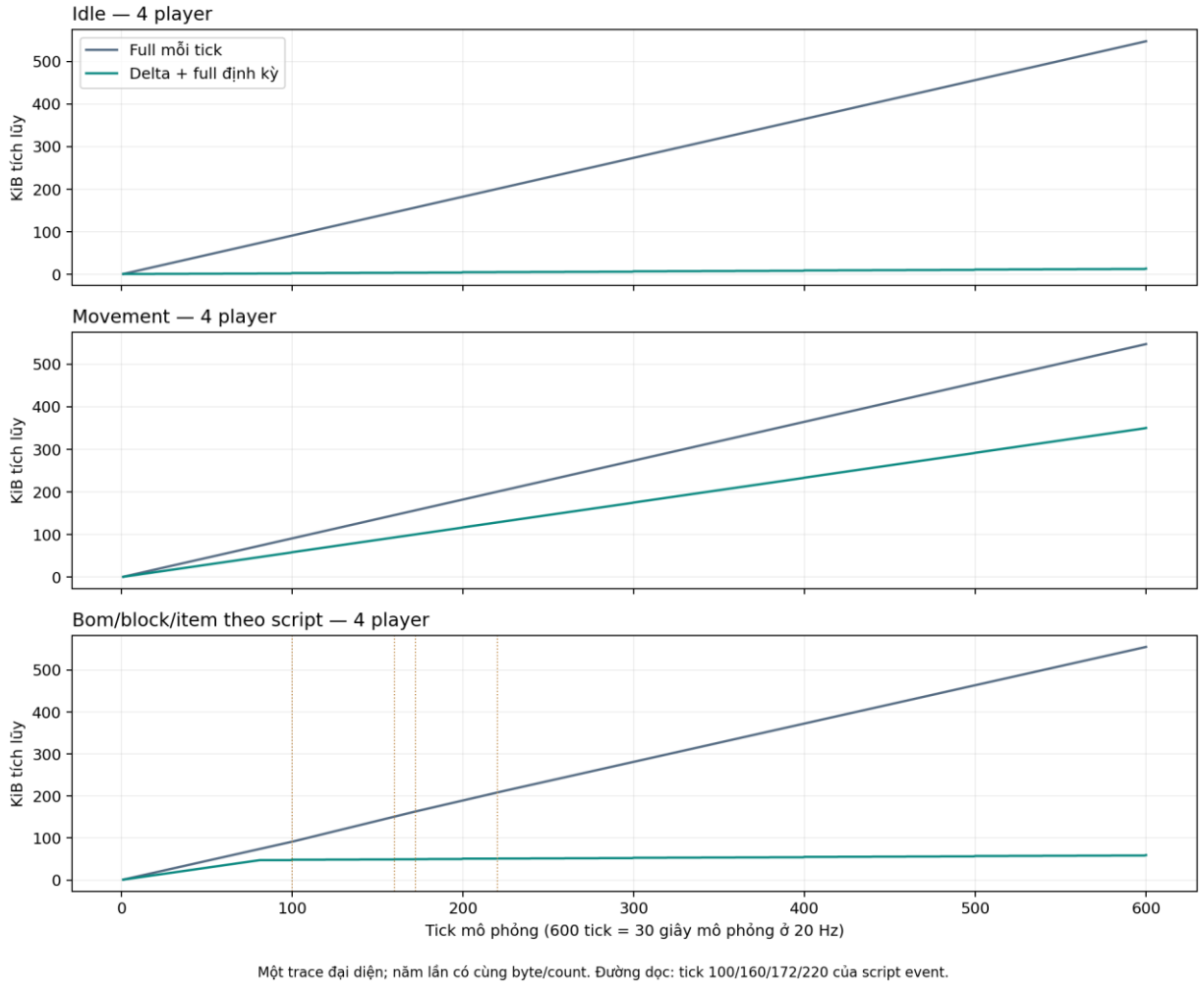
Bảng 12.8. Full so với delta trên cùng trace; số byte của một lần chạy đại diện.

Cấu hình	Tick	Tổng full B0 (byte)	Tổng delta B1 (byte)	Byte tiết kiệm	Tỷ lệ giảm
D0 v2, 1 player	200	110076	16411	93665	85,09%
D1 idle, 2 player	600	438000	11014	426986	97,49%
D1 idle, 4 player	600	560400	13786	546614	97,54%
D1 movement, 2 player	600	438000	236380	201620	46,03%
D1 movement, 4 player	600	560400	358780	201620	35,98%
D1 bom/block/item, 2 player	600	445968	41724	404244	90,64%
D1 bom/block/item, 4 player	600	568368	60648	507720	89,33%

Nguồn: summary.csv và all-runs.csv. Tính theo `100 × (1 − tổng byte B1 / tổng byte B0)`, không lấy trung bình phần trăm giảm từng packet. Với mỗi cấu hình, cả năm lần có cùng tổng byte và số packet; median, min và max tổng byte bằng giá trị trong bảng. Đây là độ lặp của phép đo size trên fixture xác định, không phải bằng chứng sai số thống kê của latency/bandwidth mạng.



Hình 12.1. Tổng frame ứng dụng theo cấu hình; phần trăm ghi trên cột delta là tỷ lệ giảm so với full của cùng hàng. Đơn vị trục là KiB = 1024 byte; D0 có 200 tick, D1 có 600 tick. Nguồn hình: script phân tích dữ liệu thực chạy.



Hình 12.2. Byte tích lũy của các trace D1 bốn player. Đường dọc ở trace sự kiện chỉ tick 100/160/172/220; đường delta vẫn có chi phí full định kỳ và update timer. Trục tick là thời gian mô phỏng, không phải thời gian đo mạng.

Bản vector để ghép DOCX/in: hình 12.1 SVG, PDF, hình 12.2 SVG, PDF.

12.5.4. Số packet và lý do mức giảm khác nhau

Bảng 12.9. Chi phí phát packet và phân biệt byte/tick với byte/packet.

Cấu hình	Full của B1	Delta của B1	Tick không gửi	Byte/tick B1	Byte/packet B1 đã gửi
D0 v2	3	80	117	82,06	197,72
D1 idle, 2 player	7	24	569	18,36	355,29
D1 idle, 4 player	7	24	569	22,98	444,71
D1 movement, 2 player	7	593	0	393,97	393,97
D1 movement, 4 player	7	593	0	597,97	597,97
D1 bom/block/item, 2 player	7	101	492	69,54	386,33
D1 bom/block/item, 4 player	7	101	492	101,08	561,56

Byte/tick lấy tổng byte chia toàn bộ tick, gồm cả tick không gửi. Byte/packet lấy tổng byte chia số packet B1 thật sự phát, gồm full và delta; không phải size trung bình chỉ của DELTA. Do đó không thể dùng 18–23 byte/tick ở idle hoặc 70–101 byte/tick ở trace sự kiện để tuyên bố mỗi gói v5 chỉ có 50–100 byte.

Idle tiết kiệm nhiều vì chỉ phát 31 packet trong 600 tick, gồm cả update timer và full định kỳ. Trace movement phát đủ 600 packet: mỗi player đổi vị trí/ACK liên tục nên builder gửi lại PlayerData, trong khi MatchFrame và metadata vẫn chiếm byte. Khi tăng từ hai lên bốn player, phần PlayerData phải phát lớn hơn; tỷ lệ giảm còn 35,98% dù tổng byte vẫn thấp hơn full. Trace bom/block/item có 492 tick không phát nên mức giảm cao hơn movement; kết quả này chịu ảnh hưởng mạnh của đoạn ít thay đổi sau sự kiện cuối.

12.5.5. Giới hạn và kết luận trong phạm vi đo

Các kết quả chứng minh implementation giảm tổng byte snapshot so với full mỗi tick trên **bảy fixture đã đo**, đồng thời giữ state replica theo các assertion của từng harness. Mức giảm 35,98–97,54% của các trace D1 không phải khoảng đảm bảo cho mọi trận. Không lấy một tỷ lệ trung bình chung cho các workload khác nhau để quảng bá mức tiết kiệm cố định.

Phép đo đồng thời hưởng lợi từ diff và bỏ tick rỗng; chưa có ablation tách riêng đóng góp hai cơ chế. Không đo CPU/GC, thời gian scan map, latency encode/decode hoặc bộ nhớ baseline. Chưa có trace map thay đổi dày đặc, nhiều entity hoặc resync lặp để xác định trường hợp delta có thể lớn hơn full. Builder hiện không tự chọn packet nhỏ nhất sau encode, nên không kết luận delta luôn tối ưu.

D1 là codec harness một process, không chạy queue/loop/network, không có backpressure, không phát notice packet và không nhân traffic cho nhiều connection nhận broadcast. Đối với cùng một danh sách notice ở hai nhánh, thêm cùng số byte notice sẽ làm tỷ lệ giảm của **toàn luồng** thấp hơn tỷ lệ snapshot trong bảng. Muốn kết luận bandwidth/latency hệ thống phải đo TCP riêng ở 12.6. Số 600 tick là 30 giây mô phỏng theo 20 Hz, không phải mỗi process chạy đủ 30 giây thực. Năm lần lặp cho size xác định không thay cho nhóm loopback/LAN nhiều lần của mục 11.

Phân công: TV4 thu thập và tổng hợp; TV1 xác nhận transport, TV2 xác nhận input/ACK/snapshot, TV3 xác nhận workload bom.

12.6. Latency/ACK/throughput/message rate theo workload

12.6.1. Môi trường, số run và quy tắc tổng hợp

Thực nghiệm ngày 26/10/2026 chạy NetworkBenchmark với classpath `target/classes` đã kiểm tra hash, JDK 25.0.1, Windows 11/amd64, 16 logical processors theo metadata runner. Server và client trong cùng JVM nhưng giao tiếp TCP thật qua 127.0.0.1; mỗi scenario tạo server/client mới. Năm suite process độc lập, mỗi suite chín scenario, tạo **45 scenario-run và 125 hàng peer**. Mỗi scenario warm-up 2 giây, đo 10 giây yêu cầu; tỷ suất dùng window_seconds thực đo. Không chạy test Maven hoặc plotting đồng thời với benchmark. CPU model/RAM chưa được xác minh; vì vậy không dùng kết quả này làm mốc hiệu năng phần cứng chuẩn.

Nguồn: started-manifest.json, all-peer-rows.csv, per-run-metrics.csv và network-summary.csv. Không trộn CSV lịch sử ngày 22/10 hoặc dữ liệu codec D0/D1 vào các bảng mạng này.

Trong từng run, mean RTT/ACK được tính từ mean từng peer với trọng số số mẫu; RX/TX byte/s và message/s được cộng theo các client trong cùng cửa sổ. P95 theo run lấy giá trị p95 cao nhất của các peer, gọi là **worst-peer p95**; không gọi nó là pooled p95 của mọi ping/input. Mỗi ô bên dưới trình bày **median (min–max) qua năm run**, không phải confidence interval. Stat đầu vào đã được runner làm tròn ba chữ số thập phân, nên số tổng hợp cũng chịu sai số làm tròn.

12.6.2. Ping RTT và input ACK delay

Bảng 12.10. Latency theo cấu hình, đơn vị ms.

Cấu hình	Run PASS/attempt	RTT mean theo run, ms	Worst-peer RTT p95, ms	ACK mean theo run, ms	Worst-peer ACK p95, ms
transport-1	5/5	1,94 (1,89–2,04)	2,31 (2,12–2,80)	—	—
transport-2	5/5	1,85 (1,80–1,96)	2,42 (2,33–2,52)	—	—
transport-4	5/5	1,70 (1,62–1,95)	2,27 (2,05–3,02)	—	—
idle-2	5/5	1,79 (1,72–2,00)	2,23 (2,09–2,53)	—	—
idle-4	5/5	1,66 (1,63–1,79)	2,26 (2,13–2,52)	—	—
movement-2	5/5	1,74 (1,70–1,78)	2,27 (2,05–2,49)	26,11 (25,68–27,63)	48,34 (47,00–49,32)
movement-4	5/5	1,76 (1,73–1,82)	2,40 (2,32–2,68)	27,07 (26,04–30,11)	49,02 (47,99–50,13)
bombs-2	5/5	1,68 (1,62–1,83)	2,23 (2,09–2,93)	27,22 (26,19–27,95)	49,56 (47,36–50,52)
bombs-4	5/5	1,69 (1,61–1,97)	2,17 (2,11–2,52)	26,22 (25,70–28,62)	49,19 (47,56–49,97)

Mean RTT có median theo cấu hình khoảng 1,66–1,94 ms trong môi trường loopback này; không có xu hướng đơn điệu tăng theo số client và không đủ căn cứ nói thêm client làm latency thấp hơn. RTT nhỏ phản ánh đường TCP cục bộ và polling của runner, chưa đại diện LAN/Wi-Fi hoặc render latency.

Các mode có MOVE đạt median mean ACK khoảng 26,11–27,22 ms và median worst-peer ACK p95 khoảng 48,34–49,56 ms. Khoảng này phù hợp với việc server lấy mẫu theo tick mục

tiêu 50 ms, nhưng phép đo không tách riêng thời gian queue, chờ tick, socket và poll nên không gán toàn bộ độ trễ cho GameLoop. Không chia RTT đôi để suy one-way latency, không dùng ACK cuối process smoke thay ACK duration. Ô không có MOVE được ghi "—", không thay bằng 0 ms.

Mode bombs chỉ có một placement/client/run. Median qua run của median peer plant observation là 53,01 ms ở hai client (min–max 49,20–54,19) và 46,28 ms ở bốn client (45,43–47,30). Đó là thời gian đến khi replica thấy bom authoritative, không có packet ACK placement riêng. Số mẫu ít và khác pha tick, nên không kết luận bốn client đặt bom nhanh hơn hai client.

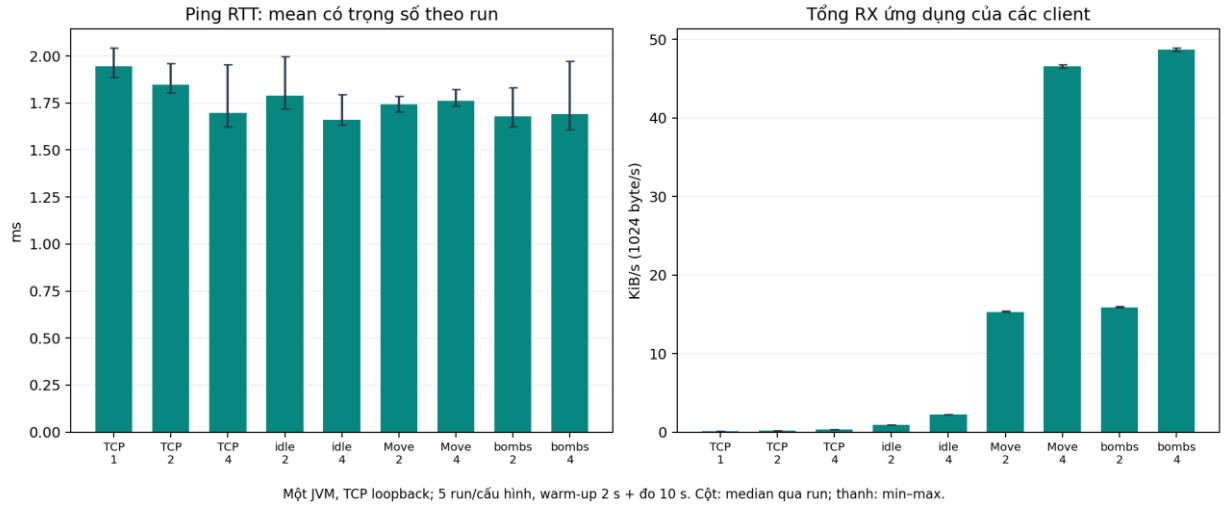
12.6.3. Throughput và message rate ứng dụng

Bảng 12.11. Tổng lưu lượng phía client; median (min–max) theo run.

Cấu hình	Tổng TX KiB/s	Tổng RX KiB/s	TX message/s	RX message/s	Mẫu RTT / mẫu ACK, cả 5 run
transport-1	0,09 (0,09–0,09)	0,09 (0,09–0,09)	5,00 (5,00–5,00)	5,00 (5,00–5,00)	250 / 0
transport-2	0,19 (0,19–0,19)	0,19 (0,19–0,19)	10,00 (10,00–10,00)	10,00 (10,00–10,00)	500 / 0
transport-4	0,37 (0,37–0,37)	0,37 (0,37–0,37)	20,00 (20,00–20,00)	20,00 (20,00–20,00)	1000 / 0
idle-2	0,19 (0,19–0,19)	0,95 (0,95–0,95)	10,00 (10,00–10,00)	12,40 (12,40–12,40)	500 / 0
idle-4	0,37 (0,37–0,37)	2,28 (2,28–2,28)	20,00 (20,00–20,00)	24,80 (24,80–24,80)	1000 / 0
movement-2	4,15 (4,13–4,15)	15,32 (15,32–15,40)	49,59 (49,40–49,60)	49,40 (49,39–49,60)	500 / 1974
movement-4	8,29 (8,25–8,30)	46,57 (46,34–46,81)	99,19 (98,80–99,20)	99,20 (98,80–99,59)	1000 / 3952
bombs-2	4,25 (4,23–4,25)	15,88 (15,87–16,00)	49,80 (49,60–49,80)	50,19 (50,19–50,60)	500 / 1974
bombs-4	8,49 (8,45–8,49)	48,68 (48,46–48,91)	99,59 (99,19–99,60)	102,39 (101,99–102,79)	1000 / 3948

KiB/s là 1024 byte/giây, đo frame ứng dụng sau framing/envelope, không phải tổng byte TCP/IP. Tổng RX là tổng frame mà các client nhận/poll; nếu một snapshot được broadcast cho bốn client, bốn bản nhận được tính bốn lần. Không cộng TX vào RX rồi gọi đó là lượng message duy nhất. Số mẫu cuối bảng là tổng sample count qua năm run, không phải năm lần lặp mới của cùng một peer.

Transport chủ yếu là PING/PONG nên tổng rate tăng gần với số client và nhịp mục tiêu 5 PING/giây/client. Idle vẫn có snapshot do timer trận/full định kỳ, nên RX cao hơn transport dù không gửi MOVE. Movement bốn client có tổng RX median 46,57 KiB/s, so với 15,32 KiB/s ở hai client; đây là lưu lượng do workload tạo ra, không phải khả năng xử lý tối đa của server. Snapshot chứa nhiều PlayerData hơn và được gửi cho nhiều receiver hơn khi roster tăng, nên không kỳ vọng tổng RX luôn tăng tuyến tính theo số player. Mode bombs cao hơn movement ở cùng số client vì có entity/effect/notice, nhưng không đại diện spam bom liên tục.



Hình 12.3. Mean RTT có trọng số và tổng RX client; cột là median qua năm run, thanh là min–max. Nhãn TCP tương ứng mode transport, Move tương ứng movement. Đo một JVM/loopback; không suy thành tải tối đa hoặc LAN.

Bản xuất để ghép DOCX/in: SVG, PDF.

12.6.4. Lỗi, cleanup và request cuối cửa sổ

Bảng 12.12. Trạng thái và số đếm qua năm run/cấu hình.

Cấu hình	FAIL / attempt	Cleanup lỗi	Pending PING cuối cửa sổ, tổng	Pending MOVE cuối cửa sổ, tổng	Stale / resync, tổng
transport-1	0/5	0	0	0	0 / 0
transport-2	0/5	0	0	0	0 / 0
transport-4	0/5	0	0	0	0 / 0
idle-2	0/5	0	0	0	0 / 0
idle-4	0/5	0	0	0	0 / 0
movement-2	0/5	0	0	2	0 / 0
movement-4	0/5	0	0	4	0 / 0
bombs-2	0/5	0	0	4	0 / 0
bombs-4	0/5	0	0	4	0 / 0

45/45 scenario-run PASS, 0/45 FAIL và không có cleanup lỗi theo kiểm tra local runner; cả năm suite process trả exit code 0. Các giá trị status/cleanup ở hàng peer cùng scenario chia sẻ trạng thái run, không phải 125 lần kiểm chứng độc lập. Không có stale snapshot hoặc resync request trong nhóm workload bình thường này; khả năng phục hồi baseline được kiểm tra bằng fixture riêng, không phải do benchmark đã gây mất delta.

Tổng 14 MOVE vẫn pending tại mốc kết thúc của các cửa sổ, còn PING pending bằng 0. Những input chưa ACK không được đưa vào mean/p95 ACK; bảng giữ số pending để lộ giới hạn quan sát này. Runner ngừng đo rồi cleanup, không theo dõi completion của mọi input sau cửa sổ. Do đó không gọi 14 request này là message loss, không tuyên bố loss rate bằng 0 từ số FAIL bằng 0, và không coi các mean ACK chỉ của mẫu hoàn tất là độ trễ của toàn bộ input đã gửi.

Ở bombs-2, tổng năm run quan sát 20 notice đặt bom và 20 notice nổ; ở bombs-4 là 80 và 80, do mỗi peer quan sát bom của mọi player. Đó là số quan sát phía client, không phải 20/80 quả bom

duy nhất; mỗi run thực tế chỉ đặt 2/4 bom. Runner cũng kiểm tra local player còn sống và baseline hợp lệ ở cuối cửa sổ.

12.6.5. Kết luận trong phạm vi chạy được

Trong cấu hình loopback đã đo, hệ thống hoàn tất workload transport 1/2/4 client và gameplay 2/4 player với latency/throughput và trạng thái như các bảng trên; các fixture lỗi lặp lại cũng đạt assertion. Đây là bằng chứng cho những mức tải và điều kiện cụ thể, không phải chứng nhận reliability tổng quát. Không đo trên nhiều máy, mạng có jitter/loss chủ động, tải vượt bốn client, trận dài hạn, CPU/GC hoặc tài nguyên phần cứng đầy đủ. Warm-up hiện tại chủ yếu poll/PING, chưa làm nóng mọi đường movement/bom. Suite có thứ tự mode cố định, không randomize và không ước lượng confidence interval từ năm run. Các giới hạn này được giữ khi diễn giải novelty và đề xuất mở rộng ở các chương sau.

Phân công: cả nhóm xác nhận; TV4 theo dõi bằng chứng còn thiếu.

12.7. Test chưa đạt và phần chưa có bằng chứng

Bảng 12.13. Bằng chứng cần bổ sung trước khi nộp.

Nội dung	Hiện trạng	Log/ảnh/biên bản cần thêm	Phụ trách
Lỗi smoke lịch sử	Đạt một lượt đợt kiểm thử chức năng và ba lượt đợt thực nghiệm mạng/reliability; lỗi cũ chưa có nguyên nhân được xác minh	Giữ log fail/pass; cần điều tra nguyên nhân nếu muốn kết luận đã xử lý flakiness	TV2/TV4
GUI phòng hai/bốn người	Test controller/TCP đạt; chưa có biên bản người thao tác	Ảnh roster/ready/start/chat; ghi run/match, tên máy, người kiểm tra và kết quả từng bước	TV1/TV4
Movement và mất focus	Logic/state test đạt; chưa nghiệm thu điều khiển GUI	Video/phân đoạn log nhấn/thả/mất focus, chạm tường/block/biên và STOP; không suy từ ảnh tĩnh	TV2
Bom, item, shield/respawn và HUD	Fixture/integration đạt; chưa có ảnh trận mạng nghiệm thu	Ảnh trước/sau placement, block/item, lives/shield/respawn và HUD cùng match; log server/client tương ứng	TV3/TV2
Kết quả và lượt chơi lại	Test authoritative result/replay đạt	Ảnh kết quả hai client, trở lại phòng và match mới; đối chiếu result DTO/log	TV1/TV3/TV4
LAN nhiều máy	Chưa thực hiện trong lượt này	Topology/hardware/IP/port, lệnh, CSV từng process, log và ảnh hai đầu	Cả nhóm
Lặp timing/concurrency/ fault	Đã hoàn thành ba lượt, 84/84 lần thực thi test đạt	Đã lưu XML/log từng attempt; bổ sung soak hoặc tải khác nếu mở rộng kết luận	TV4 và owner module
Hiệu năng full/delta và mạng	Đã đo codec D0/D1 và 45 scenario-run TCP loopback; chưa đo LAN hoặc tải dày đặc	Đã lưu CSV/sample/pending/cleanup; cần LAN/trace thực tế nếu mở rộng	TV2/TV4

Không có failure ở suite đợt kiểm thử chức năng hoặc các lượt test đợt thực nghiệm mạng/reliability; điều đó không xóa lịch sử fail hoặc các nhóm chưa nghiệm thu. Lỗi Maven repository, execution policy và JAVA_HOME ở bước chuẩn bị được lưu như lỗi môi trường/cách gọi lệnh, không trộn vào thống kê test case. Không sửa test/source để đổi kết quả và không tạo ảnh giả cho kịch bản chưa chạy. Các ảnh/video sẽ gắn caption gồm thời điểm,

topology, run/match ID và điều kiện quan sát; ảnh thiếu log đối chiếu chỉ minh họa giao diện, chưa chứng minh đồng bộ hay cleanup.

13. Thảo luận — Discussion

Phân công: TV2 phân tích full/delta, tick, ACK và đồng bộ; TV4 tổng hợp metric, concurrency/reliability và giới hạn suy luận; TV1 đối chiếu transport/lobby; TV3 đối chiếu gameplay/event.

Chương này giải thích các kết quả ở mục 12 dựa trên cơ chế đã triển khai ở mục 7–10. Căn cứ định lượng là fixture codec D0/D1 và benchmark TCP loopback; căn cứ tính đúng là assertion cùng log/XML. Các khả năng gây nghẽn được suy từ code sẽ được ghi rõ là giả thuyết, không coi là kết quả profiler. Phạm vi dữ liệu giữ nguyên revision và điều kiện thực nghiệm ở mục 11–12, không thực hiện phép đo mới trong lượt viết này.

13.1. Mức đáp ứng mục tiêu và điều kiện thực nghiệm

Bảng 13.1. Mục tiêu, bằng chứng và mức kết luận được hỗ trợ.

Mục tiêu trong mục 6	Bằng chứng đã có	Kết luận và phần còn thiếu
Phòng, quyền, movement và gameplay authoritative — FR01–FR09	Suite 133/133 JUnit đạt ở lượt mới; test lobby/TCP, collision, bom/item/lives	Các fixture đã chạy đạt assertion; chưa nghiệm thu mọi thao tác GUI/LAN
Replica và lượt trận đúng contract — FR10–FR12, NFR01	Test baseline/resync/match scope/respawn; 18.000 đối chiếu state chuẩn hóa D1 đạt	Có bằng chứng logic và tích hợp; equality D1 không kiểm tra metadata tick/clock hoặc notice
Đồng thời, giới hạn queue và cleanup — NFR02/NFR06/NFR07	84/84 lần thực thi test qua ba lượt; queue đầy, thread owner, writer/client chậm	Phản ứng theo fixture đạt; không chứng minh mọi race, mọi tải hoặc không còn memory leak
Giảm byte đồng bộ — NFR05	B0/B1 cùng trace; D1 giảm 35,98–97,54% tùy workload	Chứng minh giảm frame snapshot ở fixture đã đo; chưa tách lợi ích diff và bỏ tick rỗng
Phản hồi và nhịp cập nhật — NFR03/NFR04	45/45 scenario-run loopback PASS; mean ACK/RTT và pending được lưu	Có số đo phía ứng dụng; chưa đo residual correction, frame time hoặc mức trễ tick bằng profiler
Khả năng kiểm tra/tái lập — NFR08	Raw CSV/XML/trace, revision, class hash và script	Truy vết được các kết quả; CPU model/RAM chưa xác minh, nhóm LAN/soak chưa thực hiện

Nguồn: kết quả chức năng và thực nghiệm, delta summary, network summary và reliability summary. Không cộng số test, scenario-run, peer và tick thành một tổng "số lần thành công" vì chúng là các đơn vị khác nhau.

Những kết quả trên hỗ trợ kết luận hệ thống đã ghép được transport, runtime, gameplay và replica trong phạm vi đã kiểm tra. Chưa đủ cơ sở kết luận toàn bộ sản phẩm đã nghiệm thu: game chạy đúng assertion trong một fixture khác với bốn người thao tác trên bốn máy, nơi còn có input focus, lịch EDT và điều kiện mạng thực. Tương tự, không có ngưỡng SLA đã xác lập để từ RTT thấp ở loopback kết luận đáp ứng yêu cầu mọi mạng.

13.2. Hiệu quả/chi phí và trường hợp delta kém lợi

13.2.1. Vì sao hiệu quả phụ thuộc workload

Trong D1 idle bốn player, nhánh full gửi 560400 byte, nhánh delta gửi 13786 byte trong 600 tick, giảm 97,54%. B1 chỉ phát 31 packet, gồm bảy full định kỳ/bắt buộc và 24 delta; 569 tick

không phát. Lợi ích lớn vì world ít thay đổi và harness không sinh input ACK mới mỗi tick. Match timer vẫn tạo update, nên đây không phải trạng thái hoàn toàn không có dữ liệu nền.

Với D1 movement bốn player, B1 phát đủ 600 packet, gồm bảy full và 593 delta. Tổng byte giảm từ 560400 xuống 358780, tương ứng 35,98%. Ở trace này mọi player có input/ACK thay đổi, nên PlayerData tiếp tục xuất hiện dù map không đổi. Builder chỉ loại phần record không thay đổi, không mã hóa từng field riêng trong PlayerData. Như vậy delta vẫn có lợi trên trace chuyển động liên tục, nhưng mức giảm thấp hơn nhiều so với idle.

Hai trace movement có cùng số byte tiết kiệm tuyệt đối 201620 khi tăng từ hai lên bốn player, trong khi phần full và delta đều lớn hơn với bốn player. Điều này phù hợp với code: map cố định được loại khỏi các delta, còn các player đang đổi vẫn phải gửi. Không dùng con số này để suy một quy luật tuyến tính cho mọi kích thước map hoặc roster. Nguồn cơ chế: StateDeltaBuilder; nguồn từng tick: movement-4, run 1.

Bảng 13.2. Cách đọc hiệu quả delta theo loại dữ liệu.

Điều kiện	Kết quả đã đo hoặc suy luận từ code	Ý nghĩa
Idle không phát input	D1 giảm 97,49–97,54%, 569/600 tick không gửi	Lợi ích gồm cả giảm số packet, không chỉ giảm size delta
Mọi player đổi vị trí/ACK	D1 giảm 46,03% ở hai player, 35,98% ở bốn player; không skip	PlayerData/metadata còn là chi phí thường xuyên
Bom/block/item theo script	D1 giảm 89,33–90,64%; 492 tick không gửi	Phụ thuộc đoạn ít thay đổi sau sự kiện; không đại diện đặt bom liên tục
Nhiều tile/entity cùng đổi	Chưa đo; list upsert/tombstone và chỉ mục có thể lớn	Chưa xác lập ngưỡng delta bắt đầu kém full
Resync lặp hoặc FINISHED	Code có thể ép full; không có trong D1	Kết quả fixture không bao gồm mọi nguồn chi phí full

13.2.2. Idle của harness khác người chơi không giữ phím

InputController vẫn lấy mẫu và gửi moving=false khi không giữ phím, nếu controller còn sẵn sàng. PredictionController cấp sequence mới, còn StateDeltaBuilder so sánh cả lastProcessedInput. Do đó một player có vị trí đứng yên vẫn có thể tạo PlayerData thay đổi vì ACK. Ngược lại, D1 idle và mode idle của benchmark không gửi MOVE.

Đây là giới hạn đối chiếu workload quan trọng: mức tiết kiệm idle cao đã đo không phải lời hứa cho mọi client GUI đứng yên. Suy luận từ code cho thấy mẫu đứng yên liên tục có thể làm giảm số tick skip; mức ảnh hưởng chính xác cần trace/benchmark riêng. Chưa có phép đo đó nên không gán phần trăm cụ thể hoặc tự bỏ mẫu input trong production. Mẫu đứng yên hiện còn phục vụ contract input/ACK, vì vậy tối ưu nó phải xét cả tính đúng, không chỉ số byte.

13.2.3. Chi phí còn lại và trường hợp có thể kém lợi

Builder quét/chụp state và giữ baseline; snapshot còn có metadata, MatchFrame và Base64/JSON envelope. Đây là đổi chác giữa lượng dữ liệu gửi, công việc so sánh và bộ nhớ; phép đo byte chưa xác định chi phí CPU/GC. Không thể từ "giảm 35,98% byte" kết luận "giảm 35,98% thời gian xử lý" hoặc "tăng FPS tương ứng".

Nếu phần lớn tile/entity đổi, danh sách tọa độ/chỉ mục, record mới và khóa xóa có thể làm delta kém lợi. Code chưa encode thử cả full/delta để chọn bản nhỏ hơn, nên không có bảo đảm delta

luôn nhỏ nhất. Full định kỳ còn cần cho chính sách đồng bộ và phải được tính trong tổng, như các bảng kết quả đã làm. Việc chọn chu kỳ full khác có thể thay đổi byte/phục hồi, nhưng chưa có sweep tham số để gọi 100 tick là tối ưu.

13.3. Tick, FIFO, backlog, prediction correction và độ trễ

13.3.1. RTT thấp không loại bỏ thời gian chờ mô phỏng

Median mean RTT theo cấu hình ở loopback khoảng 1,66–1,94 ms, trong khi median mean input ACK ở workload có MOVE khoảng 26,11–27,22 ms. Hai metric có timestamp và đường xử lý khác nhau: PING/PONG không chứng minh input đã được mô phỏng, còn ACK chỉ xuất hiện khi snapshot phản ánh sequence server đã xử lý. Khoảng ACK p95 gần một chu kỳ 50 ms phù hợp với việc có thể chờ tick, nhưng chưa có trace nội bộ để tách thời gian queue, tick, socket và poll.

GameLoop đọc tối đa 256 command/tick và chỉ áp dụng một mẫu movement dương thành công/player/tick. Nếu đầu queue là mẫu tiếp theo của player vừa được xử lý, loop dừng đọc để giữ FIFO; command của player khác phía sau cũng phải chờ. Đây là một khả năng gây backlog/fairness hạn chế, không phải bottleneck đã được chứng minh ở tải bốn client. Con số 256 không đồng nghĩa server xử lý được 256 movement của cùng một player trong một tick. Nguồn: GameLoop.

13.3.2. Backpressure bảo vệ state nhưng có chi phí trải nghiệm

Command queue dùng offer có giới hạn; outbound chỉ commit baseline sau enqueue thành công. Writer/client queue đầy hoặc write deadline có thể đóng connection để tránh tiếp tục với chuỗi snapshot thiếu dữ liệu. Cách xử lý này bảo vệ một số invariant về tài nguyên và thứ tự, nhưng có thể kết thúc lượt theo ABORTED. Không nên gọi đây là phục hồi trong suốt cho người chơi hoặc exactly-once toàn hệ thống.

TCP cung cấp byte stream có thứ tự và tin cậy [1]; contract ứng dụng vẫn phải kiểm tra match/baseline. (RFC 9293, §2.2). Việc update vào outbound queue không xác nhận client đã nhận, và ACK TCP không xác nhận replica đã áp dụng. Trong nhóm mạng đã đo, không có stale/resync nhưng còn tổng 14 MOVE pending cuối cửa sổ. Đó là giới hạn quan sát completion, chưa chứng minh message loss hoặc đã mất tính nhất quán.

13.3.3. Prediction và interpolation chưa có bằng chứng UX định lượng

Prediction/reconciliation dùng input chưa ACK để mô phỏng lại theo hướng tiếp cận đã có [4]. (Gambetta, Part II). Test hiện có kiểm tra replay/reset đúng giá trị, không đo thời gian từ nhấn phím tới pixel thay đổi. Giá trị sai lệch trả về trước reconciliation cũng không phải residual sau replay, vì vị trí predicted và authoritative có thể thuộc hai thời điểm mô phỏng khác nhau.

Nội suy remote cần dữ liệu các mẫu trước để dựng chuyển động giữa update [5]. (Gambetta, Part III). Client hiện lùi render timeline khoảng hai tick với phần tiến thời gian có giới hạn; đó là lựa chọn presentation, không phải lời bảo đảm lag hiển thị luôn 100 ms. Reset khi respawn giúp tránh nội suy qua một bước nhảy vị trí không liên tục, nhưng độ mượt trên GUI và mạng có jitter vẫn cần phép đo/quan sát riêng.

13.4. Tính nhất quán gameplay, event và respawn

Một game loop/room sắp thứ tự movement → extension gameplay → snapshot. Nhờ đó snapshot phản ánh kết quả sau bước gameplay thay vì network thread đọc world đang sửa. Đây là điểm ghép quan trọng: tối ưu byte chỉ có ý nghĩa khi replica biểu diễn đúng kết quả server. Các test runtime, MatchLifecycle và harness D1 hỗ trợ các phần khác nhau của kết luận này; D1 không chạy toàn luật bom/item nên không được dùng một mình để xác nhận gameplay.

BombSystem gom hit trước khi damage, giữ barrier của block trong cùng batch và đánh một lần/player/batch. Match extension xử lý item theo thứ tự xóa item trong blast → drop → pickup → quyết định kết quả. Các fixture đã kiểm tra chain, shield, respawn, cap và chết cùng tick; chúng giúp tránh kết quả phụ thuộc việc một tia được xử lý trước rồi cho player respawn vào tia kế tiếp của cùng batch. Không suy rằng mọi thứ tự input hoặc mọi vị trí respawn trong mọi map đã được bao phủ.

Snapshot là nguồn state client, notice không thực thi damage/pickup lần nữa. Respawn counter làm client bỏ pending input cũ và lịch sử nội suy, trong khi sequence vẫn tăng. Match ID mới ngăn dữ liệu lượt cũ áp vào lượt mới. Các cơ chế này giải quyết các ranh giới khác nhau; event ID, input sequence và match ID không thay thế nhau. Nguồn: MatchGameplayExtension, BombSystem, MovementClientController và các test được dẫn tại mục 12.2–12.4.

13.5. Bottleneck, lỗi quan sát được và giới hạn kết luận

Bảng 13.3. Phân biệt quan sát thực tế với khả năng cần đo thêm.

Điểm xem xét	Căn cứ	Kết luận hiện tại	Phép đo cần để xác nhận bottleneck
PlayerData/ACK trong delta	D1 movement không skip, samePlayer xét ACK	Chi phí byte thường xuyên đã quan sát	Tách byte theo field và workload đứng yên có input
Broadcast nhiều receiver	Tổng RX movement tăng 15,32 → 46,57 KiB/s khi 2 → 4 client	Lưu lượng nhận tăng; chưa phải server bão hòa	CPU, queue depth, nhiều room/client và throughput khi tăng tải
FIFO chặn mẫu tiếp theo	Điều kiện peek/break trong loop	Nguy cơ chờ command phía sau suy từ code	Timestamp enqueue/dequeue, fairness theo player và phân bố backlog
Scan/copy/encode/Base64	Builder/codec thực hiện các bước này	Có công việc/bộ nhớ phát sinh; chưa biết bước chiếm ưu thế	Profiling CPU/allocation/GC, encode/decode duration
Writer/UI chậm	Fixture timeout/budget test đạt	Cơ chế giới hạn hoạt động trong fixture	Soak, queue occupancy và latency dưới chậm/jitter kéo dài
Smoke đa process	Fail lịch sử; bốn lần mới đạt	Chưa tái hiện mới, chưa xác định nguyên nhân	Lưu trace setup/full deadline và điều kiện môi trường khi tái hiện

Tổng RX trong bảng là median tổng phía client, không phải phép đo năng lực đường truyền hay server processing rate. Các so sánh latency bốn client thấp hơn hai client ở một số mode không chứng minh scale tốt hơn: chỉ có năm run, cùng máy, thứ tự mode cố định và chưa tách ảnh hưởng scheduling/GC. Benchmark full/delta là đối chứng codec; benchmark TCP chỉ đo cấu hình hiện hành. Chưa có A/B TCP giữa full mỗi tick và delta nên không gán RTT/ACK đang đo là kết quả cải thiện do compression.

Không có profiler hoặc phép đo bão hòa để chọn một bottleneck duy nhất. Ưu tiên kiểm tra tiếp theo là trace input/ACK/queue và chi phí serialization, rồi mới thay đổi tick, chu kỳ full hoặc format. Thay đổi cần được đánh giá lại cả tính đúng và byte/latency trên cùng workload. Các giới hạn chức năng, GUI/LAN/soak và hướng xử lý được trình bày tập trung ở mục 15–16.

14. Điểm mới và đóng góp — Novelty and Contributions

Phân công: TV2 chủ trì State Delta Compression và contract đồng bộ; TV4 tổng hợp bằng chứng/đánh giá; TV1 và TV3 xác nhận phần tích hợp của module mình. Phân công nội dung không phải xác nhận tác giả từng dòng code hoặc phần trăm đóng góp cá nhân.

Theo Instruction, mục 7 và 8.14, điểm mới có thể là cải tiến phương pháp, protocol/message format hoặc cách tích hợp; không bắt buộc là thuật toán chưa từng có trên thế giới. Vì vậy đóng góp chính được xác định là **hiện thực và đánh giá cơ chế đồng bộ state có delta trong Bomb It Multiplayer**, cùng các điểm ghép bảo đảm ranh giới state/runtime/client. Không gọi TCP, prediction, interpolation hoặc bản thân delta encoding là phát minh của nhóm.

14.1. Baseline kỹ thuật, phần kế thừa và phần tự phát triển

14.1.1. Baseline đo lường

Baseline B0 là chính sách gửi full snapshot mỗi tick trên cùng trace, cùng codec và lớp đếm byte với B1. B1 dùng StateDeltaBuilder, full ở tick 1/bội 100 và chỉ phát delta khi có thay đổi có ý nghĩa. Đây là đối chứng kỹ thuật được dựng trong harness, không phải một phiên bản lịch sử có commit cụ thể đã được chứng minh từng gửi full theo đúng cách đó trong sản phẩm. Số ví dụ "1000 byte so với 50–100 byte/gói" của ý tưởng ban đầu không được dùng thay số đo.

So sánh B0/B1 đã kiểm soát chuỗi state, số tick, codec, envelope và yêu cầu phục hồi replica của harness. Nó chưa kiểm soát một cặp server/client TCP A/B hoặc đo thay đổi latency/CPU. Nhóm không tuyên bố hệ thống nhanh hơn một game thương mại, một engine hoặc một implementation bên ngoài chưa được chạy đối chứng.

14.1.2. Kế thừa phương pháp và lựa chọn của project

Delta theo baseline đã được mô tả trong tài liệu trước, chẳng hạn Fiedler [3]. (Snapshot Compression). Prediction/reconciliation và entity interpolation cũng là các phương pháp có sẵn [4], [5]. Giá trị của project nằm ở contract, cách hiện thực và kiểm chứng trong game này.

Bảng 14.1. Phương pháp kế thừa và phần nhóm hiện thực trong project.

Nội dung	Kiến thức/phương pháp kế thừa	Lựa chọn cụ thể được hiện thực
Đồng bộ delta	Dùng snapshot nền để biểu diễn thay đổi [3]	Tile/player/entity diff, tombstone, tick/baseTick, full policy và codec v2–v5
Client movement	Prediction/replay và nội suy [4], [5]	Controller dùng ACK, bản sao world, giới hạn pending/buffer và reset khi respawn
Runtime và concurrency	Queue, thread ownership và vòng lặp cập nhật đã có trong kiến thức nền mục 5	Runtime theo room, command có kiểu, extension hook, snapshot bất biến và stop/failure API
Giao tiếp/tài nguyên	TCP và kỹ thuật giới hạn queue phổ biến	Match envelope, schema/gate, writer budget/deadline và chính sách đóng khi không giữ được chuỗi

Những API legacy còn tồn tại không phải bằng chứng đóng góp mới hoặc mốc before/after. Tương tự, tên tác giả tài liệu tham khảo không chứng minh nhóm đã sao chép code của họ; báo cáo chỉ xác nhận sự kế thừa ý tưởng/phương pháp được trích và các file hiện có trong repository. Muốn xác nhận lịch sử tác giả/công sức cần hồ sơ nhóm riêng, không suy từ ownership note trong chương.

14.2. State Delta Compression: phương pháp và bằng chứng

14.2.1. Phạm vi đóng góp chính

StateDeltaBuilder so sánh state hiện tại với baseline đã commit, tạo danh sách thay đổi hoặc bỏ update rỗng. GameLoop chỉ commit sau outbound enqueue thành công; client kiểm tra base tick và yêu cầu full khi không thể áp dụng. Entity có key và danh sách xóa để việc bom/item biến mất không bị bỏ sót. Full định kỳ và full khởi tạo được tính vào byte đối chứng, không chỉ chọn các delta nhỏ để báo cáo.

So với ví dụ baseline có ACK snapshot phía nhận trong [3], project không có cơ chế receipt ACK snapshot theo từng client để lựa chọn baseline mới nhất mà từng receiver xác nhận. Baseline ở đây là mốc xuất bản chung của runtime, ghép với TCP có thứ tự, writer queue, match scope và full-resync. Hai thiết kế không được coi là cùng mức bảo đảm; enqueue thành công chưa xác nhận remote replica đã áp dụng. Sự khác nhau này là ranh giới contract của implementation, không phải bằng chứng project tốt hơn phương pháp trong tài liệu.

14.2.2. Bằng chứng hiệu quả và tính đúng

Ở D0 v2, tổng full 110076 byte so với delta 16411 byte trong 200 tick, giảm 85,09%. Ở sáu cấu hình D1 v5/envelope, tổng frame giảm 35,98–97,54% tùy trace, mỗi cấu hình chạy năm lần cho cùng số byte/count. Cả 18.000 đối chiếu state chuẩn hóa D1 đạt. Các con số được truy vết tới summary.csv, raw/trace và công cụ và bảng/hình ở mục 12.5.

Bằng chứng này hỗ trợ đóng góp giảm lượng frame snapshot trong các workload đã đo và giữ state theo harness. Nó không hỗ trợ lời hứa mọi packet còn 50–100 byte, mọi trận tiết kiệm tối thiểu một tỷ lệ cố định, hoặc giảm latency mạng tương ứng. Mức giảm gồm cả diff và bỏ tick rỗng, chưa có ablation tách riêng. Trace idle không phát MOVE, khác GUI có mẫu đứng yên; trace sự kiện được dựng trực tiếp, không phải trận tự nhiên. Các giới hạn được phân tích ở mục 13.2.

14.3. Các đóng góp triển khai bổ sung được chứng minh

14.3.1. Ranh giới runtime/state và mở rộng gameplay

Runtime riêng cho từng room cùng command queue có kiểu cho phép movement và extension gameplay dùng chung thứ tự cập nhật. Network nhận snapshot thay vì sửa/read trực tiếp mutable GameState. Test đã kiểm tra room isolation, FIFO từng producer, queue full, snapshot giữ nguyên sau world đổi và thread cleanup. Match extension còn ghép damage batch, block barrier, drop/pickup/result vào trước bước tạo snapshot.

Đây là đóng góp tích hợp và kiểm chứng invariant của sản phẩm, không phải thuật toán concurrency hoặc luật Bomberman mới. Không có đối chứng kiến trúc đa luồng khác để tuyên bố cấu trúc này nhanh nhất hoặc scale tuyến tính theo số room.

14.3.2. Contract client và xử lý gián đoạn

Client có replica full/delta, local replay theo ACK, remote interpolation và reset khi respawn; match ID tách lượt, event notice không áp lại world effect đã có trong snapshot. Các test

controller/TCP kiểm tra resync, shield/respawn, final snapshot và replay trên connection đang dùng. Queue/budget/deadline được kiểm tra với client/writer/UI chậm hoặc lỗi.

Đóng góp được nhận ở mức cách ghép và kiểm thử các thành phần này trong một flow chơi/replay. Chưa có phép đo correction nhìn thấy hoặc nghiệm thu GUI/LAN nên không đưa "mượt hơn", "không lag", "không mất kết nối" hoặc reconnect resume vào danh sách đóng góp đã chứng minh.

14.3.3. Đánh giá có đối chứng và dữ liệu truy vết

Project lưu trace/byte của 35 scenario-run codec, 45 scenario-run TCP loopback, XML/log của suite 133 test và 84 lần thực thi test reliability bổ sung. Các đơn vị này được giữ riêng; raw data, script phân tích, artifact hash, số mẫu/pending và lỗi chuẩn bị được lưu. Điều này hỗ trợ kiểm tra lại phép tính và giải thích kết luận thay vì chỉ đưa ảnh hoặc số trung bình không có nguồn.

Khả năng truy vết là đóng góp đánh giá/engineering bổ sung, không tự chứng minh một phương pháp đo mới. Benchmark không bao gồm LAN, CPU/RAM đầy đủ hoặc profiling; việc giữ các lỗi và giới hạn giúp xác định phạm vi kết luận, không biến dữ liệu thiếu thành điểm mạnh kỹ thuật.

14.4. Bảng Contribution — Description — Evidence

Bảng 14.2. Đóng góp được nhận trong phạm vi báo cáo.

Contribution	Description	Evidence	Phạm vi/đơn vị phụ trách
C1 — Đồng bộ State Delta Compression, đóng góp chính	Diff tile/player/entity và tombstone, full policy, commit baseline và replica validation; giảm byte so với B0 cùng trace	StateDeltaBuilder/codec/replica; mục 12.5: D0 85,09%, D1 35,98–97,54%, 18.000 equality check	TV2 chủ trì, TV4 đo/tích hợp; fixture frame snapshot, không bandwidth toàn luồng
C2 — Runtime theo room và điểm mở rộng gameplay	Một owner state/room, command FIFO có giới hạn, snapshot bất biến; ghép batch gameplay trước snapshot	RuntimeContractCheck, MatchGameplay/ExtendedItems/BombRuntimeIntegration; mục 12.2–12.4	TV2/TV3, TV1/TV4 tích hợp; invariant đã test, không có benchmark kiến trúc đối chứng
C3 — Contract đồng bộ client và xử lý gián đoạn	ACK/replay, full-resync, match scope, respawn reset; giới hạn queue/deadline và cleanup khi lỗi	Controller/replica; MatchLifecycle/LobbyLifecycle, Client/TransportStability; ba lượt reliability 84/84 đạt	TV1–TV4 theo module; không tuyên bố exactly-once, reconnect resume hoặc UX đã nghiệm thu
C4 — Đánh giá tái kiểm tra được	Cùng trace cho full/delta, raw per-tick/per-peer, script, hash, sample/pending và log lỗi	Hồ sơ đợt kiểm thử chức năng–14; 35 codec scenario-run, 45 TCP scenario-run; chương 11–12	TV4 tổng hợp, cả nhóm cung cấp; loopback/fixture và phạm vi công cụ hiện có

Bằng chứng C2/C3 không phải các nhóm test rời hoàn toàn: cùng một test tích hợp có thể hỗ trợ nhiều invariant. Không cộng bốn đóng góp thành một con số định lượng hoặc gán tỷ lệ công sức cá nhân từ bảng này. Các file và test cụ thể đã được dẫn ở chương 10–12; hồ sơ đợt kiểm thử chức năng, 13 và 14 chứa log/trace để xác minh.

Phát biểu đóng góp chính dùng trong báo cáo là: **nhóm hiện thực và đánh giá cơ chế đồng bộ full/delta cho state Bomb It, gắn với runtime theo room và replica client; trên các trace được công bố, cơ chế giảm tổng frame snapshot so với gửi full mỗi tick mà giữ state theo**

các kiểm tra tương ứng. Phát biểu này không yêu cầu coi delta encoding là phát minh mới, không thay kết quả thực nghiệm bằng ví dụ byte ban đầu, và giữ các giới hạn đã nêu khi trình bày hoặc bảo vệ.

15. Hạn chế — Limitations

Phân công: TV1 xác nhận session/room và chính sách disconnect; TV2 xác nhận đồng bộ/movement/client; TV3 xác nhận gameplay; TV4 tổng hợp phạm vi bằng chứng và công cụ đo.

Các kết quả ở mục 12 cho thấy implementation đạt các kiểm tra đã thực hiện, nhưng chưa bao phủ toàn bộ điều kiện sử dụng của một game multiplayer. Chương này phân biệt chức năng chưa cung cấp, cơ chế còn giới hạn và phần chưa được đo/nghiệm thu. Một nhóm chưa có bằng chứng không tự động được coi là lỗi chức năng; ngược lại, test PASS cũng không thay cho bằng chứng còn thiếu.

15.1. Hạn chế chức năng và reconnect

15.1.1. Mất kết nối kết thúc lượt, chưa khôi phục trận

Ứng dụng hiện kết thúc lượt theo ABORTED khi người chơi disconnect/leave theo flow lobby. Route RECONNECT chưa được đăng ký trong ứng dụng; connect lại từ UI là một phiên mới, không đưa player trở lại world đang chơi. Đây là chính sách hiện hành, không phải một flow resume đã nghiệm thu.

SessionManager có token, thời gian giữ metadata và API reconnect legacy; runtime cũng có command đóng băng/mở đóng băng player. Các thành phần này chưa tạo thành reconnect end-to-end. Khi trận đã bị dọn, chỉ gắn socket mới hoặc gửi lại full không thể phục hồi world cũ. Nguồn: LobbyGameServer, đánh giá reconnect và test transport/lifecycle được dẫn ở mục 12.4.

15.1.2. Phản hồi gameplay và phạm vi client

PlantBombCommand được xử lý authoritative, nhưng callback BombCommandResult ở factory lobby hiện để rỗng. Client thấy kết quả thành công qua entity/notice; chưa có response nghiệp vụ riêng gắn request placement để phân biệt đầy đủ các lý do từ chối gameplay như hết quota hoặc ô đã có bom. Không nhầm điều này với các ERROR ở gate schema/quyền/queue: chúng thuộc các lớp xử lý khác. Metric plant_ack_ms vì vậy là thời gian quan sát bom trên replica, không phải ACK placement riêng.

Prediction hiện chỉ dành cho movement. Local correction chưa có lớp visual smoothing riêng; renderer sử dụng vị trí sau reconcile, còn remote dùng lịch sử nội suy có giới hạn. Đây là mức hiện thực hiện tại, chưa đủ để khẳng định chuyển động luôn mượt khi mạng biến động. Khoảng cách debug trước reconcile cũng chưa phải residual sau replay hoặc sai số người dùng nhìn thấy. Nguồn: ReconciliationController và phần triển khai client ở mục 10.5/10.7.

15.1.3. Giới hạn thiết kế đồng bộ và xử lý quá tải

Bảng 15.1. Các giới hạn cơ chế hiện hành.

Mã	Cơ chế/giới hạn	Hệ quả trong phạm vi project	Bằng chứng/căn cứ
L01	Chưa có resume trận; disconnect có thể ABORTED	Người chơi không tiếp tục cùng match sau mất kết nối	Lobby flow và RECONNECT_ASSESSMENT
L02	Chưa có response placement nghiệp vụ riêng	Client chưa nhận đủ lý do kết quả luật bom qua một reply theo request	Callback lobby rỗng, nguồn metric plant observation

Mã	Cơ chế/giới hạn	Hệ quả trong phạm vi project	Bằng chứng/căn cứ
L03	Diff theo tile/record, quét/chụp state; chưa tự chọn full/delta theo byte	Player ACK đổi có thể buộc gửi record; delta không được bảo đảm nhỏ nhất	StateDeltaBuilder và D1 movement
L04	Baseline commit sau enqueue nội bộ, không có receipt ACK snapshot theo từng client	Cần giữ thứ tự/chuỗi update và full-resync; enqueue chưa chứng minh remote đã áp dụng	GameLoop/replica và contract mục 9
L05	FIFO có thể dừng đọc khi đầu queue là mẫu tiếp theo cùng player; queue/deadline hữu hạn	Có khả năng chờ command phía sau; quá tải có thể từ chối/đóng connection và hủy lượt	Code loop, writer/client fault tests
L06	Chưa đo/smoothing correction local; lịch sử nội suy remote hữu hạn	Có thể giữ endpoint hoặc hiệu chỉnh rõ khi dữ liệu thiếu/nhảy	Client controller code; chưa có phép đo UX

Các giới hạn L03–L05 là đánh đổi của thiết kế đã chọn, không được dùng để suy một bottleneck CPU hoặc tỷ lệ lỗi chưa đo. Một game thread mỗi room giúp tách state nhưng các room vẫn chia sẻ tài nguyên process/server; chưa có cơ sở tuyên bố scale tuyến tính theo số phòng. Các invariant được bảo vệ trong test không đồng nghĩa delivery exactly-once hoặc phục hồi trong suốt cho mọi lỗi.

15.2. GUI/LAN/tải/soak chưa nghiệm thu

Bảng 15.2. Khoảng trống nghiệm thu và khả năng khái quát kết quả.

Mã	Phần chưa hoàn tất	Bằng chứng hiện có	Kết luận chưa được phép rút ra
L07	GUI có người thao tác hai/bốn người, focus và HUD	Test controller/component và renderer BufferedImage	Chưa xác nhận toàn bộ trải nghiệm điều khiển, bố cục/độ mượt hoặc ảnh trận GUI thực tế
L08	LAN/Internet, nhiều máy, jitter/loss có kiểm soát	TCP loopback một JVM; smoke ba process cùng máy	Chưa có latency/bandwidth thực tế trên LAN hoặc bù trễ mạng đã nghiệm thu
L09	Nhiều room đồng thời, tải vượt bốn client và soak dài hạn	Workload 1/2/4 client, fixture room isolation và vòng lifecycle	Chưa biết tải tối đa, điểm bão hòa hoặc xu hướng tài nguyên theo giờ
L10	Profiling, phần cứng đầy đủ và độ trễ nội bộ	Raw CSV có OS/JDK/16 processor logic; queue/thread assertions	Chưa xác minh CPU model/RAM; chưa phân rã queue/tick/encode/GC hoặc chứng minh không có leak

45 scenario-run mạng có cửa sổ yêu cầu 10 giây, warm-up 2 giây. Năm run mỗi cấu hình giúp quan sát độ lặp ở điều kiện đó nhưng không thay một đợt vận hành dài hạn. Ba lượt reliability có 84 lần thực thi test; 20 lượt trận/10 chu kỳ server trong fixture vẫn là kiểm tra hữu hạn. Không có profiler hoặc theo dõi heap kéo dài để suy từ thread cleanup thành bộ nhớ luôn ổn định.

Nhóm đo mạng dùng server/client chung JVM, có chung CPU/GC và thứ tự mode cố định. Warm-up chủ yếu poll/PING; các đường MOVE/bom chưa được làm nóng đầy đủ trước cửa sổ riêng. Vì vậy các khác biệt nhỏ giữa hai/bốn client có thể chịu ảnh hưởng scheduling hoặc điều kiện chạy, không đủ để xác lập quan hệ nhân quả hoặc kết luận hệ thống phục vụ nhiều client nhanh hơn.

15.3. Giới hạn số liệu lịch sử, fixture và công cụ đo

15.3.1. Giới hạn đối chứng full/delta

D0 là fixture v2 một player, map trống, 200 tick; D1 dùng v5/envelope, arena và 2/4 player, 600 tick. Chỉ so full với delta trong cùng trace, không so chéo hai bộ như cùng workload. D1 dựng trực tiếp sự kiện bom/block/item, không chạy toàn luật gameplay hoặc mô phỏng pickup hợp lệ. State equality D1 chuẩn hóa ordering và bỏ tick/serverTimeMillis của BombFrame, không kiểm tra notice.

Idle không phát MOVE, khác GUI còn gửi mẫu đứng yên có sequence/ACK mới. Trace sự kiện có nhiều tick ít thay đổi sau sự kiện cuối. Hai điều kiện này làm lợi ích bỏ tick rỗng nổi bật. Chưa có ablation tách riêng diff với skip, trace tự nhiên của nhiều người, thay đổi map dày đặc hoặc resync lặp. Năm lần size xác định bằng nhau không tạo bằng chứng phân bố latency/bandwidth mạng. Nguồn: thực nghiệm codec và thảo luận mục 13.2.

15.3.2. Ranh giới metric mạng

Byte được đếm ở frame ứng dụng, không gồm TCP/IP, retransmission hoặc traffic tầng dưới. Số packet trong CSV là message ứng dụng, không phải TCP segment. Mean RTT/ACK được tính trên mẫu hoàn tất; tổng 14 MOVE pending ở cuối cửa sổ chưa có completion quan sát được. Không dùng số pending hoặc zero FAIL để suy packet-loss rate. P95 tổng hợp theo run là worst-peer p95, không phải pooled p95 vì raw latency sample chưa được xuất. Plant observation chỉ có một yêu cầu/client/run, chưa đủ mô tả phân bố placement dưới tải dày.

Benchmark mạng không có nhánh TCP full-every-tick đối chứng; do đó chưa chứng minh giảm byte ở harness dẫn tới giảm RTT/ACK hay CPU trong hệ thống. Các phạm vi này được giữ riêng trong kết quả mạng.

15.3.3. Revision, lỗi lịch sử và tính tái lập

CSV lịch sử ở revision khác được giữ riêng với lần đo mới. Suite khảo sát đợt khảo sát ban đầu có một smoke failure và retry riêng cũng fail; suite đợt kiểm thử chức năng cùng ba lượt reliability mới đều pass smoke với source không đổi. Chưa xác minh nguyên nhân lỗi cũ nên không gọi kết quả mới là bằng chứng một bản sửa đã giải quyết flakiness.

Lỗi repository, execution policy và truyền JAVA_HOME trong bước chuẩn bị được lưu riêng, không tính thành failure của test chưa chạy hoặc âm thầm bỏ khỏi hồ sơ. Manifest/hash hỗ trợ xác định source/artifact, nhưng bản sao file/log vẫn cần được giữ khi đóng gói để người khác kiểm tra. Phạm vi bằng chứng hiện tại được công bố ở mục 12.7; các giới hạn trên là căn cứ cho hướng phát triển mục 16.

16. Hướng phát triển — Future Work

Phân công: mỗi thành viên đề xuất và kiểm chứng phần mình; TV4 tổng hợp thứ tự ưu tiên, protocol và phép đo liên module. Đây là đề xuất, chưa phải chức năng hoặc kết quả đã triển khai.

Hướng phát triển được chọn từ hạn chế ở mục 15 và bằng chứng ở mục 12–13. Ưu tiên đầu tiên là hoàn thiện nghiệm thu và khả năng quan sát trước khi mở rộng lời hứa hiệu năng hoặc thêm tính năng lớn. Các tiêu chí dưới đây là điều kiện kiểm chứng đề xuất, không phải SLA hoặc lịch cam kết đã được nhóm phê duyệt.

16.1. Hoàn thiện và mở rộng chức năng

16.1.1. Hoàn thiện luồng chơi và phản hồi người dùng

Tổ chức nghiệm thu GUI hai/bốn người cho tạo/vào phòng, ready/start, chat, movement/STOP, mất focus, placement, item/lives/shield/respawn, kết quả và replay. Mỗi lượt cần run/match ID, người kiểm tra, ảnh/video và log hai đầu; lỗi UI phải được đối chiếu với state authoritative. Chỉ sau nghiệm thu mới kết luận các thao tác đã hoạt động với người dùng thật trong topology tương ứng.

Một cải tiến chức năng cụ thể là response placement có request ID và mã kết quả nghiệp vụ rõ ràng. Cần phân biệt request đã qua gate/được enqueue với bom thật sự được luật gameplay chấp nhận, giữ dedupe khi nhận lại request và không tạo bom dự đoán chỉ để che độ trễ. TV3 cung cấp kết quả luật, TV2 nối command/runtime, TV1/TV4 thống nhất contract/gửi phản hồi và client hiển thị. Phải kiểm tra cả thành công, quota, ô có bom, player chết, request lặp và match cũ trước khi nhận tính năng.

16.1.2. Thứ tự ưu tiên và điều kiện kiểm chứng

Bảng 16.1. Các nhóm công việc đề xuất gắn với hạn chế.

Ưu tiên	Hạn chế liên quan	Công việc	Điều kiện kiểm chứng đề xuất	Phụ trách
P1	L07/L08/L10	Nghiệm thu GUI/LAN, bổ sung CPU/RAM/topology và raw log	Biên bản thao tác, kết quả hai đầu, metadata đủ cho từng máy	Cả nhóm, TV4 ghép
P1	L02	Placement response theo request	Test kết quả luật/dedupe/match scope và kiểm tra client không hiểu enqueue là placement thành công	TV3/TV2/TV1/TV4
P1	L05/L10 và lỗi smoke lịch sử	Ghi enqueue/dequeue/tick/full-ready/deadline, tái hiện lỗi setup	Log phân biệt lỗi môi trường, handshake, setup và snapshot; giữ attempt fail/pass	TV2/TV4, TV1 hỗ trợ
P2	L03/L04/L06	Tối ưu đồng bộ/correction dựa trên profiling và trace	State đúng, byte/CPU/latency/UX được đo trên cùng workload trước–sau	TV2/TV4
P2	L01	Thiết kế resume có xác thực và generation	Qua ma trận race/resume/full-baseline và luật hết hạn đã thống nhất	TV1/TV2/TV4, TV3 xác nhận gameplay
P3	L09/L10	Tải nhiều room và vận hành dài hạn	Có đường cong tải, queue/tick/tài nguyên và cleanup sau chạy	TV4, owner từng module

P1/P2/P3 là thứ tự ưu tiên tương đối của đề xuất, không phải xác nhận đã làm hoặc ước lượng số ngày. Mở rộng map/chế độ chơi chỉ nên theo sau các bước trên và giữ test contract theo kích thước/luật mới; không coi việc thêm nội dung đồ họa là thay thế đóng góp kỹ thuật đã xác định ở mục 14.

16.2. Tối ưu đồng bộ/client correction và tài nguyên

16.2.1. Đo trước khi đổi format hoặc lịch gửi

Bổ sung timestamp enqueue/dequeue, thời gian từng phần tick, kích thước từng nhóm field, encode/decode duration, allocation/GC và queue occupancy. Thu trace người chơi thật và trace đứng yên vẫn gửi input để tách tác dụng của ACK khỏi thay đổi vị trí. Xuất raw RTT/ACK sample và correction theo cùng mốc mô phỏng để có thể tính phân bố phù hợp, thay vì suy pooled p95 từ các thống kê đã tổng hợp.

Các hướng cần thử gồm dirty tracking để tránh quét toàn map, field-level delta cho PlayerData, hoặc chọn full khi encode delta lớn hơn full. Mỗi hướng cần đối chứng cùng trace và cùng lớp đếm byte, tính cả full/resync; đo thêm CPU/bộ nhớ để tránh đổi ít byte hơn lấy chi phí mô phỏng vượt ngân sách tick. Nếu đổi precision/quantization phải đặt tiêu chí sai số có ý nghĩa cho collision/gameplay và kiểm tra replica, không giảm độ chính xác chỉ để làm biểu đồ byte tốt hơn.

16.2.2. Bảo toàn input và baseline

Tối ưu mẫu đứng yên có thể giảm ACK update, nhưng phải định nghĩa lại rõ contract lấy mẫu, STOP và replay. Không chỉ xóa input không giữ phím vì server hiện dựa vào mẫu có sequence để tiến bước/ACK theo tick. Tương tự, nếu thay FIFO bằng cơ chế công bằng hơn theo player cần xác định thứ tự liên loại command và kiểm tra lại MOVE/plant/disconnect cùng tick, tránh đổi ý nghĩa gameplay ngoài dự kiến.

Nếu xét baseline riêng từng receiver hoặc cơ chế coalesce snapshot, phải bảo đảm delta chỉ tham chiếu nền receiver có thể dùng, có fallback full và test queue drop/backpressure. Không bỏ một delta giữa chuỗi rồi tiếp tục gửi delta phụ thuộc nó. Các lựa chọn này là hướng thử nghiệm, chưa chứng minh tốt hơn baseline xuất bản chung hiện tại.

16.2.3. Correction và khả năng quan sát client

Đo error trước/sau replay tại cùng thời điểm thích hợp, số lần correction, frame time và đoạn video tương ứng. Sau đó mới thử visual smoothing local hoặc buffer nội suy thích ứng. Smoothing phải reset khi respawn/match mới, không làm sai state collision hoặc hiển thị đi xuyên tường; khi mất dữ liệu vẫn cần giới hạn ngoại suy/thời gian giữ endpoint. So sánh bật/tắt cơ chế trên cùng trace delay/jitter, giữ nguyên authority server.

16.3. Reconnect có xác thực/generation an toàn

Resume là thay đổi nhiều lớp, không chỉ thêm handler RECONNECT. Đề xuất cần bắt đầu bằng việc nhóm chốt chính sách giữ ghế/world trong grace, player bị đứng yên hay vẫn chịu damage, khi nào hết hạn và xử lý kết quả nếu trận kết thúc trong lúc mất kết nối. Chính sách này sẽ thay một phần hành vi ABORTED hiện tại nên phải được đặc tả và test cùng gameplay.

Thiết kế phiên cần xác thực quyền nhận lại player, quy định hạn/đổi token, request lặp và reconnect đồng thời. Không tin player ID client tự khai làm bằng chứng sở hữu phiên. Socket/writer mới cần gắn với connection generation; EOF/write failure/timer muộn của generation cũ không được xóa phiên mới hoặc tiếp tục gửi dữ liệu như owner hiện tại.

Sau khi attach thành công, server cung cấp match context hiện tại và full state phù hợp. Client chỉ bật input sau khi có nền hợp lệ; sequence/ACK/request dedupe được tiếp nối theo match, không reset về 1 như lượt mới hoặc replay lại placement/damage/pickup. Nếu trận đã kết thúc, trả kết quả đã chốt thay vì mở lại world.

Bảng 16.2. Ma trận kiểm tra tối thiểu đề xuất cho resume.

Nhóm	Tình huống	Điều kiện cần xác nhận
Quyền/đồng thời	Token sai/hết hạn/dùng lại; hai reconnect cùng lúc	Không chiếm ghế; chỉ owner được nhận theo policy
Generation	Socket cũ EOF, writer cũ fail hoặc expiry timer cũ chạy sau attach	Không đóng/xóa connection generation mới
Deadline	Trước/sau grace; reconnect khi GAME_OVER đã chốt	State/ghế/kết quả tuân theo chính sách, không hồi sinh trận đã kết thúc
Input	MOVE đã/chưa ACK, request placement lặp, match cũ	Không dùng lại sequence sai hoặc áp dụng command hai lần
World	Còn bom/item/lửa, shield/respawn đang đổi	Full baseline hợp lệ, không cộng effect lại; input chỉ mở sau sync
Cleanup	Resume lỗi giữa handshake/attach/full	Không để hai writer active, ghost player hoặc tài nguyên không có owner

Căn cứ phạm vi: RECONNECT_ASSESSMENT. Các dòng trong bảng là tiêu chí tương lai; API legacy và test session hiện tại chưa chứng minh chúng đã đạt.

16.4. Đánh giá nhiều máy, tải lớn hơn và vận hành dài hạn

Mở rộng theo thứ tự loopback nhiều JVM → LAN nhiều máy → môi trường có delay/jitter có kiểm soát. Mỗi topology cần metadata phần cứng, nguồn điện, kết nối, server/client version và công cụ tạo tải; không trộn nhiều process cùng máy vào số liệu LAN hoặc trừ timestamp hai máy để suy one-way delay khi chưa kiểm soát clock. Giữ baseline full/delta và metric definition cố định nếu mục tiêu là so sánh tác động mạng của compression.

Tăng tải bằng nhiều room với roster hợp lệ trước khi thay cap bốn người của một match. Theo dõi tick overrun, queue depth/rejection, writer backlog, CPU/RAM/GC, RTT/ACK cả mẫu hoàn tất lẫn request timeout/pending, error và cleanup sau đợt chạy. Điểm bão hòa cần được xác định bằng tiêu chí rõ ràng, không suy từ giới hạn queue hoặc số thread cấu hình.

Soak phải lưu duration, số vòng, xu hướng tài nguyên và sự kiện lỗi theo thời gian; test đóng/mở nhanh không thay phép đo này. Đợt đánh giá cũng nên làm nóng workload thực, đổi thứ tự chạy có kiểm soát và tăng số run khi biến thiên yêu cầu. Mọi sửa lỗi hoặc thay property tạo một batch mới có revision/artifact hash riêng, giữ nguyên attempt cũ để so sánh và không chỉ chọn các lượt đạt.

17. Kết luận — Conclusion

Phân công: TV1 chủ trì ghép kết luận; TV2/TV3 xác nhận giải pháp và phạm vi chức năng, TV4 xác nhận số liệu.

Đề tài giải quyết bài toán tổ chức game Bomb It nhiều người chơi, trong đó các client cần cùng quan sát map, chuyển động, bom/item và kết quả trận do server quyết định. Hệ thống sử dụng TCP với framing/contract dùng chung, runtime theo room, command queue có giới hạn và game loop ghép movement, gameplay rồi tạo snapshot. Client giữ replica, dự đoán movement local, replay input chưa ACK và nội suy remote; match scope, full-resync và reset khi respawn hỗ trợ duy trì đúng ranh giới đồng bộ.

Đóng góp chính là hiện thực và đánh giá State Delta Compression trong flow này: gửi thay đổi tile/player/entity theo baseline, biểu diễn xóa, bỏ update rỗng và kết hợp full theo chính sách. Đối chứng dùng cùng trace với full mỗi tick, không giả định mọi delta đều nhỏ hơn full. Ở fixture D0 v2, tổng byte giảm từ 110076 xuống 16411, tương ứng 85,09%. Ở các trace D1 v5 có match envelope, mức giảm tổng frame snapshot đạt 35,98–97,54% tùy workload; 18.000 đối chiếu state chuẩn hóa của D1 đạt. Kết quả cho thấy lợi ích phụ thuộc mức thay đổi của world và input/ACK, đặc biệt các trace ít thay đổi hưởng lợi thêm từ việc bỏ tick rỗng. Đây là ứng dụng/tích hợp kỹ thuật đã có, không phải tuyên bố phát minh delta encoding.

Kiểm thử mới ghi nhận suite 133/133 JUnit đạt và 84/84 lần thực thi test reliability bổ sung đạt. Benchmark TCP loopback hoàn tất 45/45 scenario-run với transport 1/2/4 client và gameplay 2/4 player. Trong môi trường đó, median mean RTT theo cấu hình khoảng 1,66–1,94 ms, median mean input ACK ở workload có MOVE khoảng 26,11–27,22 ms. Các kết quả cùng raw log/CSV/XML, script và hash được trình bày ở mục 12; số test, tick và scenario-run được giữ riêng, không cộng thành một thước đo chất lượng duy nhất.

Phạm vi kết luận còn giới hạn ở các fixture và môi trường đã chạy. Phép đo codec không chứng minh giảm latency/CPU TCP tương ứng; benchmark loopback không thay LAN hoặc tải dài hạn. GUI nhiều máy và profiling chưa được nghiệm thu/đo đầy đủ; resume trận chưa được triển khai trong ứng dụng; nguyên nhân lỗi smoke lịch sử cũng chưa được xác minh dù các lượt mới đạt. Vì vậy kết quả hỗ trợ khả năng hoạt động và giảm byte snapshot trong điều kiện công bố, chưa xác lập tải tối đa, SLA hoặc độ mượt cho mọi điều kiện mạng.

Hướng tiếp theo là hoàn thiện nghiệm thu GUI/LAN và quan sát hệ thống, đánh giá tối ưu đồng bộ/correction trên trace thực, rồi mở rộng resume và tải nhiều room theo tiêu chí ở mục 16. Những kết quả hiện có tạo cơ sở để tiếp tục các bước đó với baseline và bằng chứng có thể kiểm tra lại.

18. Tài liệu tham khảo — References

Phân công: cả nhóm cung cấp nguồn; TV4 chuẩn hóa và đối chiếu citation.

Danh mục chỉ giữ nguồn được trích trong nội dung, theo thứ tự xuất hiện đầu tiên. `n.d.` chỉ ngày xuất bản chưa xác định; ngày truy cập là 26/10/2026.

[1] W. Eddy, Ed., "Transmission Control Protocol (TCP)," IETF, STD 7, RFC 9293, Aug. 2022. [Online]. Available: RFC 9293. Accessed: Oct. 26, 2026.

[2] G. Gambetta, "Fast-Paced Multiplayer (Part I): Client-Server Game Architecture," n.d. [Online]. Available: Client-Server Game Architecture. Accessed: Oct. 26, 2026.

[3] G. Fiedler, "Snapshot Compression," *Gaffer On Games*, Jan. 4, 2015. [Online]. Available: Snapshot Compression. Accessed: Oct. 26, 2026.

[4] G. Gambetta, "Fast-Paced Multiplayer (Part II): Client-Side Prediction and Server Reconciliation," n.d. [Online]. Available: Client-Side Prediction and Server Reconciliation. Accessed: Oct. 26, 2026.

[5] G. Gambetta, "Fast-Paced Multiplayer (Part III): Entity Interpolation," n.d. [Online]. Available: Entity Interpolation. Accessed: Oct. 26, 2026.

[6] Oracle, "Class Socket," *Java Platform, Standard Edition 25 API Specification*, n.d. [Online]. Available: java.net.Socket — Java SE 25. Accessed: Oct. 26, 2026.

[7] G. Almes, S. Kalidindi, and M. J. Zekauskas, "A Round-trip Delay Metric for IPPM," IETF, RFC 2681, Sep. 1999. [Online]. Available: RFC 2681. Accessed: Oct. 26, 2026.

[8] R. Nystrom, "Game Loop," *Game Programming Patterns*, online chapter, n.d. [Online]. Available: trang chương sách; nội dung được kiểm tra qua bản nguồn trong repository của tác giả. Accessed: Oct. 26, 2026.

[9] G. Fiedler, "Fix Your Timestep!," *Gaffer On Games*, Jun. 10, 2004. [Online]. Available: Fix Your Timestep!. Accessed: Oct. 26, 2026.

[10] Oracle, "Interface BlockingQueue<E>," *Java Platform, Standard Edition 25 API Specification*, n.d. [Online]. Available: java.util.concurrent.BlockingQueue — Java SE 25. Accessed: Oct. 26, 2026.

[11] Oracle, "Package javax.swing," *Java Platform, Standard Edition 25 API Specification*, n.d. [Online]. Available: javax.swing — Java SE 25. Accessed: Oct. 26, 2026.

[12] G. Fiedler, "Snapshot Interpolation," *Gaffer On Games*, Nov. 30, 2014. [Online]. Available: Snapshot Interpolation. Accessed: Oct. 26, 2026.

[13] S. Josefsson, "The Base16, Base32, and Base64 Data Encodings," IETF, RFC 4648, Oct. 2006. [Online]. Available: RFC 4648. Accessed: Oct. 26, 2026.

[14] Oracle, "Class System," *Java Platform, Standard Edition 25 API Specification*, n.d. [Online]. Available: java.lang.System — Java SE 25, currentTimeMillis/nanoTime. Accessed: Oct. 26, 2026.

19. Phụ lục — Appendix

Phân công: TV4 ghép contract/cấu hình/bằng chứng; TV1 cung cấp session/room và thông tin nhóm, TV2 runtime/đồng bộ, TV3 gameplay. Phụ lục trích các phần cần để kiểm tra báo cáo, không chép toàn bộ repository.

A. Đặc tả message và sequence bổ sung

A.1. Framing và lớp phiên bản

Frame ứng dụng dùng big-endian: `length:int32` → `type:int16` → payload JSON UTF-8. Length tính từ type, không tính prefix bốn byte; giới hạn length 2..1048576 byte. Handshake protocol hiện là 2; snapshot encoder phát v2/v3/v5 theo frame dữ liệu, decoder nhận v2/v3/v4/v5. Hai số phiên bản có vai trò riêng, không thay thế nhau. Đặc tả đầy đủ nằm ở mục 9, nguồn trực tiếp là PacketEncoder, PacketType và IntegrationContract.

A.2. Tra cứu message quan trọng

Bảng A.1. Nhóm message dùng để đối chiếu log và source.

Message	Type hex	Hướng/ý nghĩa	Điểm cần kiểm tra
CONNECT / CONNECT_ACK	0001 / 0002	Handshake và phiên	Hoàn tất handshake trước request lobby
CREATE_ROOM / JOIN_ROOM / ROOM_STATE	0010 / 0011 / 0012	Quản lý phòng	Membership, capacity, roster authoritative
READY / START_GAME / GAME_STARTED	0013 / 0014 / 0015	Ready và khởi tạo match	Quyền host, context và match ID mới
MOVE	0020	C→S input	Sequence dương, direction/moving; không nhận tọa độ từ client
PLANT_BOMB	0021	C→S request placement	requestId/dedupe và luật game, không coi submit là đã đặt thành công
BOMB_PLANTED / BOMB_EXPLODE	0022 / 0023	S→C notice	eventId/tick, không áp lại world effect
ITEM_SPAWN / ITEM_PICKED_UP / PLAYER_DIE	0024 / 0025 / 0026	S→C notice	Snapshot vẫn là nguồn state
GAME_SNAPSHOT / FULL_STATE_SYNC	0030 / 0033	S→C snapshot	Kind, tick/baseTick, version và replica apply
REQUEST_FULL_STATE	0034	C→S resync	Match scope, lastAppliedTick, reason và rate gate
END_GAME / GAME_OVER	0035 / 0031	Kết thúc theo quyền / kết quả	Result authoritative, không tự tính winner ở UI
PING / PONG	0040 / 0041	Probe/heartbeat	pingId, phân biệt RTT với ACK input
ERROR	00FF	Phản hồi lỗi control/gate	Phân biệt lỗi schema/quyền/queue với luật gameplay

Không phải toàn bộ constant trong PacketType đều là tính năng ứng dụng đã bật: RECONNECT có mã 0043 nhưng chưa có flow resume trong lobby. Các field chi tiết của notice/result và giới hạn timeout/budget được dẫn ở bảng mục 9, không sao chép một schema thứ hai có thể lệch với codec.

A.3. Ví dụ canonical, chỉ để đọc contract

Các object dưới đây là ví dụ định dạng, **không phải packet capture của một lần chạy**. UUID minh họa không thay context được server cấp. TCP frame còn bao gồm prefix/type như A.1.

```
{"matchId":"00000000-0000-0000-0000-
000000000013","sequence":1,"direction":"RIGHT","moving":true}
{"matchId":"00000000-0000-0000-0000-000000000013","requestId":1}
{"matchId":"00000000-0000-0000-0000-
000000000013","lastAppliedTick":42,"reason":"BASELINE_MISMATCH"}
```

Client nên dùng codec thay vì tự tạo JSON tùy ý vì một số payload có định dạng canonical. Nguồn: MovePacketCodec, BombPacketCodec, ResyncPacketCodec, MatchEnvelopeCodec. Các sequence diagram kết nối, movement/resync, placement và replay đã có ở mục 9.6; nguồn Mermaid nằm tại figures/network.

B. Cấu hình/build/run/test/demo

B.1. Yêu cầu và classpath

Sử dụng JDK 25 và Maven phù hợp với POM; JAVA_HOME trỏ tới thư mục gốc JDK, không phải thư mục bin. Build production bằng build-demo.cmd; kiểm thử qua test.cmd, các check TV2 có thể chạy riêng. Không giả định project có Maven Wrapper hoặc cần database để chạy flow hiện tại.

Wrapper run chọn classpath theo root JAR → target JAR → target/demo-classes. Khi cần chắc chắn chạy bản source vừa build, chỉ định trực tiếp target/demo-classes; đợt đo cũ đã chỉ rõ classpath riêng trong từng manifest. Không xóa JAR cũ chỉ để chọn classpath; lưu hash artifact thực dùng khi thu kết quả.

B.2. Lệnh build và chạy lobby

Lệnh ví dụ cho PowerShell ở root repository; không phải log đã được thực thi ở nhiệm vụ viết phụ lục.

```
cmd /c scripts\build-demo.cmd
$reportJava = if ($env:JAVA_HOME) { Join-Path $env:JAVA_HOME 'bin/java.exe' } else { 'java'
}
# Chỉ chạy tiếp sau khi build exit code 0. Mở terminal server riêng:
& $reportJava -cp target/demo-classes com.bombit.server.app.ServerApplication 8888 --lobby
```

Ở từng terminal client riêng, đặt reportJava như trên rồi chạy:

```
& $reportJava -cp target/demo-classes com.bombit.client.app.ClientApplication 127.0.0.1
8888 'Player A'
```

GUI cho phép tạo/vào phòng, guest ready và host start khi đủ điều kiện. Client gameplay dùng phím mũi tên, Space đặt bom theo cạnh nhấn, rời cửa sổ theo flow ứng dụng. Server arg thứ hai dạng số chọn profile demo riêng; client --headless/--demo cũng là profile riêng. Không dùng headless như một tùy chọn tự động gia nhập lobby chính ở ví dụ trên. Nguồn bootstrap: ServerApplication, ClientApplication.

B.3. Test và thực nghiệm

```
cmd /c scripts\test.cmd
```

```
# Nếu Maven fail, các bước sau trong wrapper có thể chưa chạy; lưu exit/log rồi kiểm riêng:
cmd /c scripts\check-tv2.cmd
& $reportJava -cp 'target/classes;target/test-classes' com.bombit.integration.NetworkTest
```

Maven cần dependency có sẵn hoặc được tải ở máy thực hiện; offline chỉ dùng khi cache đã đủ. Lượt đo cũ dùng Maven/cache local cụ thể, không yêu cầu máy khác hard-code đường dẫn người dùng đó. Không sửa timeout/test để thay kết quả fail bằng pass.

Bảng B.1. Điểm vào tái lập các nhóm thực nghiệm đã công bố.

Nhóm	Công cụ/lệnh tham chiếu	Bằng chứng được giữ
Suite chức năng	test.cmd và main/check riêng theo task12	Maven log, 38 XML, summary và smoke log
D0 v2	DeltaCompressionCheck, classpath target/tv2-check	Năm stdout và tick CSV
D1 v5	report/experiments/DeltaExperiment.java	30 scenario-run, trace full-frame và byte/kind/base/equality
Phân tích delta	report/experiments/analyze_delta.py	all-runs/summary và hai hình PNG/SVG/PDF
TCP loopback	NetworkBenchmark --local --mode suite --seconds 10 --warmup 2	Năm suite, 45 scenario-run/125 peer rows
Reliability lặp	report/experiments/run_reliability.ps1 với Maven/cache/OutputRoot theo máy	Ba lượt có test, XML/log và lỗi chuẩn bị tách riêng
Phân tích mạng	report/experiments/analyze_network.py	Per-run metrics/summary và hình loopback

Các đường dẫn report/experiments là công cụ báo cáo, không phải module production. Chi tiết command/attempt và giới hạn script nằm trong README evidence của nhóm tương ứng ở phụ lục C. Khi tái lập, dùng OutputRoot/run ID mới để không append/ghi đè dữ liệu gốc; script phân tích hiện đọc bộ evidence đã chỉ định trong source, cần xác nhận đường dẫn trước khi dùng batch mới.

C. Test cases, log, CSV và kết quả bổ sung

C.1. Bản đồ bằng chứng

Bảng C.1. Nguồn dữ liệu thực tế và giới hạn sử dụng.

Bộ hồ sơ	Nội dung chính	Cách sử dụng
task01	Manifest, inventory, summary suite đầu với smoke failure	Dữ liệu khảo sát/lịch sử; kiểm tra file thực có trước khi đóng gói log cũ
task12	133/133 JUnit mới, check/main, XML và smoke log	Một lượt suite đầy đủ; không xóa lịch sử fail
task13	Raw D0/D1, trace, summary và script/hình	Byte frame, không TCP/IP hoặc latency mạng
task14	Raw CSV/log network, reliability XML, manifest/checksum và lỗi môi trường	45 network run và 84 thực thi test bổ sung; không cộng vào 133 test
task15	Claim ledger và đối chiếu đóng góp	Kiểm tra lập luận, không phải lần đo mới
task16	Hạn chế, future work, số liệu kết luận	Phân biệt đề xuất với trạng thái đã làm

Kết quả số chi tiết và biểu đồ nằm ở mục 12, không cần chép toàn bộ raw CSV vào bản in. Khi nộp, thư mục evidence/script phải đi cùng source theo cấu trúc đường dẫn hoặc có manifest ánh xạ tương ứng. Không chỉ gửi một bảng summary rồi bỏ raw data dùng để tính nó.

C.2. Từ điển dữ liệu tối thiểu

Bảng C.2. Các cột cần để đọc hoặc phân tích lại.

Bộ dữ liệu	Cột/khóa chính	Định nghĩa
D0 tick CSV	tick, full_frame_bytes, delta_frame_bytes	Null update tính 0; stdout gốc còn dòng PASS ngoài CSV
D1 tick CSV	tick, full_frame_bytes, delta_frame_bytes, kind, base_tick, replica_equal	Có full định kỳ, delta hoặc SKIP; equality chuẩn hóa theo harness
D1 trace	tick, event, full_frame_base64	Full frame đầu vào cùng tick, event script; không phải packet capture
Network raw	run_id/scenario/client_index, scope, revision, window_seconds, status/error/cleanup_ok	Định danh peer và phạm vi attempt
Network latency	rtt_samples/mean/p95, move_ack_samples/mean/p95, plant_ack_ms	RTT khác input ACK, plant là observation; không có sample không thay 0 ms
Network lưu lượng	tx/rx_packets, tx/rx_bytes, snapshot_bytes, full/delta_snapshots	Frame/message ứng dụng, chưa gồm TCP/IP
Network cuối cửa sổ	pending_ping_end, pending_move_end, stale_snapshots, resync_requests	Chưa hoàn tất khác packet loss; không gộp các số này vào lỗi cùng nghĩa
Reliability	attempt, suites, testcase, exit_code, failures/errors/skipped	XML mới theo attempt; invocation chưa chạy test tách khỏi test fail

Đơn vị tổng hợp và công thức được nêu ở mục 11.6/12.6: mean có trọng số sample trong run, worst-peer p95, rồi median/min–max qua run. Không suy pooled p95 từ CSV aggregate hoặc coi 125 peer rows là 125 run độc lập.

C.3. Bằng chứng còn cần trước bản nộp

Nhóm cần bổ sung thông tin bìa/họ tên/MSSV, metadata CPU/RAM nếu công bố môi trường đầy đủ, biên bản GUI và LAN nếu muốn nhận phạm vi đó. Không tạo ảnh thay cho trận chưa chạy. Ảnh/video phải có run/match, thời điểm, topology, người kiểm tra và log đối chiếu; nếu chưa làm thì giữ trạng thái chưa nghiệm thu như mục 12.7/15. Các sơ đồ và biểu đồ được đưa vào báo cáo; cần kiểm tra lại mục lục/số trang sau khi nhóm chỉnh sửa thông tin bìa.

D. Mã giả/đoạn code quan trọng

D.1. Ranh giới xuất bản baseline

Mã giả sau rút gọn từ GameLoop, không thay implementation đầy đủ:

```
update = builder.buildUpdate(state, tick, forceFull)
if update != null:
    if outbound.offer(update):
        builder.commit(update)
        clearPublishedNotices()
        latest = update
```

```
    else:
        countOutboundFull()
        if forceFull: requestFullAgain()
```

Điểm cần bảo vệ là không commit nền của update chưa vào queue; commit này vẫn không phải receipt ACK từ client. Nguồn: GameLoop.

D.2. Nhận state và replay local

```
check match scope; decode snapshot
result = replica.apply(update)
if result == NEED_FULL_STATE: request full through sender; keep old replica
if result == STALE: return
if result == APPLIED:
    if respawn changed: discard pre-respawn pending input
    refresh prediction map/entities
    restore local authoritative state; drop inputs <= ACK; replay remaining inputs
    record complete replica into interpolation history
```

Đây là tóm tắt control flow, không mô tả correction smoothing hoặc resume đã triển khai. Nguồn: MovementClientController và PredictionController. Mã giả thuật toán đầy đủ hơn và thứ tự batch gameplay ở mục 7/10 được dẫn lại thay vì chép toàn source vào phụ lục.

E. Bảng đóng góp thực tế — nhóm xác nhận

Bảng E.1. Trách nhiệm theo tài liệu; phần xác nhận cá nhân còn thiếu.

Thành viên theo tên gọi	Phạm vi module/nội dung đã phân công	Bằng chứng cần nhóm bổ sung để xác nhận cá nhân	Trạng thái xác nhận
TV1 — Mai	Transport/session/room/chat/heartbeat, UI lobby/kết quả; tổng hợp bìa/kết luận	Commit/PR hoặc biên bản phần thực hiện/review và giải thích khi bảo vệ	Chưa có xác nhận cá nhân
TV2 — Chinh	Model/core/movement/runtime, delta/replica, prediction/reconciliation/interpolation, map/player/HUD	Commit/PR, phần code/test/report thực hiện và review	Chưa có xác nhận cá nhân
TV3 — Duy	Bom/chain/block, item/damage/lives/result, event và effect presentation	Commit/PR, test luật và phần code/report thực hiện	Chưa có xác nhận cá nhân
TV4 — Giang	Protocol/router/ClientNetwork, metrics/runner/evaluation, ghép báo cáo và evidence	Commit/PR, log/script/analysis/report thực hiện và review	Chưa có xác nhận cá nhân

Nguồn phân công: nhiemvu.md, FILE_OWNERSHIP_NOTES và ghi chú đầu các chương. Bảng này không tự chứng nhận tác giả mọi dòng code, số giờ hoặc phần trăm đóng góp. Trước khi nộp, mỗi người xác nhận nội dung thật, họ tên/MSSV và minh chứng tương ứng; không tự chia phần trăm bằng nhau hoặc suy công sức từ số file.